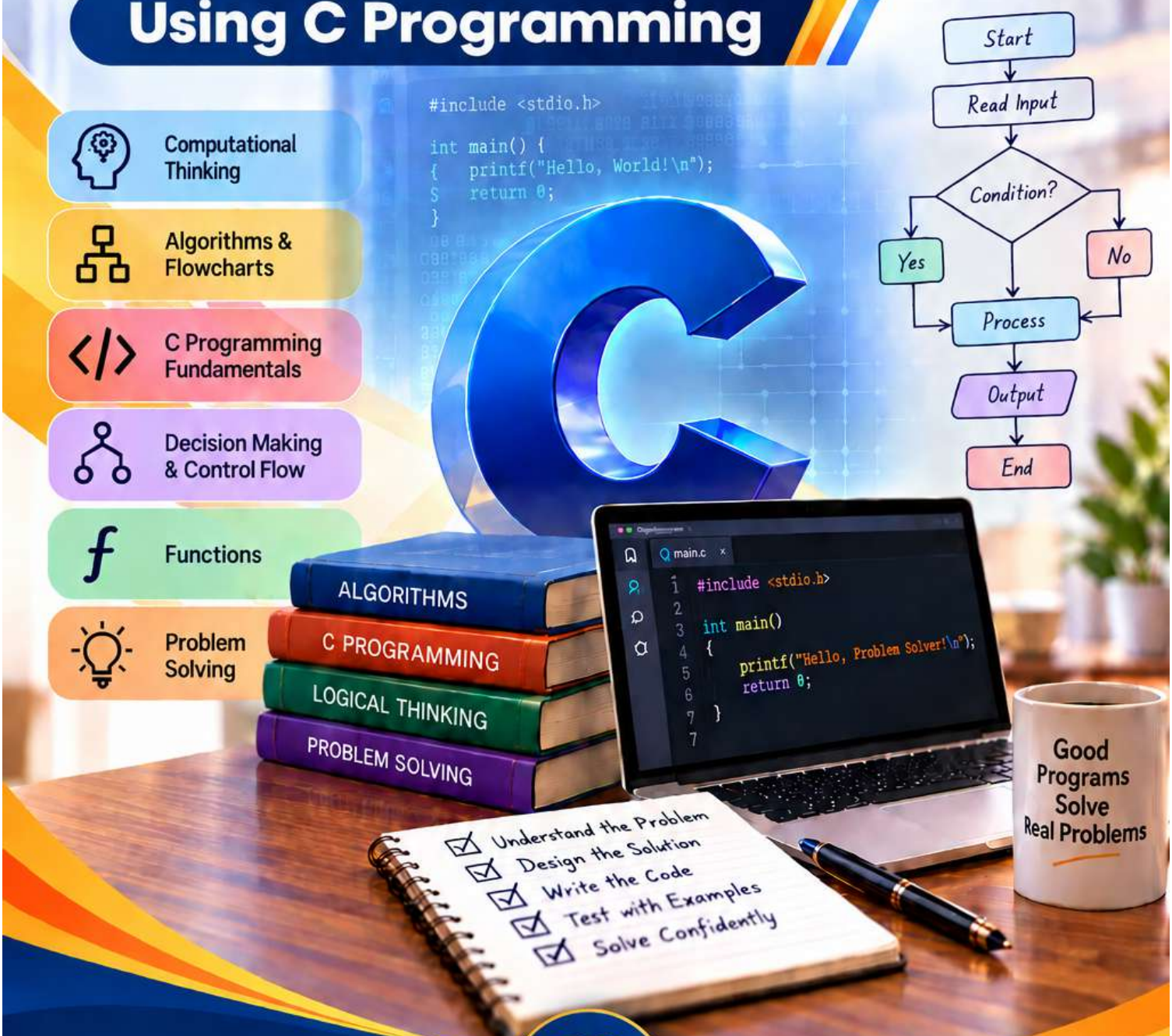
Computational Intelligence (SoC)
B.E. / B.Tech. Programme

Programming for Problem Solving
Course Code: 26CSE1002J

PROGRAMMING FOR PROBLEM SOLVING

Using C Programming

- Computational Thinking
- Algorithms & Flowcharts
- C Programming Fundamentals
- Decision Making & Control Flow
- Functions
- Problem Solving



PREMANAND S

A Note before we begin!

This book is written to be read like a story, not memorised like a manual. It has two units and six parts, and every chapter follows the same rhythm, so you always know what is coming next.

Part I and Part II (Chapters 1 to 6, from "Why Programming?" through "Why Learn C in 2026?") are pure orientation: the big ideas you need in your head before you touch code: what a problem is, how a computer understands instructions, how paradigms differ, and where C comes from. These chapters flow like a plain, continuous explanation, grounded in apps you already use, so there is nothing to practise here, only something to understand.

Part III and Part IV (Chapters 7 to 11, from "Algorithm" onward) are where thinking turns into doing. Part V (Chapters 12 to 16) then teaches the building blocks of C: the anatomy of a program, header files, data types and variables, operators and decision making. Part VI (Chapters 17 to 27, Unit 2) teaches functions, pointers and recursion. From Chapter 7 onward every topic follows the same rhythm:

- **Analogy** — an everyday comparison that builds intuition first.
- **Technical Concept** — the formal definition, stated precisely.
- **Where It Is Used** — the real systems and situations you will meet this idea in.
- **Why We Use It** — the reason this idea exists at all; the problem it solves.
- **Example** — a worked example, often step by step (the Solved Examples: Easy, Medium and HOTS).
- **Try It Yourself** — a small set of problems so the idea becomes yours, not just something you read.
- **Important Questions** — the questions most likely to be asked in exams and interviews, with model answers (every chapter from Chapter 12 onward).
- **Multiple Choice Questions** — a quick self-test with the answer and the reason shown.
- **Solved Programs** — complete programs with the problem, the approach, the code and its verified output.
- **Practice Problems — Try It Yourself** — unsolved problems to attempt on your own before moving to the next chapter.

The coloured boxes mean the same thing everywhere in the book: blue is the Analogy, grey is the Technical Concept, green is Where It Is Used and yellow is Why We Use It.

Read the chapters in order the first time through. They are written as one continuous journey — from the very first question ('Why do we even need programming?') to the moment your compiled C

program finally moves electrons inside the CPU. By the last page, you should be able to explain the whole journey to someone else, in your own words, without notes.

Let us begin.

PART I

The World Before Code

Chapter 1
Why Programming?

We solve problems every day — waking up on time, planning a route, splitting a bill between friends. Most of these problems are small enough that our own brain, unaided, gets the job done in a second. But what happens when a problem becomes too large, too repetitive, or too complex for a human to handle alone? What happens when a business needs to check a million bank balances every night, or a website needs to recommend the right movie to ten million different people, all at the same time?

This is the big question every programmer eventually asks: **can we teach a computer how to solve the problem for us?** That single question is where programming begins.

What is Programming?

Programming is the process of telling a computer, in painstaking detail, how to solve a problem. It always follows the same four-step flow:

PROBLEM → INSTRUCTIONS → COMPUTER → RESULT

We start with a **problem** — something we want solved. We express the solution as a set of **instructions**, written precisely enough that nothing is left to guesswork. We hand those instructions to a **computer**, which carries them out exactly as written, at enormous speed and without ever getting tired or bored. And out comes the **result** — the answer, the action, the outcome we originally wanted.

This is exactly how a good programmer thinks, every single time, no matter how big or small the problem is: **understand the problem, break it into steps, express the steps precisely, then execute and test.** A *program*, in the end, is nothing more than a precise set of instructions that a computer can execute — the middle two stages of that flow, written down in a language the machine can follow.

Where This Flow Shows Up — Real Apps You Already Use

This problem → instructions → computer → result flow is not an abstract classroom idea — it is quietly running behind almost every app on your phone right now.

BookMyShow. The problem is "find an available seat for tonight's 8 p.m. show and confirm my booking before someone else takes that seat." The instructions check the seating layout, mark your chosen seat as reserved, calculate the total price with taxes and convenience fees, and process your payment — all as a strict, ordered sequence of steps. The computer (a server, not your phone) executes those instructions in a fraction of a second. The result is your seat, locked in, with a ticket sent to your phone.

Swiggy. The problem is "get a specific meal from a specific restaurant to my specific address, as fast as possible." The instructions match your order to a nearby restaurant, calculate the best available delivery partner based on distance and current load, track the order's status in real time, and update your estimated delivery time as conditions change. The computer runs all of this simultaneously for thousands of orders across a city. The result is a hot meal at your door, and a live map showing exactly how it got there.

WhatsApp. The problem is "get this message from my phone to my friend's phone, instantly, and let both of us know it arrived." The instructions break your message into small data packets, route them across the internet through the fastest available path, reassemble them correctly on the other end, and send back a delivery confirmation (the blue ticks). The computer does this billions of times a day, for billions of users. The result is a message that feels instantaneous, even though a small ocean of instructions ran underneath it.

Notice something important: in every one of these examples, the *problem* was something a normal person could describe in one sentence, but the *instructions* needed to solve it precisely enough for a computer had to be broken down into many small, unambiguous, ordered steps. Learning to make that jump — from a one-sentence wish to a precise sequence of steps — is what the rest of this book is about.

Chapter 2

How Does a Computer Understand Our Instructions?

> **Analogy:** Imagine instructing a robot arm on a factory floor. You could flip the individual electrical switches yourself to move each motor directly — painfully tedious, but it is the only language the robot's motors truly understand. Or you could use a small remote control with clearly labelled buttons — ROTATE, GRIP, LIFT — short commands that still map one-to-one onto specific movements. Or, best of all, you could simply type a plain instruction into a control screen: "Pick up the box from Station A and place it on the belt at Station B" — and let the control software work out every low-level motor command for you. All three approaches move the same robot arm. They differ only in how close to the raw hardware you, the instructor, are forced to think.

> **Technical Concept:** This is exactly the choice every programmer has, formalised into three levels of programming language. **Machine language** is the raw electrical switches — patterns of 0s and 1s wired directly into what the CPU executes. **Assembly language** is the labelled remote control — short mnemonic codes such as MOV and ADD that stand in one-to-one for specific machine instructions, translated by a program called an **assembler**. **High-level language** is the plain typed instruction — human-readable code such as C, translated into machine language by a compiler or interpreter, so the programmer can think about the problem instead of the hardware.

The Three Levels of Programming Languages

1. Machine Language

Machine language is a stream of pure binary — 0s and 1s — wired directly into what a specific CPU family can execute. It is never portable: a pattern of bits that means "add two numbers" on an Intel processor means something completely different, or nothing at all, on an ARM processor.

Where it's useful: Virtually nobody writes machine language by hand today. It matters when reverse-engineering software, analysing malware, debugging at the hardware level, or simply understanding what a compiler eventually produces for you.

Languages under this category: There isn't a single "machine language" — each processor family has its own: x86 machine code (most laptops and desktops), ARM machine code (most phones), RISC-V machine code (many modern embedded chips).

Example — Machine Language (conceptual)

```
10110000 01100001
```

On an x86 processor, a bit pattern close to this loads the value 97 into a small storage location inside the CPU called a register. No words, no structure a human can casually read — just a fixed, hardware-specific pattern.

2. Assembly Language

Assembly language replaces raw bit patterns with short, human-readable mnemonics — MOV, ADD, JMP — each of which still corresponds to exactly one machine instruction. A program called an **assembler** converts this mnemonic code into machine language.

Where it's useful: Writing tiny, performance-critical routines; device drivers; bootloaders (the first code that runs when a computer powers on); embedded firmware with extremely limited memory; and the lowest levels of an operating-system kernel.

Languages under this category: x86 Assembly, ARM Assembly, MIPS Assembly, RISC-V Assembly — each tied to one specific processor family.

Example — Assembly Language

```
MOV AL, 5
ADD AL, 3
```

This loads the value 5 into register AL, then adds 3 to it — register AL now holds 8. Two short, readable lines, but still exactly one instruction each for the CPU, with no translation ambiguity.

3. High-Level Language

A high-level language lets a programmer describe *what* should happen using readable words, structured logic, and named variables, without specifying which register or which exact machine instruction to use. A compiler or interpreter handles that translation, which is also what makes high-level code portable across very different processors.

Where it's useful: Essentially all application software — web apps, mobile apps, games, data science, business systems — and, in C's case, systems programming too, since C stays unusually close to the hardware for a high-level language.

Languages under this category: C, C++, Java, Python, JavaScript, Go, Rust, and effectively every language taught in a modern computing course.

Example — High-Level Language (C)

```
int sum = 5 + 3;
```

This single, readable line does the same job as the entire assembly snippet above. The compiler expands it into whichever exact machine instructions the target processor needs — you never have to think about registers at all.

> **Why We Use It:** If machine language is the only thing the CPU truly understands, why don't we all just write in 0s and 1s? Because it is extraordinarily slow, error-prone, and unreadable for humans. That pain directly motivated assembly language, and later, high-level languages like C — each level trading a little distance from the hardware for a large gain in human readability, speed of writing, and portability. **A program written by humans must eventually become instructions the computer can execute** — every convenience a high-level language gives you exists purely for your benefit; the machine only ever sees the final, translated binary.

Chapter 3
How Is a Program Translated?

We write programs for **humans** to understand. The **CPU** needs machine instructions. Bridging that gap is the job of a **translator** — and every translator is either a compiler or an interpreter.

The overall path is always: **source code** (a C program, written by you) passes through a **translator** (compiler or interpreter), which produces **machine code** (binary instructions), which the **CPU** finally executes.

Compiler

A **compiler** translates the entire program **before** execution begins. It produces a separate executable or object-code file, and once that file exists, it can be run directly, any number of times, without the compiler needing to be present again. **C is typically compiled.**

Interpreter

An **interpreter**, by contrast, executes a program through a language implementation that is present every time the program runs — translation and execution are closely combined, happening line by line, on the spot. This style is commonly associated with **Python**.

Aspect	Compiler	Interpreter
Translation timing	Entire program, before execution	Line-by-line, during execution
Output produced	A separate executable file	No separate executable file
Execution speed	Fast (already translated)	Slower (translated every run)
Error reporting	All errors listed after full compilation	Stops at the first error it meets
Typical languages	C, C++, Rust	Python, JavaScript (classic style)

It is worth being precise about one thing here, because it is a common point of confusion: **compiler and interpreter describe *how* a program is translated and executed — they are not programming paradigms.** A paradigm is a way of *thinking* about a solution, which is a completely different idea, and the one we turn to next.

Chapter 4
Programming Paradigms

Analogy: Ask four different people to organise a birthday party and you will likely see four different styles of thinking. One writes a strict step-by-step to-do list — "first book the hall, then order the cake, then send the invites." Another thinks in terms of self-contained roles that interact — "the caterer handles food, the decorator handles decor, they just talk to each other when needed." Another thinks in pure transformations — "guest list goes in, seating chart comes out, nothing else changes." And another simply states the outcome and lets someone else work out how — "there must be enough seats for everyone invited." None of these styles is wrong; they are simply different **paradigms**, different lenses for thinking about the very same problem.

Technical Concept: A **programming paradigm** is a fundamental style or approach to structuring and thinking about a program's solution. Four paradigms dominate modern programming, and — importantly — **a single language can often support more than one of them.**

1. Procedural — Step-by-Step Instructions

A procedural program is simply a sequence of steps, grouped into functions, executed in order, operating on shared data. There are no objects, no classes — just data and the functions that act on it.

Where it's useful: operating-system kernels, embedded firmware, simple utilities, and any situation demanding predictable, linear control flow. **C is the classic procedural language**; Pascal and Fortran are close relatives.

Example — Procedural (C) — summing 1 to 5

```
int total = 0;
for (int i = 1; i <= 5; i++) {
   total = total + i;
}
printf("%d", total);
```

A running total is updated step by step inside a loop. There is no object being modelled here — just a sequence of instructions acting on a shared variable, total.

2. Object-Oriented — Objects and Behaviour

An object-oriented program is organised around **objects** — self-contained bundles of data (called attributes) and behaviour (called methods) — built from a **class**, which acts as the blueprint.

Where it's useful: large software systems, GUI applications, game engines, enterprise software — anywhere you naturally model real-world entities such as a car, an employee, or a bank account. **Java, C++, Python, and C# all support this style.**

Example — Object-Oriented (Java-style) — a Car class

```
class Car {
    String color;
    void drive() {
        System.out.println(color + " car is driving");
    }
}

Car myCar = new Car();
myCar.color = "Red";
myCar.drive();
```

Car is the class (the blueprint). myCar is an object — one specific car built from that blueprint. drive() is a method describing behaviour. Bundling data (color) together with behaviour (drive()) inside one unit is the essence of object-oriented thinking.

3. Functional — Functions and Transformations

A functional program is built from pure functions that transform input into output, closely following mathematical function composition, while deliberately avoiding changes to shared, mutable state.

Where it's useful: data-processing pipelines, parallel and concurrent systems, and situations (like financial systems) where predictable, side-effect-free behaviour matters. **Haskell** is built entirely around this style; modern JavaScript and Python both support a functional flavour.

Example — Functional (JavaScript-style) — doubling a list

```
const numbers = [1, 2, 3, 4, 5];
const doubled = numbers.map(n => n * 2);
```

map takes a small function (n => n * 2) and applies it to every element, producing a brand-new list. There is no loop counter and no variable being changed in place — just one transformation applied uniformly.

4. Logic / Declarative — Facts and Rules

In a logic program, the programmer states **facts** and **rules**, and the system works out how to derive an answer through logical inference — the programmer never spells out the step-by-step search process.

Where it's useful: expert systems, natural-language processing research, puzzle solvers, and AI planning tools. **Prolog** is the best-known example.

Example — Logic (Prolog) — who is a grandparent

```
parent(john, mary).
parent(mary, susan).
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).
```

We only state two facts and one rule ("X is a grandparent of Z if X is a parent of some Y, and Y is a parent of Z"). We never say *how* to search — the system works out on its own that grandparent(john, susan) is true.

Why We Use It: No single paradigm is "best" for every problem — each simply organises complexity differently, and **a programming language can support multiple paradigms** at once. C, the language this course is built around, sits firmly in the procedural camp: it expresses a solution as an ordered sequence of steps operating on data, which is exactly why the algorithmic thinking you will build in the coming chapters translates so directly and cleanly into C code.

PART II

Enter C

Chapter 5
Where Does C Fit?

C is a programming language designed to provide both programmer convenience and low-level control — a general-purpose language that is procedural/imperative in style (step-by-step computation), statically typed (every type is known at compile time), and typically compiled (source code is turned into machine code ahead of execution).

What makes C special is a combination of qualities that few languages manage to hold onto at once: it is **efficient**, staying close to the hardware; it is **portable**, so programs can be adapted across very different platforms; it gives the programmer direct **memory control** through pointers and dynamic memory; it is **structured**, built around functions and modular programming; and it serves as the **foundation** for systems and embedded programming. In short, **C sits between human-friendly abstraction and machine-level control** — closer to the machine than most languages you will meet later, yet still readable enough for a human to write and reason about.

Why Was C Created?

C was born from a real problem: **how can we build powerful system software efficiently?** The lineage that leads to C runs through three names — **BCPL**, created by Martin Richards, influenced **B**, created by Ken Thompson, which evolved into **C**, created by Dennis Ritchie. Ritchie then used C to build **UNIX**, establishing systems programming as C's natural home.

The problem C was solving was a genuine tension in the tools available at the time. **Assembly language** offered low-level control but was difficult to develop and maintain. **Higher-level languages** offered abstraction but were not ideal for certain systems-programming tasks. **C aimed to provide a practical balance between abstraction, efficiency, and hardware control** — and it was developed at **Bell Labs during the early 1970s**, exactly to fill that gap.

C and UNIX — A Turning Point in Computing

C became powerful because it was used to build real system software. **Hardware** (CPU and memory) runs the **UNIX** operating system, and **C**, as a systems language, implemented much of UNIX itself — with everyday **applications** built on top of that operating system.

Why did this matter so much? Because C allowed large portions of system software to be written far more conveniently than assembly ever allowed. Programs written in C could be adapted to different computer architectures with far less rewriting. And as UNIX itself proved successful and portable, that success carried C's reputation well beyond its original environment. **C was no longer just a**

language experiment — it became a practical tool for building systems, and that reputation is exactly why it is still taught and used today.

The Evolution of C

C did not stay frozen after its creation at Bell Labs — it moved from a systems language to a modern, internationally governed standard over five decades of steady refinement.

Standard	Year	Key Highlights
K&R C	1978	First widely used version, documented in Kernighan & Ritchie's book "The C Programming Language"
C89 (ANSI)	1989	First major ANSI standardization of C
C90 (ISO)	1990	ISO standardization of C
C99	1999	// comments, declarations inside for loops, variable-length arrays, inline, improved integer types
C11	2011	Multithreading support, atomics, _Generic, improved Unicode support and safer library features
C17	2018	Clarifications, corrections and defect fixes — largely a refinement of C11
C23	2024	Further modernization, including nullptr, true/false as keywords, digit separators and other improvements

The important point running underneath this whole table is this: **new standards keep adding capabilities and improving the language, but C continues to emphasize efficiency, portability, and control** — the same three values it was built around at Bell Labs, more than fifty years ago.

Chapter 6
Why Learn C in 2026?

If Python, Java, and modern AI tools already exist, a fair question to ask is: why are we still learning C? The answer lies in what learning C actually trains in you, not merely in what C can build.

Learning C builds a connected chain of skills: it strengthens **problem solving** — thinking before coding; it makes **memory** visible — understanding what actually happens underneath a variable; it develops **algorithms** — the ability to build efficient solutions; and it cultivates **systems thinking** — connecting software and hardware in your head instead of treating one as a black box for the other.

C gives you this foundation in several concrete ways: it builds **computational thinking** rather than only syntax knowledge; it makes **memory, data, and execution** more visible than higher-level languages typically allow; it strengthens your understanding of **arrays, pointers, functions, and recursion**; it provides a direct foundation for **C++, operating systems, embedded systems, and computer architecture**; and it helps you understand what higher-level languages often quietly **hide from you**.

The goal is not to become "a C programmer." The goal is to become a better problem solver. Every idea you build here — breaking a problem into precise steps, thinking about what memory a variable actually occupies, reasoning about how a loop or a function behaves — will still be exactly as useful the day you move on to Python, Java, or whatever comes after them.

PART III

Thinking Before Coding

We now know what C is, where it came from, and why it still matters. From here on, this book changes gears. **Good programmers don't start with code — they start by understanding the problem.** The path every one of them follows is the same: Problem → Analyze → Design → Algorithm → Program → Test → Improve. That mindset — understand, design, implement, test, improve — is what turns code from guesswork into engineering, and it is exactly what the next four chapters teach, one stage at a time, starting with the algorithm.

Chapter 7
Algorithm — The Recipe Behind the Code

Ask any adult to make a cup of tea and they will do it without a second thought — boil water, add tea leaves, wait, strain, add milk and sugar. It feels effortless because you have done it, or seen it done, a hundred times before. Now hand that exact same task to a robot, or to a computer. Nothing happens. Machines have no common sense, no intuition, and no memory of having watched tea being made. **Unless every single step is spelled out precisely, in the right order, with absolutely no gaps left for guesswork, a machine cannot do anything at all.** This precise, ordered list of steps — the thing that turns "make tea" from an instinct into something a mindless-but-obedient machine can actually follow — is called an **algorithm**.

Analogy: A recipe card for a cake does not depend on whose kitchen you are in, which brand of oven you own, or which language the cookbook happens to be printed in. It is a language-independent, step-by-step method that works anywhere, as long as it is followed exactly. An algorithm is exactly this — a recipe for solving a computational problem, written so precisely that it does not matter whether a human, a robot, or a program in C ends up carrying it out.

Technical Concept: An **algorithm** is a finite, ordered sequence of well-defined, unambiguous steps that takes some input and produces the desired output in a finite amount of time. A correct algorithm must have: **finiteness** (it must terminate, eventually), **definiteness** (each step is precisely and unambiguously defined), **input** (zero or more values supplied before it begins), **output** (at least one value produced), and **effectiveness** (each step must be simple enough to be carried out, in principle, exactly).

Where It Is Used: Every piece of software — from a search engine ranking billions of pages to a traffic light choosing when to turn green — is built on an underlying algorithm, whether or not the programmer ever writes it down formally.

Why We Use It: We design the algorithm *before* writing code because it lets us reason about correctness and efficiency independently of any particular programming language, catch logical mistakes early and cheaply (on paper, rather than after hours of debugging), and communicate the solution to other people (or to ourselves, months later) clearly.

Five Algorithms, Step by Step

The best way to internalise what an algorithm really is happens to be the simplest: work through several of them, side by side, until writing precise, ordered steps starts to feel natural.

Algorithm 1 — Making a Cup of Tea

Example — Making Tea

Step 1: Start.

Step 2: Boil 200 ml of water.

Step 3: Add one teaspoon of tea leaves to the boiling water.

Step 4: Wait two minutes for the tea to brew.

Step 5: Strain the tea into a cup.

Step 6: Add milk and sugar to taste.

Step 7: Stop.

Even this everyday task, once written out, satisfies every property of a good algorithm — it is finite (seven steps, then it ends), definite (no step is ambiguous), and effective (each step can actually be carried out).

Algorithm 2 — Find the Largest of Three Numbers

Example — Largest of Three Numbers

Step 1: Start.

Step 2: Read three numbers, a, b, and c.

Step 3: If a > b and a > c, then largest = a.

Step 4: Else if b > c, then largest = b.

Step 5: Else, largest = c.

Step 6: Display largest.

Step 7: Stop.

Algorithm 3 — Check Whether a Number Is Even or Odd

Example — Even or Odd

Step 1: Start.

Step 2: Read a number, n.

Step 3: Divide n by 2 and find the remainder.

Step 4: If the remainder is 0, then n is even.

Step 5: Else, n is odd.

Step 6: Display the result.

Step 7: Stop.

Algorithm 4 — Sum of the First N Natural Numbers

Example — Sum of First N Natural Numbers

Step 1: Start.

Step 2: Read a number, n.

Step 3: Set sum = 0 and counter i = 1.

Step 4: While i is less than or equal to n, repeat: add i to sum, then increase i by 1.

Step 5: Display sum.

Step 6: Stop.

Algorithm 5 — Swap the Values of Two Variables

Example — Swap Two Numbers

Step 1: Start.

Step 2: Read two numbers, a and b.

Step 3: Store a in a temporary variable, temp.

Step 4: Copy the value of b into a.

Step 5: Copy the value of temp into b.

Step 6: Display the new values of a and b.

Step 7: Stop.

Notice how the temporary variable, temp, is essential — without it, the original value of a would be lost the moment you copied b into it.

How to design an algorithm, step by step

1. Understand the problem completely — restate it in your own words, and identify exactly what is given (input) and exactly what is wanted (output).

2. Work out a solution by hand first, on paper, using small concrete numbers, before thinking about any code.

3. Break your hand-solution into small, precise, ordered steps — the level of detail a very literal-minded assistant would need.

4. Check your steps against edge cases (what if two numbers are equal? what if the input is empty?).

5. Only now translate the finished, checked algorithm into a programming language.

Try It Yourself

1. Write an algorithm to check whether a given number is even or odd, without looking back at Algorithm 3.

2. Write an algorithm to calculate the sum of the first N natural numbers, without looking back at Algorithm 4.

3. Write an algorithm to swap the values of two variables without using a third variable. (Hint: think about addition and subtraction, or think about what a calculator can do that a recipe can't.)

4. Write an algorithm to find the factorial of a given number n.

5. Write an algorithm to check whether a given number is a prime number.

6. Write an algorithm to find the smallest of three given numbers.

7. Write an algorithm to reverse the digits of a given number (for example, 123 becomes 321).

8. Write an algorithm to calculate the simple interest given the principal, rate, and time.

9. Write an algorithm to count how many vowels appear in a given word.

10. Explain, using the five properties of a good algorithm, why "keep guessing numbers until you find the answer" is a poor algorithm for finding the square root of 49.

Chapter 8
Pseudocode — Writing Like a Programmer, Without a Programming Language

Analogy: Before an architect draws a fully precise blueprint with exact measurements, they often first sketch the layout of rooms informally on paper — "kitchen here, bedroom there, door on this wall" — using an informal but structured notation everyone in the profession understands, without worrying yet about exact millimetres. Pseudocode plays this same role between an algorithm and real code.

Technical Concept: Pseudocode is a structured, informal, language-agnostic way of describing an algorithm using a mixture of plain human language and programming-style structural keywords (IF, ELSE, WHILE, FOR, READ, PRINT), without following the strict syntax rules of any specific real programming language such as C or Python. It sits deliberately between plain English (too loose) and real code (too strict, too early). Because pseudocode is meant to be read comfortably by anyone — not just someone who already knows a language's symbols — this book deliberately spells conditions out in words, such as **GREATER THAN**, **LESS THAN**, and **EQUAL TO**, instead of using symbols like >, <, and ==. Once you move to a real program in Chapter 10, those words become the familiar C symbols.

Where It Is Used: Pseudocode appears in textbooks, technical interviews, design documents, and whiteboard discussions — anywhere the logic of a solution needs to be communicated clearly to programmers who may work in entirely different real languages.

Why We Use It: Writing pseudocode forces you to nail down the *logic* of a solution before getting distracted by a real language's punctuation rules (semicolons, curly braces, exact function names). It also means the same pseudocode can be handed to a C programmer, a Python programmer, or a Java programmer, and each can translate it correctly into their own language.

Five Examples, in Pseudocode

Here are the same five tasks from Chapter 7, now written as pseudocode — structured enough to guide a program, but still in plain words rather than symbols.

Example 1 — Making Tea

Example — Pseudocode: Making Tea

```
BEGIN
  BOIL 200 ml of water
  ADD one teaspoon of tea leaves
  WAIT for two minutes
  STRAIN into a cup
  ADD milk and sugar to taste
END
```

Even a everyday task like this reads naturally as pseudocode — plain instructions, one per line, in a fixed order.

Example 2 — Largest of Three Numbers

Example — Pseudocode: Largest of Three Numbers

```
BEGIN
  READ a, b, c
  IF a GREATER THAN b AND a GREATER THAN c THEN
     largest = a
  ELSE IF b GREATER THAN c THEN
     largest = b
  ELSE
     largest = c
  ENDIF
  PRINT largest
END
```

Notice how close this already looks to real code, yet it uses no language-specific syntax — no semicolons, no comparison symbols, no data types. Writing GREATER THAN in words instead of > keeps this readable to absolutely anyone, even someone who has never seen a line of code.

Example 3 — Even or Odd

Example — Pseudocode: Even or Odd

```
BEGIN
  READ n
  remainder = n MOD 2
  IF remainder EQUAL TO 0 THEN
     PRINT "n is even"
  ELSE
     PRINT "n is odd"
  ENDIF
END
```

MOD (short for modulus) gives the remainder after division — a common pseudocode keyword you will meet again in real C code as the % operator.

Example 4 — Sum of the First N Natural Numbers

Example — Pseudocode: Sum of First N Natural Numbers

```
BEGIN
  READ n
  sum = 0
  i = 1
  WHILE i LESS THAN OR EQUAL TO n REPEAT
     sum = sum + i
     i = i + 1
  ENDWHILE
  PRINT sum
END
```

The WHILE loop repeats its two inner steps — add i to sum, then increase i — for as long as the condition stays true, exactly the same repeating idea from Algorithm 4 in the previous chapter.

Example 5 — Swap Two Numbers

Example — Pseudocode: Swap Two Numbers

```
BEGIN
  READ a, b
  temp = a
  a = b
  b = temp
  PRINT a, b
END
```

Three plain assignment lines are all it takes — temp temporarily holds a's original value so it is not lost when b is copied into a.

Try It Yourself

1. Convert your algorithm for "check even or odd" from Chapter 7 into pseudocode, using words such as EQUAL TO rather than symbols.

2. Write pseudocode to find the sum of the first N natural numbers, using a WHILE loop, without looking back at Example 4.

3. Write pseudocode for a simple ATM PIN check: read a PIN, compare it to a stored correct PIN using EQUAL TO, and print "Access Granted" or "Access Denied."

4. Write pseudocode to find the factorial of a given number n, using a loop.

5. Write pseudocode to check whether a given number is a prime number.

6. Write pseudocode to find the smallest of three given numbers, using LESS THAN instead of any symbol.

7. Write pseudocode to reverse the digits of a given number.

8. Write pseudocode to calculate simple interest given the principal, rate, and time.

9. Write pseudocode to check whether a given year is a leap year (divisible by 4, but if divisible by 100 it must also be divisible by 400).

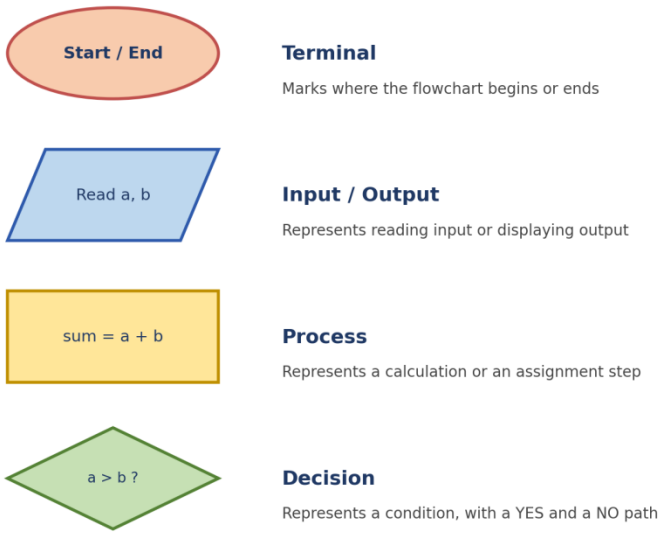
10. Take any one of the five pseudocode examples in this chapter and rewrite it using symbols (>, <, ==) instead of words. Which version do you personally find easier to read, and why?

Chapter 9
Flowchart — Seeing the Logic Instead of Reading It

Analogy: A metro (subway) map does not describe your journey in a paragraph of text — "take the blue line, then change at the interchange station to the red line, then get off two stops later." Instead, it shows you the same information as a picture: stations as points, lines as connections, interchanges as junctions. Some people grasp a route far faster from the map than from the paragraph. A flowchart is this same idea, applied to the logic of an algorithm.

Technical Concept: A **flowchart** is a diagrammatic representation of an algorithm, using standardised geometric symbols connected by arrows to show the order in which steps are executed. Reading a flowchart is a skill in itself — before drawing one, you first need to know exactly what each shape means and when to reach for it.

The Symbols, and Where Each One Is Used



The four core flowchart symbols, with a sample of what each one holds.

The **Terminal** (oval) marks where a flowchart begins or ends — every flowchart has exactly one Start and at least one End. The **Input/Output** shape (a slanted parallelogram) represents reading a value in or displaying a value out — anywhere your algorithm says READ or PRINT. The **Process** shape (a

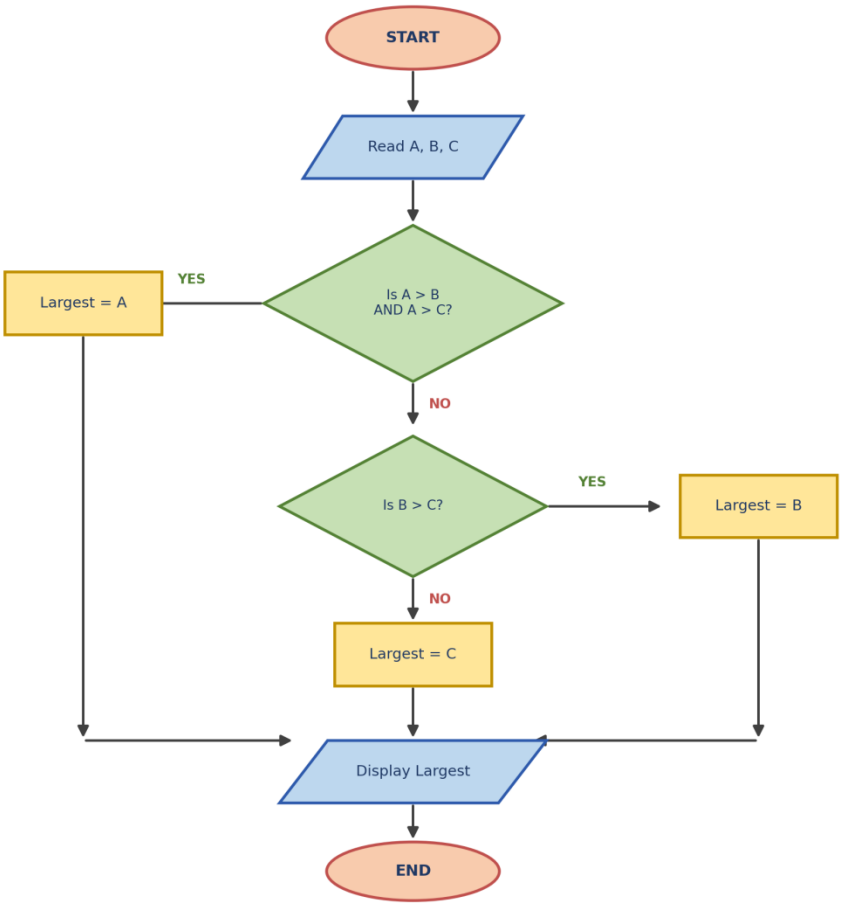
plain rectangle) represents a calculation or an assignment — anywhere a value is computed or stored, such as sum = a + b. And the **Decision** shape (a diamond) represents a condition with exactly two exits, labelled YES and NO — anywhere your algorithm branches, such as "is a greater than b?". Plain **arrows** connect all of these, and the direction of the arrow is what tells you which step happens next.

> **Where It Is Used:** Flowcharts are used in software design documents, business-process mapping, teaching logic to beginners, and documenting decision-heavy procedures (like a hospital's triage process or a customer-support script).

> **Why We Use It:** A flowchart makes the *shape* of the logic visible at a glance — you can immediately see branches, loops, and the overall structure of a solution in a way that is harder to see in a wall of text or code. This makes it an excellent tool for spotting logical gaps (like a missing ELSE branch) before any code is written.

Example: Largest of Three Numbers

Now that the symbols are familiar, here is the full flowchart for the running example from the last two chapters — the same logic as the algorithm in Chapter 7 and the pseudocode in Chapter 8, this time as a picture.

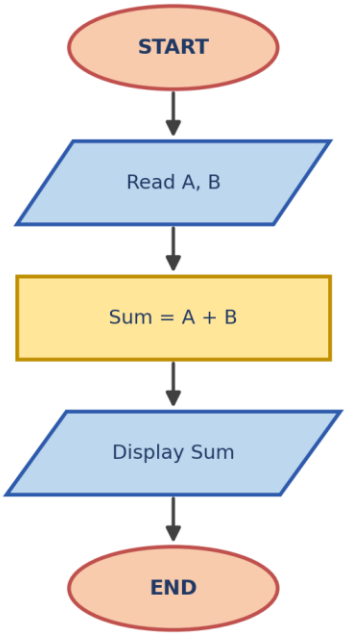


Flowchart for finding the largest of three numbers, A, B, and C.

Trace it with your finger, top to bottom: **Start**, then **Read A, B, C**, then the first **Decision** — is A greater than both B and C? If YES, Largest = A. If NO, a second **Decision** asks whether B is greater than C — YES gives Largest = B, NO gives Largest = C. All three possible outcomes then flow back together into one **Display Largest** step, and finally **End**. Every arrow you can trace in this picture corresponds to exactly one line of the pseudocode from Chapter 8 — nothing has been added, and nothing has been left out.

From One Idea to Any Language

Here is the payoff for everything built up across Chapters 7, 8, and 9: once a problem has been worked out as an algorithm, a pseudocode, and a flowchart, **the underlying logic no longer belongs to any one programming language** — it can be typed into C, Python, Java, or almost anything else, and the translation is almost mechanical. Consider the simplest possible example: adding two numbers.



Flowchart for adding two numbers, A and B.

Algorithm: Step 1: Start. Step 2: Read two numbers, A and B. Step 3: Compute Sum = A + B. Step 4: Display Sum. Step 5: Stop.

Pseudocode: BEGIN READ A, B Sum = A + B PRINT Sum END.

Watch how directly that same logic drops into three completely different languages:

Example — C

```
#include <stdio.h>

int main() {
    int a, b, sum;
    scanf("%d %d", &a, &b);
    sum = a + b;
    printf("Sum = %d", sum);
    return 0;
}
```

Example — Python

```
a = int(input())
b = int(input())
sum = a + b
```

```
print("Sum =", sum)
```

Example — Java

```
import java.util.Scanner;

public class AddTwo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int a = sc.nextInt();
        int b = sc.nextInt();
        int sum = a + b;
        System.out.println("Sum = " + sum);
    }
}
```

Three languages, three very different-looking syntaxes — yet the exact same four steps (read A and B, compute the sum, display it) sit underneath every single one of them. This is precisely why the discipline of this chapter matters: **for a problem this simple, you could reasonably jump straight to code without drawing all three stages.** But the moment a problem grows more complex — nested conditions, loops inside loops, several possible outcomes — skipping the algorithm, pseudocode, and flowchart stops being a shortcut and starts being a way to trap logical mistakes inside a wall of syntax where they are far harder to find. The more complex the problem, the more this upfront thinking pays for itself.

Try It Yourself

1. Draw a flowchart for the algorithm "check whether a number is even or odd" from Chapter 7.

2. Draw a flowchart for finding the sum of the first N natural numbers using a loop.

3. Draw a flowchart for swapping two numbers using a temporary variable.

4. Explain, using the shapes from this chapter, why a diamond (decision) shape always needs at least two arrows leaving it, while a rectangle (process) shape needs only one.

5. Take the "Add Two Numbers" example from this chapter and extend its algorithm, pseudocode, and flowchart to instead find the **average** of three numbers.

6. For which of the following would you skip straight to code, and for which would you insist on an algorithm, pseudocode, and flowchart first — and why? (a) printing "Hello, World" (b) processing a class of 60 students' exam results into pass/fail/distinction categories.

Chapter 10
Program — The Algorithm, Finally Speaking a Real Language

Analogy: If the algorithm was the recipe idea in your head, and the pseudocode was the recipe scribbled in your own shorthand, and the flowchart was the recipe drawn as a diagram — the **program** is finally the recipe written in the exact, strict language a particular kitchen (a particular compiler) requires: precise measurements, precise terms, no ambiguity, no missing punctuation. Chapter 9 showed that a *simple* problem, like adding two numbers, can jump straight to code in any language. The largest-of-three-numbers problem is only slightly more complex — but that small jump in complexity is exactly where writing the full C program, carefully, syntax and all, starts to matter.

Technical Concept: A **program** is the algorithm expressed in the strict, formal syntax of a specific programming language (here, C), ready to be translated by a compiler into machine instructions and executed by a computer. Unlike pseudocode, a program must obey every syntax rule of the language exactly — a missing semicolon or a misspelled keyword will stop it from compiling at all.

Where It Is Used: Every app on your phone, every website you browse, every piece of firmware inside your devices exists, at some point in its life, exactly in this form: a program written in some formal language's syntax.

Why We Use It: We go through problem -> algorithm -> pseudocode -> flowchart -> program deliberately, in that order, because each stage catches a different category of mistake cheaply. Logical mistakes are far easier to catch in an algorithm or flowchart (on paper, in minutes) than after you've already written and are trying to debug 100 lines of syntactically correct C code.

Example — The Largest-of-Three-Numbers Program, in C

```
#include <stdio.h>

int main() {
   int a, b, c, largest;

   printf("Enter three numbers: ");
   scanf("%d %d %d", &a, &b, &c);
```

```
    if (a > b && a > c) {
        largest = a;
    } else if (b > c) {
        largest = b;
    } else {
        largest = c;
    }

    printf("The largest number is %d\n", largest);
    return 0;
}
```

Compare this, line by line, to the pseudocode in Chapter 8 and the flowchart in Chapter 9 — they are, quite deliberately, the exact same logic, only now wearing C's strict syntax: #include <stdio.h> brings in input/output tools, scanf reads input, if/else if/else mirrors the diamond decisions, and printf produces the output.

From problem to program: the whole journey, side by side

1. **Problem**: "Find the largest of three given numbers."
2. **Algorithm**: the seven numbered steps in Chapter 7.
3. **Pseudocode**: the BEGIN/READ/IF/PRINT/END structure in Chapter 8.
4. **Flowchart**: the oval-parallelogram-diamond-rectangle diagram in Chapter 9.
5. **Program**: the C code above, ready to be compiled and executed.

Try It Yourself

1. Write the complete C program for "check whether a number is even or odd", based on your algorithm, pseudocode, and flowchart from the previous three chapters.

2. Write the complete C program to find the sum of the first N natural numbers, using a for loop.

3. Take your ATM PIN-check pseudocode from Chapter 8 and convert it into a working C program.

You have now walked the entire path from a vague real-world wish to a finished, syntactically correct C program. But one mystery remains — the very moment you type a command and press Enter, what actually happens, in order, before your program's output appears on the screen? This is the final stage of our journey.

PART IV
From Code to Execution

Chapter 11
What Happens When We Run a C Program?

Analogy: Think about sending a letter written in Hindi to a friend in Germany who only reads German, through a courier system. First, a translator converts your Hindi draft, expanding any shorthand and abbreviations you used, into a full Hindi document (this is like **preprocessing**). Then, a professional translator converts the whole document into German (this is like **compiling**). Then, a binding service combines your translated letter with any standard attachments — like a pre-printed cover page or an official stamp sheet kept in the courier office (this is like **linking**). Only then does the courier collect the finished, bound package and physically place it (this is like **loading**) into your friend's hands (**execution**) — and only your friend's brain, not the courier, not the translator, actually *reads and acts on* the message.

Technical Concept: Running a C program is not one single step — it is a pipeline of five distinct stages, each handled by a different specialised tool. Rather than explain these stages in the abstract, this chapter follows **one single program** through every stage, from the moment it is typed in, to the moment its answer appears on screen.

1. Source Code *program.c — your C statements*
2. Preprocessor *expands #include and #define*
3. Compiler *C source -> object code*
4. Linker *object code + libraries -> executable*
5. Loader *copies executable into memory*
6. CPU Execution *fetch -> decode -> execute cycle*

The Program We Will Follow

Here is a small C program that reads two numbers from the user and performs every basic arithmetic operation on them — addition, subtraction, multiplication, division, and remainder. It is simple enough to keep in your head completely, yet it touches every part of the C syntax you have seen so far

— variables, scanf, printf, and arithmetic operators — which makes it the perfect single example to trace through the whole pipeline.

Example — arithmetic.c

```
#include <stdio.h>

int main() {
    int a, b;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    printf("Sum = %d\n", a + b);
    printf("Difference = %d\n", a - b);
    printf("Product = %d\n", a * b);
    printf("Quotient = %d\n", a / b);
    printf("Remainder = %d\n", a % b);

    return 0;
}
```

Suppose the user enters 10 and 3. By the end of this chapter, you will be able to explain exactly how these two typed numbers turn into five printed lines — Sum = 13, Difference = 7, Product = 30, Quotient = 3, Remainder = 1 — by passing through every stage below.

11.1 Source Code

The file above, saved as arithmetic.c, is the **source code** — the human-written text containing your C statements exactly as you typed them into your editor. At this stage, it is nothing more than plain text; no computer can execute it yet.

11.2 The Preprocessor

Technical Concept: The **preprocessor** is a text-substitution stage that runs *before* real compilation begins. It processes every line beginning with # (called a **directive**). In our program, the very first line, #include <stdio.h>, literally copies the entire content of the stdio.h header file into the source code at that point — this is what makes the declarations for printf and scanf available to the compiler, since both functions are used later in main(). The output of this stage is a single, expanded piece of pure C source code, with no # directives left in it.

Why We Use It: Preprocessing exists so that you can reuse standard, tested declarations (like the printf and scanf declarations inside stdio.h) without retyping them in every file you write.

11.3 The Compiler

Technical Concept: The **compiler** takes the fully expanded source and translates it into **object code** — machine instructions specific to the target CPU's architecture, but not yet a runnable program on its own. Along the way, it checks our program line by line: it confirms a and b are declared as int before they are used, that %d correctly matches an integer in both scanf and every printf, that every opening brace { has a matching closing brace }, and that each arithmetic expression (a + b, a - b, a * b, a / b, a % b) is legal for two integers. Only once every one of these checks passes does the compiler generate object code for arithmetic.c.

Why We Use It: This is the stage that catches almost every mistake a beginner makes. If you had typed scanf("%d %d" &a, &b) and forgotten the comma, or written Int a, b; with a capital I, the compiler would stop right here and report an error — long before the program ever gets a chance to run.

11.4 The Linker

Technical Concept: Our program calls printf five times and scanf once, but nowhere in arithmetic.c did we write the actual code that draws characters on a screen or reads characters from a keyboard — that code already exists, pre-compiled, inside the C **standard library**. The **linker** combines our object code with that library code, resolves every call to printf and scanf to its real address, and produces a single, complete, standalone **executable file** (such as arithmetic.exe on Windows, or a.out on Linux).

Why We Use It: Linking exists so that common, heavily reused code — like the entire input/output library — does not need to be rewritten or even recompiled by every single programmer; it is written once, compiled once, and linked into thousands of different programs, including ours.

11.5 The Loader and Execution

Technical Concept: When you finally run arithmetic.exe, the operating system's **loader** copies its machine instructions and data from disk into RAM, sets up the initial execution environment, and hands control to the CPU at the start of main(). The **CPU** then performs its **fetch–decode–execute cycle** on our program's instructions in order: it fetches the instructions behind printf("Enter two numbers: ") and displays the prompt; it fetches the instructions behind scanf and waits for you to type, say, 10 and 3, storing them at the memory locations reserved for a and b; it then fetches the instructions for each arithmetic operation in turn — computing 10 + 3, 10 − 3, 10 × 3, 10 ÷ 3, and 10

mod 3 — and fetches the instructions behind each printf call to display Sum = 13, Difference = 7, Product = 30, Quotient = 3, and Remainder = 1, one line at a time.

Why We Use It: This final stage is the entire point of everything before it: preprocessing, compiling, and linking all exist purely to produce something the loader can place into memory and the CPU can actually execute, one raw binary instruction at a time.

The CPU never directly executes our C source code.

This single sentence is worth remembering above almost everything else in this chapter. Not one word of arithmetic.c — not printf, not scanf, not a + b — is ever seen by the CPU. By the time our program actually runs, it has been rewritten, layer by layer, into something entirely different: expanded text, then object code, then a linked executable, then raw binary instructions sitting in memory. The C language exists purely for *you* — to let you think, write, and reason clearly about adding two numbers. The machine only ever sees the very last translation of that thought.

11.6 The Big Picture — One Program, Five Stages

Stage	Tool Responsible	What Happens to arithmetic.c
1. Write	You, the programmer	You type out arithmetic.c in an editor
2. Translate	Preprocessor + Compiler	#include <stdio.h> is expanded, then the whole file becomes object code
3. Link	Linker	The real code for printf and scanf is attached, producing arithmetic.exe
4. Load	Loader (part of the OS)	arithmetic.exe is copied from disk into RAM
5. Execute	CPU (fetch-decode-execute)	The CPU runs the instructions, printing Sum, Difference, Product, Quotient, Remainder

Epilogue — Where You Stand Now

Trace the path we have walked. We began with a fridge missing its milk, and ended with a CPU silently fetching and executing raw binary instructions, billions of times a second, on your behalf. In between, we met problems, instructions, computers and results; discovered that machines only truly understand 0s and 1s; met the translators — compilers and interpreters — that bridge human thought to machine action; surveyed the different lenses (paradigms) programmers use to think about solutions; traced C's lineage from BCPL through B to its birth alongside UNIX at Bell Labs; justified, with concrete reasons, why C remains essential in 2026; and finally walked the disciplined path from a vague real-world wish, through algorithm, pseudocode and flowchart, into working C code — before watching that code get preprocessed, compiled, linked, loaded, and finally executed.

Nothing in the chapters that follow this book — variables, data types, operators, control statements, functions, arrays, pointers, structures, and files — will make sense in isolation. Each new C feature you learn from here is simply a more powerful way of writing an instruction, inside the very same problem-to-execution pipeline you now understand completely. Whenever a new topic feels confusing, come back to this simple question: **which of the four ingredients — problem, instruction, computer, result — am I actually working with right now?**

You are ready to begin writing real C programs.

PART V
Programming in C — The Building Blocks

With the thinking tools of Parts III and IV in hand, we now meet the language itself: the anatomy of a C program, header files, data types and variables, operators, and decision making.

Chapter 12
Anatomy of a C Program

Before We Begin

So far we have talked about what C is and how it is set up on your computer. Now it's time to actually look at how a C program is written — the exact format, order, and structure that every single C program follows, no matter how big or small it is.

Think of it like writing a formal letter. A letter always has a fixed structure — sender's address, date, greeting, body, and signature — in a fixed order. Similarly, a C program always has a fixed structure: header files, a main function, a body enclosed in braces, and statements that end with a semicolon. Once you recognise this skeleton, reading and writing any C program becomes far easier.

In this chapter, we will look at one complete, working C program and break it down piece by piece so that every symbol and every keyword makes sense before we move further.

12.1 Anatomy of a C Program

> **Analogy:** A formal letter always has the same skeleton: address, greeting, body, sign-off. A C program is the same: a fixed skeleton in which only the body changes.

> **Technical Concept:** A C program is built from a header section, one main() function, curly braces that mark its body, and statements that each end with a semicolon. Execution always starts at main().

Every C program follows a fixed skeleton. Before learning individual commands, it helps to see one complete program and identify what each line is doing. Look at the program below:

```
#include <stdio.h>          // Header file section

/* This program prints a welcome
   message and adds two numbers */

int main()                  // Main function - execution starts here
{
    int a, b, sum;          // Declaration statement

    a = 10;                 // Assignment statement
    b = 20;
    sum = a + b;            // Expression statement

    printf("Welcome to C Programming!\n");
```

```
    printf("Sum = %d", sum); // Output statement

    return 0;              // Return statement
}
```

Output:

```
Welcome to C Programming!
Sum = 30
```

Explanation of Each Part

1. Header File Section

#include <stdio.h> tells the compiler to bring in the Standard Input Output library before compiling the program. Anything used from this library — like printf() — becomes available only after this line is written. Header inclusions are always written at the very top of the program.

2. Comments

Comments are lines that the compiler completely ignores. They exist only to help humans understand the code.

- Single-line comment: // this text is ignored till the end of the line
- Multi-line comment: /* ... */ everything between these symbols is ignored, even across many lines

Comments do not affect program output in any way. Good programmers comment their code so that others (and their future selves) can understand it quickly.

3. int main() — and why it must be written exactly this way

main() is a special function — every C program must have exactly one, and program execution always starts from main(), no matter where it is physically placed in the file. But why is it written as int main() and not float main(), char main(), or bool main()? This confuses a lot of beginners, so let's break it down.

- A function's return type is a promise about what kind of value it will hand back once it finishes running. main() hands its value back not to another function, but to the operating system, in the form of an exit status.
- The exit status tells the OS whether the program ran successfully (conventionally 0) or failed (any non-zero value). Operating systems expect this status as a whole number — so the return type of main() must be int.

- float main() is not used because exit statuses are never fractional — "0.5 successful" means nothing to an operating system. The C standard does not even permit float as a valid return type for main().
- char main() is not used because a character does not represent a meaningful success/failure code the way a small integer does.
- bool main() is not standard either — C does not even have a built-in bool type unless you include <stdbool.h>, and even then, true/false does not map onto the range of exit codes an OS understands.
- void main() is sometimes seen in very old textbooks and old Turbo C compilers. It means "return nothing." This is NOT part of the official ISO C standard and most modern compilers (like GCC) will either warn about it or reject it. Always prefer int main().

So, in short: int main() is used because it is the only return type that correctly communicates a program's success/failure status back to the operating system, and it is the form required by the C language standard.

Syntax Variations of main()

```
int main()                    // most common form used by beginners
int main(void)                  // explicitly says the function takes no arguments
int main(int argc, char *argv[])   // used when the program needs command-line arguments
```

*int main() and int main(void) behave identically in practice for beginner programs. int main(int argc, char *argv[]) is used later when a program needs to accept inputs directly from the command line — you do not need to worry about this form yet.*

4. Curly Braces { }

The opening brace { marks the beginning of the function body and the closing brace } marks its end. Everything a function does must be written between its own pair of braces. Braces are also used later for loops, conditions, and other blocks of code. Every opening brace must have exactly one matching closing brace — a missing brace is one of the most common beginner errors.

5. Statements

A statement is a single instruction given to the computer. In C, every statement ends with a semicolon (;). There are different kinds of statements:

- Declaration statement — creates a variable, e.g. int a, b, sum;
- Assignment statement — stores a value in a variable, e.g. a = 10;
- Expression statement — performs a calculation, e.g. sum = a + b;
- Output/Function-call statement — calls a function to do work, e.g. printf("Sum = %d", sum);

6. printf() Function

printf() is used to display output on the screen. Text inside double quotes is printed exactly as written. Special codes like %d (integer) and \n (new line) control formatting — printf() and its counterpart scanf() are covered in full detail with syntax and examples in Chapter 14.

7. return 0;

This line ends the main() function and sends the value 0 back to the operating system, which by convention means the program finished successfully. This is exactly why main() had to be declared as int in the first place — it needs somewhere to send this value.

12.2 Important Questions

Q1. Name the main parts of a C program and state the purpose of each.

The header section (#include) brings in library declarations; comments explain the code and are ignored by the compiler; main() is the entry point where execution begins; curly braces mark the body; statements are instructions that end with a semicolon; printf() displays output; return 0; tells the operating system that the program ended successfully.

Q2. Why is int main() preferred over void main()?

main() returns an exit status to the operating system, and the C standard defines that status as an int, with 0 meaning success. void main() is non-standard and only worked on old compilers such as Turbo C.

Q3. What are comments? Write the two kinds used in C.

Comments are notes for humans that the compiler ignores. A single-line comment starts with // and runs to the end of the line; a multi-line comment is written between /* and */. Comments never change the output.

Q4. What happens if the semicolon at the end of a statement is missing?

The compiler reports a syntax error, often pointing at the next line, because it only realises the statement has not ended when it meets the next token.

Q5. What is the role of return 0; in main()?

It ends main() and sends 0 back to the operating system, meaning normal termination. A non-zero value signals an error to scripts and tools that run the program.

12.3 Multiple Choice Questions

1. Execution of every C program begins from

- a) printf()
- b) main()
- c) #include
- d) return

Answer: b) main() (main() is the entry point.)

2. Which symbol starts a single-line comment in C?

- a) #
- b) //

- c) <>
- d) $$

Answer: b) // (// comments out the rest of the line.)

3. Every simple statement in C ends with a

- a) full stop
- b) colon
- c) semicolon
- d) comma

Answer: c) semicolon (The semicolon terminates a statement.)

4. The standard return type of main() is

- a) void
- b) int
- c) float
- d) char

Answer: b) int (The exit status is an int.)

5. The line #include <stdio.h> is processed by the

- a) linker
- b) preprocessor
- c) loader
- d) debugger

Answer: b) preprocessor (The preprocessor handles every line that starts with #.)

6. return 0; at the end of main() tells the operating system that

- a) the program failed
- b) the program ended successfully
- c) zero must be printed
- d) the program must restart

Answer: b) the program ended successfully (0 conventionally means success.)

12.4 Solved Programs

Program 1: A Two-Line Greeting

Problem: Print a greeting and a second line on separate lines.

Approach: printf() prints exactly what is inside the quotes; \n moves to the next line.

```
#include <stdio.h>
int main() {
    printf("Hello, C!\n");
    printf("My first program works.\n");
    return 0;
}
```

Output:

```
Hello, C!
My first program works.
```

Program 2: Anatomy in Action

Problem: Add two numbers and label every part of the program in comments.

Approach: Each comment names the part of the skeleton that the line belongs to.

```
#include <stdio.h>              // header file section
int main()                      // execution starts here
{
    int a = 12, b = 8;          // declaration statement
    int sum = a + b;            // expression statement
    printf("Sum = %d\n", sum);  // output statement
    return 0;                   // tell the OS: success
}
```

Output:

```
Sum = 20
```

12.5 Practice Problems — Try It Yourself

1. **Write a program that prints your name, department and college on three separate lines.**
2. **Remove the semicolon after one printf(), compile the program and note where the compiler reports the error.**

3. **Write a program that contains one single-line and one multi-line comment and confirm that neither changes the output.**
4. **Using only printf(), print the pattern `*  **  ***` on three lines.**
5. **Make main() return 5 instead of 0 and find the exit status (echo $? on Linux, echo %errorlevel% on Windows).**

Chapter 13
Header Files in C

13.1 What is a Header File?

> **Analogy:** A header file is a toolbox someone has already packed for you. You open it and use the tools instead of forging them yourself.

> **Technical Concept:** A header file is a file with the extension .h that holds ready-made declarations of functions, macros and constants. #include pastes it into your program before compilation.

A header file is a file with the extension .h that contains ready-made declarations of functions, macros, and constants which programmers can reuse instead of writing that code themselves. When a header file is included using #include, the compiler is told: "the functions I am about to use are defined somewhere else — go find their declarations here."

In simple words, a header file is like a toolbox that someone has already built for you. Instead of forging your own hammer and screwdriver every time you build something, you simply open the toolbox and use the tools that are already there.

13.2 Why Are Header Files Needed? Their Importance

> **Analogy:** A restaurant menu lets you order a dish without knowing the recipe. Header files let your program call functions without knowing how they are written.

> **Technical Concept:** Header files supply function prototypes, which let the compiler check every call for the right name, argument types and return type before the program runs.

- Saves time and effort — common, frequently used functions (like printing to the screen or finding a square root) are already written and tested; you do not need to rewrite them.
- Avoids errors — reusing a well-tested standard function is far safer than writing your own version from scratch every time.
- Improves code organisation — the declarations of functions are kept separate from the logic of your program, keeping your .c file clean and readable.
- Enables code reuse — the exact same header can be included in hundreds of different programs.

- Supports modularity — in large, real-world software projects, code is split across many files, and header files are what let those files talk to each other and share functions.
- Helps the compiler check your code — a header file's function declarations (called prototypes) let the compiler check that you are calling a function correctly, catching mistakes before the program even runs.

13.3 Types of Header Files

> **Analogy:** Some tools come in the box when you buy the machine; others you build in your own workshop. Header files are the same: built-in or self-made.

> **Technical Concept:** Standard header files (stdio.h, math.h) ship with the compiler. User-defined header files are written by the programmer and included with double quotes.

- Standard (built-in) header files — supplied along with the compiler itself, e.g. stdio.h, math.h. These are included using angle brackets.
- User-defined header files — created by the programmer to organise their own project's functions, usually shared across multiple .c files in a large project. These are included using double quotes.

Syntax of Including a Header File

```
#include <filename.h>     // for standard/built-in header files
#include "filename.h"     // for user-created/custom header files
```

- Angle brackets <> → compiler looks in the standard system library folder.
- Double quotes "" → compiler first looks in the current project folder, then the system folder.

A Quick Look at a User-Defined Header File

In bigger projects, programmers write their own header files to share functions across multiple files. Here is a minimal example — you don't need to use this yet, but it's good to recognise it:

```
// greet.h  (custom header file)
void greet();

// main.c
#include <stdio.h>
```

```
#include "greet.h"      // includes our own header file

int main() {
    greet();
    return 0;
}
```

13.4 Commonly Used Header Files in C

> **Analogy:** A phone's home screen groups apps by purpose: camera, calls, maps. Header files group library functions by purpose too.

> **Technical Concept:** Each standard header declares one family of functions: stdio.h for input/output, string.h for strings, math.h for mathematics, stdlib.h for utilities, ctype.h for characters.

The table below lists the header files you will use most often as a beginner and, later, in placement-level coding rounds.

Header File	What It Provides
stdio.h	Standard input/output functions — printf, scanf, getchar, etc.
string.h	Functions to handle and manipulate strings — length, copy, compare, etc.
math.h	Mathematical functions — square root, power, trigonometry, rounding.
stdlib.h	General utility functions — memory allocation, random numbers, conversions, exit.
ctype.h	Functions to test and convert single characters — check if a character is a digit, letter, uppercase, etc.
time.h	Functions to work with system date and time.
conio.h	Console I/O functions like clrscr() and getch() — only on older Turbo C compilers.

13.5 stdio.h — Standard Input Output Header

> **Analogy:** stdio.h is the reception desk of your program: everything that talks to the keyboard and the screen goes through it.

> **Technical Concept:** stdio.h declares the standard input/output functions such as printf(), scanf(), getchar(), putchar(), fopen() and fclose().

This is the most frequently used header file. It contains declarations for all functions related to taking input from the user and displaying output on the screen.

Function	Purpose	Example Syntax
printf()	Displays output on the screen	printf("Value = %d", x);
scanf()	Reads input entered by the user	scanf("%d", &x);
getchar()	Reads a single character from input	ch = getchar();
putchar()	Prints a single character	putchar(ch);
gets()	Reads a full line of text (unsafe, avoid)	gets(name);
puts()	Prints a string followed by a new line	puts(name);

Quick example:

```
#include <stdio.h>
int main() {
    char ch = getchar();   // reads one character typed by user
    putchar(ch);           // prints that same character back
    return 0;
}
```

13.6 string.h — String Handling Functions

> **Analogy:** A word processor lets you count letters, copy a sentence or compare two names. string.h gives C the same text-handling abilities.

> **Technical Concept:** string.h declares functions that work on character arrays: strlen(), strcpy(), strcat(), strcmp() and strchr().

Function	Purpose	Example
strlen()	Finds length of a string	strlen(name)
strcpy()	Copies one string into another	strcpy(dest, src)
strcat()	Joins (concatenates) two strings	strcat(str1, str2)

Function	Purpose	Example
strcmp()	Compares two strings	strcmp(str1, str2)

Quick example:

```
#include <stdio.h>
#include <string.h>
int main() {
    char name[20] = "Kumar";
    printf("Length = %d", strlen(name));   // Length = 5
    return 0;
}
```

13.7 math.h — Mathematical Functions

> **Analogy:** A scientific calculator has keys for square root, power and rounding. math.h is that keypad for your program.

> **Technical Concept:** math.h declares mathematical functions such as sqrt(), pow(), ceil(), floor(), fabs(), sin() and cos(); on many systems they need the -lm linker option.

Function	Purpose	Example
sqrt()	Square root of a number	sqrt(25) → 5.0
pow()	Raises a number to a power	pow(2,3) → 8.0
abs() / fabs()	Absolute value (int / float)	fabs(-4.5) → 4.5
ceil() / floor()	Rounds up / rounds down	ceil(4.2) → 5.0

Quick example:

```
#include <stdio.h>
#include <math.h>
int main() {
    printf("%.2f", sqrt(49));   // 7.00
    return 0;
}
```

13.8 stdlib.h — Standard Library Functions

> **Analogy:** A general store sells the odds and ends that fit no single shelf. stdlib.h is the general store of C.

> **Technical Concept:** stdlib.h declares general utilities: malloc() and free() for memory, atoi() for conversion, rand() for random numbers, abs() and exit().

Function	Purpose	Example
malloc() / free()	Allocates / releases memory dynamically	ptr = malloc(10);
exit()	Terminates the program immediately	exit(0);
rand()	Generates a pseudo-random number	rand() % 100
atoi()	Converts a string to an integer	atoi("25") → 25

13.9 ctype.h — Character Testing Functions

> **Analogy:** A security guard at a gate checks whether each person is an adult, a child or a visitor. ctype.h checks what kind of character each one is.

> **Technical Concept:** ctype.h declares character-testing and conversion functions: isalpha(), isdigit(), isupper(), islower(), toupper() and tolower().

Function	Purpose	Example
isdigit()	Checks if a character is a digit (0-9)	isdigit('5') → true
isalpha()	Checks if a character is a letter	isalpha('a') → true
toupper()	Converts a character to uppercase	toupper('a') → 'A'
tolower()	Converts a character to lowercase	tolower('A') → 'a'

13.10 conio.h — Console Input/Output (Turbo C / older compilers)

> **Analogy:** An old typewriter had its own special keys that no modern keyboard has. conio.h belongs to that older era of Turbo C.

> **Technical Concept:** conio.h is a non-standard console header from Turbo C/DOS that offers clrscr() and getch(). It is not portable and is missing in GCC on Linux and macOS.

- clrscr() — clears the output screen
- getch() — reads one character without showing it and without needing Enter

Note: conio.h is not part of the standard C library and does not work on most modern compilers (like GCC). It is included here because some older college labs still use Turbo C.

13.11 Important Questions

Q1. List four benefits of header files.

They save time by reusing tested functions, avoid errors, keep declarations separate from logic, enable code reuse across programs and let the compiler check every function call through prototypes.

Q2. Differentiate standard and user-defined header files with an example of each.

Standard headers such as stdio.h ship with the compiler and are included with angle brackets. User-defined headers such as myfile.h are written by the programmer for their own project and are included with double quotes.

Q3. Name the header file that declares printf, sqrt, strlen, malloc and toupper.

printf is in stdio.h, sqrt in math.h, strlen in string.h, malloc in stdlib.h and toupper in ctype.h.

Q4. Why is conio.h not recommended?

It is non-standard: it comes from Turbo C/DOS, is missing in GCC on Linux and macOS, and programs that use it are not portable.

Q5. What is a function prototype and how does a header file relate to it?

A prototype declares a function's name, return type and parameter types, for example int printf(...). A header file collects such prototypes so the compiler can check calls; the actual code is added later by the linker.

13.12 Multiple Choice Questions

1. A header file has the extension

- a) .c
- b) .h
- c) .exe
- d) .obj

Answer: b) .h (Header files end in .h.)

2. Which header file declares sqrt()?

- a) stdio.h
- b) string.h
- c) math.h
- d) ctype.h

Answer: c) math.h (Mathematical functions live in math.h.)

3. For #include "myfile.h" the compiler searches first in

- a) the system library folder only
- b) the current project folder
- c) the internet
- d) the desktop

Answer: b) the current project folder (Double quotes mean: project folder first, then system folders.)

4. Which header file declares strlen()?

- a) string.h
- b) stdlib.h
- c) math.h
- d) stdio.h

Answer: a) string.h (String functions are in string.h.)

5. malloc() is declared in

- a) stdio.h
- b) stdlib.h
- c) ctype.h
- d) math.h

Answer: b) stdlib.h (Memory utilities are in stdlib.h.)

6. Which of these is NOT a standard C header?

- a) stdio.h
- b) math.h
- c) conio.h
- d) string.h

Answer: c) conio.h (conio.h is a compiler-specific extension.)

13.13 Solved Programs

Program 1: Mathematical Functions with math.h

Problem: Print the square root of 144, 2 raised to 10, and the ceiling and floor of two decimals.

Approach: Include math.h and compile with the -lm option on Linux.

```
#include <stdio.h>
#include <math.h>
int main() {
    double x = 144.0;
    printf("sqrt(%.0f) = %.1f\n", x, sqrt(x));
    printf("2 raised to 10 = %.0f\n", pow(2, 10));
    printf("ceil(4.2) = %.0f, floor(4.8) = %.0f\n", ceil(4.2), floor(4.8));
    return 0;
}
```

Output:

```
sqrt(144) = 12.0
2 raised to 10 = 1024
ceil(4.2) = 5, floor(4.8) = 4
```

Program 2: Strings and Characters with string.h and ctype.h

Problem: Find the length of a word, convert a letter to upper case and test a digit.

Approach: strlen() comes from string.h; toupper() and isdigit() from ctype.h.

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
int main() {
    char word[] = "Header";
    char first = 'h';
    printf("Length of %s = %d\n", word, (int)strlen(word));
    printf("%c -> %c\n", first, toupper(first));
    printf("Is 7 a digit? %d\n", isdigit('7') != 0);
    return 0;
}
```

Output:

```
Length of Header = 6
h -> H
Is 7 a digit? 1
```

13.14 Practice Problems — Try It Yourself

1. **Read a number and print its square root using sqrt().**
2. **Use strlen() to print the length of the string "Programming".**
3. **Use toupper() and tolower() to convert 'a' and 'Z' and print the results.**
4. **Create a header file greet.h that defines a constant with #define and include it in a program using double quotes.**
5. **Find out which header declares abs(), rand(), isalpha() and strcpy().**

Chapter 14
Data Types & Variables in C

14.1 What is a Data Type? Why Do We Need It?

> **Analogy:** A school locker is sized for a particular thing: a tall one for an umbrella, a small one for a pen. A data type tells the compiler which size of locker to reserve.

> **Technical Concept:** A data type specifies the kind of value a variable holds, the memory it occupies and the operations allowed on it. C's basic types are int, float, double, char and void.

A data type tells the compiler three things at once: what kind of value a piece of data is, how much memory to set aside for it, and what operations are valid on it. Without data types, the computer would have no way to know whether a group of bits in memory represents a number, a letter, or something else entirely.

For example, the bit pattern representing the number 65 and the character 'A' can be stored using the exact same bits — the data type is what tells the compiler how to interpret and display those bits correctly.

- Data types ensure the correct amount of memory is reserved — no more, no less.
- Data types ensure operations make sense — you can multiply two int values, but multiplying two "words" makes no sense unless the type allows it.
- Data types help the compiler catch mistakes early — like trying to store text inside a variable meant for numbers.

14.2 What is a Variable? Why Do We Need It?

> **Analogy:** A variable is a labelled box. The label (name) stays the same while the contents can be swapped as often as you like.

> **Technical Concept:** A variable is a named memory location whose value can change during execution. It must be declared with a data type before use: data_type name;

A variable is a named location in memory that can hold a value, and — unlike a fixed constant — that value is allowed to change while the program is running. Think of a variable as a labelled box: the label (variable name) never changes, but what you keep inside the box can be replaced any time.

We need variables because a program constantly has to store, update, compare, and retrieve data as it runs — a running total, a user's input, the result of a calculation, and so on. Without variables, a program would have no way to remember anything between one line of code and the next.

Syntax of Declaring a Variable

```
data_type variable_name;
data_type variable_name = value;      // declaration with initialization
data_type var1, var2, var3;           // multiple variables together

// Examples:
int age = 20;
float price = 99.50;
char grade = 'A';
```

14.3 Rules for Naming a Variable

> **Analogy:** Every person in a class needs a unique roll number. Variable names need rules so the compiler can tell them apart without confusion.

> **Technical Concept:** A variable name must begin with a letter or underscore, may contain letters, digits and underscores, is case-sensitive, and cannot be a reserved keyword.

- A variable name must begin with a letter (a–z, A–Z) or an underscore (_). It cannot begin with a digit.
- After the first character, it may contain letters, digits, and underscores.
- No spaces or special symbols (@, #, %, -, etc.) are allowed anywhere in the name.
- C is case-sensitive, so age, Age, and AGE are treated as three completely different variables.
- A variable name cannot be a reserved keyword of the C language (e.g. int, return, for, while).
- There is no strict length limit, but it is good practice to keep names short, clear, and meaningful.

Valid vs Invalid Variable Names

Variable Name	Valid / Invalid	Reason
Age	Valid	Starts with a letter, all lowercase letters
_count	Valid	Starts with an underscore, which is allowed

Variable Name	Valid / Invalid	Reason
total_marks	Valid	Letters and underscore only
student2	Valid	Digit is allowed after the first character
2ndYear	Invalid	Cannot start with a digit
total-marks	Invalid	Hyphen (-) is a special symbol, not allowed
Float	Invalid	float is a reserved C keyword
roll no	Invalid	Contains a space, which is not allowed

Reserved Keywords in C (cannot be used as variable names)

auto, break, case, char, const, continue, default, do, double, else, enum, extern, float, for, goto, if, int, long, register, return, short, signed, sizeof, static, struct, switch, typedef, union, unsigned, void, volatile, while

Dos and Don'ts While Naming Variables

✔ Do	✘ Don't
Use meaningful names, e.g. totalMarks	Use meaningless names, e.g. x1, temp99
Start with a letter or underscore	Start with a digit, e.g. 1value
Keep names reasonably short but clear	Use extremely long, hard-to-type names
Follow one consistent style (e.g. camelCase)	Mix styles randomly across the same program
Check that the name is not a keyword	Name a variable int, return, or for

Solved Examples — Variables and Declaration

Define a variable. Write the syntax to declare an integer variable age and initialize it to 20.

A **variable** is a named location in memory that stores a value which can change while the program runs. **Declaring** tells the compiler the type and the name; **initializing** gives it its first value. Both can be done in one statement.

```
#include <stdio.h>
int main() {
    int age = 20;      // declaration + initialization in one statement
    int limit;         // declaration only: value is NOT set yet
    limit = 25;        // assignment: first value stored later
    printf("age = %d, limit = %d", age, limit);
    return 0;
}
```

Output: age = 20, limit = 25

Syntax: data_type variable_name = value; Example: int age = 20; A declaration without a value (int limit;) reserves memory but leaves whatever garbage was already there until you assign.

What is the output of the following code? int x = 5; printf("%d", x);

%d is a placeholder that printf() replaces with the **value** stored in x. The letter x itself is never printed.

```
#include <stdio.h>
int main() {
    int x = 5;
    printf("%d", x);
    return 0;
}
```

Output: 5

The format string decides HOW a value is shown; the variable supplies WHAT is shown. printf("x") would print the letter x, while printf("%d", x) prints 5.

Practice Problems — Try It Yourself

1. Declare and initialise suitable variables to store a student's roll number, the first letter of their name and their CGPA. Print all three using a single printf().
2. Which of these are valid variable names? Give a reason for every invalid one: _count, 9lives, my-var, float, Float, total_marks, student name, marks#.
3. Predict the output, then run it: int x = 5; int y = x; x = 10; printf("%d %d", x, y); (Does y change when x changes?)

Debugging Corner — Find the Bug (Solved)

DEBUG 1

Every line of this program declares a variable. Find every naming error.

```
#include <stdio.h>
int main() {
    int 2marks = 90;
    float total-price = 250.50;
    int int = 5;
    char first name = 'R';
    return 0;
}
```

Compiler says: invalid suffix "marks" on integer constant

What is wrong?

- **2marks** — a name cannot start with a digit. Use marks2 or twoMarks.
- **total-price** — the hyphen is the minus operator. Use total_price.
- **int int** — int is a reserved keyword and cannot be a name. Use count.
- **first name** — spaces are not allowed. Use first_name.

Corrected code:

```
#include <stdio.h>
int main() {
    int marks2 = 90;
    float total_price = 250.50;
    int count = 5;
    char first_name = 'R';
    printf("%d %.2f %d %c", marks2, total_price, count, first_name);
    return 0;
}
```

Output: 90 250.50 5 R

DEBUG 2

This program compiles, but its answer is unpredictable. Why?

```
#include <stdio.h>
int main() {
    int price = 40, qty = 3;
    int sum;
    sum = sum + price * qty;
    printf("Total = %d", sum);
    return 0;
}
```

Output of the buggy program: an unpredictable number — different on different runs or machines

What is wrong?

• **sum is used before it has any value.** A local variable that is only declared holds garbage, so sum + price*qty adds 120 to garbage.

• Compiling with -Wall makes GCC warn: "sum is used uninitialized". Always read warnings.

Corrected code:

```
#include <stdio.h>
int main() {
    int price = 40, qty = 3;
    int sum = 0;            // initialise before use
    sum = sum + price * qty;
    printf("Total = %d", sum);
    return 0;
}
```

Output: Total = 120

Debug It Yourself — Practice (answers in Appendix A)

DEBUG IT 1

The compiler reports an error on the line after the mistake. Where is the real bug?

```
#include <stdio.h>
int main() {
    int age = 20
    float height = 5.8;
    printf("%d %.1f", age, height);
    return 0;
}
```

Hint: *Look at the END of the line before the one the compiler complains about.*

DEBUG IT 2

The variable is declared, yet the compiler says it is undeclared.

```
#include <stdio.h>
int main() {
    int Total = 100;
    printf("Total = %d", total);
    return 0;
}
```

Hint: *Compare the spelling of the two names letter by letter, including capitals.*

14.4 Classification of Data Types in C

> **Analogy:** Vehicles are classified as two-wheelers, cars and trucks. C classifies data types into primary, derived and user-defined groups in the same way.

> **Technical Concept:** C data types are classified as primary (int, float, char, double, void), derived (arrays, pointers, functions) and user-defined (struct, union, enum, typedef).

- Primary (Basic) data types — int, float, char, double, void
- Derived data types — array, pointer, function, structure
- User-defined data types — struct, union, enum, typedef

As beginners, our focus for now is on the primary data types, since almost every program starts with these.

14.5 Primary Data Types in Detail

> **Analogy:** Different containers hold different things: a glass for water, a box for books. int, float and char are containers for whole numbers, decimals and single letters.

> **Technical Concept:** int stores whole numbers (usually 4 bytes), float and double store real numbers (4 and 8 bytes), char stores one character (1 byte) and void means no value.

1. int (Integer)

Used to store whole numbers — numbers with no decimal part, whether positive, negative, or zero.

```
int age = 21;
int temperature = -5;
int score = 0;
```

2. float (Floating Point)

Used to store numbers that include a decimal part, with moderate precision (about 6-7 accurate digits).

```
float price = 499.50;
float pi = 3.14f;
```

3. double (Double Precision Floating Point)

Also stores decimal numbers, but with roughly double the precision of float (about 15-16 accurate digits). Used when high accuracy is needed, such as in scientific or financial calculations.

```
double distance = 384400.256789;
double interestRate = 7.256321;
```

4. char (Character)

Used to store a single character — a letter, digit, symbol, or space — enclosed in single quotes. Internally, C stores a char as a small integer, which is its ASCII value.

```
char grade = 'A';
char symbol = '$';
printf("%d", grade);   // prints 65, the ASCII value of 'A'
```

5. void (Empty / No Value)

void represents the absence of a value. It is not used to store data in a variable; instead, it is mainly used as a function's return type (to say the function returns nothing) or to indicate a function takes no parameters.

```
void printMessage() {        // this function returns nothing
    printf("Hello!");
}
```

Size and Range of Primary Data Types

(Sizes shown are the common values on a standard 32-bit/64-bit compiler such as GCC. Always confirm using the sizeof() operator, since size can vary slightly across compilers.)

Data Type	Size	Range	Format Specifier
Char	1 byte	-128 to 127	%c
unsigned char	1 byte	0 to 255	%c
short int	2 bytes	-32,768 to 32,767	%hd
unsigned short int	2 bytes	0 to 65,535	%hu
Int	4 bytes	-2,147,483,648 to 2,147,483,647	%d
unsigned int	4 bytes	0 to 4,294,967,295	%u
long int	4 or 8 bytes	-2,147,483,648 to 2,147,483,647 (or larger)	%ld
long long int	8 bytes	≈ -9.2×10^18 to 9.2×10^18	%lld
Float	4 bytes	≈ 1.2×10^-38 to 3.4×10^38 (6 decimal digits precision)	%f
Double	8 bytes	≈ 2.3×10^-308 to 1.7×10^308 (15 decimal digits precision)	%lf
long double	10/16 bytes	greater range than double	%Lf
Void	0 bytes	represents "no value" / empty type	—

Type Modifiers

- signed — allows both positive and negative values (this is the default)

- unsigned — allows only zero and positive values, doubling the positive range
- short — reduces the memory allotted to a data type
- long — increases the memory allotted to a data type

The sizeof() Operator

sizeof() tells you exactly how much memory (in bytes) a data type or variable occupies on your compiler. It is good practice to check this rather than memorising sizes blindly.

```
#include <stdio.h>
int main() {
    printf("Size of int = %lu bytes\n", sizeof(int));
    printf("Size of float = %lu bytes\n", sizeof(float));
    printf("Size of char = %lu bytes\n", sizeof(char));
    printf("Size of double = %lu bytes\n", sizeof(double));
    return 0;
}
```

Output (typical GCC compiler):

```
Size of int = 4 bytes
Size of float = 4 bytes
Size of char = 1 bytes
Size of double = 8 bytes
```

Solved Examples — Data Types in Action

What is the difference between int, float and char? Give one real-life example variable for each.

Type	Stores	Typical size	Real-life variable	Specifier
int	whole numbers	4 bytes	int studentsInClass = 60;	%d
float	numbers with a decimal part	4 bytes	float cgpa = 8.75;	%f
char	one single character	1 byte	char grade = 'A';	%c

```
#include <stdio.h>
int main() {
    int studentsInClass = 60;
    float cgpa = 8.75;
    char grade = 'A';
    printf("Students: %d (size %lu bytes)\n", studentsInClass, sizeof(studentsInClass));
    printf("CGPA    : %.2f (size %lu bytes)\n", cgpa, sizeof(cgpa));
    printf("Grade   : %c (size %lu bytes)\n", grade, sizeof(grade));
    return 0;
}
```

Output: Students: 60 (size 4 bytes) / CGPA : 8.75 (size 4 bytes) / Grade : A (size 1 bytes)

Choose the type by asking: is it a count (int), a measurement with decimals (float/double) or a single symbol (char)?

List the basic data types in C along with the format specifiers used in printf() and scanf().

Data type	Typical size	Stores	printf	scanf
Int	4 bytes	whole numbers	%d	%d
Float	4 bytes	decimals, about 6–7 digits	%f	%f
Double	8 bytes	decimals, about 15–16 digits	%f (or %lf)	%lf
Char	1 byte	one character (ASCII code)	%c	%c
Void	—	no value (used for "returns nothing")	—	—

Memory trick: scanf needs the exact type, so a double needs %lf there. printf() accepts %f for both float and double because floats are automatically widened to double when passed to it.

Predict the output. Explain why it is not 2.5 and how to fix it. int a = 5, b = 2; float c; c = a / b; printf("%f", c);

Both a and b are int, so a / b is an **integer division** and gives 2 (the .5 is discarded) BEFORE the result is stored in c. Storing 2 into a float only turns it into 2.0.

```
#include <stdio.h>
int main() {
   int a = 5, b = 2;
   float c1 = a / b;         // integer division first -> 2 -> 2.0
   float c2 = (float)a / b;   // cast: a becomes 5.0 -> real division
   float c3 = a / 2.0;        // one operand is already real
   printf("%f %f %f", c1, c2, c3);
   return 0;
}
```

Output: 2.000000 2.500000 2.500000

The type of the variable on the left NEVER changes how the right-hand side is calculated. To get real division, make at least one operand real.

What is implicit type conversion? Predict the output. char ch = 'A'; int x = ch + 1; printf("%d %c", x, x);

When different types are mixed in one expression, C automatically converts (promotes) the smaller type to the larger one. Here the char 'A' (ASCII 65) becomes an int, so ch + 1 = 66. Printed with %d it shows 66; with %c the same number is shown as its character, B.

```
#include <stdio.h>
int main() {
   char ch = 'A';
   int x = ch + 1;
   printf("%d %c\n", x, x);
   printf("%.1f", 10 + 2.5);      // int 10 is promoted to double
   return 0;
}
```

Output: 66 B / 12.5

Promotion order (lower to higher): char, short → int → long → float → double. The lower type always converts up, never down, inside an expression.

A student writes float x = 1 / 3; printf("%f", x); and gets 0.000000 instead of 0.333333. Give two separate reasons and correct the code.

Reason 1: 1 and 3 are both int literals, so / does integer division and the result is 0. **Reason 2:** the conversion to float happens only AFTER that division; turning 0 into 0.0 cannot bring back the lost fraction.

```
#include <stdio.h>
int main() {
    float bad   = 1 / 3;       // 0
    float good  = 1.0 / 3;     // 0.333333...
    double best = 1.0 / 3;     // same value, more digits kept
    printf("%f\n%f\n%.10f", bad, good, best);
    return 0;
}
```

Output: 0.000000 / 0.333333 / 0.3333333333

Rule: to get real division make at least one operand real — 1.0/3, 1/3.0 or (float)1/3. Use double when you need more than 6–7 correct digits.

A junior programmer claims char can never store a negative number. Prove or disprove with a code example and explain how char is stored.

Disproved. char is a small integer type. As a **signed char** it holds -128 to 127; as an **unsigned char** it holds 0 to 255. The same 8 bits, 11111011, mean -5 when read as signed and 251 when read as unsigned.

```
#include <stdio.h>
int main() {
    signed char s = -5;
    int neg = -5;
    unsigned char u = neg;       // same bits, read as unsigned
    printf("%d %d\n", s, u);
    printf("%d %d", (int)sizeof(char), (int)(unsigned char)-1);
    return 0;
}
```

Output: -5 251 / 1 255

Whether a plain char (without signed/unsigned) is signed depends on the compiler and CPU: signed on most x86 compilers, unsigned on many ARM ones. Write signed char if you need negatives and unsigned char for raw bytes.

Practice Problems — Try It Yourself

1. Write a program that stores your age, your height in metres (with decimals) and the first letter of your name, choosing int / float / char appropriately. Print each one with its correct specifier.
2. Predict the output, then check: int x = 7, y = 2; printf("%d %f %f", x / y, (float)x / y, x / 2.0);
3. Predict: char c = 'a'; c = c - 32; printf("%c %d", c, c); What does subtracting 32 do to a lowercase letter?
4. Run: int big = 300; unsigned char c = big; printf("%d", c); Explain the printed value using the idea that unsigned char keeps only 8 bits.

Debugging Corner — Find the Bug (Solved)

Find every error in this program, explain why it is wrong and correct it.

```
#include <stdio.h>
int main() {
    float pi = 3.14;
    int radius = 5;
    float area;
    area = pi * radius * radius
    printf("Area = %d", area);
    return 0;
}
```

Compiler says: expected ';' before 'printf'

What is wrong?

• **Missing semicolon** after pi * radius * radius.

• **Wrong format specifier:** area is a float, but %d expects an int. The bytes are misinterpreted and the printed number is meaningless. Use %f (or %.2f).

Corrected code:

```
#include <stdio.h>
int main() {
    float pi = 3.14;
    int radius = 5;
    float area;
    area = pi * radius * radius;
    printf("Area = %.2f", area);
    return 0;
}
```

Output: Area = 78.50

The programmer wants to store the grade A. What is wrong?

```
#include <stdio.h>
int main() {
    char grade = "A";
    printf("Grade: %s", grade);
    return 0;
}
```

Output of the buggy program: a compiler warning, then a wrong result or a crash

What is wrong?

• **"A" is a string, 'A' is a character.** Double quotes make a string (several characters plus a hidden end marker); a char variable needs single quotes.

• **%s expects a string,** but grade is a single char, so %c is the right specifier.

Corrected code:

```
#include <stdio.h>
int main() {
    char grade = 'A';
    printf("Grade: %c", grade);
    return 0;
}
```

Output: Grade: A

Debug It Yourself — Practice (answers in Appendix A)

DEBUG IT 1

The average of 70, 75, 80 and 90 should be 78.75. Why is 78.00 printed?

```
#include <stdio.h>
int main() {
    int m1 = 70, m2 = 75, m3 = 80, m4 = 90;
    float avg = (m1 + m2 + m3 + m4) / 4;
    printf("Average = %.2f", avg);
    return 0;
}
```

Hint: *Which type is the expression on the right of = BEFORE it reaches avg?*

DEBUG IT 2

50000 times 50000 should be 2500000000. The output is wrong. Why?

```
#include <stdio.h>
int main() {
    int a = 50000, b = 50000;
    int product = a * b;
    printf("Product = %d", product);
    return 0;
}
```

Hint: *What is the largest value an int can hold (about 2.1 billion)?*

14.6 Format Specifiers

> **Analogy:** A form field says 'enter digits only' or 'enter text only'. A format specifier is that instruction for printf() and scanf().

> **Technical Concept:** A format specifier is a % code that tells printf() and scanf() how to interpret a value: %d for int, %f for float, %lf for double, %c for char, %s for string.

A format specifier is a special code, always beginning with %, that tells printf() how to display a value or tells scanf() what type of value to expect while reading input. Using the wrong format specifier for a data type is one of the most common bugs in C, so learning this table well is important — including for placement tests.

Specifier	Used For	Purpose

Specifier	Used For	Purpose
%d	int	Prints or reads a signed decimal integer
%u	unsigned int	Prints or reads an unsigned decimal integer
%f	float	Prints or reads a floating-point number
%lf	double	Prints or reads a double-precision floating-point number
%c	char	Prints or reads a single character
%s	string / char array	Prints or reads a sequence of characters
%ld	long int	Prints or reads a long integer
%lld	long long int	Prints or reads a very large integer
%x	int	Prints an integer in hexadecimal form
%o	int	Prints an integer in octal form
%e	float / double	Prints a number in scientific notation
%p	pointer	Prints a memory address

14.7 The printf() Function

> **Analogy:** A shop's display board shows prices to customers. printf() is the display board of your program.

> **Technical Concept:** printf() writes formatted output to the screen. Syntax: printf("format string", values); The format string mixes plain text, format specifiers and escape sequences.

printf() ("print formatted") is used to display output on the screen. It comes from stdio.h.

Syntax

```
printf("format string", value1, value2, ...);
```

The format string is written inside double quotes and can mix plain text with format specifiers. Each format specifier is filled in, in order, by the values listed after the comma.

Examples

```
int age = 20;
```

```
float marks = 88.5;
char grade = 'A';

printf("Age = %d\n", age);
printf("Marks = %.1f\n", marks);       // .1 limits to 1 decimal place
printf("Grade = %c\n", grade);
printf("Age: %d, Grade: %c\n", age, grade);   // multiple values together
```

Output:

```
Age = 20
Marks = 88.5
Grade = A
Age: 20, Grade: A
```

Common Escape Sequences Used Inside printf()

Escape Sequence	Meaning
\n	Moves the cursor to a new line
\t	Inserts a horizontal tab (spacing)
\\	Prints a single backslash character
\"	Prints a double quote character
\'	Prints a single quote character
\0	Represents the null character (end of a string)

14.8 The scanf() Function

> **Analogy:** A cashier asks for your card and waits until you hand it over. scanf() waits until the user types a value.

> **Technical Concept:** scanf() reads formatted input from the keyboard and stores it at the address of a variable. Syntax: scanf("%d", &number); the & operator supplies the address.

scanf() ("scan formatted") is used to read input typed by the user at the keyboard. It also comes from stdio.h.

Syntax

```
scanf("format specifier(s)", &variable1, &variable2, ...);
```

Notice the & (address-of operator) placed before each variable name. scanf() needs to know the exact memory address of a variable so that it can store the typed value directly into it — without &, scanf() would not know where to put the value.

Examples

```
int age;
float marks;
char grade;

printf("Enter your age: ");
scanf("%d", &age);

printf("Enter your marks: ");
scanf("%f", &marks);

printf("Enter your grade: ");
scanf(" %c", &grade);   // note the space before %c — it skips leftover newline
```

Common mistake: forgetting the & before a variable name in scanf(). This is one of the most frequent beginner errors and can cause a program to crash or behave unpredictably.

Exception: when reading a whole string into a character array, the & is not used, because an array name already represents the address of its first element — e.g. scanf("%s", name);

Solved Examples — Format Specifiers, printf() and scanf()

Write the correct format specifier for: (a) integer (b) float (c) character (d) double (e) string.

```
#include <stdio.h>
int main() {
    int    a = 42;
    float  b = 3.5;
    char   c = 'K';
    double d = 2.718281828;
    char  *e = "Hello";          // a string literal
    printf("(a) int    : %d\n", a);
    printf("(b) float  : %f\n", b);
    printf("(c) char   : %c\n", c);
    printf("(d) double : %lf\n", d);
    printf("(e) string : %s\n", e);
    return 0;
}
```

Output:

```
(a) int    : 42
(b) float  : 3.500000
(c) char   : K
(d) double : 2.718282
(e) string : Hello
```

Quick reference: int → %d float → %f char → %c double → %lf (scanf) / %f (printf) string → %s

Write a C program to declare two integer variables a and b, take input from the user and print their sum.

```
#include <stdio.h>
int main() {
    int a, b;
    printf("Enter two integers: ");
    scanf("%d %d", &a, &b);
    printf("Sum = %d", a + b);
    return 0;
}
```

Sample Run:

```
Enter two integers: 12 8
Sum = 20
```

&a means "the address of a": scanf needs to know WHERE in memory to store what the user typed, so every ordinary variable in scanf() carries an & in front.

What is the purpose of scanf()? Write its syntax to read an integer into variable n.

scanf() reads formatted input from the keyboard and stores it in variables. Syntax: **scanf("%d", &n);** It also **returns the number of values it read successfully**, which lets a program check the input.

```
#include <stdio.h>
int main() {
    int n;
    printf("Enter n: ");
    if (scanf("%d", &n) == 1)
        printf("You entered %d", n);
    else
        printf("Invalid input!");
    return 0;
}
```

Sample Run 1:

```
Enter n: 25
You entered 25
```

Sample Run 2:

```
Enter n: abc
Invalid input!
```

A user enters 3.9 for int x; scanf("%d", &x); What does x store and why? What happens to the leftover input and how can it affect a later scanf("%f", &y)?

%d reads only the longest valid integer prefix, so it takes **3** and stops at the decimal point (x = 3, nothing is rounded). The characters **.9** and the Enter key stay in the input buffer. The next scanf("%f") finds them waiting, reads .9 as 0.9 and never asks the user to type anything.

```
#include <stdio.h>
int main() {
    int x;
    float y;
    printf("Enter an integer: ");
    scanf("%d", &x);
    printf("Enter a float: ");
    scanf("%f", &y);
    printf("x = %d, y = %.1f", x, y);
    return 0;
}
```

Sample Run:

```
Enter an integer: 3.9
Enter a float: x = 3, y = 0.9
```

The second prompt was "answered" by leftover text. Fixes: read the value as float and convert with (int), or clear the leftover input before the next read (with a loop, taught in the next unit).

Practice Problems — Try It Yourself

1. Read two numbers a and b and print their sum, difference and product, each on its own line with a label.
2. Read a float and print it (i) with 2 decimal places and (ii) right-aligned in a field 10 characters wide (%10.2f).
3. Read a single character and print it both as a character and as its ASCII value.
4. What do n and the value returned by scanf() become when the user types 12abc for scanf("%d", &n)? And when the user types abc? Print both.

Debugging Corner — Find the Bug (Solved)

DEBUG 1

Identify the errors (syntax and logical), explain them and write the corrected code.

```
#include <stdio.h>
int main() {
   int a, b
   printf("Enter two numbers: ");
   scanf("%d %d", a, b);
   if (a > b)
      printf("A is greater);
   return 0;
}
```

Compiler says: expected '=', ',', ';', 'asm' or '__attribute__' before 'printf'

What is wrong?

- **Missing semicolon** after int a, b.
- **scanf without &:** a and b must be written as &a and &b. Without them scanf tries to store the numbers at the addresses a and b (garbage) and the program crashes.
- **Unterminated string:** the closing quote is missing in "A is greater".
- **Logical gap:** when b is greater or equal nothing is printed. Add an else.

Corrected code:

```
#include <stdio.h>
int main() {
   int a, b;
   printf("Enter two numbers: ");
   scanf("%d %d", &a, &b);
   if (a > b)
      printf("A is greater");
   else
      printf("B is greater or both are equal");
   return 0;
}
```

Sample Run:

```
Enter two numbers: 9 4
A is greater
```

DEBUG 2

The user types 99.5 but a wrong price is printed. Find the bug.

```
#include <stdio.h>
int main() {
    double price;
    printf("Enter price: ");
    scanf("%f", &price);
    printf("Price = %.2f", price);
    return 0;
}
```

Output of the buggy program: a wrong value (garbage) instead of 99.50

What is wrong?

• **%f in scanf reads a 4-byte float,** but price is an 8-byte double, so only half of the variable is filled. scanf needs **%lf** for double. (printf is different: it accepts %f for double.)

Corrected code:

```
#include <stdio.h>
int main() {
    double price;
    printf("Enter price: ");
    scanf("%lf", &price);
    printf("Price = %.2f", price);
    return 0;
}
```

Sample Run:

```
Enter price: 99.5
Price = 99.50
```

Debug It Yourself — Practice (answers in Appendix A)

DEBUG IT 1

The compiler rejects this printf(). What is missing?

```
#include <stdio.h>
int main() {
    int a = 4, b = 6;
    printf("Sum = %d\n" a + b);
    return 0;
}
```

Hint: *How does printf() separate the format string from the values?*

DEBUG IT 2

The user types 20 and Enter, but the program never waits for the grade. Why?

```
#include <stdio.h>
int main() {
    int age; char grade;
    printf("Age: ");
    scanf("%d", &age);
    printf("Grade: ");
    scanf("%c", &grade);
    printf("%d %c", age, grade);
    return 0;
}
```

Hint: *What character did the Enter key leave in the input? Which specifier reads whitespace?*

14.9 Important Questions

Q1. What is a header file? Why is it used in C programs?

A header file (.h) is a file containing pre-written declarations of functions, macros, and constants. It is used so that programmers do not need to rewrite common functionality — the compiler simply includes these declarations wherever #include is written, giving access to standard functions like printf() and sqrt().

Q2. Differentiate between #include <stdio.h> and #include "myfile.h".

Angle brackets tell the compiler to search for the file only in the standard system library directory (used for built-in headers). Double quotes tell the compiler to first search the current project directory and then the system directory (used for custom, user-written headers).

Q3. What is a variable? Why do we need variables in a program?

A variable is a named memory location whose stored value can change while the program runs. Variables are needed because a program must store, update, and reuse data — such as user input or calculation results — as it executes.

Q4. State any four rules for naming a variable in C.

A variable name must start with a letter or underscore, cannot start with a digit, cannot contain spaces or special symbols, is case-sensitive, and cannot be a reserved keyword such as int or return.

Q5. Why does float have less precision than double?

float occupies 4 bytes and stores about 6-7 significant decimal digits accurately, while double occupies 8 bytes and stores about 15-16 significant decimal digits, giving it far greater precision for the same calculation.

Q6. Why is the & symbol required before a variable name in scanf()?

scanf() needs the exact memory address of a variable to store the typed input directly into that location. The & (address-of) operator supplies that address; without it, scanf() has no valid place to write the value.

Q7. What happens if you store a value larger than int's range inside an int variable?

The value overflows. Since the variable does not have enough bits to hold the number, the extra bits are discarded and the stored value wraps around, producing an incorrect and often negative result. The fix is to use long int, long long int, or unsigned int as appropriate.

Q8. What is the purpose of the sizeof() operator? Give an example.

sizeof() returns the number of bytes occupied by a data type or variable in memory, e.g. sizeof(int) typically returns 4 on modern compilers, helping programmers write portable code instead of assuming a fixed size across all machines.

14.10 Multiple Choice Questions

1. Which header file must be included to use printf() and scanf()?

- a) math.h
- b) stdio.h
- c) string.h
- d) conio.h

Answer: b) stdio.h

2. What is the size of an int data type on most modern (32-bit/64-bit) compilers?

- a) 1 byte
- b) 2 bytes
- c) 4 bytes
- d) 8 bytes

Answer: c) 4 bytes

3. Which data type would you use to store a student's grade such as 'A' or 'B'?

- a) int
- b) float
- c) char
- d) double

Answer: c) char

4. Which format specifier is used to print a double value?

- a) %d
- b) %f
- c) %c
- d) %lf

Answer: d) %lf

5. Which of the following correctly includes a user-created header file named myheader.h?

- a) #include <myheader.h>
- b) #include "myheader.h"
- c) #import myheader.h
- d) include myheader.h

Answer: b) #include "myheader.h"

6. Which symbol must appear before a variable name inside scanf()?

- a) * (asterisk)
- b) & (ampersand)
- c) # (hash)
- d) @ (at symbol)

Answer: b) & (ampersand)

7. Which function from string.h is used to find the length of a string?

- a) strcpy()
- b) strcat()
- c) strlen()
- d) strcmp()

Answer: c) strlen()

8. Why must main() return an int and not a float?

- a) float takes more memory
- b) The OS expects a whole-number exit status
- c) int is easier to type
- d) There is no real reason

Answer: b) The OS expects a whole-number exit status

9. Which of these is an invalid C variable name?

- a) totalMarks
- b) _count
- c) 2ndValue
- d) student1

Answer: c) 2ndValue

10. Which escape sequence moves the cursor to a new line inside printf()?

- a) \t
- b) \0
- c) \n

- d) \\

Answer: c) \n

14.11 Solved Programs

The five scenarios below train you to think about which data type fits a real situation before writing a single line of code.

Problem 1: Student Roll Number and Marks

Scenario: A college wants to store a student's roll number and the marks scored in an exam (out of 100), then display the total and average of three subjects.

Reasoning: Roll numbers and marks are whole numbers with no decimal part and stay well within a small range, so int is the right choice for them. However, the average of three subjects can result in a decimal value, so it must be stored in a float.

```
#include <stdio.h>
int main() {
    int roll = 101;
    int marks1 = 78, marks2 = 85, marks3 = 92;
    int total;
    float average;

    total = marks1 + marks2 + marks3;
    average = total / 3.0;      // 3.0 forces decimal division

    printf("Roll No: %d\n", roll);
    printf("Total = %d\n", total);
    printf("Average = %.2f\n", average);
    return 0;
}
```

Output: Roll No: 101 Total = 255 Average = 85.00

Problem 2: Area of a Circle

Scenario: A program must calculate the area of a circle when the radius is entered by the user, and the radius can include decimal values like 3.5 cm.

Reasoning: Since the radius and the resulting area involve decimal precision, both must be declared as float (or double for higher precision).

```
#include <stdio.h>
int main() {
    float radius, area;
    const float PI = 3.14159;

    printf("Enter radius: ");
    scanf("%f", &radius);

    area = PI * radius * radius;
    printf("Area = %.2f\n", area);
    return 0;
}
```

Sample Run: Enter radius: 4 → Area = 50.27

Problem 3: Grade Based on Marks

Scenario: A program should store and display a student's grade, which is a single letter such as 'A', 'B', or 'F'.

Reasoning: A single letter needs only 1 byte of storage, so char is the correct data type — using int or float here would waste memory and would not even represent a letter directly.

```
#include <stdio.h>
int main() {
    int marks = 88;
    char grade;

    if (marks >= 90) grade = 'A';
    else if (marks >= 75) grade = 'B';
    else grade = 'C';

    printf("Marks = %d\n", marks);
    printf("Grade = %c\n", grade);
    return 0;
}
```

Output: Marks = 88 Grade = B

Problem 4: Storing a Country's Population

Scenario: A program needs to store the population of a country, which can be well over 1.4 billion — a number bigger than what int can safely hold.

Reasoning: A regular int can safely hold values only up to about 2.14 billion, and using it for population figures close to or beyond that limit risks overflow. long long int is the safer choice, since it comfortably stores very large whole numbers.

```
#include <stdio.h>
int main() {
    long long int population = 1428600000LL;

    printf("Population = %lld\n", population);
    printf("Size in memory = %lu bytes\n", sizeof(population));
    return 0;
}
```

Output: Population = 1428600000 Size in memory = 8 bytes

Problem 5: Precise Compound Interest Calculation

Scenario: A banking program must calculate compound interest on a large deposit amount, where even a tiny rounding error in the interest rate can lead to a noticeably wrong final amount.

Reasoning: Since this requires high decimal precision on a potentially large amount, double is preferred over float — it keeps far more accurate digits after the decimal point during repeated multiplication.

```
#include <stdio.h>
#include <math.h>
int main() {
    double principal = 250000.00;
    double rate = 7.25;      // percent per year
    int years = 5;
    double amount;

    amount = principal * pow((1 + rate / 100), years);

    printf("Final Amount = %.2lf\n", amount);
    return 0;
}
```

Output: Final Amount = 353948.34

14.12 Practice Problems — Try It Yourself

Attempt the following on your own. For each one, first decide which data type(s) you would use and why, then write and run the complete program. These are written in a style similar to what you may see in placement coding rounds.

1. Write a program to read a person's weight in kilograms (with a decimal value) and convert it to pounds (1 kg = 2.20462 lbs).
2. Write a program to read a single character from the user and print its ASCII value.
3. Write a program to read the number of items purchased and price per item, then calculate and display the total bill.
4. Write a program to store an employee's ID (a large number issued by the company), basic salary, and print the ID along with the salary formatted to two decimal places.
5. Write a program to swap the values of two integer variables without using a third variable.
6. Write a program that reads the length and breadth of a rectangle (with decimals) and prints its area and perimeter.
7. Write a program to check the size of every primary data type on your own compiler using sizeof() and print the results in a neat format.
8. Write a program to read a temperature in Celsius (which can be negative or a decimal) and convert it to Fahrenheit using the formula F = (C × 9/5) + 32.

Placement Tip: Interviewers often ask "why did you choose this data type?" for a given scenario rather than just asking you to write the code. Always be ready to justify your choice in one sentence, the way the reasoning is explained in the solved problems above.

Chapter 15
Operators in C

15.1 What is an Operator? Why Do We Need Operators?

> **Analogy:** A calculator button, + or x, tells the machine what to do with the numbers. Operators tell C what to do with its operands.

> **Technical Concept:** An operator is a symbol that performs an operation on one or more operands. An expression combines operators and operands to produce a value.

An operator is a special symbol that tells the compiler to perform a specific mathematical, relational, or logical action on one or more values (called operands). For example, in the expression a + b, the + symbol is the operator, and a and b are the operands it acts upon.

We need operators because a program is rarely just about storing data — it constantly has to calculate totals, compare values, make decisions, and combine conditions. Without operators, a variable would just sit there holding a value; operators are what let a program actually do something useful with that value.

- Operators let us perform calculations — totals, averages, discounts, interest, and so on.
- Operators let us compare values — deciding who scored higher, whether a number is within range, whether a password matches.
- Operators let us combine multiple conditions — such as checking if a person is above 18 AND has a valid ID before allowing entry.
- Operators let us make quick decisions in a single line — using the ternary operator instead of writing a full if-else block.

15.2 Types of Operators in C

> **Analogy:** Tools in a toolbox are grouped: cutting, measuring, fastening. C operators are grouped by the job they do.

> **Technical Concept:** C has arithmetic, relational, logical, assignment, increment/decrement, bitwise, conditional and special operators. By operand count they are unary, binary or ternary.

- Arithmetic Operators — for basic mathematical calculations
- Relational Operators — for comparing two values

- Logical Operators — for combining multiple conditions
- Assignment Operators — for storing and updating values
- Increment / Decrement Operators — for increasing or decreasing a value by 1
- Bitwise Operators — for working directly on the individual bits of a number
- Conditional (Ternary) Operator — a shortcut for simple if-else decisions
- Special Operators — sizeof, comma, address-of (&), and dereference (*)

Each type below is explained with its syntax and benefit, followed by its own full set of solved examples (Easy → Medium → HOTS) and practice problems — so you master one operator family completely before moving to the next.

Solved Examples — Identify the Operator Type

Identify the type of operator used in each case: +, &&, <=, ++, =.

Operator	Type	What it does	Operands
+	Arithmetic	adds two numbers	binary
&&	Logical (AND)	true only if both conditions are true	binary
<=	Relational	compares: less than or equal to	binary
++	Increment / Decrement	increases a variable by 1	unary
=	Assignment	stores the right-hand value in the left variable	binary

```
#include <stdio.h>
int main() {
    int a = 7, b = 3, r;
    r = a + b;                // '=' assignment, '+' arithmetic
    if (r >= 10 && a <= 9)    // '>=' and '<=' relational, '&&' logical
        r++;                  // '++' increment
    printf("r = %d", r);
    return 0;
}
```

Output: r = 11

Operators are also grouped by how many operands they take: unary (one: ++, !, sizeof), binary (two: +, &&, <=) and ternary (three: ?:).

Practice Problems — Try It Yourself

1. Classify each operator into its family and say whether it is unary, binary or ternary: %, !, ?:, sizeof, &, >>, *=.

Debugging Corner — Find the Bug (Solved)

DEBUG 1

These comparisons look natural but will not compile. Why?

```
#include <stdio.h>
int main() {
   int age = 20;
   if (age => 18)
      printf("Adult");
   return 0;
}
```

Compiler says: expected expression before '>' token

What is wrong?

• **=> is not an operator in C.** The relational operators are written >= and <= (the = always comes second). Likewise =< is wrong.

Corrected code:

```
#include <stdio.h>
int main() {
   int age = 20;
   if (age >= 18)
      printf("Adult");
   return 0;
}
```

Output: Adult

DEBUG 2

A student writes English words for the logical operators. What goes wrong?

```
#include <stdio.h>
int main() {
    int age = 20, citizen = 1;
    if (age >= 18 and citizen == 1)
        printf("Can vote");
    return 0;
}
```

Compiler says: expected ‘)’ before ‘and’

What is wrong?

• **C uses symbols, not words:** && for AND, || for OR, ! for NOT. The word "and" is treated as an unknown name.

Corrected code:

```
#include <stdio.h>
int main() {
    int age = 20, citizen = 1;
    if (age >= 18 && citizen == 1)
        printf("Can vote");
    return 0;
}
```

Output: Can vote

Debug It Yourself — Practice (answers in Appendix A)

DEBUG IT 1

Find the invalid operator.

```
#include <stdio.h>
int main() {
    int a = 5, b = 5;
    if (a === b)
        printf("Equal");
    return 0;
}
```

Hint: *How many equal signs does the equality operator have?*

DEBUG IT 2

Where do the brackets of an if belong?

```
#include <stdio.h>
int main() {
    int marks = 70, attendance = 80;
    if (marks >= 40) && (attendance >= 75)
        printf("Eligible");
    return 0;
}
```

Hint: *The WHOLE condition must sit inside one pair of brackets.*

15.3 Arithmetic Operators

> **Analogy:** A shop bill is pure arithmetic: add the prices, subtract the discount, multiply by quantity. Arithmetic operators do exactly this.

> **Technical Concept:** The arithmetic operators are +, -, *, / and %. Division of two integers gives an integer (the fraction is dropped) and % gives the remainder, defined only for integers.

Used to perform standard mathematical operations between two operands.

Operator	Meaning	Syntax	Example (a=10, b=3)
+	Addition	a + b	13
-	Subtraction	a - b	7
*	Multiplication	a * b	30
/	Division (quotient)	a / b	3
%	Modulus (remainder)	a % b	1

Benefit: These are the operators you will use in almost every calculation-based program — bills, averages, totals, conversions, and more.

Important: The % (modulus) operator only works with integers in C, not with float or double. Also, when both operands of / are integers, the result is an integer (the decimal part is dropped), not rounded.

Solved Examples — Easy

Build a simple calculator that reads two integers and prints their sum, difference, product, quotient, and remainder.

A direct, one-to-one use of all five arithmetic operators on the same pair of numbers.

```
#include <stdio.h>
int main() {
    int a, b;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);
    printf("Sum=%d Diff=%d Prod=%d Quot=%d Rem=%d", a+b, a-b, a*b, a/b, a%b);
    return 0;
}
```

Output: Input: 20 6 → Sum=26 Diff=14 Prod=120 Quot=3 Rem=2

A teacher wants the average of a student's marks in three subjects.

Add the three marks and divide by 3.0 (not 3) so the result keeps its decimal part.

```
#include <stdio.h>
int main() {
    int m1=72, m2=85, m3=91;
    float avg = (m1+m2+m3)/3.0;
    printf("Average = %.2f", avg);
    return 0;
}
```

Output: Average = 82.67

Evaluate the following expressions (a = 10, b = 3): (a) a + b (b) a % b (c) a / b (d) a * b.

```
#include <stdio.h>
int main() {
    int a = 10, b = 3;
    printf("a + b = %d\n", a + b);
    printf("a %% b = %d\n", a % b);
    printf("a / b = %d\n", a / b);
    printf("a * b = %d\n", a * b);
    return 0;
}
```

Output: a + b = 13 / a % b = 1 / a / b = 3 / a * b = 30

*10 / 3 gives 3 because both operands are int, so the fraction is dropped; 10 % 3 gives the remainder 1. They always fit together: a == (a / b) * b + a % b, i.e. 10 = 3*3 + 1. (%% prints a single % symbol.)*

Solved Examples — Medium

A cinema counter receives ₹500 from a customer for tickets priced ₹120 each. Find how many tickets can be bought and how much change must be returned.

Division gives the number of whole tickets; modulus gives the leftover money that cannot buy another ticket.

```
#include <stdio.h>
int main() {
    int amount = 500, price = 120;
    int tickets = amount / price;
    int change = amount % price;
    printf("Tickets = %d, Change = Rs.%d", tickets, change);
    return 0;
}
```

Output: Tickets = 4, Change = Rs.20

A weather app must convert a temperature reading from Celsius to Fahrenheit.

Apply the standard conversion formula directly using arithmetic operators.

```
#include <stdio.h>
int main() {
    float celsius = 36.5;
    float fahrenheit = (celsius * 9/5) + 32;
    printf("Fahrenheit = %.2f", fahrenheit);
    return 0;
}
```

Output: Fahrenheit = 97.70

A restaurant bill of ₹875 must be split equally among 4 friends. Find each friend's fair share, and how much extra the first friend should pay so the full bill is exactly covered.

Integer division gives the equal share each person owes; modulus gives the leftover rupees someone must cover.

```
#include <stdio.h>
int main() {
    int bill = 875, friends = 4;
    int share = bill / friends;
    int remainder = bill % friends;
    printf("Each pays Rs.%d, first friend pays Rs.%d extra", share, remainder);
    return 0;
}
```

Output: Each pays Rs.218, first friend pays Rs.3 extra

Solved Examples — HOTS

Check whether a given 3-digit number is an Armstrong number (the sum of the cubes of its digits equals the number itself) — a very common placement question.

Extract each digit using / and % (no loop needed for a fixed 3-digit number), cube each digit, and compare the sum to the original.

```
#include <stdio.h>
int main() {
    int n = 153, orig = 153;
    int d1 = n / 100;
    int d2 = (n / 10) % 10;
    int d3 = n % 10;
    int sum = d1*d1*d1 + d2*d2*d2 + d3*d3*d3;
    if (sum == orig) printf("%d is an Armstrong number", orig);
    else printf("%d is NOT an Armstrong number", orig);
    return 0;
}
```

Output: 153 is an Armstrong number

An ATM must dispense ₹2380 using the fewest notes from denominations ₹2000, ₹500, ₹200, ₹100. Find how many notes of each are needed.

Repeatedly use / to find how many notes of the current denomination fit, then %= to carry forward only the remaining amount to the next, smaller denomination.

```
#include <stdio.h>
int main() {
    int amount = 2380;
    int n2000 = amount / 2000; amount %= 2000;
```

```
    int n500 = amount / 500; amount %= 500;
    int n200 = amount / 200; amount %= 200;
    int n100 = amount / 100; amount %= 100;
    printf("2000x%d 500x%d 200x%d 100x%d Remaining=%d", n2000, n500, n200, n100,
amount);
    return 0;
}
```

Output: 2000x1 500x0 200x1 100x1 Remaining=80

Convert a video's total duration given in seconds (e.g. 9045 seconds) into hours, minutes, and seconds.

Use integer division and modulus together, layer by layer, to peel off hours, then minutes, then leftover seconds.

```
#include <stdio.h>
int main() {
    int totalSeconds = 9045;
    int hours = totalSeconds / 3600;
    int minutes = (totalSeconds % 3600) / 60;
    int seconds = totalSeconds % 60;
    printf("%d hr %d min %d sec", hours, minutes, seconds);
    return 0;
}
```

Output: 2 hr 30 min 45 sec

What do / and % return when the operands are negative?

Since C99, integer division **truncates toward zero** and the result of % takes the **sign of the left operand** (the dividend).

```
#include <stdio.h>
int main() {
    printf("%d %d\n", 7 / 2, 7 % 2);
    printf("%d %d\n", -7 / 2, -7 % 2);
    printf("%d %d\n", 7 / -2, 7 % -2);
    printf("%d %d", -7 / -2, -7 % -2);
    return 0;
}
```

Output: 3 1 / -3 -1 / -3 1 / 3 -1

Consequence: n % 2 == 1 is NOT a safe test for odd numbers — for n = -7 the remainder is -1. Use n % 2 != 0 instead. This fact returns in the parity questions of Chapter 16.

Practice Problems — Try It Yourself

1. Write a program to calculate the area and perimeter of a rectangle given its length and breadth.
2. A pizza costs ₹450 in total, plus a fixed ₹50 delivery charge, to be split equally among a given number of friends. Write a program to compute each friend's share.
3. Write a program to extract and print the last digit and the last two digits of a given number using the modulus operator.
4. Take a = 17 and b = 5. Print a+b, a-b, a*b, a/b and a%b. Then repeat with a = -17 and compare the signs.
5. Predict, then verify: 7/2*2, 7/(2*2), 7.0/2*2, 2*7/2.
6. Read a 4-digit number (e.g. 1234) and print the sum of its digits using only / and % (no loops).

Debugging Corner — Find the Bug (Solved)

DEBUG 1

The average of 80, 90 and 100 should be 90, but 203 is printed. Why?

```
#include <stdio.h>
int main() {
   int a = 80, b = 90, c = 100;
   float avg = a + b + c / 3;
   printf("Average = %.2f", avg);
   return 0;
}
```

Output of the buggy program: Average = 203.00

What is wrong?

• **Precedence:** / is done before +, so only c is divided: 80 + 90 + 33 = 203. The sum needs brackets.

Corrected code:

```
#include <stdio.h>
int main() {
    int a = 80, b = 90, c = 100;
    float avg = (a + b + c) / 3.0;
    printf("Average = %.2f", avg);
    return 0;
}
```

Output: Average = 90.00

DEBUG 2

This program does not compile. Why, and what is the fix?

```
#include <stdio.h>
int main() {
    float weight = 5.5;
    int r = weight % 2;
    printf("%d", r);
    return 0;
}
```

Compiler says: invalid operands to binary % (have ‘float’ and ‘int’)

What is wrong?

• **% works only on integers.** For real numbers use fmod() from math.h.

Corrected code:

```
#include <stdio.h>
#include <math.h>
int main() {
    float weight = 5.5;
    printf("%.1f", fmod(weight, 2));
    return 0;
}
```

Output: 1.5

Debug It Yourself — Practice (answers in Appendix A)

DEBUG IT 1

45 marks out of 60 should give 75 percent. Why is 0 printed?

```
#include <stdio.h>
int main() {
    int obtained = 45, total = 60;
    float percent = (obtained / total) * 100;
    printf("Percentage = %.2f", percent);
    return 0;
}
```

Hint: *Work out obtained / total for two ints.*

DEBUG IT 2

This program crashes when the count is 0. Guard it.

```
#include <stdio.h>
int main() {
    int total = 300, count = 0;
    int average = total / count;
    printf("Average = %d", average);
    return 0;
}
```

Hint: *What does the computer do when asked to divide an integer by zero?*

15.4 Relational Operators

> **Analogy:** A referee compares two scores and announces who is ahead. Relational operators compare two values and report the result.

> **Technical Concept:** The relational operators are <, >, <=, >=, == and !=. Each gives 1 when the comparison is true and 0 when it is false.

Used to compare two values. The result of a relational expression is always either 1 (true) or 0 (false).

Operator	Meaning	Syntax	Example (a=10, b=3)
==	Equal to	a == b	0 (false)
!=	Not equal to	a != b	1 (true)

Operator	Meaning	Syntax	Example (a=10, b=3)
>	Greater than	a > b	1 (true)
<	Less than	a < b	0 (false)
>=	Greater than or equal to	a >= b	1 (true)
<=	Less than or equal to	a <= b	0 (false)

Benefit: Relational operators are the backbone of every decision-making statement — if, while, for — since a condition must evaluate to true or false before the program can decide what to do next.

Common mistake: Using = (assignment) instead of == (comparison) inside a condition. if (marks = 35) assigns 35 to marks instead of comparing it, and always evaluates as true — this is one of the most common bugs beginners write.

Solved Examples — Easy

Compare two numbers and print whichever is greater.

A direct chain of relational comparisons.

```
#include <stdio.h>
int main() {
    int a=45, b=38;
    if (a > b) printf("%d is greater", a);
    else if (b > a) printf("%d is greater", b);
    else printf("Both are equal");
    return 0;
}
```

Output: 45 is greater

Check whether a student has passed, given a pass mark of 40.

A single relational comparison decides the result.

```
#include <stdio.h>
int main() {
    int marks = 32;
    if (marks >= 40) printf("Result: Pass");
    else printf("Result: Fail");
    return 0;
}
```

Output: Result: Fail

What will be the value of c after execution? int a = 4, b = 2, c; c = a > b;

A relational operator produces **1 for true and 0 for false** — an ordinary int. Since 4 > 2 is true, c becomes 1.

```
#include <stdio.h>
int main() {
    int a = 4, b = 2, c;
    c = a > b;
    printf("c = %d, a < b gives %d, a == 4 gives %d", c, a < b, a == 4);
    return 0;
}
```

Output: c = 1, a < b gives 0, a == 4 gives 1

Differentiate between = (assignment) and == (equality) with an example.

	= (assignment)	== (equality)
Purpose	stores the right-hand value into the left variable	compares two values
Changes a variable?	**Yes**	**No**
Result	the value that was stored	1 (true) or 0 (false)

```
#include <stdio.h>
int main() {
    int x = 10;
    printf("x == 10 ? %d\n", x == 10);   // comparison, x unchanged
    x = 25;                              // assignment, x becomes 25
    printf("x is now %d\n", x);
    printf("x == 10 ? %d", x == 10);
    return 0;
}
```

Output: x == 10 ? 1 / x is now 25 / x == 10 ? 0

Solved Examples — Medium

A scholarship is open only to students born between 2003 and 2006 (inclusive). Check eligibility for a given birth year.

Since logical operators (&&) are covered in the next section, this is solved using two nested relational checks to isolate the concept.

```
#include <stdio.h>
int main() {
    int birthYear = 2004;
    if (birthYear >= 2003) {
        if (birthYear <= 2006) printf("Eligible");
        else printf("Not Eligible");
    } else printf("Not Eligible");
    return 0;
}
```

Output: Eligible

A bonus feature needs the smaller of two exam scores — the "minimum score to beat".

A single < comparison picks out the smaller value.

```
#include <stdio.h>
int main() {
    int s1=72, s2=65;
    if (s1 < s2) printf("Smallest = %d", s1);
    else printf("Smallest = %d", s2);
    return 0;
}
```

Output: Smallest = 65

A signup form requires a password's length to be between 8 and 16 characters. Validate a given password's length.

Use strlen() from string.h to get the length, then apply relational checks on that length.

```
#include <stdio.h>
#include <string.h>
int main() {
    char password[] = "Secure123";
    int len = strlen(password);
    if (len >= 8) {
        if (len <= 16) printf("Valid password length");
        else printf("Password too long");
```

```
    } else printf("Password too short");
    return 0;
}
```

Output: Valid password length

Predict the output and explain the behaviour of x == y == 1.

== is **left-associative**, so x == y == 1 is read as (x == y) == 1. The inner comparison gives 1 or 0, and THAT is compared with 1. It does not mean "x, y and 1 are all equal".

```
#include <stdio.h>
int main() {
    int x = 4, y = 4;
    printf("%d\n", x == y == 1);     // (4==4)==1 -> 1==1 -> 1
    int p = 5, q = 5;
    printf("%d", p == q == 5);       // (5==5)==5 -> 1==5 -> 0 !
    return 0;
}
```

Output: 1 / 0

The second line prints 0 even though p, q and 5 are all equal. To test three values, write p == q && q == 5.

Solved Examples — HOTS

Classify a temperature reading into alert levels: below 0 = "Freezing Alert", 0–15 = "Cold", 16–30 = "Normal", above 30 = "Heat Alert".

A chain of relational comparisons in an if-else ladder, evaluated top to bottom — each check only runs if the ones above it failed.

```
#include <stdio.h>
int main() {
    int temp = 34;
    if (temp < 0) printf("Freezing Alert");
    else if (temp <= 15) printf("Cold");
    else if (temp <= 30) printf("Normal");
    else printf("Heat Alert");
    return 0;
}
```

Output: Heat Alert

Given three numbers, check whether the second lies strictly between the first and the third, using only relational operators (no logical operators).

Chain two relational checks using nested if statements instead of &&.

```
#include <stdio.h>
int main() {
    int low=10, mid=25, high=40;
    if (mid > low) {
        if (mid < high) printf("%d lies between %d and %d", mid, low, high);
        else printf("Not in range");
    } else printf("Not in range");
    return 0;
}
```

Output: 25 lies between 10 and 40

Given three side lengths, determine whether they can form a valid triangle. (Rule: each side must be less than the sum of the other two.)

Three separate relational checks, each comparing one side against the sum of the other two, nested together.

```
#include <stdio.h>
int main() {
    int a=7, b=10, c=5;
    if (a < b + c) {
        if (b < a + c) {
            if (c < a + b) printf("Valid Triangle");
            else printf("Not a Valid Triangle");
        } else printf("Not a Valid Triangle");
    } else printf("Not a Valid Triangle");
    return 0;
}
```

Output: Valid Triangle

Explain what happens if you use = instead of == in an if condition. Give an example where the compiler gives no error.

The condition becomes an **assignment**. It stores the value, and the value stored (non-zero = true, zero = false) decides the branch. The variable is silently overwritten and the branch no longer depends on the data.

```
#include <stdio.h>
int main() {
    int marks = 30;
    if (marks = 100)          // assigns 100 -> value 100 is non-zero -> TRUE
        printf("Full marks!");
    else
        printf("Not full marks");
    printf("\nmarks is now %d", marks);
    return 0;
}
```

Output: Full marks! / marks is now 100

The program compiles (GCC only warns if you compile with -Wall) and reports "Full marks!" for a student with 30. Defensive habit: write the constant first — if (100 == marks) — because a mistyped 100 = marks is a compile error.

Practice Problems — Try It Yourself

1. Write a program to check whether a given ASCII value lies within the printable character range (32 to 126).
2. Write a program to check whether three given numbers are already arranged in ascending order, using only relational operators.
3. Write a program to check whether a person is eligible to vote (age is 18 or above).
4. Evaluate by hand, then verify with printf: 5 > 3, 5 == 5.0, 'a' < 'b', 3 != 3, 10 >= 10, 'A' == 65.
5. Predict 10 < 5 < 3 and explain why the answer is not 0.
6. Read two integers and print 1 if they are equal, otherwise 0, without using if.

Debugging Corner — Find the Bug (Solved)

DEBUG 1

This program should say "x is not 5". What does it print and why?

```
#include <stdio.h>
int main() {
    int x = 10;
    if (x = 5)
        printf("x is 5");
    else
        printf("x is not 5");
    return 0;
}
```

Output of the buggy program: x is 5

What is wrong?

• **= instead of ==.** x = 5 assigns 5 to x; the value 5 is non-zero, so the if is always true and x is destroyed.

Corrected code:

```
#include <stdio.h>
int main() {
    int x = 10;
    if (x == 5)
        printf("x is 5");
    else
        printf("x is not 5");
    return 0;
}
```

Output: x is not 5

DEBUG 2

150 is accepted as valid marks. Why?

```
#include <stdio.h>
int main() {
   int marks = 150;
   if (0 <= marks <= 100)
      printf("Valid marks");
   else
      printf("Invalid marks");
   return 0;
}
```

Output of the buggy program: Valid marks

What is wrong?

• **Chained comparison does not exist in C.** (0 <= 150) is 1, and then 1 <= 100 is also true. Two conditions must be joined with &&.

Corrected code:

```
#include <stdio.h>
int main() {
   int marks = 150;
   if (marks >= 0 && marks <= 100)
      printf("Valid marks");
   else
      printf("Invalid marks");
   return 0;
}
```

Output: Invalid marks

Debug It Yourself — Practice (answers in Appendix A)

DEBUG IT 1

Mathematically 0.1 + 0.2 equals 0.3, yet the program prints "Not equal".

```
#include <stdio.h>
int main() {
    double x = 0.1 + 0.2;
    if (x == 0.3)
        printf("Equal");
    else
        printf("Not equal");
    return 0;
}
```

Hint: *Real numbers are stored approximately in binary.*

DEBUG IT 2

The programmer wants "a is not equal to b". What does =! do?

```
#include <stdio.h>
int main() {
    int a = 3, b = 3;
    if (a =! b)
        printf("Different");
    else
        printf("Same");
    return 0;
}
```

Hint: *Read =! as two separate operators.*

15.5 Logical Operators

> **Analogy:** To enter a club you need a ticket AND an ID. For a discount, a student card OR a senior card is enough. Logical operators combine conditions like this.

> **Technical Concept:** The logical operators are && (AND), || (OR) and ! (NOT). They evaluate left to right with short-circuiting: the right side is skipped when the left side already decides the result.

Used to combine two or more conditions into a single expression.

Operator	Meaning	Syntax	Result Rule

Operator	Meaning	Syntax	Result Rule
&&	Logical AND	cond1 && cond2	True only if BOTH conditions are true
||	Logical OR	cond1 || cond2	True if AT LEAST ONE condition is true
!	Logical NOT	!cond	Reverses true to false and false to true

Benefit: Real-world decisions almost always depend on more than one condition — logical operators let a program check several rules at once instead of writing nested if statements for each one.

Solved Examples — Easy

A bank approves a personal loan only if the applicant's age is 21 or above AND their salary is ₹25,000 or above.

Combine two relational checks with &&.

```
#include <stdio.h>
int main() {
    int age=24; float salary=32000;
    if (age >= 21 && salary >= 25000) printf("Loan Approved");
    else printf("Loan Rejected");
    return 0;
}
```

Output: Loan Approved

A student qualifies for the next round if they passed EITHER Math or Science.

Combine two flags with ||.

```
#include <stdio.h>
int main() {
    int passedMath=0, passedScience=1;
    if (passedMath || passedScience) printf("Eligible for next round");
    else printf("Not eligible");
    return 0;
}
```

Output: Eligible for next round

Solved Examples — Medium

Check whether a number is divisible by 3 OR by 5 — a building block of the classic FizzBuzz problem.

Combine two modulus checks using ||.

```
#include <stdio.h>
int main() {
    int n = 15;
    if (n % 3 == 0 || n % 5 == 0) printf("Divisible by 3 or 5");
    else printf("Not divisible by either");
    return 0;
}
```

Output: Divisible by 3 or 5

Check whether a character entered by the user is a consonant (a letter that is NOT a vowel).

Chain the five vowel checks with ||, then reverse the whole result using !.

```
#include <stdio.h>
int main() {
    char ch = 'k';
    if (!(ch=='a'||ch=='e'||ch=='i'||ch=='o'||ch=='u')) printf("%c is a consonant", ch);
    else printf("%c is a vowel", ch);
    return 0;
}
```

Output: k is a consonant

An employee qualifies for a bonus if (experience > 5 years AND rating >= 4) OR they received a special award.

Combine && inside brackets first, then join the result with || — precedence matters here.

```
#include <stdio.h>
int main() {
    int experience=6, rating=3, specialAward=1;
    if ((experience > 5 && rating >= 4) || specialAward == 1) printf("Bonus Approved");
    else printf("Bonus Not Approved");
    return 0;
}
```

Output: Bonus Approved

Write a program to check whether a number is divisible by both 3 and 5 using the logical && operator.

```
#include <stdio.h>
int main() {
    int n = 45;
    if (n % 3 == 0 && n % 5 == 0)
        printf("%d is divisible by both 3 and 5", n);
    else
        printf("%d is not divisible by both", n);
    return 0;
}
```

Output: 45 is divisible by both 3 and 5

Both remainders must be 0 at the same time. This is the same as testing n % 15 == 0.

Check whether a triangle is valid given its three angles (the sum must equal 180 and each angle must be greater than 0).

```
#include <stdio.h>
int main() {
    int a = 60, b = 70, c = 50;
    if (a > 0 && b > 0 && c > 0 && a + b + c == 180)
        printf("Valid triangle");
    else
        printf("Not a valid triangle");
    return 0;
}
```

Output: Valid triangle

Why check > 0 as well? Angles 90, 90, 0 add up to 180 but do not form a triangle.

Solved Examples — HOTS

Check whether a given year is a leap year. (Rule: divisible by 4 AND not divisible by 100, OR divisible by 400.)

This is the single most-asked logical-operator placement question — it combines &&, ||, and != in one compact expression.

```
#include <stdio.h>
int main() {
   int year = 2024;
   if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)) printf("%d is a Leap Year",
year);
   else printf("%d is NOT a Leap Year", year);
   return 0;
}
```

Output: 2024 is a Leap Year

A pedestrian "WALK" signal should turn ON only if the light is red AND (the pedestrian button was pressed OR it is a designated school-zone crossing).

Group the OR condition in brackets first since it must be evaluated as one unit before being ANDed with the red-light check.

```
#include <stdio.h>
int main() {
   int isRed=1, buttonPressed=0, isSchoolZone=1;
   if (isRed && (buttonPressed || isSchoolZone)) printf("WALK signal ON");
   else printf("WALK signal OFF");
   return 0;
}
```

Output: WALK signal ON

A store gives a discount if (the customer is a member AND total purchase exceeds ₹1000) OR (it is a festival sale AND the customer is NOT a returning customer).

Trace both bracketed groups independently, then combine with ||. This tests whether you can evaluate a compound condition step by step rather than guessing.

```
#include <stdio.h>
int main() {
   int isMember=0, totalPurchase=1500, isFestivalSale=1, isReturning=0;
   if ((isMember && totalPurchase > 1000) || (isFestivalSale && !isReturning))
printf("Discount Applied");
   else printf("No Discount");
   return 0;
}
```

Output: Discount Applied

Trace: isMember=0, so the first group is false regardless of totalPurchase. isFestivalSale=1 and !isReturning=1, so the second group is true. false || true = true → Discount Applied.

Predict the output and explain short-circuit evaluation. Why is there no divide-by-zero error?
int a = 0, b = 5, c; c = (a != 0) && (b / a > 1);

&& evaluates left to right and **stops as soon as the answer is known**. Here a != 0 is false, so the whole && must be false and the right side b / a is never evaluated. || behaves the same way when its left side is true.

```
#include <stdio.h>
int main() {
    int a = 0, b = 5, c;
    c = (a != 0) && (b / a > 1);
    printf("c = %d\n", c);
    int x = 5;
    int r = (x > 10) && (x++ > 0);   // x++ is skipped
    printf("r = %d, x = %d", r, x);
    return 0;
}
```

Output: c = 0 / r = 0, x = 5

Practical use: put the safe check on the left and the risky one on the right, e.g. (b != 0) && (a / b > 2).

Predict the output: int a = 5, b = 10, c = 15; int result = (a < b) && (b < c) || (a > c);

&& has higher precedence than ||, so the expression is ((a < b) && (b < c)) || (a > c) = (1 && 1) || 0 = 1.

```
#include <stdio.h>
int main() {
    int a = 5, b = 10, c = 15;
    int result = (a < b) && (b < c) || (a > c);
    printf("result = %d\n", result);
    printf("%d %d", 1 || 0 && 0, (1 || 0) && 0);
    return 0;
}
```

Output: result = 1 / 1 0

The last line shows why precedence matters: 1 || 0 && 0 is 1 because 0 && 0 is done first, but (1 || 0) && 0 is 0. Use brackets to make your intention obvious.

A bulb must turn ON only if exactly two of three switches are ON (not zero, one or three). Use only if-else and logical operators.

S1	S2	S3	Bulb
0	0	0	OFF
0	0	1	OFF
0	1	0	OFF
0	1	1	**ON**
1	0	0	OFF
1	0	1	**ON**
1	1	0	**ON**
1	1	1	OFF

```
#include <stdio.h>
int main() {
   int s1 = 1, s2 = 0, s3 = 1;
   if ((s1 && s2 && !s3) || (s1 && !s2 && s3) || (!s1 && s2 && s3))
      printf("Bulb ON");
   else
      printf("Bulb OFF");
   return 0;
}
```

Output: Bulb ON

Read the three ON rows of the truth table and join them with ||. A shortcut is (s1 + s2 + s3) == 2, because 0/1 values can be added.

A scholarship has two independent ways to qualify: Merit (percentage ≥ 90, any family income) OR Need-based (percentage ≥ 75 AND family income ≤ 200000). Combine them in one condition.

```
#include <stdio.h>
int main() {
    float percentage = 82.5;
    int familyIncome = 150000;
    if (percentage >= 90 || (percentage >= 75 && familyIncome <= 200000))
        printf("Eligible for scholarship");
    else
        printf("Not eligible");
    return 0;
}
```

Output: Eligible for scholarship

Percentage	Family income	Merit?	Need-based?	Result
92	500000	yes	—	Eligible
80	150000	no	yes	Eligible
80	300000	no	no	Not eligible
70	100000	no	no	Not eligible

The brackets around the need-based pair make the precedence explicit even though && would bind first anyway.

Practice Problems — Try It Yourself

1. Write a program to check whether a number is both even AND positive.
2. Given three flags (0 or 1) representing whether three different judges approved a proposal, write a program to check if at least two of the three approved it.
3. Write a program to validate that a password is at least 8 characters long AND contains at least one digit (assume a flag hasDigit is already given as 0 or 1).
4. Three switches control a lamp. Write the condition for "exactly one switch is ON" and for "at least two switches are ON".
5. A gym discount applies if age ≥ 60, or if age ≥ 55 and the person is a member. Write the single condition and test it with 3 cases.

6. Predict i, j and r: int i = 0, j = 5, r; r = (i++ && j++); then with int p = 0, q = 5; r = (p++ || q++); Explain which side was skipped.

Debugging Corner — Find the Bug (Solved)

DEBUG 1

Any number is accepted as a valid option. Why?

```
#include <stdio.h>
int main() {
   int choice = 7;
   if (choice == 1 || 2 || 3)
      printf("Valid option");
   else
      printf("Invalid option");
   return 0;
}
```

Output of the buggy program: Valid option

What is wrong?

• **choice == 1 || 2 || 3** means (choice == 1) || 2 || 3. The constants 2 and 3 are non-zero, so they are "true" on their own and the condition is always true. Each comparison must be written out.

Corrected code:

```
#include <stdio.h>
int main() {
   int choice = 7;
   if (choice == 1 || choice == 2 || choice == 3)
      printf("Valid option");
   else
      printf("Invalid option");
   return 0;
}
```

Output: Invalid option

DEBUG 2

Both conditions are true (non-zero), yet the entry is denied. Why?

```
#include <stdio.h>
int main() {
    int hasID = 2, paid = 1;
    if (hasID & paid)
        printf("Entry allowed");
    else
        printf("Entry denied");
    return 0;
}
```

Output of the buggy program: Entry denied

What is wrong?

• **& is the bitwise AND, not the logical one.** 2 (binary 10) & 1 (binary 01) = 0 = false. For "both true" use &&.

Corrected code:

```
#include <stdio.h>
int main() {
    int hasID = 2, paid = 1;
    if (hasID && paid)
        printf("Entry allowed");
    else
        printf("Entry denied");
    return 0;
}
```

Output: Entry allowed

Debug It Yourself — Practice (answers in Appendix A)

DEBUG IT 1

This will not compile. Fix the range check.

```
#include <stdio.h>
int main() {
    int marks = 95;
    if (marks > 90 && < 100)
        printf("Excellent");
    return 0;
}
```

Hint: *Each side of && must be a complete condition.*

DEBUG IT 2

x is 3, so "x is not 5" should print. It does not.

```
#include <stdio.h>
int main() {
    int x = 3;
    if (!x == 5)
        printf("x is not 5");
    return 0;
}
```

Hint: *! is applied before ==. What is !3?*

15.6 Assignment Operators

> **Analogy:** Putting a number in a drawer replaces what was there before. Assignment stores a value in a variable, replacing the old one.

> **Technical Concept:** The assignment operator = stores the right-hand value in the left-hand variable. Compound forms (+=, -=, *=, /=, %=) combine an operation with assignment.

Used to assign a value to a variable, often while also performing a calculation in the same step.

Operator	Meaning	Example	Equivalent To
=	Assigns value	a = 5;	a = 5;
+=	Adds then assigns	a += 5;	a = a + 5;

Operator	Meaning	Example	Equivalent To
-=	Subtracts then assigns	a -= 5;	a = a - 5;
*=	Multiplies then assigns	a *= 5;	a = a * 5;
/=	Divides then assigns	a /= 5;	a = a / 5;
%=	Modulus then assigns	a %= 5;	a = a % 5;

Benefit: Compound assignment operators make code shorter and easier to read, especially inside loops where a value is updated repeatedly, such as adding to a running total.

Solved Examples — Easy

Add 5 bonus marks to a student's existing score of 78.

A single += updates the variable in place.

```
#include <stdio.h>
int main() {
    int score = 78;
    score += 5;
    printf("Final Score = %d", score);
    return 0;
}
```

Output: Final Score = 83

Deduct a ₹250 penalty charge from an account balance of ₹5000.

A single -= updates the balance in place.

```
#include <stdio.h>
int main() {
    float balance = 5000.0;
    balance -= 250.0;
    printf("Balance = %.2f", balance);
    return 0;
}
```

Output: Balance = 4750.00

Swap the values of two variables using a temporary variable.

Think of two glasses of juice: to exchange their contents you need a third, empty glass.

```
#include <stdio.h>
int main() {
    int a = 5, b = 10, temp;
    temp = a;     // save a
    a = b;        // a takes b's value
    b = temp;     // b takes the saved value
    printf("a = %d, b = %d", a, b);
    return 0;
}
```

Output: a = 10, b = 5

Solved Examples — Medium

A shopping cart adds items one at a time. Track the running total as three items are added.

Each item's price is added to the same total using += in sequence.

```
#include <stdio.h>
int main() {
    float total = 0;
    total += 250.00;
    total += 899.00;
    total += 149.50;
    printf("Cart Total = Rs.%.2f", total);
    return 0;
}
```

Output: Cart Total = Rs.1298.50

A jacket costs ₹2000. A festival sale applies a 10% discount, followed by an additional 5% loyalty discount. Find the final price.

Apply each discount as a multiplication factor using *=, one after another.

```
#include <stdio.h>
int main() {
    float price = 2000.0;
    price *= 0.90;
    price *= 0.95;
    printf("Final Price = Rs.%.2f", price);
    return 0;
}
```

Output: Final Price = Rs.1710.00

Predict the output of a chained assignment, and explain the associativity involved.

The = operator is right-to-left associative, so the rightmost assignment happens first.

```
#include <stdio.h>
int main() {
   int a, b, c;
   a = b = c = 10;
   c += 5;
   printf("a=%d b=%d c=%d", a, b, c);
   return 0;
}
```

Output: a=10 b=10 c=15

Because c=10 is evaluated first, then b=c makes b=10, then a=b makes a=10. Only c is changed afterwards by +=5, so a and b remain untouched at 10.

What is the difference between x *= 2 + 3; and x = x * 2 + 3; when x = 4?

In a compound assignment the **entire right side is evaluated first**, so x *= 2 + 3 means x = x * (2 + 3).

```
#include <stdio.h>
int main() {
   int x = 4, y = 4;
   x *= 2 + 3;       // x = 4 * (2 + 3) = 20
   y = y * 2 + 3;    // y = 4 * 2 + 3   = 11
   printf("x = %d, y = %d", x, y);
   return 0;
}
```

Output: x = 20, y = 11

Assignment operators have the lowest precedence, so everything to their right is grouped first.

Solved Examples — HOTS

A fixed deposit of ₹10,000 earns 8% interest per year, compounded annually. Using only *= (no loop, 3 fixed years), calculate the amount after 3 years.

Multiply the amount by 1.08 three times in a row, once per year, updating the same variable each time.

```
#include <stdio.h>
int main() {
    float amount = 10000.0;
    amount *= 1.08;
    amount *= 1.08;
    amount *= 1.08;
    printf("Amount after 3 years = Rs.%.2f", amount);
    return 0;
}
```

Output: Amount after 3 years = Rs.12597.12

A game score starts at 50. Apply this exact sequence and predict the final score: score += 20; score *= 2; score -= 30; score %= 100;

Trace the score through each compound assignment operation, one line at a time, in order.

```
#include <stdio.h>
int main() {
    int score = 50;
    score += 20;
    score *= 2;
    score -= 30;
    score %= 100;
    printf("Final Score = %d", score);
    return 0;
}
```

Output: Final Score = 10

*Trace: 50+20=70 → 70*2=140 → 140-30=110 → 110%100=10.*

A playlist has 7 songs (indexed 0–6). Each "next song" press increases the index by 1, wrapping back to 0 after the last song. Starting at index 5, simulate 3 presses and predict the final index.

Use %= (or the equivalent full expression) after every increment so the index automatically wraps around the playlist size — a real technique used in circular queues.

```
#include <stdio.h>
int main() {
    int index = 5;
    index = (index + 1) % 7;
    index = (index + 1) % 7;
    index = (index + 1) % 7;
    printf("Current Song Index = %d", index);
    return 0;
}
```

Output: Current Song Index = 1

Trace: 5→6 (press1), 6→0 (press2, wraps around), 0→1 (press3).

Swap two integers without a third variable in two different ways: (i) arithmetic operators (+, -), (ii) bitwise XOR (^).

```
#include <stdio.h>
int main() {
    int x = 7, y = 3;          // method (i): + and -
    x = x + y;                 // x = 10
    y = x - y;                 // y = 7
    x = x - y;                 // x = 3
    printf("Arithmetic: x = %d, y = %d\n", x, y);

    int p = 7, q = 3;          // method (ii): XOR
    p = p ^ q;
    q = p ^ q;
    p = p ^ q;
    printf("XOR      : p = %d, q = %d", p, q);
    return 0;
}
```

Output: Arithmetic: x = 3, y = 7 / XOR : p = 3, q = 7

Step (p = 7 = 0111, q = 3 = 0011)	p	q
start	0111	0011
p = p ^ q	0100	0011
q = p ^ q	0100	0111
p = p ^ q	0011	0111

XOR trick: a ^ b ^ b = a. The arithmetic version can overflow for very large values. In real code a temporary variable is clearer; the tricks are interview classics. The pointer-based danger of these tricks is shown in section 15.10.

Practice Problems — Try It Yourself

1. A taxi fare starts at ₹50 and increases by ₹12 for every km. Using +=, add the fare for 5 km travelled one km at a time and print the final fare.
2. Trace the final value of x after: x=8; x+=5; x-=2; x*=3; x/=4; — write the program and verify your hand-traced answer.
3. Using %=, simulate a 24-hour clock that starts at hour 22 and advances by 5 hours; print the resulting hour (it should wrap around correctly).
4. Swap a = 25 and b = 40 three ways: with a temp variable, with + and -, and with ^. Confirm all three give the same answer.
5. A balance is 5000.0. Using only compound assignments, add a deposit of 2000, subtract a withdrawal of 1500, then add 5% interest. Print the balance.
6. Explain why `a = a + b; b = a - b; a = a - b;` can fail for very large ints but a temp-variable swap never does.

Debugging Corner — Find the Bug (Solved)

DEBUG 1

This program is meant to swap two numbers but prints the wrong result. Find the bug.

```
#include <stdio.h>
int main() {
    int a = 5, b = 10, temp;
    temp = a;
    a = b;
    b = temp;
    printf("a = %d, b = %d", a, a);
    return 0;
}
```

Output of the buggy program: a = 10, b = 10

What is wrong?

• **The swap itself is correct;** the bug is in printf(): it prints a twice instead of a and b.

Corrected code:

```
#include <stdio.h>
int main() {
    int a = 5, b = 10, temp;
    temp = a;
    a = b;
    b = temp;
    printf("a = %d, b = %d", a, b);
    return 0;
}
```

Output: a = 10, b = 5

DEBUG 2

The total should become 125 but shows 25. Why?

```
#include <stdio.h>
int main() {
    int total = 100;
    total =+ 25;
    printf("Total = %d", total);
    return 0;
}
```

Output of the buggy program: Total = 25

What is wrong?

• **=+ is not a compound operator.** total =+ 25 is read as total = +25 (assign positive 25). The correct operator has the sign first: +=.

Corrected code:

```
#include <stdio.h>
int main() {
    int total = 100;
    total += 25;
    printf("Total = %d", total);
    return 0;
}
```

Output: Total = 125

Debug It Yourself — Practice (answers in Appendix A)

DEBUG IT 1

Why does this not compile?

```
#include <stdio.h>
int main() {
    int a = 3, b = 4, c;
    a + b = c;
    printf("%d", c);
    return 0;
}
```

Hint: *Which side of = receives the value?*

DEBUG IT 2

The chained declaration fails. Why?

```
#include <stdio.h>
int main() {
    int a = b = 10;
    printf("%d %d", a, b);
    return 0;
}
```

Hint: *Has b been declared before it is used?*

15.7 Increment and Decrement Operators

> **Analogy:** A lift's floor counter goes up by one when it rises and down by one when it descends. ++ and -- do the same to a variable.

> **Technical Concept:** ++ and -- increase or decrease a variable by 1. In prefix form (++x) the change happens before the value is used; in postfix form (x++) it happens after.

These operators increase (++) or decrease (--) the value of a variable by exactly 1, and can be used in pre or post form — a classic placement interview topic.

Form	Syntax	Behaviour
Pre-increment	++a	Value is increased first, then used in the expression
Post-increment	a++	Value is used in the expression first, then increased

Form	Syntax	Behaviour
Pre-decrement	--a	Value is decreased first, then used in the expression
Post-decrement	a--	Value is used in the expression first, then decreased

Solved Examples — Easy

Show the exact difference between post-increment and pre-increment on the same starting value.

Print a++ first (uses old value, then increases), then ++a (increases first, then uses new value).

```
#include <stdio.h>
int main() {
    int a = 5;
    printf("a++ = %d\n", a++);
    printf("++a = %d\n", ++a);
    return 0;
}
```

Output: a++ = 5 / ++a = 7

Display a launch countdown: 3, 2, 1, Liftoff! using post-decrement.

Each printf uses the current value of t, then decreases it afterwards.

```
#include <stdio.h>
int main() {
    int t = 3;
    printf("%d ", t--);
    printf("%d ", t--);
    printf("%d ", t--);
    printf("Liftoff!");
    return 0;
}
```

Output: 3 2 1 Liftoff!

Solved Examples — Medium

A queue starts with 10 people waiting. Track people processed (post-increment) and people waiting (pre-decrement) as 2 people are served, printed one at a time.

Use two different variables so the expression stays well-defined, and observe how post vs pre forms behave differently even in the same line.

```
#include <stdio.h>
int main() {
    int processed = 0, waiting = 10;
    printf("processed=%d waiting=%d\n", processed++, --waiting);
    printf("processed=%d waiting=%d\n", processed++, --waiting);
    printf("Totals: processed=%d waiting=%d", processed, waiting);
    return 0;
}
```

Output: processed=0 waiting=9 / processed=1 waiting=8 / Totals: processed=2 waiting=8

Simulate a lift moving up floor by floor, using pre-increment, and stopping after 3 presses.

Each ++floor increases the floor before printing it, so the very first print already shows floor 1, not floor 0.

```
#include <stdio.h>
int main() {
    int floor = 0;
    printf("Floor %d\n", ++floor);
    printf("Floor %d\n", ++floor);
    printf("Floor %d\n", ++floor);
    printf("Arrived!");
    return 0;
}
```

Output: Floor 1 / Floor 2 / Floor 3 / Arrived!

A countdown timer is checked twice using post-decrement; store each checked value and the timer's value afterwards.

Each t-- returns the current value into a variable before reducing t.

```
#include <stdio.h>
int main() {
    int timer = 5;
    int a = timer--;
    int b = timer--;
    printf("First check=%d, Second check=%d, Timer now=%d", a, b, timer);
    return 0;
}
```

Output: First check=5, Second check=4, Timer now=3

Trace the values: int a = 5; int b = a++ + 10; int c = ++a * 2;

Statement	What happens	a	b / c
int a = 5;		5	
b = a++ + 10;	uses 5, THEN a becomes 6	6	b = 15
c = ++a * 2;	a becomes 7 FIRST, then used	7	c = 14

```
#include <stdio.h>
int main() {
    int a = 5;
    int b = a++ + 10;
    int c = ++a * 2;
    printf("a = %d, b = %d, c = %d", a, b, c);
    return 0;
}
```

Output: a = 7, b = 15, c = 14

Solved Examples — HOTS

A warehouse starts with 20 units in stock. Simulate, in order: 2 sales, 1 return, 3 sales, 1 restock — and predict the final stock.

Chain -- and ++ statements in the exact order the events happened, tracking the running value.

```
#include <stdio.h>
int main() {
    int stock = 20;
    stock--; stock--;
    stock++;
    stock--; stock--; stock--;
    stock++;
    printf("Final Stock = %d", stock);
    return 0;
}
```

Output: Final Stock = 17

Trace: 20-2=18, +1=19, -3=16, +1=17.

Explain why an expression like i = i++ + ++i; should never be written in real code, and show the well-defined way to achieve an equivalent result.

Modifying the same variable more than once between sequence points without a defined order is undefined behaviour in C — different compilers can legally produce different answers. Break the steps apart explicitly instead.

```
#include <stdio.h>
int main() {
    int i = 5;
    int step1 = i++;
    int step2 = ++i;
    i = step1 + step2;
    printf("i = %d", i);
    return 0;
}
```

Output: i = 12

This is the safe, well-defined equivalent: step1=5 (i becomes 6), step2=7 (i becomes 7), then i=5+7=12. Never rely on the order in which ++ and -- apply to the SAME variable within one unsequenced expression — interviewers ask this to test if you recognise it as undefined behaviour, not to get "the" answer.

A vending machine tracks coinsInserted (post-increment) and itemsLeft (pre-decrement) as a customer buys 2 items. Trace and predict all printed values.

Since the two counters are different variables, the expression is well-defined — trace each line carefully in order.

```
#include <stdio.h>
int main() {
    int coinsInserted = 0, itemsLeft = 15;
    printf("%d %d\n", coinsInserted++, --itemsLeft);
    printf("%d %d\n", coinsInserted++, --itemsLeft);
    printf("Final: coins=%d items=%d", coinsInserted, itemsLeft);
    return 0;
}
```

Output: 0 14 / 1 13 / Final: coins=2 items=13

Predict the output and explain the operator behaviour: int a = 5; printf("%d %d %d", a++, ++a, a);

This is **undefined behaviour**. The order in which printf's arguments are evaluated is not specified by C, and a is modified and read in the same statement without any guaranteed order. Different compilers give different answers, so the only correct exam answer is "unpredictable".

```
#include <stdio.h>
int main() {
    int a = 5;
    printf("%d %d %d", a++, ++a, a);    // DO NOT write code like this
    return 0;
}
```

Output:

6 7 7 (this is what GCC happened to print; a compiler that evaluates strictly left to right would print 5 7 7)

The safe way is one change per statement:

```
#include <stdio.h>
int main() {
    int a = 5, first, second;
    first = a++;       // first = 5, a = 6
    second = ++a;      // a = 7, second = 7
    printf("%d %d %d", first, second, a);
    return 0;
}
```

Output: 5 7 7

The old exam question B12 and C10 (x = x++ + ++x) are the same idea: never modify a variable twice, or modify and read it, in one expression. The compiler flag -Wall warns about it.

Practice Problems — Try It Yourself

1. Predict the output of: int p=0, q=8; printf("%d %d", p++, --q); printf("%d %d", ++p, q--);
2. Write a program simulating a lift that goes down from floor 5 to floor 1 using post-decrement, printing each floor reached.
3. In your own words, explain why combining ++ and -- on the SAME variable more than once in a single expression is risky, and rewrite this risky line safely: x = x++ - --x;
4. Run int x = 5; x = x++ + ++x; printf("%d", x); with gcc -Wall on your machine. Note the warning and the result, then explain why examiners call this undefined behaviour.
5. Predict: int a = 2, b = 3; int c = a++ * --b; printf("%d %d %d", a, b, c);

6. A cricket scoreboard starts at 0. Using only ++ and --, model 4 singles and 1 correction (subtract one). Print the score.

Debugging Corner — Find the Bug (Solved)

DEBUG 1

The programmer expects count to become 6.

```
#include <stdio.h>
int main() {
    int count = 5;
    count = count++;
    printf("count = %d", count);
    return 0;
}
```

Output of the buggy program: count = 5

What is wrong?

• **count = count++ modifies count twice** (the ++ and the assignment) in one statement. The post-increment result is overwritten by the old value, so count stays 5 (and the C standard calls it undefined). Just write count++;.

Corrected code:

```
#include <stdio.h>
int main() {
    int count = 5;
    count++;
    printf("count = %d", count);
    return 0;
}
```

Output: count = 6

DEBUG 2

This will not compile. Why?

```
#include <stdio.h>
int main() {
    int a = 3, b = 4;
    (a + b)++;
    printf("%d", a + b);
    return 0;
}
```

Compiler says: lvalue required as increment operand

What is wrong?

• **++ needs a variable to change.** (a + b) is a temporary value with nowhere to store the new value. Store the sum first: int sum = a + b; sum++;

Corrected code:

```
#include <stdio.h>
int main() {
    int a = 3, b = 4;
    int sum = a + b;
    sum++;
    printf("%d", sum);
    return 0;
}
```

Output: 8

Debug It Yourself — Practice (answers in Appendix A)

DEBUG IT 1

The next token should be 101 but 100 is printed.

```
#include <stdio.h>
int main() {
    int token = 100;
    printf("Next token: %d", token++);
    return 0;
}
```

Hint: *Which value does post-increment hand to printf?*

DEBUG IT 2

The programmer expects 10 or 11. Why is the result unreliable?

```
#include <stdio.h>
int main() {
    int n = 5;
    n = n++ * 2;
    printf("%d", n);
    return 0;
}
```

Hint: *How many times is n modified in the statement?*

15.8 Bitwise Operators

> **Analogy:** A bank of light switches: each switch is a bit that is either on or off. Bitwise operators flip and test those switches directly.

> **Technical Concept:** Bitwise operators &, |, ^, ~, << and >> work on the individual bits of integers. They are used for masks, flags, fast multiplication by powers of two and low-level control.

These operators work directly on the individual bits (0s and 1s) that make up a number's binary form. They are widely used in system programming, optimisation, and placement coding rounds.

Operator	Meaning	Syntax	Example (a=5, b=3 → 0101, 0011)
&	Bitwise AND	a & b	0001 → 1
|	Bitwise OR	a | b	0111 → 7
^	Bitwise XOR	a ^ b	0110 → 6
~	Bitwise NOT (complement)	~a	flips every bit of a
<<	Left shift	a << 1	1010 → 10 (doubles the value)
>>	Right shift	a >> 1	0010 → 2 (halves the value)

Benefit: Bitwise operators are extremely fast because they work directly at the hardware level, and they power some of the most popular placement coding questions.

Solved Examples — Easy

Check whether a number is even or odd using a bitwise operator instead of modulus.

A number's very last bit is 1 for odd numbers and 0 for even numbers, so ANDing with 1 isolates just that bit.

```
#include <stdio.h>
int main() {
    int n = 27;
    if (n & 1) printf("%d is Odd", n);
    else printf("%d is Even", n);
    return 0;
}
```

Output: 27 is Odd

Double a number and halve a number using shift operators instead of * and /.

Shifting left by 1 doubles a number; shifting right by 1 halves it (dropping any remainder).

```
#include <stdio.h>
int main() {
    int n = 9;
    printf("Double = %d\n", n << 1);
    printf("Half = %d", n >> 1);
    return 0;
}
```

Output: Double = 18 / Half = 4

Solved Examples — Medium

Swap two integers using bitwise XOR only, with no temporary variable.

XOR-ing a value with itself gives 0, and XOR-ing with 0 returns the original — applying XOR three times between two variables swaps them safely.

```
#include <stdio.h>
int main() {
    int a = 12, b = 25;
    a = a ^ b;
    b = a ^ b;
    a = a ^ b;
    printf("a=%d b=%d", a, b);
    return 0;
}
```

Output: a=25 b=12

A game stores 8 power-up flags as bits in one integer (bit0 and bit2 are currently ON). Toggle (flip) the flag at bit position 2 without touching any other flag.

XOR-ing a number with (1 << position) flips only that one bit, leaving every other bit unchanged.

```
#include <stdio.h>
int main() {
    int flags = 5;   // binary 101: bit0 and bit2 are ON
    flags = flags ^ (1 << 2);
    printf("Flags after toggle = %d", flags);
    return 0;
}
```

Output: Flags after toggle = 1

An access-control system stores permission bits in one integer. Set (turn ON) bit1 to grant "write" access, then clear (turn OFF) bit0 to revoke "read" access.

OR-ing with (1 << position) sets a bit ON; AND-ing with the complement of (1 << position) clears a bit OFF.

```
#include <stdio.h>
int main() {
    int perm = 4;   // binary 100
    perm = perm | (1 << 1);
    perm = perm & ~(1 << 0);
    printf("Permission bits = %d", perm);
    return 0;
}
```

Output: Permission bits = 6

Check whether a number is divisible by 4 using only bitwise operators (hint: look at the last 2 bits). How does the trick generalise?

A number is divisible by 4 exactly when its last two bits are 00. The mask 3 (binary 11) isolates them: n & 3 == 0. In general, a number is divisible by 2^k when its last k bits are all 0, so the mask is 2^k − 1.

Divisor	Mask	Test
2	1	(n & 1) == 0
4	3	(n & 3) == 0
8	7	(n & 7) == 0
16	15	(n & 15) == 0

```
#include <stdio.h>
int main() {
   int n = 28;
   if ((n & 3) == 0) printf("%d is divisible by 4\n", n);
   else              printf("%d is NOT divisible by 4\n", n);
   if ((n & 7) == 0) printf("%d is divisible by 8", n);
   else              printf("%d is NOT divisible by 8", n);
   return 0;
}
```

Output: 28 is divisible by 4 / 28 is NOT divisible by 8

The brackets around n & 3 are essential: == binds tighter than &. See the Debugging Corner below.

Check whether two integers a and b have opposite signs using XOR. Explain why (a ^ b) < 0 works.

The XOR of two numbers has a 1 in the sign bit exactly when the sign bits of the two numbers are different. A number with sign bit 1 is negative, so (a ^ b) < 0 means "opposite signs". (Zero counts as non-negative.)

```
#include <stdio.h>
int main() {
   int a = 12, b = -7;
   printf("a ^ b = %d\n", a ^ b);
   if ((a ^ b) < 0)
      printf("Opposite signs");
   else
      printf("Same sign");
   return 0;
}
```

Output: a ^ b = -11 / Opposite signs

	8-bit pattern	Sign bit
12	00001100	0
-7	11111001	1
12 ^ -7	11110101	1 → negative → opposite

Now try the same-sign check in Practice Problem 1 of this section yourself: it is the mirror image of this example.

Solved Examples — HOTS

A memory allocator only hands out block sizes that are powers of 2. Check whether a given block size is a valid power of 2, without loops or division.

A power of 2 has exactly one bit set (e.g. 8 = 1000). Subtracting 1 flips every bit after that set bit (7 = 0111), so ANDing the two always gives 0 only for powers of 2.

```
#include <stdio.h>
int main() {
    int n = 64;
    if (n > 0 && (n & (n - 1)) == 0) printf("%d is a power of 2", n);
    else printf("%d is NOT a power of 2", n);
    return 0;
}
```

Output: 64 is a power of 2

Count the number of set bits (1s) in a number's binary form — one of the most frequently asked bit-manipulation placement questions.

Brian Kernighan's technique: n & (n-1) always clears exactly the lowest set bit, so counting how many times this can be done before n becomes 0 gives the total count of set bits.

```
#include <stdio.h>
int main() {
    int n = 29;   // binary 11101
    int count = 0;
    while (n) {
        n = n & (n - 1);
        count++;
    }
    printf("Number of set bits = %d", count);
    return 0;
}
```

Output: Number of set bits = 4

Loops are formally covered in the next chapter — this is shown here early because it is such a common placement question, and the while loop is only used to repeat the bit-clearing

step.

Find the value representing only the rightmost (lowest) set bit of a number — a technique used in structures like Binary Indexed Trees in competitive programming.

n & (-n) isolates the lowest set bit because of how negative numbers are stored in two's complement form.

```
#include <stdio.h>
int main() {
    int n = 12;   // binary 1100
    int rightmostSetBit = n & (-n);
    printf("Rightmost set bit value = %d", rightmostSetBit);
    return 0;
}
```

Output: Rightmost set bit value = 4

For x = 5 predict the output of each line before compiling, then repeat for a negative number.

```
#include <stdio.h>
int main() {
    int x = 5;
    printf("%d %d %d %d\n", x & 1, x % 2, x >> 1, x / 2);
    int y = -5;
    printf("%d %d %d %d", y & 1, y % 2, y >> 1, y / 2);
    return 0;
}
```

Output: 1 1 2 2 / 1 -1 -3 -2

Pair	x = 5	x = -5	Same?
x & 1 and x % 2	1 and 1	1 and -1	positive only
x >> 1 and x / 2	2 and 2	-3 and -2	positive only

For negative numbers & 1 still detects odd (-5 ends in bit 1), but % 2 returns -1. Right shift rounds toward minus infinity (-2.5 → -3) while / truncates toward zero (-2). Shifting a negative number right is implementation-defined, though arithmetic on virtually every compiler.

Classify an integer as Positive even, Positive odd, Negative even, Negative odd or Zero using relational, logical and bitwise operators. Restriction: do not use %.

```
#include <stdio.h>
int main() {
   int x = -7;
   if (x == 0)
      printf("Zero");
   else if (x > 0 && (x & 1) == 0)
      printf("Positive even");
   else if (x > 0)
      printf("Positive odd");
   else if ((x & 1) == 0)
      printf("Negative even");
   else
      printf("Negative odd");
   return 0;
}
```

Output: Negative odd

x	0	12	7	-12	-7
Result	Zero	Positive even	Positive odd	Negative even	Negative odd

The last bit decides odd/even in two's complement for positive AND negative numbers, so x & 1 works where x % 2 == 1 fails.

An integer n between 0 and 15 (4 bits) is given. Count how many bits are 1 without loops or arrays, using only if-else and bitwise &.

```
#include <stdio.h>
int main() {
    int n = 11;              // binary 1011
    int count = 0;
    if (n & 1) count++;      // bit 0
    if (n & 2) count++;      // bit 1
    if (n & 4) count++;      // bit 2
    if (n & 8) count++;      // bit 3
    printf("Set bits in %d = %d", n, count);
    return 0;
}
```

Output: Set bits in 11 = 3

The masks 1, 2, 4, 8 are the four powers of 2; each one isolates a single bit. Compare with the second HOTS example above, which works for any number.

Find the absolute value of an integer without abs() and without an if statement, using a bitwise right-shift and XOR trick.

```
#include <stdio.h>
int main() {
    int x = -25;
    int mask = x >> 31;          // all 1s if x is negative, all 0s otherwise
    int absX = (x ^ mask) - mask;    // flip bits and add 1 for negatives
    printf("|%d| = %d", x, absX);
    return 0;
}
```

Output: |-25| = 25

For a negative x the mask is -1: x ^ -1 = ~x = -x - 1, and subtracting the mask (-1) adds 1, giving -x. For positive x the mask is 0 and nothing changes. The ternary version, x < 0 ? -x : x, is in section 15.9. Edge case: INT_MIN has no positive counterpart and stays negative. Assumes a 32-bit int.

Rewrite the vowel test as ONE compound condition and explain the trick of converting a letter to lowercase with ch | 0x20.

In ASCII an uppercase letter and its lowercase partner differ in exactly one bit, the bit worth 0x20 ('A' = 0x41, 'a' = 0x61). OR-ing with 0x20 sets that bit: uppercase becomes lowercase and lowercase stays unchanged.

```
#include <stdio.h>
int main() {
    char ch = 'E';
    char lower = ch | 0x20;              // 'E' (0x45) -> 'e' (0x65)
    if (lower == 'a' || lower == 'e' || lower == 'i' || lower == 'o' || lower == 'u')
        printf("'%c' is a vowel", ch);
    else
        printf("'%c' is a consonant", ch);
    return 0;
}
```

Output: 'E' is a vowel

ch	ch | 0x20	Verdict
'A'	'a'	correct
'a'	'a'	unchanged (fine)
'1'	'1'	unchanged by luck: digits already have that bit set
'@'	'`'	WRONG — symbol silently changed
'['	'{'	WRONG — symbol silently changed

The trick is only meaningful for letters. Symbols between 0x40 and 0x5F are silently turned into different symbols, so validate that the character is a letter first (see the fifth Medium example in section 16.5).

Practice Problems — Try It Yourself

1. Write a program to check whether two integers have the same sign using the XOR operator.
2. Write a program to clear the last 3 bits of a given number using a bitwise AND mask.
3. Write a program to check whether the 4th bit from the right (bit position 3) of a number is set or not.
4. Check divisibility by 8 and by 16 with masks. Test 48, 100 and 256.

5. Write the absolute-value trick for x = 0, x = -1 and x = 100. Then explain, in one sentence, what happens for INT_MIN.
6. Predict and explain: printf("%d %d", -9 >> 1, -9 / 2);

Debugging Corner — Find the Bug (Solved)

DEBUG 1

A student checks whether x is odd with if (x & 1 == 1). It appears to work for every test case. Explain what the condition really evaluates, why it seems right, and rewrite it correctly.

```
#include <stdio.h>
int main() {
    int x = 7;
    if (x & 1 == 1)
        printf("Odd");
    else
        printf("Even");
    return 0;
}
```

Output (looks correct): Odd

What is really evaluated: in C's precedence table == comes BEFORE &, so the condition is x & (1 == 1), i.e. x & 1. Because 1 == 1 happens to be exactly 1, the wrong grouping gives the right answer for every x — there is no x that breaks THIS line. That is why the habit is dangerous: it stops working the moment the constant changes.

```
#include <stdio.h>
int main() {
    int x = 6;                    // binary 110: bit 1 is set, bit 0 is clear
    printf("%d\n", x & 2 == 2);       // x & (2==2) = 6 & 1 = 0   WRONG: bit 1 IS set
    printf("%d\n", (x & 2) == 2);     // correct: 2 == 2  -> 1
    printf("%d\n", x & 1 == 0);       // x & (1==0) = 6 & 0 = 0   WRONG: 6 is even
    printf("%d", (x & 1) == 0);       // correct: 1
    return 0;
}
```

Output (0 = wrong answers hiding): 0 / 1 / 0 / 1

What is wrong?

• **Precedence:** == binds tighter than &, so unparenthesised bit tests test the wrong thing.

• **Fix:** always bracket the bitwise part: (x & 1) == 1 or simply (x & 1). GCC's -Wall even warns "suggest parentheses around comparison in operand of &".

DEBUG 2

The programmer wants the square of 5 but gets 7.

```
#include <stdio.h>
int main() {
    int x = 5;
    int square = x ^ 2;
    printf("Square = %d", square);
    return 0;
}
```

Output of the buggy program: Square = 7

What is wrong?

• **^ is bitwise XOR, not "power".** 5 ^ 2 = 101 ^ 010 = 111 = 7. C has no exponent operator: use x * x (or pow() from math.h, which returns a double).

Corrected code:

```
#include <stdio.h>
int main() {
    int x = 5;
    int square = x * x;
    printf("Square = %d", square);
    return 0;
}
```

Output: Square = 25

Debug It Yourself — Practice (answers in Appendix A)

DEBUG IT 1

Bit 3 of 12 (binary 1100) IS set, but the program says it is clear.

```
#include <stdio.h>
int main() {
    int n = 12;
    if (n & (1 << 3) == 1)
        printf("Bit 3 is set");
    else
        printf("Bit 3 is clear");
    return 0;
}
```

Hint: *Two problems: precedence of ==, and what value a set bit 3 really has.*

DEBUG IT 2

The result should be 21 (5 shifted left by 2, plus 1).

```
#include <stdio.h>
int main() {
    int y = 5 << 2 + 1;
    printf("%d", y);
    return 0;
}
```

Hint: *Which has higher precedence: + or <<?*

15.9 Conditional (Ternary) Operator

> **Analogy:** 'If it is raining take an umbrella, else take sunglasses' is a one-line decision. The ternary operator is the one-line decision of C.

> **Technical Concept:** The conditional operator has the form condition ? value_if_true : value_if_false. It is the only ternary operator in C and can replace a short if-else.

This is the only operator in C that works on three operands, which is why it is called "ternary." It offers a compact shortcut for a simple if-else decision.

Syntax

```
condition ? value_if_true : value_if_false;
```

Benefit: It replaces a small if-else block with a single readable line, especially handy when assigning one of two values to a variable.

Solved Examples — Easy

Find the greater of two numbers using the ternary operator.

A single condition decides which value is picked.

```
#include <stdio.h>
int main() {
   int a=18, b=25;
   int greater = (a > b) ? a : b;
   printf("Greater = %d", greater);
   return 0;
}
```

Output: Greater = 25

Print "Even" or "Odd" for a number using the ternary operator.

The condition is a modulus check; the ternary directly supplies the string to print.

```
#include <stdio.h>
int main() {
   int n = 17;
   printf("%s", (n % 2 == 0) ? "Even" : "Odd");
   return 0;
}
```

Output: Odd

Solved Examples — Medium

Find the largest of three numbers using a nested ternary operator (no if-else at all).

The result of one ternary becomes an operand of another, nested inside it.

```
#include <stdio.h>
int main() {
   int a=45, b=78, c=62;
   int largest = (a > b) ? ((a > c) ? a : c) : ((b > c) ? b : c);
   printf("Largest = %d", largest);
   return 0;
}
```

Output: Largest = 78

Assign a grade (A/B/C/F) based on marks, using nested ternary operators only.

Chain the ternary operators exactly as you would chain else-if conditions.

```
#include <stdio.h>
int main() {
    int marks = 82;
    char grade = (marks >= 90) ? 'A' : (marks >= 75) ? 'B' : (marks >= 50) ? 'C' : 'F';
    printf("Grade = %c", grade);
    return 0;
}
```

Output: Grade = B

Find the absolute value of a number using the ternary operator, without calling abs().

If the number is negative, flip its sign; otherwise leave it as is.

```
#include <stdio.h>
int main() {
    int n = -17;
    int absolute = (n < 0) ? -n : n;
    printf("Absolute value = %d", absolute);
    return 0;
}
```

Output: Absolute value = 17

Solved Examples — HOTS

A courier service charges: up to 1kg → ₹40, 1–5kg → ₹70, above 5kg → ₹120. Compute the shipping cost using ONLY nested ternary operators.

Chain the weight-range checks exactly as a slab-based if-else ladder would, but written entirely as nested ternaries.

```
#include <stdio.h>
int main() {
    float weight = 3.2;
    float cost = (weight <= 1) ? 40 : (weight <= 5) ? 70 : 120;
    printf("Shipping Cost = Rs.%.2f", cost);
    return 0;
}
```

Output: Shipping Cost = Rs.70.00

Predict the output of this code and explain the implicit type conversion that happens.

When a ternary expression's two possible results have different types (here int and float), C converts the whole expression to the common, "wider" type — even though the int branch is chosen.

```
#include <stdio.h>
int main() {
    int a = 5;
    float b = 5.0;
    printf("%f", (1) ? a : b);
    return 0;
}
```

Output: 5.000000

Even though a (an int) is the value actually selected, the ternary expression's overall type becomes float because the other branch is float — so it must be printed with %f, not %d. Always match your format specifier to the expression's resulting type, not just to the branch you expect to be chosen.

Without storing the result in a variable, use the ternary operator directly inside printf() to print "Fizz" if a number is divisible by 3, or the number itself otherwise.

Both branches of the ternary here are used as printf's format-string argument; extra unused arguments passed to printf are simply ignored, which is legal in C.

```
#include <stdio.h>
int main() {
    int n = 9;
    printf(n % 3 == 0 ? "Fizz" : "%d", n);
    return 0;
}
```

Output: Fizz

Since 9 is divisible by 3, the string literal "Fizz" (with no %d in it) is used as the format string, so n is simply passed but never consumed — this is valid C, though such tricks are best understood, not overused, in real code.

Find the maximum of three integers using only the ternary operator (no if). Follow-up: print "Tie" if two or more numbers equal the maximum.

The basic nested-ternary maximum is shown in the first Medium example above. The follow-up counts how many numbers equal the maximum; because a comparison gives 0 or 1, the counts can be added.

```
#include <stdio.h>
int main() {
    int a = 12, b = 30, c = 30;
    int max = (a > b) ? ((a > c) ? a : c) : ((b > c) ? b : c);
    int count = (a == max) + (b == max) + (c == max);
    printf("Max = %d\n", max);
    printf("%s", count > 1 ? "Tie" : "Unique maximum");
    return 0;
}
```

Output: Max = 30 / Tie

Given an integer x, decide whether it is even/odd AND positive/negative/zero without using if, %, < or >. Use only bitwise operators, arithmetic and the ternary operator.

```
#include <stdio.h>
int main() {
    int x = -14;
    unsigned int sign = (unsigned int)x >> 31;      // sign bit: 1 if negative, 0 otherwise
    printf("Parity: %s\n", (x & 1) ? "Odd" : "Even");
    printf("Sign  : %s", x == 0 ? "Zero" : (sign ? "Negative" : "Positive"));
    return 0;
}
```

Output: Parity: Even / Sign : Negative

(x & 1) is the last bit: 1 = odd, 0 = even. Shifting the value right by 31 moves the sign bit to position 0 (an unsigned shift fills with zeros, so the result is exactly 0 or 1). The == comparison is allowed; only < and > are banned. Assumes a 32-bit int.

Practice Problems — Try It Yourself

1. Use the ternary operator to check whether a given character is a vowel or a consonant (chain the checks for all five vowels).
2. Use the ternary operator to compute a simplified 2-slab electricity bill in a single line: ₹0 for units ≤ 100, ₹5/unit beyond that.
3. Predict the output of: int a=5, b=10; printf("%d", a > b ? a : (b > 5 ? b : 0));
4. Use nested ternary operators to print "Positive", "Negative" or "Zero" for an integer (no if).
5. Extend F2 to four numbers a, b, c, d: find the maximum using ternary operators only.
6. Print the parity and the sign of -9, 0 and 16 using the E4 method.

Debugging Corner — Find the Bug (Solved)

DEBUG 1

The result of the ternary is stored in the wrong kind of variable.

```
#include <stdio.h>
int main() {
    int marks = 72;
    int result = (marks >= 40) ? "Pass" : "Fail";
    printf("%s", result);
    return 0;
}
```

Output of the buggy program: a compiler warning, then garbage or a crash

What is wrong?

• **"Pass" and "Fail" are strings (addresses of text),** not integers, so they cannot be stored in an int; and %s applied to an int is undefined. Store text in a char pointer.

Corrected code:

```
#include <stdio.h>
int main() {
    int marks = 72;
    char *result = (marks >= 40) ? "Pass" : "Fail";
    printf("%s", result);
    return 0;
}
```

Output: Pass

DEBUG 2

The programmer assigns inside the two branches of ?: and gets an error.

```
#include <stdio.h>
int main() {
    int a = 8, b = 5, big;
    a > b ? big = a : big = b;
    printf("%d", big);
    return 0;
}
```

Compiler says: lvalue required as left operand of assignment

What is wrong?

• **Precedence:** = is lower than ?:, so the compiler reads it as (a > b ? big = a : big) = b, which tries to assign to a value. Put the ternary on the right of a single assignment.

Corrected code:

```
#include <stdio.h>
int main() {
    int a = 8, b = 5, big;
    big = (a > b) ? a : b;
    printf("%d", big);
    return 0;
}
```

Output: 8

Debug It Yourself — Practice (answers in Appendix A)

DEBUG IT 1

The maximum of 20, 5 and 30 should be 30. Why is 20 printed?

```
#include <stdio.h>
int main() {
    int a = 20, b = 5, c = 30;
    int max = a > b ? a : b > c ? b : c;
    printf("Max = %d", max);
    return 0;
}
```

Hint: *Follow the true branch of the first condition: is a ever compared with c?*

DEBUG IT 2

The text should be printed, but a number appears.

```
#include <stdio.h>
int main() {
    int x = 8;
    printf("%d", x > 5 ? "big" : "small");
    return 0;
}
```

Hint: *What does %d do with an address?*

15.10 Special Operators

> **Analogy:** A Swiss-army knife has odd tools that fit no other group. C's special operators are the odd tools: sizeof, comma, address-of and dereference.

> **Technical Concept:** Special operators include sizeof (size in bytes), the comma operator (evaluates left to right), & (address of), * (value at address), type casts and the member operators . and ->.

Operator	Name	Purpose	Example
sizeof()	Size-of operator	Returns the memory size (in bytes) of a data type or variable	sizeof(int) → 4
,	Comma operator	Lets multiple expressions be evaluated where only one is expected	for(i=0, j=5; ...)
&	Address-of operator	Returns the memory address of a variable	&a
*	Dereference operator	Accesses the value stored at a given memory address (used with pointers)	*ptr

Solved Examples — Easy

Print the memory size of every primary data type using sizeof().

sizeof() is applied directly to each type name.

```
#include <stdio.h>
int main() {
   printf("int=%lu float=%lu char=%lu double=%lu",
      sizeof(int), sizeof(float), sizeof(char), sizeof(double));
   return 0;
}
```

Output: int=4 float=4 char=1 double=8

Print both the value stored in a variable and its memory address.

The & operator, placed before the variable, returns its address.

```
#include <stdio.h>
int main() {
   int marks = 90;
   printf("Value = %d, Address = %p", marks, &marks);
   return 0;
```

```
}
```

Output: Value = 90, Address = 0x7ffee2b1a05c (address varies each run)

Solved Examples — Medium

Understand how the comma operator evaluates multiple expressions but keeps only the last one's value.

All expressions inside the parentheses are evaluated left to right, but only the final one becomes the result.

```
#include <stdio.h>
int main() {
    int x = (5, 10, 15);
    printf("x = %d", x);
    return 0;
}
```

Output: x = 15

Use a pointer to read a variable's value, and then modify that variable directly through the pointer.

Store the variable's address in a pointer using &, then use * on the pointer to read or change the value it points to.

```
#include <stdio.h>
int main() {
    int score = 70;
    int *ptr = &score;
    printf("Before: %d\n", *ptr);
    *ptr = 95;
    printf("After: %d", score);
    return 0;
}
```

Output: Before: 70 / After: 95

Find the number of elements in an integer array without hardcoding the count.

Dividing the total size of the array by the size of one element gives the element count — a very common placement technique.

```
#include <stdio.h>
int main() {
```

```
    int marks[] = {56, 78, 90, 45, 67};
    int count = sizeof(marks) / sizeof(marks[0]);
    printf("Number of elements = %d", count);
    return 0;
}
```

Output: Number of elements = 5

Does sizeof evaluate the expression inside its brackets?

```
#include <stdio.h>
int main() {
    int x = 5;
    int size = sizeof(x++);      // x++ is NOT executed
    printf("size = %d, x = %d", size, x);
    return 0;
}
```

Output: size = 4, x = 5

sizeof is decided at compile time from the TYPE of its operand, so the expression inside is never run. Never put real work inside sizeof.

Solved Examples — HOTS

Write a function that swaps two integers using pointers, demonstrating how C achieves "pass by reference" using & and * together.

Passing addresses (&a, &b) into a function lets that function modify the original variables directly through * — this is a very common placement/viva question.

```
#include <stdio.h>
void swap(int *p, int *q) {
    int temp = *p;
    *p = *q;
    *q = temp;
}
int main() {
    int a = 15, b = 40;
    swap(&a, &b);
    printf("a=%d b=%d", a, b);
    return 0;
}
```

Output: a=40 b=15

Predict the output of x = (2, 4, 6); printf("x = %d", x); and explain why.

The comma operator discards every value except the last one in the sequence, even inside an assignment.

```
#include <stdio.h>
int main() {
    int x;
    x = (2, 4, 6);
    printf("x = %d", x);
    return 0;
}
```

Output: x = 6

The expressions 2 and 4 are evaluated (here, simply computed and thrown away) and only the final value, 6, is actually assigned to x.

Explain what happens when you add 1 to an int pointer, and demonstrate the relationship with sizeof().

Pointer arithmetic does not add "1 byte" — it adds "1 element's worth of bytes", so ptr+1 jumps forward by sizeof(int) bytes, not by 1 byte. This is a very common placement/viva conceptual question.

```
#include <stdio.h>
int main() {
    int n = 100;
    int *ptr = &n;
    printf("Difference in bytes = %lu (equals sizeof(int))",
        (unsigned long)(ptr + 1) - (unsigned long)ptr);
    return 0;
}
```

Output: Difference in bytes = 4 (equals sizeof(int))

What goes wrong if the XOR (or arithmetic) swap is used when both pointers hold the SAME address, i.e. a = a ^ a? Trace it and explain why this is a real bug.

```
#include <stdio.h>
void swapXor(int *p, int *q) {
    *p = *p ^ *q;
    *q = *p ^ *q;
    *p = *p ^ *q;
}
void swapSafe(int *p, int *q) {
    if (p == q) return;         // guard: same variable, nothing to swap
    *p = *p ^ *q;
    *q = *p ^ *q;
    *p = *p ^ *q;
}
int main() {
    int a = 7, b = 3, s1 = 7, s2 = 7;
    swapXor(&a, &b);
    printf("Different variables: a = %d, b = %d\n", a, b);
    swapXor(&s1, &s1);
    printf("Same variable, unsafe: %d\n", s1);
    swapSafe(&s2, &s2);
    printf("Same variable, safe  : %d", s2);
    return 0;
}
```

Output: Different variables: a = 3, b = 7 / Same variable, unsafe: 0 / Same variable, safe : 7

*Trace: with p and q pointing to the same cell, *p ^ *q is 7 ^ 7 = 0. That 0 is stored into the one shared cell, and every later step just XORs 0 with 0. The value is destroyed. The + and - version fails the same way (7 + 7, then 14 - 14 = 0). Real code such as sorting calls swap(&a[i], &a[j]) with i == j and silently zeroes an element — which is why production swaps check p != q or simply use a temporary variable.*

Practice Problems — Try It Yourself

1. Write a program to print the size of a double variable, and note (as a comment) how many decimal digits of precision that generally represents.
2. Write a program that uses the comma operator to increment two separate counters within a single expression, then prints both.
3. Write a program that declares an integer variable, prints its value and address using &, then declares a pointer to it and prints the value read back through the pointer using *.

4. Write swap(int *p, int *q) with the p == q guard and test it with two different variables and with the same variable twice.
5. Predict the output: int x = 10; printf("%lu %d", sizeof(x + 1.5), x);

Debugging Corner — Find the Bug (Solved)

DEBUG 1

This program crashes. Where does the pointer point?

```
#include <stdio.h>
int main() {
    int *p;
    *p = 10;
    printf("%d", *p);
    return 0;
}
```

Output of the buggy program: crash (segmentation fault) or garbage

What is wrong?

• **p was never given an address.** It holds a random value, and *p = 10 writes 10 into unknown memory. A pointer must point to a real variable before it is dereferenced.

Corrected code:

```
#include <stdio.h>
int main() {
    int x;
    int *p = &x;
    *p = 10;
    printf("%d", *p);
    return 0;
}
```

Output: 10

DEBUG 2

The value 10 is stored in a, yet reading through the pointer fails.

```
#include <stdio.h>
int main() {
    int a = 10;
    int *p;
    p = a;
    printf("%d", *p);
    return 0;
}
```

Output of the buggy program: warning at compile time, crash at run time

What is wrong?

• **p = a stores the VALUE 10 as an address.** A pointer must receive an address: p = &a;.

Corrected code:

```
#include <stdio.h>
int main() {
    int a = 10;
    int *p;
    p = &a;
    printf("%d", *p);
    return 0;
}
```

Output: 10

Debug It Yourself — Practice (answers in Appendix A)

DEBUG IT 1

The value of a should become 6, but it stays 5.

```
#include <stdio.h>
int main() {
    int a = 5;
    int *p = &a;
    *p++;
    printf("%d", a);
    return 0;
}
```

Hint: *Does ++ act on the pointer or on the thing it points to?*

DEBUG IT 2

An array of 3 elements should report a count of 3, but 12 is printed.

```
#include <stdio.h>
int main() {
    int arr[3] = {10, 20, 30};
    int n = sizeof(arr);
    printf("Elements = %d", n);
    return 0;
}
```

Hint: *sizeof measures BYTES. How many bytes does one int take?*

15.11 Operator Precedence and Associativity

> **Analogy:** In 2 + 3 x 4 you multiply first, just as school arithmetic taught you. C has a strict order for every operator.

> **Technical Concept:** Precedence decides which operator is applied first; associativity decides the direction (left-to-right or right-to-left) when operators of equal precedence meet. Parentheses override both.

When an expression contains more than one operator, C follows a fixed order — called precedence — to decide which operator is applied first, just like the BODMAS rule in mathematics. When two operators share the same precedence, associativity decides the direction (left-to-right or right-to-left) in which they are evaluated.

Precedence (High → Low)	Operators	Associativity
1	() [] (function call, array)	Left to Right
2	++ -- (unary) ! ~ sizeof	Right to Left
3	* / %	Left to Right
4	+ -	Left to Right
5	<<= >>=	Left to Right
6	== !=	Left to Right
7	&&	Left to Right

Precedence (High → Low)	Operators	Associativity
8	||	Left to Right
9	?: (ternary)	Right to Left
10	= += -= *= /=	Right to Left

*Placement Tip: A very common interview/test question is to evaluate an expression like x = 2 + 3 * 4 % 5 and ask for the output. Solve such expressions strictly using the precedence table above, one step at a time.*

Solved Examples — Precedence and Associativity

Explain operator precedence with an example involving +, * and (). Evaluate: int result = 10 + 2 * 3 - (4 / 2);

Step	Work done	Reason
1	(4 / 2) = 2	brackets first
2	2 * 3 = 6	* before + and -
3	10 + 6 = 16	+ and - left to right
4	16 - 2 = 14	final result

```
#include <stdio.h>
int main() {
    int result = 10 + 2 * 3 - (4 / 2);
    printf("result = %d", result);
    return 0;
}
```

Output: result = 14

When two operators have the same precedence, associativity decides. Predict: 100 / 10 * 2, 20 - 5 - 3, 2 * 3 % 4

```
#include <stdio.h>
int main() {
    printf("%d %d %d", 100 / 10 * 2, 20 - 5 - 3, 2 * 3 % 4);
    return 0;
}
```

Output: 20 12 2

*Left to right: (100/10)*2 = 20, not 100/(10*2) = 5. (20-5)-3 = 12, not 20-(5-3) = 18. (2*3)%4 = 2. Assignment and ?: are the main right-to-left operators.*

Explain the difference between x*x > 100 and x*(x > 100). For x = 12 evaluate both step by step and say why the second is almost never intended.

Expression	Step by step	Value
x*x > 100	* first: 144, then 144 > 100	1 (true)
x*(x > 100)	brackets: (12 > 100) = 0, then 12 * 0	0

```
#include <stdio.h>
int main() {
    int x = 12;
    printf("%d %d", x*x > 100, x*(x > 100));
    return 0;
}
```

Output: 1 0

A comparison yields 0 or 1, so multiplying by it acts like an on/off switch on x. That is occasionally a deliberate trick, but a programmer who wanted "is x squared over 100?" would get the number 0 or x instead of true/false.

Practice Problems — Try It Yourself

1. Fully parenthesise, then evaluate: a + b * c - d / e % f with a = 10, b = 4, c = 3, d = 20, e = 6, f = 2.
2. Write ONE expression that is true when x is between 10 and 20 (inclusive) and odd, using only relational, logical and bitwise operators (no %).
3. Predict: int a = 5, b = 3; printf("%d", a > b == b < a);

Debugging Corner — Find the Bug (Solved)

DEBUG 1

37 degrees Celsius should be 98.6 Fahrenheit, but 69 is printed.

```
#include <stdio.h>
int main() {
    float c = 37.0;
    float f = 9 / 5 * c + 32;
    printf("F = %.1f", f);
    return 0;
}
```

Output of the buggy program: F = 69.0

What is wrong?

• **9 / 5 is an integer division = 1,** and it is evaluated first (left to right). Make it real: 9.0 / 5 * c + 32, or multiply before dividing: c * 9 / 5 + 32.

Corrected code:

```
#include <stdio.h>
int main() {
    float c = 37.0;
    float f = c * 9 / 5 + 32;
    printf("F = %.1f", f);
    return 0;
}
```

Output: F = 98.6

DEBUG 2

Someone expects 1 from 8 / 2 * (2 + 2) but 16 appears.

```
#include <stdio.h>
int main() {
    int r = 8 / 2 * (2 + 2);
    printf("%d", r);
    return 0;
}
```

Output of the buggy program: 16

What is wrong?

• **Same precedence, left to right:** (8 / 2) * 4 = 16. The multiplication after the division is NOT grouped with the bracket. If 1 was intended, write 8 / (2 * (2 + 2)).

Corrected code:

```
#include <stdio.h>
int main() {
    int r = 8 / (2 * (2 + 2));
    printf("%d", r);
    return 0;
}
```

Output: 1

Debug It Yourself — Practice (answers in Appendix A)

DEBUG IT 1

The area of a ring (outer radius R, inner radius r) is wrong.

```
#include <stdio.h>
int main() {
    float R = 5, r = 3;
    float area = 3.14 * R * R - r * r;
    printf("Area = %.2f", area);
    return 0;
}
```

Hint: *Which part is multiplied by 3.14?*

DEBUG IT 2

Half of n should be 4.0 for n = 8, but 0 is printed.

```
#include <stdio.h>
int main() {
    float n = 8;
    float half = 1 / 2 * n;
    printf("%.1f", half);
    return 0;
}
```

Hint: *Evaluate 1 / 2 first.*

15.12 Chapter Summary — Placement Tips

- Interviewers rarely ask you to define an operator; they ask you to trace or predict the output of an expression — practice reading code step by step, not just writing it.
- Bitwise tricks (power of 2, XOR swap, counting set bits) appear extremely often in coding-round questions — revisit section 15.8 until they feel automatic.
- Whenever ++ or -- is applied to the same variable more than once in a single unsequenced expression, recognise it as undefined behaviour rather than trying to compute "the" answer.
- Slab-based problems (billing, tax, shipping, commission) are a favourite scenario for combining relational, logical, arithmetic, and ternary operators together — practice tracing through each slab carefully.

15.13 Important Questions

Q1. Differentiate between = and ==.

= is the assignment operator: it stores the right-hand value in the left-hand variable. == is the relational equality operator: it compares two values and gives 1 or 0. Writing if (x = 5) assigns 5 and is always true, a classic bug.

Q2. Explain pre-increment and post-increment with an example.

++x increments first and then uses the value; x++ uses the value first and then increments. With a = 5: b = a++ gives b = 5 and a = 6; then c = ++a gives a = 7 and c = 7.

Q3. Why does 7 / 2 give 3 in C?

Both operands are int, so C performs integer division and drops the fraction. Use 7 / 2.0 or (float)7 / 2 to get 3.5. The % operator gives the remainder (1) and works only on integers.

Q4. What is short-circuit evaluation?

In a && b the operand b is skipped if a is false; in a || b it is skipped if a is true. This makes expressions such as (x != 0 && 10 / x > 1) safe from division by zero.

Q5. Give three uses of bitwise operators.

n & 1 tests odd or even; n << 1 doubles a number and n >> 1 halves it; (n & (n - 1)) == 0 tests for a power of two; a ^ b ^ b gives a back, which is the idea behind XOR swap.

15.14 Multiple Choice Questions

These 20 questions are written in the style of real placement aptitude/coding rounds — mostly "predict the output" and concept-trap questions, not simple definitions. Attempt each one before checking the answer.

1. What is the output of: int a=7, b=2; printf("%d", a/b);

- a) 3.5
- b) 3
- c) 4
- d) Error

Answer: b) 3 (integer division always truncates the decimal part)

2. What is the result of 17 % 5?

- a) 2
- b) 3
- c) 3.4
- d) 0

Answer: a) 2

3. Which operator performs logical OR on two whole conditions (not individual bits)?

- a) &
- b) ||
- c) |
- d) &&

Answer: b) ||

4. What does the expression (5 & 1) evaluate to?

- a) 0
- b) 1
- c) 5
- d) Error

Answer: b) 1 (0101 & 0001 = 0001)

5. Which expression correctly doubles the value of n using a bitwise operator?

- a) n >> 1
- b) n << 1
- c) n ^ 1
- d) n % 1

Answer: b) n << 1

6. What is the value of x after: int x=10; x+=5; x-=3;

- a) 10
- b) 12
- c) 15
- d) 7

Answer: b) 12 (10+5=15, then 15-3=12)

7. Which of these operators has the HIGHEST precedence?

- a) +
- b) *
- c) ++ (unary)
- d) &&

Answer: c) ++ (unary)

8. Which of these operators works on only ONE operand?

- a) +
- b) %
- c) ++
- d) &&

Answer: c) ++

9. What is the result of the bitwise operation 8 ^ 3?

- a) 8
- b) 3
- c) 11
- d) 5

Answer: c) 11 (1000 XOR 0011 = 1011)

10. Which expression correctly checks if a number n is odd, using a bitwise operator?

- a) n % 2 == 1
- b) n & 1
- c) n | 1

- d) n ^ 1

Answer: b) n & 1

11. What is the value of x after: int x = (3, 7, 9);

- a) 3
- b) 7
- c) 9
- d) 19

Answer: c) 9 (the comma operator keeps only the last value)

12. Which of the following is NOT a valid compound assignment operator in C?

- a) +=
- b) -=
- c) =+
- d) %=

Answer: c) =+ (this is parsed as "= +", a classic beginner typo, not a real operator)

13. What is sizeof(char) on a standard compiler?

- a) 1
- b) 2
- c) 4
- d) 8

Answer: a) 1

14. Evaluate: 2 + 3 * 4 % 5

- a) 4
- b) 9
- c) 5
- d) 3

Answer: a) 4 (3*4=12, 12%5=2, then 2+2=4)

15. What does the ! operator do in C?

- a) Bitwise complement
- b) Logical NOT

- c) Decrement
- d) Address-of

Answer: b) Logical NOT

16. Which associativity does the assignment operator (=) follow?

- a) Left to Right
- b) Right to Left
- c) No associativity
- d) Depends on the compiler

Answer: b) Right to Left

17. What can correctly be said about the expression i = i++ + ++i; ?

- a) It always evaluates to 2*i
- b) Its result is undefined behaviour in C
- c) It causes a compile error
- d) It always evaluates to i+1

Answer: b) Its result is undefined behaviour in C

18. Which of these correctly finds the maximum of a and b using the ternary operator?

- a) max = a > b ? a : b;
- b) max = a > b : a ? b;
- c) max = (a, b) ? max : min;
- d) max = if(a>b) a; else b;

Answer: a) max = a > b ? a : b;

19. What is the output of: int a = 10; a %= 3; printf("%d", a);

- a) 3
- b) 1
- c) 0
- d) 10

Answer: b) 1 (10 % 3 = 1)

20. Which of these statements about operator precedence and associativity is TRUE?

- a) All operators are evaluated strictly left to right

- b) Parentheses () always have the lowest precedence
- c) When precedence is equal, associativity decides the evaluation direction
- d) Associativity never matters in C

Answer: c) When precedence is equal, associativity decides the evaluation direction

15.15 Solved Programs

Program 1: Total, Average and Remainder

Problem: Find the total, the exact average and the integer division of three marks.

Approach: Divide by 3.0 to keep the decimal part; use / and % on integers to see truncation and remainder.

```
#include <stdio.h>
int main() {
    int m1 = 78, m2 = 85, m3 = 91;
    int total = m1 + m2 + m3;
    float average = total / 3.0;
    printf("Total = %d\n", total);
    printf("Average = %.2f\n", average);
    printf("Integer division = %d\n", total / 3);
    printf("Remainder = %d\n", total % 3);
    return 0;
}
```

Output:

```
Total = 254
Average = 84.67
Integer division = 84
Remainder = 2
```

Program 2: Bitwise Tricks

Problem: Test odd/even and power of two, then multiply and divide by shifting.

Approach: A number is odd if its lowest bit is 1; a power of two has exactly one bit set.

```
#include <stdio.h>
int main() {
    int n = 64;
```

```
    printf("n is %s\n", (n & 1) ? "odd" : "even");
    printf("n is %sa power of two\n", ((n & (n - 1)) == 0) ? "" : "not ");
    printf("n * 8 = %d\n", n << 3);
    printf("n / 4 = %d\n", n >> 2);
    return 0;
}
```

Output:

```
n is even
n is a power of two
n * 8 = 512
n / 4 = 16
```

15.16 Practice Problems — Try It Yourself

More problems, at Medium and HOTS level, mixed across all the operator types covered in this chapter — solve these without looking at the earlier solved examples, then compare your reasoning with the approach used there.

Medium Level

1. A cab company charges a base fare of ₹50 plus ₹12/km for the first 10 km, and ₹15/km beyond that. Write a program to calculate the total fare for a given distance using arithmetic operators.
2. Write a program to check whether a given number lies within the valid range of a 16-bit signed integer (-32768 to 32767).
3. A gym membership discount applies if a person is a student OR is above 60 years of age. Write a program to check eligibility using logical operators.
4. A parking lot display starts with 25 available slots. Simulate 3 cars entering (decrement) and 1 car leaving (increment), and predict the final slot count.
5. Write a program to calculate the number of pages needed to print a document, given the total number of lines and how many lines fit per page — without using the ceil() function (hint: adjust the arithmetic so incomplete pages are still counted).
6. An online COD (Cash on Delivery) option is available only if the order amount is below ₹5000 AND the delivery pincode is serviceable (given as a 0/1 flag). Write the validating condition using logical and relational operators.

HOTS Level

1. Write a program to swap the upper nibble (top 4 bits) and lower nibble (bottom 4 bits) of an 8-bit number using bitwise operators.
2. A robot's battery percentage decreases by 15% every hour of continuous operation (compounding each hour). Using *=, calculate the remaining battery percentage after 4 hours, starting from 100%.
3. Rewrite the following grading logic entirely using nested ternary operators (no if-else at all): 90+ → 'A', 75–89 → 'B', 60–74 → 'C', below 60 → 'F'.
4. Write a program using pointers where a single function calculates BOTH the sum and the product of two numbers and returns both results to main() through pointer parameters (since a function can only directly return one value).
5. Write a program to check whether the 8-bit binary representation of a number is a palindrome (reads the same forwards and backwards).
6. Predict the output of: int result = 5 + 3 * 2 - 8 / 4 % 3; — then explain your answer step by step using the operator precedence table from section 15.11.

Chapter 16
Decision Making in C

16.1 What is Decision Making? Why Do We Need It?

> **Analogy:** At a crossroads you choose a road depending on where you want to go. A program too must choose between paths depending on a condition.

> **Technical Concept:** Decision making lets a program execute different statements depending on whether a condition is true (non-zero) or false (zero).

So far, every program we've written has executed line by line, from top to bottom, in exactly the same order every single time. Decision making (also called conditional control flow) is what lets a program break that pattern — choosing one path over another depending on whether a condition is true or false.

We need decision making because real programs must react differently to different situations. A login system must decide whether to grant or deny access. A banking app must decide whether to approve or reject a withdrawal. A game must decide whether the player has won or lost. Without decision making, a program would behave identically no matter what input it receives — which would make it almost useless for solving real problems.

- Decision-making statements rely entirely on the relational and logical operators covered in Chapter 15 — a condition must evaluate to true (non-zero) or false (0) before a decision can be made.
- This is also the single most heavily tested topic in placement coding rounds and vivas, since almost every real problem needs some form of conditional logic.

16.2 Types of Decision-Making Statements in C

> **Analogy:** There are several kinds of signboards: a single arrow, a fork, a multi-way junction. C offers matching decision statements.

> **Technical Concept:** C provides if, if-else, nested if, the if-else-if ladder and switch. Choose by the number of alternatives and the type of condition.

- if statement — executes a block of code only when a condition is true

- if-else statement — executes one of two blocks, depending on whether a condition is true or false
- Nested if statement — an if (or if-else) placed inside another if, for multi-level decisions
- if-else-if ladder — a chain of conditions checked in order; the first one found true is executed and the rest are skipped
- switch statement — matches one variable against several exact constant values; a clean, efficient alternative to a long if-else-if ladder

Each construct below is explained with its syntax, followed by its own complete set of solved examples (Easy → Medium → HOTS) and practice problems.

16.3 The if Statement

> **Analogy:** If the gate is open, walk in. If not, nothing happens. The simple if statement acts only when its condition is true.

> **Technical Concept:** if (condition) statement; The statement runs only when the condition is non-zero; otherwise it is skipped. Use braces for more than one statement.

The simplest decision-making statement. The block of code inside it runs only if the condition is true; if the condition is false, the block is skipped entirely and nothing inside it happens — there is no alternative path.

Syntax

```
if (condition) {
    // statements — run ONLY if condition is true
}
```

Important: A plain if has no "else" — if the condition is false, execution simply moves to whatever comes after the if block. Multiple independent if statements in a row are each checked separately, so more than one of them can run for the same input.

Solved Examples — Easy

Check whether a number is positive, and print a message only if it is.

A single relational condition inside if.

```
#include <stdio.h>
int main() {
    int n = 15;
```

```
    if (n > 0)
        printf("%d is a positive number", n);
    return 0;
}
```

Output: 15 is a positive number

Check whether a person is eligible to vote (age 18 or above).

A single if with no alternative — nothing prints if the condition fails.

```
#include <stdio.h>
int main() {
    int age = 20;
    if (age >= 18)
        printf("Eligible to vote");
    return 0;
}
```

Output: Eligible to vote

Write a program using only if statements to check whether a number is even or odd.

With a plain if there is no "otherwise", so two independent ifs are used: one for even and one for odd.

```
#include <stdio.h>
int main() {
    int n = 14;
    if (n % 2 == 0)
        printf("%d is Even", n);
    if (n % 2 != 0)
        printf("%d is Odd", n);
    return 0;
}
```

Output: 14 is Even

Note != 0 rather than == 1: for negative numbers n % 2 is -1. The if-else version in section 16.4 needs only one test.

Solved Examples — Medium

An online store gives free shipping if the cart total is ₹500 or more. Print the free-shipping message only when it applies.

A single if guards the message — for a cart total below ₹500, the program would print nothing at all.

```
#include <stdio.h>
int main() {
    float cartTotal = 650.0;
    if (cartTotal >= 500)
        printf("You qualify for FREE shipping!");
    return 0;
}
```

Output: You qualify for FREE shipping!

Check whether a number is divisible by 2, by 3, and by 5 — using three separate, independent if statements (not else-if) — so more than one message can print for the same number.

Because each if is independent, all three checks run regardless of what the earlier ones found.

```
#include <stdio.h>
int main() {
    int n = 30;
    if (n % 2 == 0) printf("Divisible by 2\n");
    if (n % 3 == 0) printf("Divisible by 3\n");
    if (n % 5 == 0) printf("Divisible by 5\n");
    return 0;
}
```

Output: Divisible by 2 / Divisible by 3 / Divisible by 5 (all three print)

An ATM only accepts withdrawal amounts that are multiples of 100. Print a warning only if the entered amount is invalid.

A single guard-style if — commonly used for input validation.

```
#include <stdio.h>
int main() {
    int amount = 450;
    if (amount % 100 != 0)
        printf("Invalid amount! Please enter a multiple of 100.");
    return 0;
}
```

Output: Invalid amount! Please enter a multiple of 100.

Solved Examples — HOTS

Using only independent if statements (no else-if), print "Fizz" if a number is divisible by 3 and "Buzz" if divisible by 5. Explain why a number divisible by both prints "FizzBuzz".

Because each if is checked independently, BOTH can fire for the same number, and their outputs simply appear one after another.

```
#include <stdio.h>
int main() {
    int n = 15;
    if (n % 3 == 0) printf("Fizz");
    if (n % 5 == 0) printf("Buzz");
    return 0;
}
```

Output: FizzBuzz

This is exactly why the classic FizzBuzz problem is solved with independent if statements rather than an else-if ladder — an else-if ladder would only ever let ONE branch run, making "FizzBuzz" impossible to print this way.

A home security system checks three independent sensors and must trigger a SEPARATE alert for each one that is currently active: door, motion, and window sensors.

Three independent if statements, each guarding its own alert message.

```
#include <stdio.h>
int main() {
    int doorOpen=1, motionDetected=1, windowOpen=0;
    if (doorOpen) printf("ALERT: Door opened!\n");
    if (motionDetected) printf("ALERT: Motion detected!\n");
    if (windowOpen) printf("ALERT: Window opened!\n");
    return 0;
}
```

Output: ALERT: Door opened! / ALERT: Motion detected!

Predict the final value of a variable after it passes through a sequence of independent if statements, each of which may update it and each checking the CURRENT value.

Trace carefully — since these are independent ifs (not else-if), each one reads whatever value the variable holds at that exact point, including changes made by earlier ifs.

```
#include <stdio.h>
int main() {
   int marks = 42;
   if (marks >= 35) marks += 5;
   if (marks >= 50) marks += 3;
   if (marks > 100) marks = 100;
   printf("Final Marks = %d", marks);
   return 0;
}
```

Output: Final Marks = 47

Trace: 42>=35 is true, so marks becomes 47. But the SECOND if then checks 47>=50, which is false, so no further bonus is added. This is a common placement trap: independent ifs always check the latest, possibly already-updated value — order matters.

Identify the logical flaw (not a syntax error) and rewrite the code correctly: int marks = 85; if (marks > 90) "Grade A"; if (marks > 80) "Grade B"; if (marks > 70) "Grade C";

Each if is **independent**, so every condition is tested. 85 is above 80 AND above 70, so two grades print. Grades are meant to be mutually exclusive, so the tests must be linked with else-if.

```
#include <stdio.h>
int main() {
   int marks = 85;
   printf("Buggy : ");
   if (marks > 90) printf("Grade A ");
   if (marks > 80) printf("Grade B ");
   if (marks > 70) printf("Grade C ");

   printf("\nFixed : ");
   if (marks > 90)      printf("Grade A");
   else if (marks > 80) printf("Grade B");
   else if (marks > 70) printf("Grade C");
   return 0;
}
```

Output: Buggy : Grade B Grade C / Fixed : Grade B

An else-if chain stops at the FIRST true condition. Real rubrics use >= (a student with exactly 90 gets A); the > here follows the question.

Practice Problems — Try It Yourself

1. Write a program using simple if to print "Warning: Low Battery" if battery percentage is below 20.
2. Write a program using three independent if statements to check whether a number is divisible by 2, by 7, and by 11 separately.
3. Write a program to check if a temperature reading exceeds a safety threshold of 100°C and print an alert (no else needed).
4. Write a program using only if statements to print "Even" when a number is even and "Positive" when it is greater than zero. Both may print.
5. Extend the C5 grade program with Grade D for marks above 60 and print exactly one grade. Test with 95, 85, 75, 65 and 55.
6. Predict the output: int x = 5; if (x > 3) printf("A"); if (x > 4) printf("B"); if (x > 5) printf("C"); if (x > 6) printf("D");

Debugging Corner — Find the Bug (Solved)

DEBUG 1

The programme should print a grade. Look carefully at the semicolon right after the if condition.

```
#include <stdio.h>
int main() {
    int marks;
    scanf("%d", &marks);
    if (marks >= 90);
        printf("Grade A");
    else if (marks >= 75)
        printf("Grade B");
    else
        printf("Grade C");
    return 0;
}
```

Compiler says: 'else' without a previous 'if'

What is wrong?

• **Stray semicolon.** The ; after the condition is an empty statement, so it ends the if. printf("Grade A") is now a separate statement, and the following else has no if to belong to — "else without a previous if".

Corrected code:

```
#include <stdio.h>
int main() {
    int marks;
    scanf("%d", &marks);
    if (marks >= 90)
        printf("Grade A");
    else if (marks >= 75)
        printf("Grade B");
    else
        printf("Grade C");
    return 0;
}
```

Sample Run:

```
82
Grade B
```

DEBUG 2

The balance is fine, yet the recharge message is printed.

```
#include <stdio.h>
int main() {
    int balance = 800;
    if (balance < 500)
        printf("Low balance! ");
        printf("Please recharge.");
    return 0;
}
```

Output of the buggy program: Please recharge.

What is wrong?

• **Without braces an if controls only ONE statement.** The indentation is misleading: the second printf is outside the if and always runs. Use { } whenever the if guards more than one statement.

Corrected code:

```
#include <stdio.h>
int main() {
    int balance = 800;
    if (balance < 500) {
        printf("Low balance! ");
        printf("Please recharge.");
    }
    return 0;
}
```

Output: (nothing is printed — the balance is fine)

Debug It Yourself — Practice (answers in Appendix A)

DEBUG IT 1

The alert appears even though the temperature is safe.

```
#include <stdio.h>
int main() {
    int temp = 30;
    if (temp > 40);
        printf("Heat alert!");
    return 0;
}
```

Hint: *Look at the end of the if line.*

DEBUG IT 2

The compiler complains about the if line.

```
#include <stdio.h>
int main() {
    int x = 8;
    if x > 5
        printf("big");
    return 0;
}
```

Hint: *What must surround every condition?*

16.4 The if-else Statement

> **Analogy:** If it rains, take an umbrella; otherwise, wear a cap. if-else guarantees that exactly one of two paths runs.

> **Technical Concept:** if (condition) { block A } else { block B } Exactly one of the two blocks executes, depending on the condition.

Extends the plain if by adding an alternative path — exactly one of the two blocks always runs, depending on whether the condition is true or false.

Syntax

```
if (condition) {
    // runs if condition is TRUE
} else {
    // runs if condition is FALSE
}
```

Solved Examples — Easy

Check whether a number is even or odd.

A modulus check decides which of the two branches runs.

```
#include <stdio.h>
int main() {
    int n = 14;
    if (n % 2 == 0) printf("Even");
```

```
    else printf("Odd");
    return 0;
}
```

Output: Even

Check whether a student has passed or failed, given a pass mark of 40.

A single relational check decides pass or fail.

```
#include <stdio.h>
int main() {
    int marks = 38;
    if (marks >= 40) printf("Pass");
    else printf("Fail");
    return 0;
}
```

Output: Fail

Check whether a year is divisible by 4 (a simplified, incomplete leap-year test).

```
#include <stdio.h>
int main() {
    int year = 2024;
    if (year % 4 == 0)
        printf("%d is a leap year", year);
    else
        printf("%d is not a leap year", year);
    return 0;
}
```

Output: 2024 is a leap year

Year	Divisible by 4?	This rule says	Truth
2024	yes	leap	leap ✔
2023	no	not leap	not leap ✔
2000	yes	leap	leap ✔
1900	yes	leap	**NOT a leap year ✘**

The rule is incomplete: century years such as 1900 and 2100 are not leap years unless divisible by 400. The complete rule is in section 15.5 (the first HOTS example) and 7.5 (the fifth Medium example).

Read a single lowercase character and decide whether it is a vowel or a consonant using if-else.

```
#include <stdio.h>
int main() {
    char ch = 'e';
    if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u')
        printf("Vowel");
    else
        printf("Consonant");
    return 0;
}
```

Output: Vowel

This assumes the input is a lowercase letter. Uppercase letters and non-letters are handled next.

Solved Examples — Medium

Find and print the larger of two numbers.

One comparison, two possible outcomes.

```
#include <stdio.h>
int main() {
    int a=56, b=72;
    if (a > b) printf("Largest = %d", a);
    else printf("Largest = %d", b);
    return 0;
}
```

Output: Largest = 72

A cinema charges ₹250 for adults and ₹120 for children (age below 12). Determine the ticket price for a given age.

The condition decides which of the two prices is assigned.

```
#include <stdio.h>
int main() {
    int age = 9;
    float price;
    if (age < 12) price = 120;
    else price = 250;
    printf("Ticket Price = Rs.%.2f", price);
    return 0;
}
```

Output: Ticket Price = Rs.120.00

Validate that a year entered by the user is a positive number before it is used in further calculations.

A simple validity check with a clear success/failure message on either side.

```
#include <stdio.h>
int main() {
    int year = -2023;
    if (year > 0) printf("Valid year entered");
    else printf("Invalid year! Must be positive.");
    return 0;
}
```

Output: Invalid year! Must be positive.

Read a character and decide whether it is a vowel or a consonant (both cases).

```
#include <stdio.h>
int main() {
    char ch = 'U';
    if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u' ||
        ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U')
        printf("%c is a vowel", ch);
    else
        printf("%c is a consonant", ch);
    return 0;
}
```

Output: U is a vowel

Weakness: the digit 5 would be reported as a "consonant". Validation is added in section 16.5.

Check whether a character is an alphabet or a digit using ASCII range checks (no ctype.h functions).

Group	Range (characters)	ASCII values
Digits	'0' to '9'	48 to 57
Uppercase	'A' to 'Z'	65 to 90
Lowercase	'a' to 'z'	97 to 122

```
#include <stdio.h>
int main() {
    char ch = '7';
    if ((ch >= 'A' && ch <= 'Z') || (ch >= 'a' && ch <= 'z'))
        printf("'%c' is an alphabet", ch);
    else if (ch >= '0' && ch <= '9')
        printf("'%c' is a digit", ch);
    else
        printf("'%c' is neither alphabet nor digit", ch);
    return 0;
}
```

Output: '7' is a digit

Solved Examples — HOTS

An ATM must check whether the account balance covers a withdrawal request. If it does, deduct the amount and show the new balance; otherwise, show how much is actually available.

The condition determines not just which message prints, but which calculation (the deduction) actually happens.

```
#include <stdio.h>
int main() {
    float balance = 5000, amount = 7500;
    if (balance >= amount) {
        balance -= amount;
        printf("Withdrawal successful. New balance = Rs.%.2f", balance);
```

```
    } else {
        printf("Insufficient balance! Available = Rs.%.2f", balance);
    }
    return 0;
}
```

Output: Insufficient balance! Available = Rs.5000.00

Packages under 5kg get Express shipping at ₹15/kg; 5kg and above get Standard shipping at ₹8/kg. Determine both the shipping method AND the cost together.

The same condition drives two related outcomes at once — the method label and the rate used in the cost formula.

```
#include <stdio.h>
int main() {
    float weight = 3.5, cost;
    char *method;
    if (weight < 5) { method = "Express"; cost = weight * 15; }
    else { method = "Standard"; cost = weight * 8; }
    printf("Method = %s, Cost = Rs.%.2f", method, cost);
    return 0;
}
```

Output: Method = Express, Cost = Rs.52.50

Predict the output of the following nested if-else, and explain exactly which if the else belongs to — the classic "dangling else" problem.

In C, an else always binds to the nearest unmatched if — regardless of how the code is indented on screen.

```
#include <stdio.h>
int main() {
    int a = 10;
    if (a > 5)
        if (a > 20)
            printf("Large");
        else
            printf("Medium");
    return 0;
}
```

Output: Medium

Even though indentation makes the else look like it belongs to the outer if(a>5), C actually attaches it to the nearest if — if(a>20). Since a>5 is true but a>20 is false, "Medium" prints. Always use braces {} around nested if-else blocks in real code to remove this ambiguity entirely.

Practice Problems — Try It Yourself

1. Write a program using if-else to check whether a number is positive or negative. (Bonus: what does your program print for the input 0 — and is that correct? You'll fix this properly with nested if in the next section.)
2. Write a program to calculate an electricity bill: if units used are 100 or below, charge a flat ₹200; otherwise, charge ₹200 plus ₹6 for every unit beyond 100.
3. Write a program to check whether a given character is uppercase or lowercase using if-else (assume the input is always a letter).
4. Read a year and test the simplified rule (divisible by 4). Run it for 1900 and 2000 and explain why the answer for 1900 is wrong.
5. Write the COMPLETE leap-year check (divisible by 4, but not by 100 unless also by 400) with if-else, and test 1900, 2000, 2023, 2024, 2100.
6. Read a character and print whether it is a vowel or consonant in both cases. Then enter 5 and observe the wrong answer (the fix is in section 16.5).

Debugging Corner — Find the Bug (Solved)

DEBUG 1

Find the errors in this program, explain them and correct the code.

```
#include <stdio.h>
int main() {
    int num;
    printf("Enter a number: ");
    scanf("%d", num);
    if (num % 2 = 0)
        printf("Even");
    else
        printf("Odd");
    return 0;
}
```

Compiler says: lvalue required as left operand of assignment

What is wrong?

• **Missing & in scanf:** it must be &num, otherwise scanf writes to a garbage address.

• **= instead of ==:** num % 2 = 0 tries to assign to an expression, which is a compile error. The comparison operator is ==.

Corrected code:

```
#include <stdio.h>
int main() {
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);
    if (num % 2 == 0)
        printf("Even");
    else
        printf("Odd");
    return 0;
}
```

Sample Run:

```
Enter a number: 8
Even
```

DEBUG 2

A programmer tries to get three outcomes from one if-else.

```
#include <stdio.h>
int main() {
    int x = 0;
    if (x > 0)
        printf("Positive");
    else
        printf("Negative");
    else
        printf("Zero");
    return 0;
}
```

Compiler says: 'else' without a previous 'if'

What is wrong?

• **An if can have only one else.** For three outcomes chain them with else if.

Corrected code:

```
#include <stdio.h>
int main() {
    int x = 0;
    if (x > 0)
        printf("Positive");
    else if (x < 0)
        printf("Negative");
    else
        printf("Zero");
    return 0;
}
```

Output: Zero

Debug It Yourself — Practice (answers in Appendix A)

DEBUG IT 1

An 18-year-old is refused although 18 is the voting age.

```
#include <stdio.h>
int main() {
    int age = 18;
    if (age > 18)
        printf("Eligible to vote");
    else
        printf("Not eligible");
    return 0;
}
```

Hint: *What happens exactly at the boundary value?*

DEBUG IT 2

The compiler reports "else without a previous if".

```
#include <stdio.h>
int main() {
    int x = 7;
    if (x > 0)
        printf("Positive");
        printf(" number");
    else
        printf("Not positive");
    return 0;
}
```

Hint: *How many statements does the if control when there are no braces?*

16.5 Nested if Statement

> **Analogy:** An airport passes you through passport control first, and only then through the security check. Nested if tests the second condition only when the first passes.

> **Technical Concept:** A nested if is an if statement inside another if or else block. The inner condition is evaluated only if the outer condition allowed it; braces avoid the dangling-else trap.

An if (or if-else) placed inside another if. This is used when a decision only makes sense to check AFTER another condition has already been confirmed true — a multi-level decision, checked one level at a time.

Syntax

```
if (condition1) {
    if (condition2) {
        // runs only if BOTH condition1 and condition2 are true
    } else {
        // condition1 true, but condition2 false
    }
} else {
    // condition1 itself is false
}
```

Solved Examples — Easy

Check whether a number is positive, negative, or exactly zero.

First separate zero-or-positive from negative; then, within that branch, separate zero from positive.

```
#include <stdio.h>
int main() {
    int n = 0;
    if (n >= 0) {
        if (n == 0) printf("Zero");
        else printf("Positive");
    } else {
        printf("Negative");
    }
    return 0;
}
```

Output: Zero

A club allows entry only to people 18 or older. Members enter free; non-members must pay an entry fee.

The outer if checks age; only if that passes does the inner if check membership.

```
#include <stdio.h>
int main() {
    int age = 22, isMember = 1;
    if (age >= 18) {
        if (isMember) printf("Welcome, Member!");
        else printf("Welcome, Guest! Please pay entry fee.");
    } else {
        printf("Entry denied: must be 18 or older.");
    }
    return 0;
}
```

Output: Welcome, Member!

Find the largest of three numbers using only nested if statements (an if inside another if, no else-if ladder).

Compare a and b first; the winner is then compared with c.

```
#include <stdio.h>
int main() {
   int a = 25, b = 40, c = 33;
   if (a > b) {
      if (a > c)
         printf("%d is the largest", a);
      else
         printf("%d is the largest", c);
   } else {
      if (b > c)
         printf("%d is the largest", b);
      else
         printf("%d is the largest", c);
   }
   return 0;
}
```

Output: 40 is the largest

Solved Examples — Medium

First check whether a student passed (marks 40 or above); only if they passed, further determine their grade (A/B/C).

The outer if isolates pass vs fail; the inner else-if ladder only runs for students who already passed.

```
#include <stdio.h>
int main() {
   int marks = 78;
   if (marks >= 40) {
      if (marks >= 90) printf("Grade A");
      else if (marks >= 60) printf("Grade B");
      else printf("Grade C");
   } else {
      printf("Fail");
   }
   return 0;
}
```

Output: Grade B

First check whether three side lengths form a valid triangle; only if valid, classify it as Equilateral, Isosceles, or Scalene.

The outer if applies the triangle-inequality check from Chapter 15; the inner nested checks only run once a valid triangle is confirmed.

```
#include <stdio.h>
int main() {
   int a=5, b=5, c=8;
   if (a+b>c && b+c>a && a+c>b) {
      if (a==b && b==c) printf("Equilateral Triangle");
      else if (a==b || b==c || a==c) printf("Isosceles Triangle");
      else printf("Scalene Triangle");
   } else {
      printf("Not a valid triangle");
   }
   return 0;
}
```

Output: Isosceles Triangle

A loan is considered only if income is ₹30,000 or above; among those applicants, it is approved only if credit score is 700 or above.

Two separate eligibility gates, checked one after another, each with its own rejection message.

```
#include <stdio.h>
int main() {
   float income = 35000; int creditScore = 680;
   if (income >= 30000) {
      if (creditScore >= 700) printf("Loan Approved");
      else printf("Loan Rejected: Low credit score");
   } else {
      printf("Loan Rejected: Insufficient income");
   }
   return 0;
}
```

Output: Loan Rejected: Low credit score

Check whether a number lies between 1 and 100; if so, classify it as belonging to the range 1–50 or 51–100.

```
#include <stdio.h>
int main() {
   int n = 72;
   if (n >= 1 && n <= 100) {
      if (n <= 50)
         printf("%d lies in the range 1-50", n);
      else
         printf("%d lies in the range 51-100", n);
   } else
      printf("%d is outside 1-100", n);
   return 0;
}
```

Output: 72 lies in the range 51-100

Check whether a given year is a leap year using nested if.

```
#include <stdio.h>
int main() {
   int year = 2024;
   if (year % 4 == 0) {
      if (year % 100 == 0) {
         if (year % 400 == 0)
            printf("%d is a leap year", year);
         else
            printf("%d is not a leap year", year);
      } else
         printf("%d is a leap year", year);
   } else
      printf("%d is not a leap year", year);
   return 0;
}
```

Output: 2024 is a leap year

Year	1900	2000	2023	2024	2100
Result	not leap	leap	not leap	leap	not leap

The same rule as ONE condition is in section 15.5 (the first HOTS example): (year%4==0 && year%100!=0) || year%400==0.

A cinema prices tickets like this: first check whether the person is a student; if so, the discount depends on whether the show is on a weekday (30%) or a weekend (15%). Non-students pay full price (base price ₹200).

```
#include <stdio.h>
int main() {
    float basePrice = 200, discount;
    int isStudent = 1, isWeekend = 0;
    if (isStudent) {
        if (isWeekend)
            discount = 15;
        else
            discount = 30;
    } else
        discount = 0;
    float finalPrice = basePrice - basePrice * discount / 100;
    printf("Discount = %.0f%%, Ticket price = Rs.%.2f", discount, finalPrice);
    return 0;
}
```

Output: Discount = 30%, Ticket price = Rs.140.00

Customer	Show	Discount	Price
Student	Weekday	30%	₹140
Student	Weekend	15%	₹170
Not a student	any	0%	₹200

Categorise a person's BMI (Underweight / Normal / Overweight) from weight and height using nested if-else.

```
#include <stdio.h>
int main() {
   float weight = 68, height = 1.75;
   float bmi = weight / (height * height);
   if (bmi < 25) {
      if (bmi < 18.5)
         printf("BMI = %.1f : Underweight", bmi);
      else
         printf("BMI = %.1f : Normal", bmi);
   } else
      printf("BMI = %.1f : Overweight", bmi);
   return 0;
}
```

Output: BMI = 22.2 : Normal

Classify an integer into exactly one of Positive Even, Positive Odd, Negative Even, Negative Odd or Zero using nested if.

```
#include <stdio.h>
int main() {
   int x = -8;
   if (x == 0)
      printf("Zero");
   else if (x > 0) {
      if (x % 2 == 0) printf("Positive Even");
      else            printf("Positive Odd");
   } else {
      if (x % 2 == 0) printf("Negative Even");
      else            printf("Negative Odd");
   }
   return 0;
}
```

Output: Negative Even

The test is x % 2 == 0 with an else, never x % 2 == 1, because -7 % 2 is -1. The bitwise version of this classifier is the fifth HOTS example in section 15.8.

Extend the vowel program: work for upper AND lowercase input, and first validate that the input is an alphabet, printing "Invalid input" for digits or symbols.

```
#include <stdio.h>
int main() {
    char ch = '5';
    if ((ch >= 'A' && ch <= 'Z') || (ch >= 'a' && ch <= 'z')) {
        if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u' ||
            ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U')
            printf("Vowel");
        else
            printf("Consonant");
    } else
        printf("Invalid input");
    return 0;
}
```

Output: Invalid input

Validate first, classify second — the same pattern interviewers look for in the triangle problem (the second Medium example above).

Solved Examples — HOTS

A theme park's ticket price depends first on age category (Child <12, Adult 12–59, Senior 60+), and then, within each category, on whether it is a weekday or a weekend.

The outer decision (age category) is handled with an else-if ladder; each branch then uses a ternary to apply the weekday/weekend rate.

```
#include <stdio.h>
int main() {
    int age = 65, isWeekend = 1;
    float price;
    if (age < 12) price = isWeekend ? 100 : 80;
    else if (age < 60) price = isWeekend ? 300 : 250;
    else price = isWeekend ? 150 : 120;
    printf("Ticket Price = Rs.%.2f", price);
    return 0;
}
```

Output: Ticket Price = Rs.150.00

Validate a password: first check its length is at least 8; only if that passes, check whether it contains a digit — and give a SPECIFIC error message for whichever check actually fails.

Nested if lets each level give its own precise error, rather than one generic "invalid password" message — this mirrors how real signup forms behave.

```
#include <stdio.h>
#include <string.h>
int main() {
    char password[] = "welcome1";
    int hasDigit = 1;
    int len = strlen(password);
    if (len >= 8) {
        if (hasDigit) printf("Password is valid");
        else printf("Password must contain at least one digit");
    } else {
        printf("Password must be at least 8 characters long");
    }
    return 0;
}
```

Output: Password is valid

A bank pays different interest rates: Savings accounts get 4% if balance is below ₹100,000, else 5%. Current accounts always get a flat 1%, regardless of balance.

The outer if separates account types; the inner if (only inside the Savings branch) applies the balance-tier rule, since Current accounts don't need it.

```
#include <stdio.h>
int main() {
    char accountType = 'S'; float balance = 150000, rate;
    if (accountType == 'S') {
        if (balance < 100000) rate = 4.0;
        else rate = 5.0;
    } else {
        rate = 1.0;
    }
    printf("Interest Rate = %.1f%%", rate);
    return 0;
}
```

Output: Interest Rate = 5.0%

Without running the code, predict the output and justify your answer (tests the dangling else).

```
#include <stdio.h>
int main() {
    int a = 10, b = 20, c = 30;
    if (a > b)
        if (b > c)
            printf("Case 1");
        else
            printf("Case 2");
    else
        printf("Case 3");
    return 0;
}
```

Output: Case 3

if	its else
if (b > c) (inner)	first else → Case 2 (nearest unmatched if)
if (a > b) (outer)	second else → Case 3 (the inner if already has one)

a > b is false, so the whole inner statement is skipped and control goes to the outer else: "Case 3". An else always pairs with the nearest earlier if that has no else yet.

Predict the output for marks = 60, explain which if the else binds to, and rewrite with braces so that 60 prints "Pass", below 50 prints "Fail" and 75+ prints "Distinction".

```
#include <stdio.h>
int main() {
    int marks = 60;
    if (marks >= 50)
        if (marks >= 75)
            printf("Distinction");
        else
            printf("Fail");
    return 0;
}
```

Output: Fail

The else pairs with the NEAREST if, if (marks >= 75). So "Fail" is printed for marks between 50 and 74, and nothing at all for marks below 50. Braces make the pairing explicit:

```
#include <stdio.h>
int main() {
    int marks = 60;
    if (marks >= 50) {
        if (marks >= 75)
            printf("Distinction");
        else
            printf("Pass");
    } else
        printf("Fail");
    return 0;
}
```

Output: Pass

Extend the classifier to handle INT_MIN (-2147483648). If the logic ever computes -x, explain why this value can silently give a wrong answer or undefined behaviour and how to guard against it.

```
#include <stdio.h>
#include <limits.h>
int main() {
    int x = INT_MIN;
    if (x == 0)
        printf("Zero");
    else if (x > 0)
        printf(x % 2 == 0 ? "Positive Even" : "Positive Odd");
    else
        printf(x % 2 == 0 ? "Negative Even" : "Negative Odd");

    long long y = x;            // widen BEFORE negating
    if (y < 0) y = -y;          // safe: +2147483648 fits in a long long
    printf("\n|x| = %lld", y);
    return 0;
}
```

Output: Negative Even / |x| = 2147483648

An int holds -2147483648 to +2147483647, so +2147483648 does not exist: -INT_MIN overflows, which is undefined behaviour (on typical hardware it stays INT_MIN, still negative, and later "x > 0" style tests give wrong answers). Guards: (1) test x == INT_MIN first, (2) avoid negating at all — x % 2 and x & 1 need no negation, (3) widen to long long before negating. The same edge case applies to abs(INT_MIN).

Practice Problems — Try It Yourself

1. Write a program using nested if to check whether a number is positive; if it is, further check whether it is even or odd.
2. Write a program using nested if to validate a login: first check whether the username matches, then (only if it does) check whether the password matches — print a specific message for each possible failure.
3. Write a program for an exam result system: first check attendance is 75% or above; if it is, check whether marks are 40 or above to decide Pass/Fail; otherwise print "Detained due to low attendance".
4. Read three numbers and find the SMALLEST using only nested if.
5. Read a character. If it is a letter, say vowel or consonant; else if it is a digit, print "Digit"; otherwise print "Special symbol" (nested if).
6. Extend the cinema program: senior citizens (age 60 or more) get 40% off on any day.
7. Test your leap-year program on 1900, 2000, 2023, 2024 and 2100 and record expected vs actual.

Debugging Corner — Find the Bug (Solved)

DEBUG 1

For a = 10, b = 20, c = 5 this program is meant to print "A is largest" only when a beats both b and c, and "something else" otherwise. It compiles but misbehaves. Find which if the else binds to and fix it.

```
#include <stdio.h>
int main() {
   int a = 10, b = 20, c = 5;
   if (a > b)
      if (a > c)
         printf("A is largest");
   else
      printf("something else");
   return 0;
}
```

Output of the buggy program: (nothing is printed)

What is wrong?

• **The else binds to the inner if (a > c),** despite the indentation. Because a > b is false, the whole inner statement (including its else) is skipped and NOTHING is printed — but "something else" was expected.

Corrected code:

```
#include <stdio.h>
int main() {
   int a = 10, b = 20, c = 5;
   if (a > b) {
      if (a > c)
         printf("A is largest");
      else
         printf("something else");
   } else
      printf("something else");
   return 0;
}
```

Output: something else

Simpler still: if (a > b && a > c) printf("A is largest"); else printf("something else");

DEBUG 2

Every equilateral triangle is reported as not equilateral.

```
#include <stdio.h>
int main() {
    int a = 5, b = 5, c = 5;
    if (a == b == c)
        printf("Equilateral");
    else
        printf("Not equilateral");
    return 0;
}
```

Output of the buggy program: Not equilateral

What is wrong?

• **a == b == c is (a == b) == c.** (5 == 5) is 1, and 1 == 5 is false. Compare pairs and join them: a == b && b == c.

Corrected code:

```
#include <stdio.h>
int main() {
    int a = 5, b = 5, c = 5;
    if (a == b && b == c)
        printf("Equilateral");
    else
        printf("Not equilateral");
    return 0;
}
```

Output: Equilateral

Debug It Yourself — Practice (answers in Appendix A)

DEBUG IT 1

The user exists but has the wrong password. The message is misleading.

```
#include <stdio.h>
int main() {
    int userOk = 1, passOk = 0;
    if (userOk)
        if (passOk)
            printf("Welcome!");
    else
        printf("Unknown user!");
    return 0;
}
```

Hint: *Which if does the else really belong to?*

DEBUG IT 2

The origin (0, 0) is reported as "On the X-axis".

```
#include <stdio.h>
int main() {
    int x = 0, y = 0;
    if (y == 0)
        printf("On the X-axis");
    else if (x == 0)
        printf("On the Y-axis");
    else if (x == 0 && y == 0)
        printf("Origin");
    return 0;
}
```

Hint: *Which branch catches (0, 0) first? Can the last branch ever run?*

16.6 if-else-if Ladder

> **Analogy:** A teacher grades by checking A+, then A, then B, then C, stopping at the first match. The if-else-if ladder works the same way.

> **Technical Concept:** The if-else-if ladder tests conditions from top to bottom and executes the block of the first true condition; the final else handles every remaining case.

A chain of conditions checked one after another, from top to bottom. As soon as one condition is found true, its block runs and the ENTIRE rest of the ladder is skipped — even if a later condition would also have been true. If none match, the final else (if present) runs.

Syntax

```
if (condition1) {
   // runs if condition1 is true
} else if (condition2) {
   // runs only if condition1 is false AND condition2 is true
} else if (condition3) {
   // runs only if condition1 and condition2 are both false, and condition3 is true
} else {
   // runs if none of the above matched
}
```

Solved Examples — Easy

Assign a grade (A/B/C/F) based on marks.

A standard 4-way grading ladder.

```
#include <stdio.h>
int main() {
   int marks = 82;
   if (marks >= 90) printf("Grade A");
   else if (marks >= 75) printf("Grade B");
   else if (marks >= 50) printf("Grade C");
   else printf("Grade F");
   return 0;
}
```

Output: Grade B

Print the action to take at a traffic signal, based on its current colour.

Each colour is one rung of the ladder.

```
#include <stdio.h>
int main() {
   char light = 'Y';
   if (light == 'R') printf("Stop");
   else if (light == 'Y') printf("Get Ready");
   else if (light == 'G') printf("Go");
```

```
    else printf("Invalid signal");
    return 0;
}
```

Output: Get Ready

Solved Examples — Medium

Classify a person's BMI: below 18.5 → Underweight, 18.5–24.9 → Normal, 25–29.9 → Overweight, 30 and above → Obese.

Each condition only needs an upper bound, since anything that failed the earlier (lower) checks is already known to be above them.

```
#include <stdio.h>
int main() {
    float bmi = 27.3;
    if (bmi < 18.5) printf("Underweight");
    else if (bmi < 25.0) printf("Normal");
    else if (bmi < 30.0) printf("Overweight");
    else printf("Obese");
    return 0;
}
```

Output: Overweight

Determine the applicable income tax slab: up to ₹250,000 → 0%, up to ₹500,000 → 5%, up to ₹1,000,000 → 20%, above → 30%.

A classic slab-based ladder, ordered from lowest to highest.

```
#include <stdio.h>
int main() {
    float income = 680000, rate;
    if (income <= 250000) rate = 0;
    else if (income <= 500000) rate = 5;
    else if (income <= 1000000) rate = 20;
    else rate = 30;
    printf("Applicable Tax Slab = %.0f%%", rate);
    return 0;
}
```

Output: Applicable Tax Slab = 20%

Convert a day number (1–7) into its weekday name.

A 7-way ladder with a default case for invalid numbers.

```
#include <stdio.h>
int main() {
    int day = 4;
    if (day==1) printf("Monday");
    else if (day==2) printf("Tuesday");
    else if (day==3) printf("Wednesday");
    else if (day==4) printf("Thursday");
    else if (day==5) printf("Friday");
    else if (day==6) printf("Saturday");
    else if (day==7) printf("Sunday");
    else printf("Invalid day number");
    return 0;
}
```

Output: Thursday

Find the largest among three numbers using if – else if – else.

```
#include <stdio.h>
int main() {
    int a = 25, b = 40, c = 33;
    if (a >= b && a >= c)
        printf("%d is the largest", a);
    else if (b >= a && b >= c)
        printf("%d is the largest", b);
    else
        printf("%d is the largest", c);
    return 0;
}
```

Output: 40 is the largest

>= is used so that ties still give an answer. Compare this flat ladder with the nested version in section 16.5 (the third Easy example).

Extend the ASCII check to a full 4-way classification: Uppercase Letter, Lowercase Letter, Digit or Special Character.

```
#include <stdio.h>
int main() {
    char ch = 'g';
    if (ch >= 'A' && ch <= 'Z')
        printf("Uppercase letter");
    else if (ch >= 'a' && ch <= 'z')
        printf("Lowercase letter");
    else if (ch >= '0' && ch <= '9')
        printf("Digit");
    else
        printf("Special character");
    return 0;
}
```

Output: Lowercase letter

For an integer x decide whether x^2 is greater than, equal to or less than 100, but treat negative input separately as "Negative input" BEFORE any comparison (arithmetic and relational operators only, no abs() or sqrt()).

```
#include <stdio.h>
int main() {
    int x = -7;
    if (x < 0)
        printf("Negative input");
    else if (x * x > 100)
        printf("x squared is greater than 100");
    else if (x * x == 100)
        printf("x squared is equal to 100");
    else
        printf("x squared is less than 100");
    return 0;
}
```

Output: Negative input

x	-7	5	10	11
Result	Negative input	less than 100	equal to 100	greater than 100

The negative check must come first: $(-10)^2$ is also 100, but the rule says negatives are set aside before comparing. Also, for $x \geq 0$, $x*x > 100$ is equivalent to $x > 10$, which avoids overflow for huge x.

Classify marks (0–100) into Fail (<40), Pass (40–59), Good (60–79) and Excellent (80–100) WITHOUT using the && operator.

The order of the ladder does the work of &&: by the time marks < 60 is tested, we already know marks >= 40, because the earlier test failed.

```
#include <stdio.h>
int main() {
   int marks = 67;
   if (marks < 0 || marks > 100)
      printf("Invalid marks");
   else if (marks < 40)
      printf("Fail");
   else if (marks < 60)
      printf("Pass");
   else if (marks < 80)
      printf("Good");
   else
      printf("Excellent");
   return 0;
}
```

Output: Good

&& is a convenience, not a necessity: chained tests in the right order (or nesting) express the same range.

Differentiate between independent ifs and an else-if ladder with an example where the ladder is more efficient.

```
#include <stdio.h>
int main() {
    int marks = 92;
    printf("Independent ifs: ");
    if (marks >= 90) printf("A");
    if (marks >= 75 && marks < 90) printf("B");
    if (marks >= 60 && marks < 75) printf("C");
    if (marks < 60) printf("Fail");

    printf("\nElse-if ladder : ");
    if (marks >= 90)      printf("A");
    else if (marks >= 75) printf("B");
    else if (marks >= 60) printf("C");
    else                  printf("Fail");
    return 0;
}
```

Output: Independent ifs: A / Else-if ladder : A

marks	Comparisons: independent ifs	Comparisons: ladder
92	6	1
80	6	2
70	5	3
50	4	3

The ladder stops at the first true condition, so it does less work AND its conditions can stay simple (no need for && marks < 90). Independent ifs re-test everything and can print several results or none when the ranges overlap or leave gaps.

Solved Examples — HOTS

Determine a loan's interest rate from a credit score: 750+ → 7%, 650–749 → 9%, 550–649 → 12%, below 550 → rejected.

Order the ladder from the HIGHEST threshold down to the lowest, so each check only needs a lower bound.

```
#include <stdio.h>
int main() {
    int creditScore = 700;
    if (creditScore >= 750) printf("Interest Rate = 7%%");
    else if (creditScore >= 650) printf("Interest Rate = 9%%");
    else if (creditScore >= 550) printf("Interest Rate = 12%%");
    else printf("Loan Rejected");
    return 0;
}
```

Output: Interest Rate = 9%

An e-commerce site applies tiered discounts: ₹5000+ → 20% off, ₹2000+ → 12% off, ₹1000+ → 5% off, below that → no discount. Compute the final price.

The ladder decides the discount rate; a follow-up arithmetic step applies it to the amount.

```
#include <stdio.h>
int main() {
    float amount = 3200, discount;
    if (amount >= 5000) discount = 0.20;
    else if (amount >= 2000) discount = 0.12;
    else if (amount >= 1000) discount = 0.05;
    else discount = 0;
    float finalPrice = amount - (amount * discount);
    printf("Final Price = Rs.%.2f", finalPrice);
    return 0;
}
```

Output: Final Price = Rs.2816.00

Predict the output of the ladder below, and explain why only ONE message prints even though a later condition would also technically be true for these marks.

The ladder stops at the FIRST true condition it finds, no matter what order the conditions were written in — this tests whether you understand "first match wins", not just the individual conditions.

```
#include <stdio.h>
int main() {
    int marks = 95;
    if (marks >= 50) printf("Grade C or above");
    else if (marks >= 75) printf("Grade B or above");
    else if (marks >= 90) printf("Grade A");
    return 0;
```

```
}
```

Output: Grade C or above

Even though marks=95 also satisfies >=75 and >=90, the ladder stops at the very first true condition it meets — marks>=50 — and never even evaluates the rest. This is exactly why conditions in a ladder must be ordered from most specific/restrictive to least specific, not the other way around.

Print "Fizz" if x is divisible by 3, "Buzz" if by 5, "FizzBuzz" if by both, and the number itself otherwise. Explain why the order of conditions matters.

```
#include <stdio.h>
int main() {
    int x = 15;
    if (x % 3 == 0 && x % 5 == 0)     // the combined case MUST come first
        printf("FizzBuzz");
    else if (x % 3 == 0)
        printf("Fizz");
    else if (x % 5 == 0)
        printf("Buzz");
    else
        printf("%d", x);
    return 0;
}
```

Output: FizzBuzz

With the checks in the wrong order, 15 is caught by the very first test and the combined case is unreachable:

```
#include <stdio.h>
int main() {
    int x = 15;
    if (x % 3 == 0)
        printf("Fizz");              // 15 stops here!
    else if (x % 5 == 0)
        printf("Buzz");
    else if (x % 3 == 0 && x % 5 == 0)
        printf("FizzBuzz");          // never reached
    return 0;
}
```

Output: Fizz

Rule: in a ladder put the most specific condition first and the most general last.

Given a point (x, y), find which quadrant (I–IV) it lies in, or whether it lies on the X-axis, the Y-axis or at the Origin. Explain why the order of conditions matters.

```
#include <stdio.h>
int main() {
    int x = 0, y = 0;
    if (x == 0 && y == 0)
        printf("Origin");
    else if (y == 0)
        printf("On the X-axis");
    else if (x == 0)
        printf("On the Y-axis");
    else if (x > 0 && y > 0)
        printf("Quadrant I");
    else if (x < 0 && y > 0)
        printf("Quadrant II");
    else if (x < 0 && y < 0)
        printf("Quadrant III");
    else
        printf("Quadrant IV");
    return 0;
}
```

Output: Origin

(x, y)	(3,4)	(-3,4)	(-3,-4)	(3,-4)	(5,0)	(0,-2)	(0,0)
Result	I	II	III	IV	X-axis	Y-axis	Origin

The origin also satisfies y == 0, so if the X-axis test came first, (0, 0) would be mislabelled. By the time the quadrant tests run, both x and y are known to be non-zero, so each quadrant needs only a sign check.

Classify a character as Uppercase, Lowercase, Digit or Special using ASCII ranges only, and convert case by arithmetic on the ASCII value (no built-in functions).

```
#include <stdio.h>
int main() {
   char ch = 'q';
   if (ch >= 'A' && ch <= 'Z')
      printf("Uppercase, lowercase form: %c", ch - 'A' + 'a');
   else if (ch >= 'a' && ch <= 'z')
      printf("Lowercase, uppercase form: %c", ch - 'a' + 'A');
   else if (ch >= '0' && ch <= '9')
      printf("Digit");
   else
      printf("Special character");
   return 0;
}
```

Output: Lowercase, uppercase form: Q

The gap between 'a' and 'A' is 32, so ch - 'a' + 'A' is the same as ch - 32, written in a self-explaining way. It is correct only for letters, which is why the range check comes first.

Write a single-character "password strength" checker: classify one character as Has Uppercase, Has Lowercase, Has Digit or Has Special Symbol. Then explain how it is reused for a full password.

```
#include <stdio.h>
int main() {
   char ch = '@';
   if (ch >= 'A' && ch <= 'Z')      printf("Has Uppercase");
   else if (ch >= 'a' && ch <= 'z') printf("Has Lowercase");
   else if (ch >= '0' && ch <= '9') printf("Has Digit");
   else                             printf("Has Special Symbol");
   return 0;
}
```

Output: Has Special Symbol

Reusing it for "Abc123!": keep four flags hasUpper, hasLower, hasDigit, hasSpecial, all starting at 0. Run the classifier on each character in turn and set the matching flag to 1. The password is strong only when all four flags are 1. Loops (next unit) will repeat the classifier automatically; for now we describe the idea by tracing it:

Character	A	b	c	1	2	3	!
Sets flag	Upper	Lower	Lower	Digit	Digit	Digit	Special

After the last character all four flags are 1, so the final decision is: hasUpper && hasLower && hasDigit && hasSpecial → "Strong".

Design a loan approval classifier with three outcomes: Approved, Approved with higher interest rate, or Rejected, based on credit_score, monthly_income and existing_loans_count. Then explain how using || where && was needed can wrongly approve an applicant.

```
#include <stdio.h>
int main() {
    int creditScore = 720, income = 40000, loans = 1;
    if (creditScore >= 750 && income >= 50000 && loans <= 1)
        printf("Approved");
    else if (creditScore >= 650 && income >= 30000 && loans <= 2)
        printf("Approved with higher interest rate");
    else
        printf("Rejected");
    return 0;
}
```

Output: Approved with higher interest rate

The colleague's bug — || instead of && between the credit and income conditions — tested with an applicant whose credit score is only 400:

```
#include <stdio.h>
int main() {
    int creditScore = 400, income = 90000, loans = 1;   // very poor credit
    if (creditScore >= 750 || income >= 50000 && loans <= 1)   // BUG: || instead of &&
        printf("Approved");
    else
        printf("Rejected");
    return 0;
}
```

Output: Approved

The applicant should be Rejected (credit 400), but the high income alone satisfies the || so the bank says "Approved". Nothing crashes and no compiler error appears — the program just makes a silently wrong business decision, which is why this class of bug is so dangerous. Test every branch with boundary and "one good, one bad" cases.

Practice Problems — Try It Yourself

1. Write an else-if ladder to classify a person's age group: 0–12 Child, 13–19 Teenager, 20–59 Adult, 60+ Senior Citizen.
2. Write an else-if ladder to compute postal shipping charges based on weight: up to 1kg ₹40, up to 5kg ₹70, up to 10kg ₹120, above 10kg ₹200.
3. Write an else-if ladder to assign a letter grade with +/- variants: 95+ → A+, 90–94 → A, 85–89 → B+, 80–84 → B, below 80 → C.
4. Grade calculator with an else-if ladder: marks ≥ 90 → A, ≥ 75 → B, ≥ 60 → C, below 60 → Fail. Test 90, 89, 75, 74, 60 and 59.
5. Modify the quadrant finder and test it on (-2, 0), (0, 5), (4, -1) and (0, 0).
6. Add a fourth outcome "Manual review" to the loan classifier for credit scores 600–649 with income ≥ 30000. Decide where in the ladder it must go and why.
7. Extend the character classifier to print "Whitespace" for a space or tab before falling back to "Special character".

Debugging Corner — Find the Bug (Solved)

DEBUG 1

The program should print Positive, Negative or Zero, but always prints "Positive" for negative numbers too. Find and fix the logic error.

```
#include <stdio.h>
int main() {
    int n = -5;
    if (n >= -100)
        printf("Positive");
    else if (n < 0)
        printf("Negative");
    else
        printf("Zero");
    return 0;
}
```

Output of the buggy program: Positive

What is wrong?

• **Wrong first condition:** n >= -100 is true for -5 (and for 0 and for -50), so the first branch grabs almost everything. "Positive" means n > 0.

Corrected code:

```
#include <stdio.h>
int main() {
    int n = -5;
    if (n > 0)
        printf("Positive");
    else if (n < 0)
        printf("Negative");
    else
        printf("Zero");
    return 0;
}
```

Output: Negative

DEBUG 2

Find the syntax and logical errors.

```
#include <stdio.h>
int main() {
    char grade = 'A'
    int marks = 85;
    if (marks > 90)
    {
        grade = 'A';
    else if (marks > 75) {
        grade = 'B';
    }
    printf("Grade: %c", grade);
    return 0;
}
```

Compiler says: expected ‘,’ or ‘;’ before ‘int’

What is wrong?

• **Missing semicolon** after char grade = 'A'.

• **Unmatched braces:** the { after if (marks > 90) is never closed before else if, so the else has no if and the braces do not balance.

• **Logical error:** grade starts as 'A' and there is no final else, so a student with 60 marks would still print Grade: A.

Corrected code:

```
#include <stdio.h>
int main() {
    char grade;
    int marks = 85;
    if (marks > 90) {
        grade = 'A';
    } else if (marks > 75) {
        grade = 'B';
    } else {
        grade = 'C';
    }
    printf("Grade: %c", grade);
    return 0;
}
```

Output: Grade: B

DEBUG 3

A student with 92 marks receives grade C.

```
#include <stdio.h>
int main() {
    int marks = 92;
    if (marks >= 60)
        printf("C");
    else if (marks >= 75)
        printf("B");
    else if (marks >= 90)
        printf("A");
    return 0;
}
```

Output of the buggy program: C

What is wrong?

• **Order of conditions.** 92 >= 60 is true, so the first branch runs and the stricter tests are never reached. A ladder must go from the most restrictive condition down to the least.

Corrected code:

```
#include <stdio.h>
int main() {
    int marks = 92;
    if (marks >= 90)
        printf("A");
    else if (marks >= 75)
        printf("B");
    else if (marks >= 60)
        printf("C");
    return 0;
}
```

Output: A

Debug It Yourself — Practice (answers in Appendix A)

DEBUG IT 1

This will not compile. What is wrong after else?

```
#include <stdio.h>
int main() {
    int marks = 30;
    if (marks >= 40)
        printf("Pass");
    else (marks < 40)
        printf("Fail");
    return 0;
}
```

Hint: *Does a plain else take a condition?*

DEBUG IT 2

a = 30, b = 30, c = 10: the maximum is 30, but 10 is printed.

```
#include <stdio.h>
int main() {
    int a = 30, b = 30, c = 10, max;
    if (a > b && a > c)
        max = a;
    else if (b > a && b > c)
        max = b;
    else
        max = c;
    printf("Max = %d", max);
    return 0;
}
```

Hint: *What happens when two values are equal?*

16.7 The switch Statement

> **Analogy:** A lift panel has one button per floor; pressing a button jumps straight to that floor. switch jumps straight to the matching case.

> **Technical Concept:** switch (expression) selects the case whose constant equals the expression. Each case should end with break; default handles unmatched values. The expression must be an integer or char.

An alternative to a long if-else-if ladder, used when ONE variable needs to be compared against several fixed, exact constant values (like a menu choice, a day number, or a grade letter). Each break statement is essential — it stops execution from "falling through" into the next case.

Syntax

```
switch (expression) {
    case value1:
        // statements
        break;
    case value2:
        // statements
        break;
    default:
```

```
        // runs if no case matches
}
```

switch only works with int or char (whole-number/character) expressions — it cannot directly switch on a float or a string, and it cannot test a range like "90 to 100" in a single case (unlike an if-else-if ladder).

Solved Examples — Easy

Build a simple calculator that performs the operation matching a given operator character.

One case per operator symbol, each ending in break.

```
#include <stdio.h>
int main() {
   char op = '*'; int a=8, b=5;
   switch (op) {
      case '+': printf("%d", a+b); break;
      case '-': printf("%d", a-b); break;
      case '*': printf("%d", a*b); break;
      case '/': printf("%d", a/b); break;
      default: printf("Invalid operator");
   }
   return 0;
}
```

Output: 40

Convert a day number (1–7) into its weekday name using switch.

One case per day number, with default catching anything invalid.

```
#include <stdio.h>
int main() {
   int day = 3;
   switch (day) {
      case 1: printf("Monday"); break;
      case 2: printf("Tuesday"); break;
      case 3: printf("Wednesday"); break;
      default: printf("...");
   }
   return 0;
}
```

Output: Wednesday

What is the purpose of break inside a switch block? Show what happens if it is left out.

break **exits the switch** as soon as a case has done its work. Without it, execution falls through into the next case (and the next) until a break or the closing brace.

```
#include <stdio.h>
int main() {
   int n = 2;
   switch (n) {
      case 1: printf("One\n");   break;
      case 2: printf("Two\n");   break;
      case 3: printf("Three\n"); break;
   }
   printf("--- without break ---\n");
   switch (n) {
      case 1: printf("One\n");
      case 2: printf("Two\n");
      case 3: printf("Three\n");
   }
   return 0;
}
```

Output: Two / --- without break --- / Two / Three

Solved Examples — Medium

Display a simple bank menu (1: Check Balance, 2: Deposit, 3: Withdraw, 4: Exit) and respond based on the user's choice.

Menu-driven programs are one of the most common real-world uses of switch.

```
#include <stdio.h>
int main() {
   int choice = 2;
   switch (choice) {
      case 1: printf("Balance = Rs.10000"); break;
      case 2: printf("Deposit successful"); break;
      case 3: printf("Withdrawal successful"); break;
      case 4: printf("Thank you, exiting..."); break;
      default: printf("Invalid choice");
   }
   return 0;
```

```
}
```

Output: Deposit successful

Print a message for a letter grade, treating both uppercase and lowercase input the same way (e.g. 'B' and 'b' should behave identically).

Deliberate, intentional fall-through: stacking case labels together with no break between them groups them into one shared block.

```
#include <stdio.h>
int main() {
    char grade = 'b';
    switch (grade) {
        case 'A': case 'a': printf("Excellent"); break;
        case 'B': case 'b': printf("Good"); break;
        case 'C': case 'c': printf("Average"); break;
        default: printf("Invalid grade");
    }
    return 0;
}
```

Output: Good

Print the action for a traffic light, and correctly handle an invalid/unrecognised signal using default.

default acts exactly like the final else in an if-else-if ladder — it catches anything that didn't match any case.

```
#include <stdio.h>
int main() {
    char light = 'X';
    switch (light) {
        case 'R': printf("Stop"); break;
        case 'Y': printf("Get Ready"); break;
        case 'G': printf("Go"); break;
        default: printf("Invalid signal - malfunction!");
    }
    return 0;
}
```

Output: Invalid signal - malfunction!

Write a menu-driven program using switch to convert temperature: 1 = C to F, 2 = F to C, 3 = C to K.

```
#include <stdio.h>
int main() {
   int choice;
   float temp, result;
   printf("1. C to F\n2. F to C\n3. C to K\nEnter choice: ");
   scanf("%d", &choice);
   printf("Enter temperature: ");
   scanf("%f", &temp);
   switch (choice) {
      case 1: result = temp * 9 / 5 + 32;
            printf("%.1f C = %.1f F", temp, result); break;
      case 2: result = (temp - 32) * 5 / 9;
            printf("%.1f F = %.1f C", temp, result); break;
      case 3: result = temp + 273.15;
            printf("%.1f C = %.2f K", temp, result); break;
      default: printf("Invalid choice");
   }
   return 0;
}
```

Sample Run:

```
1. C to F
2. F to C
3. C to K
Enter choice: 1
Enter temperature: 100
100.0 C = 212.0 F
```

*In case 2 the brackets in (temp - 32) * 5 / 9 are essential. Writing 5 / 9 * (temp - 32) would give 0, because 5 / 9 is an integer division.*

Print the number of days in a given month (1–12) using switch. For month 2 use a nested if inside the case to check for a leap year.

```
#include <stdio.h>
int main() {
   int month = 2, year = 2024, days;
   switch (month) {
      case 1: case 3: case 5: case 7: case 8: case 10: case 12:
         days = 31; break;
      case 4: case 6: case 9: case 11:
         days = 30; break;
      case 2:
         if ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0)
            days = 29;
         else
            days = 28;
         break;
      default:
         days = 0;
   }
   if (days == 0) printf("Invalid month");
   else          printf("Month %d of %d has %d days", month, year, days);
   return 0;
}
```

Output: Month 2 of 2024 has 29 days

Stacked case labels (no statement between them) deliberately share one block — the useful side of fall-through.

Solved Examples — HOTS

Predict the output of this switch statement, which is deliberately MISSING a break in one case.

Once a matching case begins executing, control "falls through" every case below it until a break is hit or the switch ends — this is a purely mechanical rule, not a smart skip.

```
#include <stdio.h>
int main() {
   int day = 2;
   switch (day) {
      case 1: printf("Mon ");
```

```
        case 2: printf("Tue ");
        case 3: printf("Wed ");
        default: printf("...more days");
    }
    return 0;
}
```

Output: Tue Wed ...more days

Because case 2 has no break, execution doesn't stop there — it keeps running straight through case 3 and default as well. Accidentally forgetting a break is one of the most common real-world C bugs; always check that every case ends in break unless you deliberately want it to fall through (as in the second Medium example above).

A college portal first asks for a department (1=CSE, 2=ECE), then, within that department, asks for a year (1=First Year, 2=Second Year). Implement this using a switch nested inside another switch.

Just like nested if, a switch can be placed inside another switch's case block for multi-level menus.

```
#include <stdio.h>
int main() {
    int dept = 1, year = 2;
    switch (dept) {
        case 1:
            printf("CSE - ");
            switch (year) {
                case 1: printf("First Year"); break;
                case 2: printf("Second Year"); break;
                default: printf("Invalid year");
            }
            break;
        case 2:
            printf("ECE - ");
            switch (year) {
                case 1: printf("First Year"); break;
                case 2: printf("Second Year"); break;
                default: printf("Invalid year");
            }
            break;
        default: printf("Invalid department");
    }
```

```
    return 0;
}
```

Output: CSE - Second Year

switch cannot directly test a range like "marks 90 to 100". Using switch, still classify marks into grade bands (90–100→A, 80–89→B, 70–79→C) by first reducing the marks to a single "band" number.

Divide the marks by 10 first to collapse the value into a small set of exact integers (a band), which switch CAN match directly — a well-known workaround for range-style checks.

```
#include <stdio.h>
int main() {
    int marks = 84;
    int band = marks / 10;
    switch (band) {
        case 10: case 9: printf("Grade A"); break;
        case 8: printf("Grade B"); break;
        case 7: printf("Grade C"); break;
        default: printf("Grade F");
    }
    return 0;
}
```

Output: Grade B

switch only ever matches exact constant values, never ranges. When you truly need range checks, an if-else-if ladder (section 16.6) is usually the more natural and readable tool — this band-reduction trick is a useful workaround, not a replacement.

Why can't switch use floats or strings? Explain why the following code fails to compile, mention how switch is typically implemented internally, and state which data types ARE legal.

```
#include <stdio.h>
int main() {
   float x = 2.5;
   switch (x) {
      case 2.5: printf("Two point five"); break;
      default:  printf("Other");
   }
   return 0;
}
```

Compiler says: switch quantity not an integer

How switch works. A compiler typically turns a switch into a **jump table** (an array of addresses indexed directly by the value) when the case values are close together, or into a fast **binary search** over the sorted case values when they are spread out. Both techniques need the switch value and every case label to be **exact integers known at compile time**.

Candidate	Legal?	Why
int, char, short, long, enum	Yes	integer types with exact values; can index a table
float, double	No	not exact (0.1 + 0.2 != 0.3) and cannot index a table
string ("abc")	No	a string is an address; comparing it would compare pointers, not text (needs strcmp)
variable as a case label	No	labels must be constants fixed at compile time
range like case 1..5	No (standard C)	a case is one exact value

```
#include <stdio.h>
int main() {
   char grade = 'B';          // char is an integer type -> legal
   switch (grade) {
      case 'A': printf("Excellent"); break;
      case 'B': printf("Good");      break;
      default:  printf("Other");
   }
   return 0;
}
```

Output: Good

Design a switch-based calculator (+ − * /) that handles two edge cases most students forget: division by zero and an invalid operator. Explain what would go wrong without each safeguard.

```
#include <stdio.h>
int main() {
    float a = 20, b = 0;
    char op = '/';
    switch (op) {
        case '+': printf("%.2f", a + b); break;
        case '-': printf("%.2f", a - b); break;
        case '*': printf("%.2f", a * b); break;
        case '/':
            if (b != 0)
                printf("%.2f", a / b);
            else
                printf("Error: division by zero");
            break;
        default:
            printf("Error: invalid operator '%c'", op);
    }
    return 0;
}
```

Output: Error: division by zero

Without the b != 0 check the same input prints:

```
#include <stdio.h>
int main() {
    float a = 20, b = 0;
    printf("%.2f", a / b);
    return 0;
}
```

Output: inf

Safeguard removed	What goes wrong
division-by-zero check	float division prints inf (garbage for the user); with int operands the program CRASHES at run time
default case	a mistyped operator such as % or x is silently ignored: nothing is printed and the user cannot tell whether the program failed

Given day, month and year, decide whether it is a legal calendar date. Reject 31-04-2024 and 29-02-2023, accept 29-02-2024 (combine the leap-year rule with days-per-month using switch).

```
#include <stdio.h>
int main() {
    int day = 29, month = 2, year = 2023, maxDays;
    int leap = (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
    switch (month) {
        case 1: case 3: case 5: case 7: case 8: case 10: case 12:
            maxDays = 31; break;
        case 4: case 6: case 9: case 11:
            maxDays = 30; break;
        case 2:
            maxDays = leap ? 29 : 28; break;
        default:
            maxDays = 0;              // invalid month
    }
    if (day >= 1 && day <= maxDays)
        printf("%02d-%02d-%d is a valid date", day, month, year);
    else
        printf("%02d-%02d-%d is NOT a valid date", day, month, year);
    return 0;
}
```

Output: 29-02-2023 is NOT a valid date

Date	31-04-2024	29-02-2024	29-02-2023	31-12-2025	15-13-2024
Result	invalid	valid	invalid	valid	invalid

Practice Problems — Try It Yourself

1. Write a switch-based program to convert a month number (1–12) into its name.
2. Write a switch-based 4-function calculator (like the earlier if-based one), making sure every case correctly ends in a break.
3. Write your own short switch statement with one case deliberately missing its break, then trace through it by hand and write down exactly what it prints — swap with a classmate and check each other's reasoning.
4. The fall-through trap. Predict the exact output, then explain "fall-through" in your own words: int x = 2; switch (x) { case 1: printf("One "); case 2: printf("Two "); case 3: printf("Three "); break; default: printf("Other "); } Rewrite it so only "Two" prints. What does x = 1 print?

5. Extend the temperature converter with options 4 = F to K and 5 = K to C, and make sure invalid choices are handled by default.
6. Read a date (dd mm yyyy) and print the NEXT day's date, handling month-end and year-end. Reuse the days-in-month logic of G3.3.

Debugging Corner — Find the Bug (Solved)

DEBUG 1

For day = 3 this program prints "WednesdayThursdayInvalid" all together. Find the bug and fix it.

```
#include <stdio.h>
int main() {
    int day = 3;
    switch (day) {
        case 1: printf("Monday");
        case 2: printf("Tuesday");
        case 3: printf("Wednesday");
        case 4: printf("Thursday");
        default: printf("Invalid");
    }
    return 0;
}
```

Output of the buggy program: WednesdayThursdayInvalid

What is wrong?

• **Missing break statements.** Once case 3 matches, execution falls through case 4 and default. Every case needs its own break (the last one is optional).

Corrected code:

```
#include <stdio.h>
int main() {
    int day = 3;
    switch (day) {
        case 1: printf("Monday");    break;
        case 2: printf("Tuesday");   break;
        case 3: printf("Wednesday"); break;
        case 4: printf("Thursday");  break;
        default: printf("Invalid");
    }
    return 0;
}
```

Output: Wednesday

DEBUG 2

A student cannot understand why a switch on a float will not even compile. Identify the rule being violated and rewrite the logic correctly with if-else.

```
#include <stdio.h>
int main() {
    float x = 2.5;
    switch (x) {
        case 2.5:
            printf("Two point five");
            break;
        default:
            printf("Other");
    }
    return 0;
}
```

Compiler says: switch quantity not an integer

What is wrong?

• **The switch expression and every case label must be integer values** (int, char, enum). A float can neither be the switch value nor a case label. Use if-else for real numbers.

Corrected code:

```
#include <stdio.h>
int main() {
    float x = 2.5;
    if (x == 2.5)
        printf("Two point five");
    else
        printf("Other");
    return 0;
}
```

Output: Two point five

Comparing floats with == is only safe for values that are exactly representable (like 2.5). For results of calculations, compare with a small tolerance instead.

Debug It Yourself — Practice (answers in Appendix A)

DEBUG IT 1

The compiler reports a duplicate case value.

```
#include <stdio.h>
int main() {
    int n = 2;
    switch (n) {
        case 1: printf("One");        break;
        case 2: printf("Two");        break;
        case 2: printf("Second two"); break;
    }
    return 0;
}
```

Hint: *Can two cases share the same value?*

DEBUG IT 2

Why is a variable not accepted as a case label?

```
#include <stdio.h>
int main() {
   int limit = 5, n = 5;
   switch (n) {
      case limit: printf("At limit"); break;
      default:    printf("Other");
   }
   return 0;
}
```

Hint: *Is limit a constant known while compiling?*

16.8 if-else-if Ladder vs switch — Which One to Use?

> **Analogy:** Choosing between a bus and a metro depends on the route. Choosing between a ladder and a switch depends on the kind of condition.

> **Technical Concept:** Use switch for exact matches of an integer or char against fixed constants; use the if-else-if ladder for ranges, floating-point values and compound conditions.

"When would you use switch instead of if-else-if?" is one of the most frequently asked conceptual questions in placement interviews and vivas. The table below summarises the key differences.

Aspect	if-else-if Ladder	switch Statement
Works with	Any data type; complex, multi-variable conditions	Only int/char/enum constants, matched against ONE variable
Range checks (e.g. marks ≥ 90)	Yes, naturally	No — needs a workaround (e.g. dividing into bands)
Combining conditions (&&, ||)	Yes, freely	No — only one expression is compared per switch
Readability with many fixed options	Can get long and repetitive	Very clean for menus, day names, grade letters, etc.
Risk of a common bug	Dangling else with nested ifs	Forgetting a break causes fall-through

Placement Answer: Use switch when comparing ONE variable against many exact, fixed

Aspect	if-else-if Ladder	switch Statement
values (like a menu or a single character). Use if-else-if when you need range checks, multiple variables, or conditions combined with && / ||.		

Solved Examples — Choosing Between Ladder and switch

Compare nested if-else and switch-case. Give one situation where switch is clearly the better choice and one where it cannot be used at all even though if-else can.

Switch is better when ONE variable is compared with several fixed integer/character values, such as a menu option:

```
#include <stdio.h>
int main() {
    int option = 2;
    switch (option) {
        case 1: printf("Balance enquiry");  break;
        case 2: printf("Cash withdrawal");  break;
        case 3: printf("Mini statement");   break;
        case 4: printf("Exit");             break;
        default: printf("Invalid option");
    }
    return 0;
}
```

Output: Cash withdrawal

Switch is impossible when the condition is a range, involves real numbers or combines several variables. The check "bill ≥ 1000" is not an exact value, and the bill is a float:

```
#include <stdio.h>
int main() {
    float bill = 1250.75;
    if (bill >= 1000)
        printf("10%% discount");
    else if (bill >= 500)
        printf("5%% discount");
    else
        printf("No discount");
    return 0;
}
```

Output: 10% discount

The deciding question is the TYPE of the condition, not the number of cases: exact integer/char matches → switch; ranges, floats, or conditions on more than one variable → if-else.

Practice Problems — Try It Yourself

1. For each situation decide switch or if-else-if and justify: (a) month number → month name, (b) BMI category, (c) vowel test on a character, (d) ticket price by age band, (e) a menu with 5 options.
2. Convert the ATM-menu switch above into an if-else-if ladder, then count how many comparisons the ladder makes for option 4.
3. Rewrite your grade ladder with switch using the marks / 10 trick. What goes wrong for marks = 100 and for marks = 105?

Debugging Corner — Find the Bug (Solved)

DEBUG 1

A student who scored exactly 100 is marked "Fail".

```
#include <stdio.h>
int main() {
    int marks = 100;
    switch (marks / 10) {
        case 9: printf("Grade A"); break;
        case 8: printf("Grade B"); break;
        case 7: printf("Grade C"); break;
        default: printf("Fail");
    }
    return 0;
}
```

Output of the buggy program: Fail

What is wrong?

• **100 / 10 = 10, and there is no case 10.** The band trick must include the top value: case 10 shares the block with case 9.

Corrected code:

```
#include <stdio.h>
int main() {
    int marks = 100;
    switch (marks / 10) {
        case 10:
        case 9: printf("Grade A"); break;
        case 8: printf("Grade B"); break;
        case 7: printf("Grade C"); break;
        default: printf("Fail");
    }
    return 0;
}
```

Output: Grade A

DEBUG 2

For n = 5 the program prints more than the default message.

```
#include <stdio.h>
int main() {
    int n = 5;
    switch (n) {
        default: printf("Invalid ");
        case 1:  printf("One");   break;
        case 2:  printf("Two");   break;
    }
    return 0;
}
```

Output of the buggy program: Invalid One

What is wrong?

• **default may appear anywhere, and it also falls through.** Here default has no break, so execution continues into case 1. Put default last, or end it with break.

Corrected code:

```
#include <stdio.h>
int main() {
    int n = 5;
    switch (n) {
        case 1:  printf("One");   break;
        case 2:  printf("Two");   break;
        default: printf("Invalid ");
    }
    return 0;
}
```

Output: Invalid

Debug It Yourself — Practice (answers in Appendix A)

DEBUG IT 1

A student with 95 marks gets no grade.

```
#include <stdio.h>
int main() {
    int marks = 95;
    switch (marks) {
        case 90: printf("Grade A"); break;
        case 80: printf("Grade B"); break;
        default: printf("No grade");
    }
    return 0;
}
```

Hint: *Does switch match ranges?*

DEBUG IT 2

Several values should share one message, but the compiler rejects this.

```
#include <stdio.h>
int main() {
    int n = 2;
    switch (n) {
        case 1, 2: printf("Low");  break;
        default:   printf("Other");
    }
    return 0;
}
```

Hint: *How do you stack several labels on one block?*

16.9 Chapter Summary — Placement Tips

- The most common interview question style for this topic is "predict the output" or "find the bug" — practice tracing through nested if and switch code by hand, one line at a time.
- Remember: an if-else-if ladder always stops at the FIRST true condition; independent if statements always check EVERY condition, even after one has already matched.
- Always use braces {} with nested if-else, even for single statements, to avoid the dangling-else trap.
- Always double-check every case in a switch ends with break, unless the fall-through is deliberate and intentional.
- Slab-based, menu-based, and eligibility-check problems (billing, tax, loans, tickets) are the favourite real-world scenario for this topic — practice identifying which construct (if, if-else, nested if, ladder, or switch) best fits a given scenario before you start coding.

16.10 Important Questions

Q1. Differentiate between the if-else-if ladder and switch.

The ladder can test ranges, floating-point values and compound conditions and stops at the first true condition. switch compares one integer or char expression with constant cases and jumps directly to the match, so it is cleaner for fixed menus.

Q2. Why is break used in a switch?

Without break, execution falls through into the following cases until a break or the end of the switch is reached. break stops after the matching case.

Q3. What is the dangling-else problem?

An else always attaches to the nearest unmatched if, which may not be the one the programmer intended. Using braces makes the pairing explicit.

Q4. When should nested if be used instead of &&?

Use nested if when the inner test only makes sense after the outer test passes, for example validating input before classifying it. Use && when all conditions are independent and equally important.

Q5. Which data types can a switch expression have?

Only integer types and char (constant case labels). float, double, strings and ranges are not allowed.

16.11 Multiple Choice Questions

These 20 questions follow the style of real placement aptitude/coding rounds — mostly "predict the output" and classic gotcha questions on if, if-else, nested if, ladders, and switch. Attempt each one before checking the answer.

1. What happens if the condition in an if statement is false and there is no else?

- a) Compile error
- b) The program crashes
- c) Control simply moves to the statement after the if block
- d) The block executes anyway

Answer: c) Control simply moves to the statement after the if block

2. In an if-else-if ladder, how many blocks can execute during a single run?

- a) All matching blocks
- b) Only the first one whose condition is true
- c) Only the last one
- d) None, ever

Answer: b) Only the first one whose condition is true

3. What is the output of: int x=5; if(x=10) printf("A"); else printf("B");

- a) A
- b) B
- c) Compile Error
- d) Infinite loop

Answer: a) A (x=10 is an ASSIGNMENT, not a comparison; it sets x to 10 and the condition becomes true — the classic = vs == trap)

4. Which statement correctly describes a "dangling else" in C?

- a) else always belongs to the outermost if
- b) else belongs to the nearest unmatched if
- c) else can be omitted from any if statement freely
- d) else must always be paired with braces

Answer: b) else belongs to the nearest unmatched if

5. Which of the following is TRUE about the switch statement in C?

- a) It can switch on float values
- b) It can test ranges directly, like case 1...10
- c) It compares int/char values against exact constants
- d) It never needs break statements

Answer: c) It compares int/char values against exact constants

6. What happens when a break statement is missing in a switch case?

- a) Compile error
- b) Execution falls through into the next case
- c) The switch exits immediately
- d) The default case runs instead

Answer: b) Execution falls through into the next case

7. Which construct is best suited for checking multiple RANGES of a variable (like BMI categories)?

- a) switch
- b) if-else-if ladder
- c) Nested switch
- d) None of the above

Answer: b) if-else-if ladder

8. In a nested if statement, when does the inner if actually get evaluated?

- a) Always, regardless of the outer condition
- b) Only when the outer if's condition is true
- c) Only when the outer if's condition is false
- d) At the same time as the outer if

Answer: b) Only when the outer if's condition is true

9. What is the output of:

```
int a=5;
if(a>0)
   if(a>10) printf("X");
   else printf("Y");
```

- a) X

- b) Y
- c) Nothing prints
- d) Compile Error

Answer: b) Y (the else binds to the nearest if, a>10, which is false since a=5)

10. Which value is treated as "false" in a C condition?

- a) 0
- b) 1
- c) -1
- d) Any negative number

Answer: a) 0

11. What is the correct behaviour of the default case in a switch statement?

- a) It always executes first
- b) It executes only if no other case matches
- c) It must always be written as the last case
- d) It is mandatory in every switch

Answer: b) It executes only if no other case matches

12. Predict the output:

```
int day=8;
switch(day){
  case 1: printf("Mon");break;
  default: printf("Invalid");break;
  case 2: printf("Tue");break;
}
```

- a) Mon
- b) Tue
- c) Invalid
- d) Compile Error

Answer: c) Invalid (default does not need to be written last — it still only runs when nothing else matches, here day=8)

13. Which of the following achieves the same logical result as checking voting eligibility (age 18+ AND citizen)?

- a) if(age>=18) OR if(isCitizen)
- b) if(age>=18 && isCitizen)
- c) if(age>=18) { if(isCitizen) ... }
- d) Both (b) and (c)

Answer: d) Both (b) and (c)

14. Which statement about ordering conditions in an if-else-if ladder is TRUE?

- a) Order never matters
- b) Conditions should generally go from most specific to least specific
- c) The final else is always mandatory
- d) Only one else-if is allowed per ladder

Answer: b) Conditions should generally go from most specific to least specific

15. What is the result of:

```
char grade='A';
switch(grade){
  case 'A':
  case 'B': printf("Pass"); break;
  case 'C': printf("Average");break;
  default: printf("Fail");
}
```

- a) Pass
- b) Average
- c) Fail
- d) Compile error

Answer: a) Pass (case 'A' has no break, so it falls through and shares case 'B''s block)

16. Which of these is the correct syntax for an if statement in C?

- a) if condition {}
- b) if (condition) {}
- c) if [condition] {}
- d) if <condition> {}

Answer: b) if (condition) {}

17. What is the key difference between a plain if and an if-else?

- a) if can hold multiple conditions, if-else cannot
- b) if-else guarantees exactly one of its two blocks always runs; a plain if may run nothing at all
- c) if-else compiles faster
- d) There is no real difference

Answer: b) if-else guarantees exactly one of its two blocks always runs; a plain if may run nothing at all

18. Which is the cleanest way to handle a menu with 6 fixed numeric choices?

- a) Nested if
- b) if-else-if ladder
- c) switch statement
- d) A single if statement

Answer: c) switch statement

19. What happens when multiple case labels in a switch share no break between them (intentionally)?

- a) Compile error
- b) Only the first matching case executes and stops
- c) All the grouped cases share the same block of code
- d) The program crashes

Answer: c) All the grouped cases share the same block of code

20. Predict the final output:

```
int n=-5;
if(n>=0){
   if(n==0) printf("Zero");
   else printf("Positive");
} else printf("Negative");
```

- a) Zero
- b) Positive
- c) Negative
- d) Nothing

Answer: c) Negative

16.12 Solved Programs

Program 1: Grade with an if-else-if Ladder

Problem: Print the grade for a mark using the ranges 90, 75, 60 and 40.

Approach: Test from the highest range downwards; the first true condition wins.

```
#include <stdio.h>
int main() {
   int marks = 76;
   if (marks >= 90)      printf("Grade A+\n");
   else if (marks >= 75) printf("Grade A\n");
   else if (marks >= 60) printf("Grade B\n");
   else if (marks >= 40) printf("Grade C\n");
   else                  printf("Fail\n");
   return 0;
}
```

Output:

```
Grade A
```

Program 2: Menu Calculator with switch

Problem: Apply the operator chosen by a character to two integers.

Approach: Each case handles one operator; default catches invalid input.

```
#include <stdio.h>
int main() {
   int a = 12, b = 4;
   char op = '*';
   switch (op) {
      case '+': printf("%d + %d = %d\n", a, b, a + b); break;
      case '-': printf("%d - %d = %d\n", a, b, a - b); break;
      case '*': printf("%d * %d = %d\n", a, b, a * b); break;
      case '/': printf("%d / %d = %d\n", a, b, a / b); break;
      default:  printf("Invalid operator\n");
   }
   return 0;
}
```

Output:

```
12 * 4 = 48
```

16.13 Practice Problems — Try It Yourself

More problems, at Medium and HOTS level, covering if, if-else, nested if, if-else-if ladders, and switch. Try to first decide which construct best fits each scenario before writing any code.

Medium Level

1. A smart irrigation system should turn on the water pump only if soil moisture is below 30%. Write the condition using a plain if.
2. Write a program to determine whether a given number is a multiple of 4, printing an appropriate message either way, using if-else.
3. Write an if-else-if ladder to categorise a person's internet speed: Slow (<2 Mbps), Average (2–10 Mbps), Fast (10–50 Mbps), Very Fast (>50 Mbps).
4. Write a switch-based program to display the number of days in a given month (1–12), treating February as a special case (assume a non-leap year, 28 days).
5. Write a program to check whether a given string's length represents a valid Indian mobile number (must be exactly 10 digits) — just validate the length using if-else for now.
6. A job application system requires the candidate's age to be between 21 and 35. Using nested if, first check the age range, then (only if it passes) check whether they have at least 2 years of experience, printing a specific message at each stage.

HOTS Level

1. An airline check-in system decides baggage fees: Business class gets 2 free bags; within Economy, loyalty members get 1 free bag and non-members get 0. Any bag beyond the free limit costs ₹500 each. Write a program (combining nested if and a ladder) to calculate the total baggage fee for a given number of bags.
2. Write a switch-based program (using intentional fall-through) to determine the number of days in a given month — group all 31-day months together, all 30-day months together, and handle February separately.
3. A cab-booking app applies surge pricing based on the ratio of active riders to available drivers: ratio < 1 → normal price, 1–2 → 1.5x, 2–3 → 2x, above 3 → 3x. Write an if-else-if ladder to compute the final fare given a base fare and the current ratio.

4. Write a switch-based program to assign a fitness category to a person's running pace (minutes per km) by first converting the pace into a "band" number, using the same band-reduction technique shown in section 16.7.
5. A parking garage charges: the first hour free, the next 2 hours at ₹20/hour, and beyond 3 hours at a cheaper long-stay rate of ₹15/hour. If the vehicle is electric, apply a flat 20% discount on the total. Write a complete program combining an if-else-if ladder with a final discount check.
6. The following code has a dangling-else bug: it is MEANT to print "A is not positive" whenever a is not greater than 0, regardless of b — but as written, the else actually binds to the wrong if. Rewrite it correctly using braces, and explain how the output changes for a=-2, b=5:

```
if(a>0)
  if(b>0)
    printf("Both positive");
else
  printf("A is not positive");
```

PART VI

Functions, Pointers and Recursion

Unit 2: how a large problem is broken into functions, how data travels between them by value and by reference, how pointers work, and how a function can call itself.

Chapter 17
Decomposition — Breaking a Problem into Sub-tasks

Decomposition is the process of breaking a large or complex problem into smaller, simpler sub-tasks. Instead of writing one long program, each sub-task becomes a separate **function** that does exactly one job.

The main idea: **Large problem → Smaller tasks → Functions → Complete solution.**

Divide a problem into sub-tasks when…	Benefit
It contains several different operations	Each operation gets its own function
The program is becoming large or hard to read	Easier to understand
The same operation is needed more than once	Write once, call many times (reuse)
Different parts can be solved independently	Parts can be tested and debugged separately

Common mistake: *putting input, calculation and output all inside main(). The program works, but it cannot be reused, tested piece by piece, or read easily. A good function does one job and has a name that says what that job is.*

17.1 Theory — Understand It First

The Big Idea

Analogy: A restaurant kitchen does not have one cook doing everything. One person chops, one cooks, one plates, one handles billing. Each station has one clear job, can be trained and checked on its own, and the head chef (main) only decides the order in which stations are called. If the soup is salty you know exactly which station to blame.

Technical Concept: **Decomposition** is the process of breaking a large, complex problem into smaller, simpler, well-defined sub-problems. In C each sub-problem is implemented as a **function**, and main() only coordinates them. Decomposition is the practical form of the **divide-and-conquer** approach to problem solving.

Where It Is Used: Every real application: a banking app splits into login, balance enquiry, fund transfer and statement modules; an e-commerce site splits into search, cart, payment and delivery tracking; a game splits into input, physics, rendering and scoring.

> **Why We Use It:** A human can keep only a handful of ideas in mind at once. Small functions fit in the head, can be written by different people, tested one by one, reused in other programs, and replaced without touching the rest.

Top-Down Design and Stepwise Refinement

In **top-down design** we start with the whole problem at the top and keep refining it into smaller steps until every step is simple enough to code as one function. This repeated splitting is called **stepwise refinement**. The result is a tree (a *structure chart*).

```
Student Scorecard                  <- Level 0 (whole problem)
 |
 +-- Read marks                    <- Level 1
 +-- Calculate total
 +-- Calculate average
 +-- Calculate grade
 +-- Display scorecard
      |
      +-- print total / average / grade  <- Level 2 (if still complex)
```

Stop refining when a sub-task can be written in a few lines, has a single clear purpose, and its inputs and outputs are obvious.

Method: How to Decompose Any Problem

Use this checklist for every exam or interview problem:

Step	Action	Scorecard example
1	Understand the problem; write inputs and final output	In: 5 marks. Out: total, average, grade
2	List the sub-tasks in the order they happen	read → total → average → grade → display
3	For every sub-task write Input – Process – Output (IPO)	See the IPO table below
4	Decide the function type: does it need data in? does it return a result?	total: args + return; display: args, no return
5	Choose parameter and return types	average → float; grade → char
6	Write and test each function alone with sample values	calculateTotal(80,70,90,85,75) = 400
7	Join the functions in main() and test the whole program	Expected: 400, 80.00, B

Function	Input	Process	Output (return)
readMarks	array, n	scanf in a loop	none (fills array)
calculateTotal	array, n	add all marks	int total
calculateAverage	total, n	total / n	float average
calculateGrade	average	if–else ladder	char grade
displayScorecard	total, average, grade	printf	none

Qualities of a Good Module — Cohesion and Coupling

Term	Meaning	Aim	Good / bad example
Cohesion	How closely the statements inside one function belong together	High	Good: calculateTotal() only adds marks. Bad: one function that reads, calculates and prints.
Coupling	How much one function depends on the others (shared globals, assumptions)	Low	Good: data passed through parameters. Bad: functions communicating through global variables.

- **Single Responsibility Principle:** one function = one job.
- Name a function with a **verb** that says the job: calculateTotal, isPrime, displayBill.
- Keep functions short (about 10–20 lines is a good target).
- Return results instead of printing inside calculation functions, so they can be reused.
- Avoid global variables; pass data through parameters.

Monolithic vs Modular Programs

Aspect	Monolithic (everything in main)	Modular (decomposed)
Structure	One long block of code	Many small functions, short main()
Readability	Hard to follow	main() reads like an outline
Repeated work	Code is copy-pasted	Function written once, called many times
Debugging	Bug could be anywhere	Bug confined to one function
Testing	Only the whole program	Each function separately
Team work	Hard to split	Each member writes some functions
Maintenance	A change touches many places	A change is made in one function

Related terms: ***Abstraction*** *— use a function by what it does without knowing how (you call printf without reading its code).* ***Modularity*** *— organising a program into independent units.* ***Reusability*** *— writing a function once and using it in many programs.*

17.2 Solved Examples and Debugging

Solved Examples — Easy

Define decomposition. State four situations in which a problem should be divided into smaller sub-tasks.

Decomposition breaks a large problem into smaller, well-defined sub-tasks, each solved by a separate function. A problem should be decomposed when (1) it has multiple different operations, (2) the program has become large or difficult to understand, (3) the same operation must be performed more than once, (4) different parts can be solved independently, or (5) we want the program to be easy to test, debug and maintain.

Decompose "calculate the area and perimeter of a rectangle and display the result" into sub-tasks and write the program.

Sub-tasks: (1) input length and breadth, (2) calculate area, (3) calculate perimeter, (4) display results. Each calculation is a function that receives values and returns a result; the display function only prints.

```
#include <stdio.h>
float calcArea(float l, float b)      { return l * b; }
float calcPerimeter(float l, float b) { return 2 * (l + b); }
void display(float area, float peri)
{
   printf("Area = %.2f\n", area);
   printf("Perimeter = %.2f", peri);
}
int main()
{
   float l = 8, b = 5;
   display(calcArea(l, b), calcPerimeter(l, b));
   return 0;
}
```

Output:

```
Area = 40.00
Perimeter = 26.00
```

Solved Examples — Medium

Decompose the problem "calculate an electricity bill from the units consumed" into functions. For each function state whether it takes arguments and whether it returns a value.

The bill needs the units, a slab calculation, optional extra charges and a printed bill. A function receives arguments only when it needs data from the caller, and returns a value only when the caller needs a result back.

Sub-task	Function	Arguments?	Returns?
Read units consumed	int readUnits(void)	No	Yes — int units
Calculate bill amount	float calculateBill(int units)	Yes	Yes — float bill
Add extra charges	float calculateCharges(float bill)	Yes	Yes — float total
Display the bill	void displayBill(int units, float bill)	Yes	No (void)

A student scores 80, 70, 90, 85 and 75. Decompose the scorecard problem into functions and trace the output. (Grade: 90+ → A, 75–89 → B, 60–74 → C, 50–59 → D, else F.)

readMarks → calculateTotal → calculateAverage → calculateGrade → displayScorecard. Total = 400, Average = 400 / 5 = 80, and 80 lies in 75–89, so the grade is B. In this version the marks are fixed in main() so the output is predictable.

```
#include <stdio.h>
int calculateTotal(int marks[], int n)
{
   int i, total = 0;
   for (i = 0; i < n; i++)
      total = total + marks[i];
   return total;
}
float calculateAverage(int total, int n)
{
   return (float)total / n;
}
char calculateGrade(float average)
{
   if (average >= 90) return 'A';
   else if (average >= 75) return 'B';
   else if (average >= 60) return 'C';
   else if (average >= 50) return 'D';
   else return 'F';
}
void displayScorecard(int total, float average, char grade)
{
   printf("Total   = %d\n", total);
   printf("Average = %.2f\n", average);
   printf("Grade   = %c", grade);
}
int main()
{
   int marks[5] = {80, 70, 90, 85, 75};
   int total = calculateTotal(marks, 5);
   float average = calculateAverage(total, 5);
   char grade = calculateGrade(average);
   displayScorecard(total, average, grade);
   return 0;
}
```

Output:

```
Total   = 400
Average = 80.00
Grade   = B
```

Why (float)total / n? If both operands are int, C performs integer division and drops the decimal part. The cast makes the division a floating-point division.

Solved Examples — HOTS

A program calculates "price + 18% GST" for three products by repeating the same five lines of code three times. Redesign it using decomposition and explain the benefit.

The repeated operation becomes one function, withGST(price), called three times. If the GST rate changes, only one line changes; with copy-pasted code, three places must be edited and one is easy to miss.

```
#include <stdio.h>
float withGST(float price)
{
   return price + price * 18 / 100;
}
int main()
{
   printf("Rs. %.2f\n", withGST(500));
   printf("Rs. %.2f\n", withGST(1200));
   printf("Rs. %.2f", withGST(250));
   return 0;
}
```

Output:

```
Rs. 590.00
Rs. 1416.00
Rs. 295.00
```

Library fine: ₹2 per day for the first 10 days late and ₹5 per day after that. Decompose the problem, choose a suitable function type for each sub-task, and write the program for a book due on day 5 and returned on day 19.

Sub-tasks: find days late (args + return), calculate fine (args + return), display (args, no return). Late = 19 − 5 = 14 days. Fine = 10 × 2 + 4 × 5 = 40.

```
#include <stdio.h>
int daysLate(int due, int returned)
{
   return returned - due;
}
float calculateFine(int late)
{
   if (late <= 0)  return 0;
   if (late <= 10) return late * 2;
   return 10 * 2 + (late - 10) * 5;
}
void displayFine(int late, float fine)
{
   printf("Days late = %d, Fine = Rs. %.2f\n", late, fine);
}
int main()
{
   int late = daysLate(5, 19);
   displayFine(late, calculateFine(late));
   displayFine(3, calculateFine(3));
   return 0;
}
```

Output:

```
Days late = 14, Fine = Rs. 40.00
Days late = 3, Fine = Rs. 6.00
```

Debugging Corner — Find the Bug (Solved)

DEBUG 1

The total of the three marks should be 245, but the program prints 0. Why?

```
#include <stdio.h>
int calculateTotal(int a, int b, int c)
{
   return a + b + c;
}
int main()
{
   int total = 0;
   calculateTotal(80, 75, 90);
   printf("Total = %d", total);
   return 0;
}
```

Output of the buggy program: Total = 0

What is wrong?

- **The returned value is thrown away.** calculateTotal() does return 245, but nobody stores it. total keeps its old value 0. A returned value must be assigned (or used) by the caller.

Corrected code:

```
#include <stdio.h>
int calculateTotal(int a, int b, int c)
{
   return a + b + c;
}
int main()
{
   int total = 0;
   total = calculateTotal(80, 75, 90);
   printf("Total = %d", total);
   return 0;
}
```

Output:

```
Total = 245
```

DEBUG 2

The average of the marks (total 245 for 3 subjects) should be 81.67, but 81.00 is printed. Find the bug.

```
#include <stdio.h>
float calculateAverage(int total, int n)
{
   return total / n;
}
int main()
{
   printf("Average = %.2f", calculateAverage(245, 3));
   return 0;
}
```

Output of the buggy program: Average = 81.00

What is wrong?

- **Integer division.** total and n are both int, so 245 / 3 is computed as 81 first, and only then converted to float. Cast one operand: (float)total / n.

Corrected code:

```
#include <stdio.h>
float calculateAverage(int total, int n)
{
   return (float)total / n;
}
int main()
{
   printf("Average = %.2f", calculateAverage(245, 3));
   return 0;
}
```

Output:

```
Average = 81.67
```

Debug It Yourself — Practice (answers in Appendix B)

DEBUG IT 1

This program does not even compile. What does the compiler complain about?

```
#include <stdio.h>
void calculateTotal(int a, int b, int c)
{
   printf("Adding...\n");
}
int main()
{
   int total;
   total = calculateTotal(80, 75, 90);
   printf("Total = %d", total);
   return 0;
}
```

Hint: *What does a void function hand back to its caller?*

DEBUG IT 2

For an average of 85 the program prints Grade = C. Why?

```
#include <stdio.h>
char findGrade(float average)
{
   if (average >= 60) return 'C';
   else if (average >= 75) return 'B';
   else if (average >= 90) return 'A';
   else return 'F';
}
int main()
{
   printf("Grade = %c", findGrade(85));
   return 0;
}
```

Hint: *An if-else ladder stops at the first condition that is true. Which one is true first for 85?*

17.3 Important Questions

Exam Questions — Model Answers (2 / 5 / 10 marks)

Q1. Define decomposition. (2 marks)

Decomposition is the process of breaking a large, complex problem into smaller, simpler sub-tasks, each of which is solved by a separate function.

Q2. List four advantages of modular programming. (2 marks)

- Improves readability.
- Simplifies debugging and testing.
- Promotes code reuse.
- Reduces complexity and eases maintenance.

Q3. Differentiate between monolithic and modular programming. (5 marks)

Monolithic code puts all statements in main(); it is long, hard to read, repeats code and is hard to debug. Modular code divides the work into functions; main() becomes a short outline, each function is tested separately, and code is reused. (Write the comparison table of aspects: structure, readability, repetition, debugging, testing, maintenance.)

Q4. Explain top-down design with the example of an electricity bill. (5 marks)

- Level 0: Electricity bill.
- Level 1 (refinement): read units → calculate bill amount → add surcharge → display bill.
- Each Level-1 step becomes a function: readUnits(), calculateBill(), calculateCharges(), displayBill().
- main() calls them in order. If the tariff changes only calculateBill() is edited.

Q5. Decompose the problem "simple ATM: opening balance ₹5000, deposit ₹700, withdraw ₹1200" into functions and write the program. (10 marks)

Sub-tasks: deposit (args + return new balance), withdraw (args + return new balance, only if balance is enough), display (args, no return). main() chains them.

```
#include <stdio.h>
float deposit(float balance, float amount)
{
   return balance + amount;
}
float withdraw(float balance, float amount)
{
   if (amount > balance)
   {
      printf("Insufficient balance\n");
      return balance;
   }
   return balance - amount;
}
void display(float balance)
{
   printf("Balance = Rs. %.2f\n", balance);
}
int main()
{
   float balance = 5000;
   balance = deposit(balance, 700);
   balance = withdraw(balance, 1200);
   display(balance);
   balance = withdraw(balance, 9000);
   display(balance);
   return 0;
}
```

Output:

```
Balance = Rs. 4500.00
Insufficient balance
Balance = Rs. 4500.00
```

Interview and Placement Questions — Short Answers

Q6. What are cohesion and coupling? Which should be high and which low?

Cohesion is how strongly the statements inside one function belong together; coupling is how much functions depend on each other. Aim for high cohesion and low coupling.

Q7. How do you decide how big a function should be?

One function, one job. If you cannot describe it in one short sentence without using "and", split it. A practical size is a screenful (10–20 lines).

Q8. Why should calculation functions return values instead of printing them?

A function that returns a value can be reused (printed, stored, compared, passed on) and unit-tested. A function that prints is tied to one output format.

Q9. How would you test a program that has been decomposed?

Test each function alone with normal and boundary values (for example 0, 1, negative, maximum), then test the integration in main(). This is the basis of unit testing.

Q10. Why is a very long main() considered bad design?

It mixes input, logic and output, repeats code, cannot be reused and makes bugs hard to locate.

17.4 Multiple Choice Questions

1. Decomposition means

- a) breaking a large problem into smaller sub-tasks
- b) converting a program into machine language
- c) writing the whole program inside main()
- d) removing all comments from a program

Answer: a) breaking a large problem into smaller sub-tasks (Each sub-task is then solved by its own function.)

2. In C, each sub-task obtained by decomposition is normally implemented as a

- a) macro constant
- b) header file
- c) comment
- d) function

Answer: d) function (A function performs one specific task.)

3. Which of the following is NOT a reason to decompose a problem?

- a) Different parts can be solved independently
- b) The program contains only one printf() statement
- c) The same operation is needed more than once
- d) The program has become large and hard to read

Answer: b) The program contains only one printf() statement (A one-line program has nothing to break down.)

4. In the student scorecard problem, the most suitable return type for the function that finds the grade letter is

- a) void
- b) int
- c) char
- d) float

Answer: c) char (A grade such as 'A' or 'B' is a single character.)

5. The Single Responsibility Principle says that each function should

- a) do exactly one job and do it well
- b) read input, calculate and print together
- c) contain at least three loops
- d) never return a value

Answer: a) do exactly one job and do it well (Small single-purpose functions are easier to test and reuse.)

17.5 Solved Programs

Program 1: Simple Interest with Three Functions

Problem: Calculate and display simple interest using separate functions for calculation and display.

Approach: calcInterest() only calculates and returns; display() only prints; main() coordinates.

```
#include <stdio.h>
float calcInterest(float p, float r, float t) { return p * r * t / 100; }
void display(float p, float interest) {
    printf("Principal = %.2f\n", p);
    printf("Interest  = %.2f\n", interest);
    printf("Total     = %.2f\n", p + interest);
```

```
}
int main() {
    float p = 5000, r = 8, t = 3;
    display(p, calcInterest(p, r, t));
    return 0;
}
```

Output:

```
Principal = 5000.00
Interest  = 1200.00
Total     = 6200.00
```

Program 2: Marks Processing

Problem: Decompose 'total, average and grade of three marks' into functions.

Approach: One function per sub-task, each with the most suitable return type.

```
#include <stdio.h>
int calcTotal(int a, int b, int c) { return a + b + c; }
float calcAverage(int total, int n) { return (float)total / n; }
char calcGrade(float avg) {
    if (avg >= 75) return 'A';
    else if (avg >= 50) return 'B';
    return 'C';
}
int main() {
    int total = calcTotal(72, 68, 80);
    float avg = calcAverage(total, 3);
    printf("Total = %d, Average = %.2f, Grade = %c\n", total, avg, calcGrade(avg));
    return 0;
}
```

Output:

```
Total = 220, Average = 73.33, Grade = B
```

17.6 Practice Problems — Try It Yourself

Attempt these without looking at the solved examples. Write the program, run it, and compare the output with your hand-trace.

1. Decompose the problem “withdraw cash from an ATM” into at least five sub-tasks. Name a function for each and say whether it needs arguments and a return value.
2. Decompose “simple calculator (add, subtract, multiply, divide two numbers)” into functions. Write only the prototypes.
3. An online food order has an item total, 5% GST, and a delivery fee of ₹40 if the item total is below ₹300. Decompose it and write a modular program.
4. Write a program to find the area and circumference of a circle using two separate functions and one display function.
5. A 60-line program reads marks, finds the grade and prints the result, all inside main(). List three signs that it should be decomposed.
6. For marks 60, 72, 88, 91 and 79, trace the scorecard program of EASY/MEDIUM above by hand and predict the total, average and grade.

Chapter 18
Functions in C — Declaration, Definition, Call and the Four Types

A **function** is a block of code designed to perform one particular task. Every function involves three parts: a **declaration (prototype)** that tells the compiler the name, return type and parameter types; a **definition** that contains the body; and a **call** that transfers control to it.

Depending on whether data goes in (arguments) and whether a result comes back (return value), functions fall into four types:

Type	Example prototype	Data in?	Result back?
1. No arguments, no return value	void display(void);	No	No
2. Arguments, no return value	void add(int, int);	Yes	No
3. Arguments, return value	int add(int, int);	Yes	Yes
4. No arguments, return value	int add(void);	No	Yes

Common mistake: *the prototype and the definition must agree exactly on the return type and the parameter types. If they differ, the compiler reports "conflicting types".*

18.1 Theory — Understand It First

The Big Idea

Analogy: A vending machine is a function. You give it inputs (coins and a selection), it does its internal work without you seeing how, and it gives back one result (the snack). You can use it again and again, and you only need to know **what** it needs and **what** it gives, not how the motor works.

Technical Concept: A **function** is a named block of statements that performs one specific task. It may receive values (**arguments**), and may send back one value (**return value**). Every C program has at least one function, main(), where execution begins.

Where It Is Used: Everywhere. printf(), scanf(), sqrt(), strlen() are functions someone else wrote. Large programs such as operating systems and compilers are millions of lines of small functions.

Why We Use It: To avoid repeating code, to divide a big task into small ones, to make programs readable, to test small parts independently, and to share code between programs.

Library Functions vs User-Defined Functions

	Library (built-in) function	User-defined function
Written by	Compiler / standard library authors	The programmer
Declared in	A header file (stdio.h, math.h, string.h)	Your own program, before main()
Definition needed?	No — already compiled in the library	Yes — you write the body
Examples	printf, scanf, sqrt, strlen, toupper	add, isPrime, calculateGrade

The Three Parts: Declaration, Definition, Call

Part	Syntax	Purpose
Declaration (prototype)	return_type name(param_types);	Tells the compiler the name, return type and parameter types. Ends with a semicolon. Parameter names are optional.
Definition	return_type name(params) { body }	Contains the actual statements. Allocated code; written once.
Call	name(arguments);	Transfers control to the function. May be used inside an expression.

```
#include <stdio.h>
int maximum(int, int);          /* declaration: names optional */

int main()
{
   int m = maximum(25, 18);     /* call */
   printf("Maximum = %d", m);
   return 0;
}

int maximum(int a, int b)       /* definition */
{
   if (a > b)
      return a;
   return b;
}
```

Output:

```
Maximum = 25
```

- If the definition appears **before** the first call, a separate prototype is not required — but putting prototypes at the top is good practice.
- A function **cannot be defined inside another function** in standard C.
- A function can be called any number of times and from any function (including itself — recursion).

What Happens During a Function Call

```
main()                          add(int a, int b)
 |                              |
 | result = add(10, 20);                |
 |--(1) evaluate arguments 10, 20       |
 |--(2) copy them into a, b ------------->|
 |--(3) control jumps to the function ---->|  (4) body runs: a + b
 |<---------(5) return value 30 -----------|
 | result = 30   (6) control resumes here  |
```

Each call creates a fresh set of parameters and local variables in a block of stack memory (a **stack frame**). When the function returns, that frame is destroyed and the caller continues from the statement after the call.

The Four Types of Functions in Detail

Type	Data flow	Use when	Drawback
1. No args, no return void f(void)	Only control moves	Fixed task: menu, banner, a demo that reads and prints inside	Not reusable; tied to fixed data
2. Args, no return void f(int, int)	Caller → function (one way)	Task only needs to show / use values: display, print table	Caller gets nothing back
3. Args, return int f(int, int)	Caller ⇄ function (two way)	Calculations: total, average, isPrime, max	Only one value returned
4. No args, return int f(void)	Function → caller (one way)	Function produces data itself: read a number, generate a value	Hidden input; harder to test

Type 3 is the most useful and most common in exams and interviews because it is reusable and testable.

The return Statement and main()

- **return expr;** sends one value to the caller and ends the function immediately.
- In a void function use **return;** (no value) or simply let the function reach the closing brace.
- The value is converted to the declared return type (return 3.9 from an int function gives 3).
- If a non-void function ends without return, using its result is **undefined behaviour**. (main() is the one exception: it returns 0 automatically in C99 and later.)
- **int main(void)** is the correct standard form. main() returns 0 to the operating system to mean "success".

Declaration	Meaning in C
int f(void)	f takes **no** parameters; any call with arguments is an error
int f()	f takes an **unspecified** number of parameters (old-style); the compiler does not check the arguments

18.2 Solved Examples and Debugging

Solved Examples — Easy

What is a function in C? What is a function prototype, and why is it written before main()?

A function is a block of statements designed to perform a specific task. A prototype declares the function's name, return type and parameter types before it is called, so the compiler can check every call (number and type of arguments, use of the result). Here add() is defined after main(), yet the call is valid because the prototype comes first.

```
#include <stdio.h>
int add(int, int);            /* declaration (prototype) */
int main()
{
   int result = add(10, 20);   /* call */
   printf("result = %d", result);
   return 0;
}
int add(int a, int b)          /* definition */
{
   return a + b;
}
```

Output:

```
result = 30
```

Name the four types of functions in C. Identify the type of: void display() { printf("Hello"); }

The four types are: (1) no arguments and no return value, (2) arguments and no return value, (3) arguments and a return value, (4) no arguments and a return value. display() takes nothing and returns nothing (its return type is void), so it is **type 1 — no arguments, no return value**. Only transfer of control takes place; no data is exchanged.

```
#include <stdio.h>
void display(void)
{
   printf("Hello");
}
int main()
{
   display();
   return 0;
}
```

Output:

```
Hello
```

Solved Examples — Medium

Write the "add two numbers" program four times — once for each type of function.

Same task, four designs. Type 1 reads/prints inside the function; type 2 receives data and prints; type 3 receives data and returns the sum; type 4 gets its data itself and returns the sum.

```
#include <stdio.h>
void add1(void)          { int a = 4, b = 5; printf("Type 1: %d\n", a + b); }
void add2(int x, int y)   { printf("Type 2: %d\n", x + y); }
int  add3(int x, int y)   { return x + y; }
int  add4(void)          { int a = 4, b = 5; return a + b; }
int main()
{
   add1();
   add2(4, 5);
   printf("Type 3: %d\n", add3(4, 5));
   printf("Type 4: %d", add4());
   return 0;
}
```

Output:

```
Type 1: 9
Type 2: 9
Type 3: 9
Type 4: 9
```

Choose the most suitable function type and write the prototype for each task: (a) print a welcome banner, (b) compute the area of a circle from its radius, (c) read and return the user's age, (d) print the multiplication table of n.

Ask two questions: does the function need data from the caller? does the caller need a result back?

Task	Needs input?	Returns result?	Prototype
(a) print a welcome banner	No	No	void banner(void);
(b) area of a circle	Yes (radius)	Yes (area)	float circleArea(float r);
(c) read and return age	No	Yes (age)	int readAge(void);
(d) print multiplication table	Yes (n)	No (only prints)	void printTable(int n);

Solved Examples — HOTS

Predict the output and explain how control moves between the functions.

Control goes main → first → second, and each function returns to the statement just after the call that invoked it. The sequence of prints therefore reflects the call/return order, not the order in which the functions are written.

```
#include <stdio.h>
void second(void) { printf("S "); }
void first(void)
{
   printf("F1 ");
   second();
   printf("F2 ");
}
int main()
{
   printf("M1 ");
   first();
   printf("M2");
   return 0;
}
```

Output:

```
M1 F1 S F2 M2
```

The program below has the function defined after main(). Why does it compile and run only because of the prototype on the first line? What happens if the prototype is removed?

The prototype tells the compiler that square takes one int and returns an int, before it meets the call. Without it, the compiler sees square(7) with no declaration. Old C (C89) silently assumed "returns int"; modern C (C99 onwards, and recent GCC versions by default) reports an error. Always declare the function before using it.

```
#include <stdio.h>
int square(int);
int main()
{
   printf("%d", square(7));
   return 0;
}
int square(int n)
{
   return n * n;
}
```

Output:

```
49
```

Debugging Corner — Find the Bug (Solved)

DEBUG 1

The program fails to compile with "conflicting types for 'add'". Why?

```
#include <stdio.h>
int add(int, int);
int main()
{
   printf("%d", add(2, 3));
   return 0;
}
float add(float a, float b)
{
   return a + b;
}
```

Output of the buggy program: Compilation error (the program does not build)

What is wrong?

- **Prototype and definition disagree.** The prototype says int add(int, int) but the definition says float add(float, float). They must match in return type and parameter types.

Corrected code:

```
#include <stdio.h>
int add(int, int);
int main()
{
   printf("%d", add(2, 3));
   return 0;
}
int add(int a, int b)
{
   return a + b;
}
```

Output:

```
5
```

DEBUG 2

The message "Squared!" was supposed to be printed, but it never appears. Why?

```
#include <stdio.h>
int square(int n)
{
   return n * n;
   printf("Squared! ");
}
int main()
{
   printf("%d", square(4));
   return 0;
}
```

Output of the buggy program: 16

What is wrong?

- **Code after return is unreachable.** return ends the function immediately, so the printf after it never runs. Put the printf before the return.

Corrected code:

```
#include <stdio.h>
int square(int n)
{
   printf("Squared! ");
   return n * n;
}
int main()
{
   printf("%d", square(4));
   return 0;
}
```

Output:

```
Squared! 16
```

Debug It Yourself — Practice (answers in Appendix B)

DEBUG IT 1

The compiler stops with "void value not ignored as it ought to be". Which line is wrong and why?

```
#include <stdio.h>
void display(void)
{
   printf("Hello");
}
int main()
{
   int r = display();
   return 0;
}
```

Hint: *What does a function whose return type is void give back?*

DEBUG IT 2

The call add(5) is rejected by the compiler. What is wrong?

```
#include <stdio.h>
int add(int a, int b)
{
   return a + b;
}
int main()
{
   printf("%d", add(5));
   return 0;
}
```

Hint: *Count the parameters in the definition and the arguments in the call.*

18.3 Important Questions

Exam Questions — Model Answers (2 / 5 / 10 marks)

Q1. What is a function? What is a function prototype? (2 marks)

A function is a block of statements designed to perform a specific task. A function prototype is a declaration that tells the compiler the function's name, return type and parameter types before the function is called.

Q2. Differentiate between function declaration and function definition. (2 marks)

A declaration (prototype) only gives the name, return type and parameter types and ends with a semicolon; a definition also contains the body of statements and allocates the code.

Q3. Explain the four types of functions in C with a suitable example of each. (5 marks)

- No arguments, no return: void display(void) { printf("Hello"); }
- Arguments, no return: void add(int a, int b) { printf("%d", a + b); }
- Arguments, return value: int add(int a, int b) { return a + b; }
- No arguments, return value: int readNum(void) { int n; scanf("%d", &n); return n; }
- (Give one sentence on the data flow of each: none, one way to function, two way, one way from function.)

Q4. Explain how a function call works in C. (5 marks)

- The arguments are evaluated.
- Their values are copied into the parameters, which are created in a new stack frame.
- Control is transferred to the first statement of the function.
- The body executes until return or the closing brace.
- The return value (if any) is passed back, the frame is destroyed and control resumes after the call.

Q5. Write a program with a function for each of the four types to find the sum of two numbers, and state the type of each. (10 marks)

See Chapter 18 the first Medium example for the complete program. The functions are add1(void) [type 1], add2(int, int) [type 2], add3(int, int) [type 3] and add4(void) [type 4].

```
#include <stdio.h>
void add1(void)        { int a = 4, b = 5; printf("Type 1: %d\n", a + b); }
void add2(int x, int y) { printf("Type 2: %d\n", x + y); }
int  add3(int x, int y) { return x + y; }
int  add4(void)        { int a = 4, b = 5; return a + b; }
int main()
{
   add1();
   add2(4, 5);
   printf("Type 3: %d\n", add3(4, 5));
   printf("Type 4: %d", add4());
   return 0;
}
```

Output:

```
Type 1: 9
Type 2: 9
Type 3: 9
Type 4: 9
```

Interview and Placement Questions — Short Answers

Q6. What is the difference between int f() and int f(void) in C?

f(void) means no parameters and the compiler rejects arguments. f() means the parameters are unspecified and the compiler does not check them — a leftover from old C. Prefer f(void).

Q7. Can we define one function inside another function in C?

No. Standard C has no nested function definitions (GCC has an extension, but it is not portable). Define functions at file level.

Q8. Why does main() return int? What does return 0 mean?

The return value is the program's exit status given to the operating system. 0 means success; non-zero means an error.

Q9. What happens if you call a function before it is declared?

Old C compilers assumed it returned int (implicit declaration). Modern compilers (C99 onwards) give an error or warning. Always declare it first.

Q10. Can a function return more than one value?

Not directly with return. Use pointer parameters, return a struct, or return a pointer to an array.

Q11. Can a function return an array?

No. A function cannot return an array type; it can return a pointer to one (never to a local array).

18.4 Multiple Choice Questions

1. A function prototype declares

- a) the output of the function
- b) the name, return type and parameter types before the function is called
- c) only the body of the function
- d) only the variables used inside main()

Answer: b) the name, return type and parameter types before the function is called (It lets the compiler check every call.)

2. How many types of functions are there, based on arguments and return values?

- a) Three
- b) Two
- c) Five
- d) Four

Answer: d) Four (No/with arguments × no/with return value = 4 combinations.)

3. void display(void) is a function with

- a) no arguments and a return value
- b) arguments and a return value
- c) no arguments and no return value
- d) arguments and no return value

Answer: c) no arguments and no return value (Both the parameter list and the return type are void.)

4. int readAge(void) is a function with

- a) arguments and a return value
- b) no arguments and no return value
- c) arguments and no return value

- d) no arguments and a return value

Answer: d) no arguments and a return value (It takes nothing in, but sends an int back.)

5. Which keyword is used as the return type of a function that returns nothing?

- a) none
- b) null
- c) void
- d) empty

Answer: c) void (void means "no value".)

6. Which is the correct prototype of a function that takes two integers and returns a float?

- a) int calc(float, float);
- b) float calc(int, int);
- c) void calc(int, int, float);
- d) calc(int, int) float;

Answer: b) float calc(int, int); (The return type comes first, then the name and parameter types.)

7. In C, int f() and int f(void) differ because

- a) f() returns void while f(void) returns int
- b) f(void) cannot be called
- c) f() leaves the parameters unspecified while f(void) means no parameters
- d) they are identical in every way

Answer: c) f() leaves the parameters unspecified while f(void) means no parameters (Only f(void) makes the compiler reject any arguments.)

18.5 Solved Programs

Program 1: Larger of Two Numbers

Problem: Write a function that returns the larger of two integers.

Approach: The function has two parameters and returns an int.

```
#include <stdio.h>
int larger(int a, int b) { return (a > b) ? a : b; }
int main() {
    printf("Larger of 17 and 25 = %d\n", larger(17, 25));
```

```
    return 0;
}
```

Output:

```
Larger of 17 and 25 = 25
```

Program 2: A void Function and a Value-Returning Function

Problem: Show one function with no result and one that returns a result.

Approach: greet() takes nothing and returns nothing; square() takes an int and returns an int.

```
#include <stdio.h>
void greet(void) { printf("Welcome to functions!\n"); }
int square(int n) { return n * n; }
int main() {
    greet();
    printf("Square of 9 = %d\n", square(9));
    return 0;
}
```

Output:

```
Welcome to functions!
Square of 9 = 81
```

18.6 Practice Problems — Try It Yourself

Attempt these without looking at the solved examples. Write the program, run it, and compare the output with your hand-trace.

1. Write a type-1 function that prints “Welcome to SRM” three times, and call it from main().
2. Write a type-2 function that receives two integers and prints the larger one.
3. Write a type-3 function cube() that receives an integer and returns its cube. Print cube(3).
4. Write a type-4 function that reads an integer from the keyboard and returns it. In main(), print the square of the value it returns.
5. Classify each prototype into one of the four types: void greet(void); int max(int, int); float readPrice(void); void printGrade(char); double power(double, int);
6. Explain the difference between declaring and defining a function. Rewrite the program of the first Easy example so that no prototype is needed.

Chapter 19
Parameters vs Arguments and Return Values

Parameters and arguments are the two ends of the same data transfer. Parameters belong to the function **definition**; arguments belong to the function **call**.

	Parameter (formal)	Argument (actual)
Where it appears	In the function definition / prototype	In the function call
What it is	A variable that receives a value	The actual value or variable that is passed
Example	int multiply(int a, int b) → a, b	multiply(5, 4) → 5, 4
Matched by	Position (first with first, second with second)	Position — names do not have to match

Common mistake: *assuming that variables with the same name in main() and in a function are the same variable. They are completely separate local variables; only the values travel between them.*

19.1 Theory — Understand It First

The Big Idea

Analogy: A form has labelled boxes (Name, Age). The labels are the **parameters**; what you write in the boxes is the **arguments**. The labels stay the same for everyone; the filled-in values change every time.

Technical Concept: **Parameters (formal parameters)** are the variables named in the function definition; they receive values. **Arguments (actual parameters)** are the values or expressions supplied in the function call. At the call, each argument is copied into the matching parameter, **by position**.

Where It Is Used: Any function that works on different data each time: sqrt(25), printf("%d", n), isPrime(17), calculateBill(units).

Why We Use It: Parameters make a function general. Instead of one function for "square of 5", we write square(n) once and use it for any value.

Parameters vs Arguments — Rules of Matching

- **Number** — a call must supply exactly as many arguments as there are parameters (when a prototype is present).
- **Order** — matched by position: first argument → first parameter.
- **Type** — each argument is converted to the parameter's type (float 3.9 passed to an int parameter becomes 3).
- **Names** — argument names and parameter names are independent; they may be the same or different.
- An argument can be a **constant** (5), a **variable** (x), an **expression** (x + 2) or **another function call** (square(3)).

```
#include <stdio.h>
void show(int n)
{
    printf("n = %d\n", n);
}
int main()
{
    int x = 4;
    show(10);          /* constant   */
    show(x);           /* variable   */
    show(x * 2 + 1);   /* expression */
    show(3.9);         /* double converted to int -> 3 */
    return 0;
}
```

Output:

```
n = 10
n = 4
n = 9
n = 3
```

*The order in which the arguments of one call are evaluated is **unspecified** in C. Never write calls such as f(i++, i++) or f(i, i++) — the result is undefined and can differ between compilers.*

Parameters Are Local Variables

A parameter behaves like a local variable that is **initialised with the argument's value** when the function starts. It exists only during the call. Its name may match a variable in the caller — they are still two different variables in different places in memory.

Parameter (formal)	Argument (actual)
In the definition / prototype	In the call
A variable that receives the value	A value, variable or expression that is sent
Created when the function is called; destroyed on return	Exists in the caller as usual
Its type must be given	Its type is whatever the expression has
Cannot be redeclared with the same name in the function's outermost block	No such restriction

Return Values

- The **return type** in the header says what kind of value comes back: int, float, char, … or void for nothing.
- The returned value is **converted** to the return type (an int function returning 5.5 gives 5).
- The caller can **store** it (r = f(x);), **print** it (printf("%d", f(x));), **use it in an expression** (f(x) + 1) or **pass it on** (g(f(x))).
- Ignoring the return value is legal, but the result is lost (printf itself returns the number of characters printed, and we usually ignore it).
- Only **one** value can be returned directly. To send back several, use pointer parameters (Chapter 22) or a structure.

Technique	How many results	Example
return statement	1	int add(int a, int b)
Pointer (reference) parameters	Any number	void minMax(int a, int b, int *min, int *max)
Returning a structure	Many, grouped	struct Result f(...) (Unit on structures)

C Does Not Have Default Arguments or Overloading

Unlike C++, C has no default argument values and no function overloading. Two functions cannot share a name, and every call must supply every argument. Variable argument lists exist only in special functions such as printf (declared with "...").

19.2 Solved Examples and Debugging

Solved Examples — Easy

Differentiate between a parameter and an argument. Identify them in: int multiply(int a, int b) { return a * b; } called as multiply(x, y).

a and b are the **parameters** (formal parameters) — variables defined in the function header. x and y are the **arguments** (actual parameters) — the values supplied in the call. The value of x is copied into a, and y into b.

```
#include <stdio.h>
int multiply(int a, int b)       /* a, b : parameters */
{
   return a * b;
}
int main()
{
   int x = 5, y = 4;
   int result = multiply(x, y); /* x, y : arguments */
   printf("result = %d", result);
   return 0;
}
```

Output:

```
result = 20
```

What is a return value? Which statement returns it? How many values can a function return directly?

A return value is the result sent back from the called function to the calling function, using the **return** statement. return also ends the function immediately. A C function can directly return **only one** value. (To send back more, use pointer parameters — see Chapter 22.) The return type in the header (int, float, char, void) states what kind of value comes back.

```
#include <stdio.h>
int square(int n)
{
   return n * n;
}
int main()
{
   int num = 6, result;
   result = square(num);
   printf("Square = %d", result);
   return 0;
}
```

Output:

```
Square = 36
```

Solved Examples — Medium

Predict the output. What does it show about how arguments are matched to parameters?

Arguments are matched to parameters by **position**, not by name. sub(a, b) puts a into the first parameter, so swapping the arguments swaps the answer. The a and b inside sub() are different variables from the a and b in main().

```
#include <stdio.h>
int sub(int a, int b)
{
   return a - b;
}
int main()
{
   int a = 3, b = 10;
   printf("%d %d %d", sub(a, b), sub(b, a), sub(10, 3));
   return 0;
}
```

Output:

```
-7 7 7
```

Why must the return type match the value being returned? Show what happens when a float result is returned from an int function.

The return type decides how the result is converted. avgOK() returns float, so 5.5 survives. avgBad() computes 5.5 but its return type is int, so the value is **truncated** to 5 on the way out.

```
#include <stdio.h>
float avgOK(int a, int b)  { return (a + b) / 2.0; }
int   avgBad(int a, int b) { return (a + b) / 2.0; }
int main()
{
   printf("%.2f %d", avgOK(7, 4), avgBad(7, 4));
   return 0;
}
```

Output:

```
5.50 5
```

Solved Examples — HOTS

Predict the output. Is the value of a in main() changed by the function?

The parameter a inside change() is a separate local variable that received a copy of 5. Adding 10 changes only that copy. When the function ends, the copy is destroyed; main()'s a still holds 5. (Scope: a local variable exists only inside its own function.)

```
#include <stdio.h>
void change(int a)
{
   a = a + 10;
   printf("inside:  a = %d\n", a);
}
int main()
{
   int a = 5;
   change(a);
   printf("outside: a = %d", a);
   return 0;
}
```

Output:

```
inside:  a = 15
outside: a = 5
```

A function may contain several return statements. Write sign(n) that returns 1, −1 or 0, and explain which return runs.

Only the first return statement that is reached is executed; the function ends at that point. For n = 25 the first if is true, so 1 is returned and the rest is skipped. For 0, both conditions fail and the last return 0 runs.

```
#include <stdio.h>
int sign(int n)
{
   if (n > 0) return 1;
   if (n < 0) return -1;
   return 0;
}
int main()
{
   printf("%d %d %d", sign(25), sign(-8), sign(0));
   return 0;
}
```

Output:

```
1 -1 0
```

Predict the output of a program in which a function call is used as an argument of another function call.

The innermost calls are evaluated first: square(2) = 4 and square(3) = 9. Those return values become the arguments of add(), which returns 13. A returned value can be used anywhere a value of that type is allowed.

```
#include <stdio.h>
int square(int n) { return n * n; }
int add(int a, int b) { return a + b; }
int main()
{
   printf("%d", add(square(2), square(3)));
   return 0;
}
```

Output:

```
13
```

Debugging Corner — Find the Bug (Solved)

DEBUG 1

The compiler reports "'x' undeclared" inside addTen(). Why?

```
#include <stdio.h>
int addTen(int n)
{
   return n + x;
}
int main()
{
   int x = 10;
   printf("%d", addTen(3));
   return 0;
}
```

Output of the buggy program: Compilation error (the program does not build)

What is wrong?

- **x is local to main().** A local variable cannot be seen from another function. The value must be passed in as an argument (or the function should use only its own parameters).

Corrected code:

```
#include <stdio.h>
int addTen(int n, int x)
{
   return n + x;
}
int main()
{
   int x = 10;
   printf("%d", addTen(3, x));
   return 0;
}
```

Output:

```
13
```

DEBUG 2

Marks lost should be 40 (scored 60 out of 100) but the program prints −40. Why?

```
#include <stdio.h>
int marksLost(int total, int scored)
{
   return total - scored;
}
int main()
{
   int scored = 60, total = 100;
   printf("Marks lost = %d", marksLost(scored, total));
   return 0;
}
```

Output of the buggy program: Marks lost = -40

What is wrong?

- **Arguments are in the wrong order.** The first parameter is total, but the first argument passed is scored. Arguments are matched by position, so the call must be marksLost(total, scored).

Corrected code:

```
#include <stdio.h>
int marksLost(int total, int scored)
{
   return total - scored;
}
int main()
{
   int scored = 60, total = 100;
   printf("Marks lost = %d", marksLost(total, scored));
   return 0;
}
```

Output:

```
Marks lost = 40
```

Debug It Yourself — Practice (answers in Appendix B)

DEBUG IT 1

The area of a circle of radius 5 should be 78.50, but the program prints 78. Why?

```
#include <stdio.h>
int calculateArea(float radius)
{
   return 3.14 * radius * radius;
}
int main()
{
   printf("Area = %d", calculateArea(5));
   return 0;
}
```

Hint: *Look at the return type. What happens to the .50?*

DEBUG IT 2

The compiler complains about "too many arguments to function 'add'". Fix it.

```
#include <stdio.h>
int add(int a, int b)
{
   return a + b;
}
int main()
{
   printf("%d", add(2, 3, 4));
   return 0;
}
```

Hint: *Compare how many values the call supplies with how many parameters exist.*

19.3 Important Questions

Exam Questions — Model Answers (2 / 5 / 10 marks)

Q1. Differentiate between formal and actual parameters. (2 marks)

Formal parameters are the variables in the function definition that receive values. Actual parameters (arguments) are the values or expressions given in the function call.

Q2. How many values can a function return? How can more values be returned? (2 marks)

A function can return only one value with return. More values can be sent back through pointer parameters (or a structure).

Q3. Explain the rules for matching arguments with parameters, with an example. (5 marks)

- Number: equal count of arguments and parameters.
- Order: matched by position, not by name.
- Type: each argument is converted to the parameter's type.
- Example: int sub(int a, int b) { return a - b; } sub(10, 3) gives 7 but sub(3, 10) gives −7.

Q4. Explain the return statement with examples. (5 marks)

- return sends a value to the caller and terminates the function.
- A function may have several returns but only the first one reached runs.
- void functions use return; with no value.
- Example: int sign(int n) { if (n > 0) return 1; if (n < 0) return -1; return 0; }

Q5. Write a program with function calculate(a, b) that returns the larger number and show calls using a constant, a variable and an expression as arguments. (10 marks)

The prototype fixes int larger(int, int). Calls larger(5, 9), larger(x, y) and larger(x + 10, y * 2) show that an argument may be a constant, a variable or an expression.

```
#include <stdio.h>
int larger(int a, int b)
{
   return (a > b) ? a : b;
}
int main()
{
   int x = 4, y = 7;
   printf("%d\n", larger(5, 9));
   printf("%d\n", larger(x, y));
   printf("%d", larger(x + 10, y * 2));
   return 0;
}
```

Output:

```
9
7
14
```

Interview and Placement Questions — Short Answers

Q6. Can a parameter have the same name as a variable declared in the function's outermost block?

No — it is a redeclaration error, because parameters belong to the same scope as the outermost block. An inner block { } may reuse the name (it then shadows the parameter).

Q7. What happens if a float is passed to a function whose parameter is int?

With a prototype in scope the value is converted (truncated toward zero): 3.9 becomes 3.

Q8. What is the output of printf("%d %d", f(), g()) if both print something?

The order of evaluation of function arguments is unspecified, so g() may run before f(). Do not rely on it.

Q9. Does C support default arguments or function overloading?

No. Every call must pass all arguments and each function needs a unique name.

Q10. What happens if you pass fewer arguments than parameters?

With a prototype the compiler gives an error. Without a prototype (old style) the missing parameter holds a garbage value — undefined behaviour.

19.4 Multiple Choice Questions

1. In int multiply(int a, int b) called as multiply(5, 4), the values 5 and 4 are

- a) arguments (actual parameters)
- b) local variables of main()
- c) formal parameters
- d) return values

Answer: a) arguments (actual parameters) (Arguments appear in the call; parameters appear in the definition.)

2. Formal parameters are the variables

- a) written in the function definition that receive the values
- b) written in the function call
- c) declared only in main()
- d) that hold the return value in the caller

Answer: a) written in the function definition that receive the values (They stand for the values passed from the caller.)

3. The return statement

- a) repeats the function
- b) prints a value on the screen
- c) sends a value back to the caller and ends the function
- d) declares a new variable

Answer: c) sends a value back to the caller and ends the function (Statements after a return in the same path are never executed.)

4. How many values can a C function return directly using return?

- a) None
- b) Two
- c) Any number
- d) One

Answer: d) One (To send back more results, pointer parameters are used.)

5. What is printed? int sub(int a, int b){ return a - b; } printf("%d", sub(10, 3));

- a) 13
- b) 7
- c) -7
- d) 3

Answer: b) 7 (10 − 3 = 7; arguments are matched by position.)

6. float f(){ return 5 / 2; } printf("%.2f", f()); prints

- a) 2.00
- b) 2
- c) 2.50
- d) 0.00

Answer: a) 2.00 (5/2 is integer division (2), converted to 2.00.)

7. int add(int a, int b){ return a + b; } add(1, 2) * add(3, 4) =

- a) 10
- b) 7
- c) 21

- d) 12

Answer: c) 21 (3 × 7 = 21.)

19.5 Solved Programs

Program 1: Celsius to Fahrenheit

Problem: Convert a temperature using a function with a float parameter and return value.

Approach: The argument 37 is copied into the parameter c; the result is returned to main().

```
#include <stdio.h>
float toFahrenheit(float c) { return c * 9 / 5 + 32; }
int main() {
    printf("37 C = %.2f F\n", toFahrenheit(37));
    return 0;
}
```

Output:

```
37 C = 98.60 F
```

Program 2: Average of Three Values

Problem: Return the exact average of three integers.

Approach: Dividing by 3.0 keeps the decimal part of the result.

```
#include <stdio.h>
float average3(int a, int b, int c) { return (a + b + c) / 3.0; }
int main() {
    printf("Average = %.2f\n", average3(10, 20, 25));
    return 0;
}
```

Output:

```
Average = 18.33
```

19.6 Practice Problems — Try It Yourself

Attempt these without looking at the solved examples. Write the program, run it, and compare the output with your hand-trace.

1. For float simpleInterest(float p, float r, float t) called as simpleInterest(5000, 5, 2), name the parameters, the arguments, the return type and the value returned.
2. Write a function perimeter(int l, int b) that returns the perimeter. Call it using variables whose names are different from the parameter names.
3. Predict the output: int sub(int a, int b){ return a - b; } printf("%d", sub(sub(10, 4), sub(9, 7)));
4. Write a function findLargest(int a, int b, int c) that returns the largest of three numbers. Test it with 15, 42 and 27.
5. Can a function use return to send back two values at the same time? Explain, and name the technique that overcomes this limit.
6. Write a function celsiusToFahrenheit(float c) that returns a float, and print the result for 37 °C.

Chapter 20
Pass by Value

In **pass by value** (call by value), a **copy** of the argument's value is placed in the parameter. The function works on the copy, so whatever it does to the parameter is local to the function and disappears when it returns. The original variable in the caller is never changed.

C passes every argument by value by default. Use it whenever the function only needs to *look at* the value (square, interest, isPrime, grade, average).

Feature	Pass by value
What is passed	A copy of the value
Parameter declared as	Normal variable, e.g. int n
Call looks like	square(x)
Can the function change the caller's variable?	No
Lifetime of changes	Only inside the called function

Common mistake: *expecting void inc(int n) { n++; } to increase the caller's variable. It increases only its own copy.*

20.1 Theory — Understand It First

The Big Idea

Analogy: You hand a **photocopy** of your assignment to a friend. They may scribble on it, cross out answers, even tear it up — your original is safe in your bag. Pass by value is exactly that: the function gets a copy of the value.

Technical Concept: In **pass by value (call by value)** the value of each argument is copied into the corresponding parameter. The parameter is a separate variable in a separate memory location. Any change to the parameter affects only the copy.

Where It Is Used: It is C's **default** method for every argument: square(x), isPrime(n), calculateBill(units), sqrt(x). Use it whenever the function only needs to read the value.

Why We Use It: It is safe — a function cannot accidentally damage the caller's data — and simple: the caller can pass constants and expressions, not only variables.

What Happens in Memory

```
main():  int x = 5;              square(int n):  n = n * n;

  address 1000         address 2000
 +---------+          +---------+
 | x = 5   |  -copy-> | n = 5   |   after n = n * n :  n = 25
 +---------+          +---------+
 (caller)             (callee)      x is STILL 5 : different memory cell

 When square() returns, the cell at 2000 (n) is destroyed.
```

*The addresses 1000 and 2000 are illustrative; real addresses are different on every machine. The point is that x and n live at **different** addresses.*

```
#include <stdio.h>
void show(int n)
{
   n = n + 100;
   printf("Inside : n = %d\n", n);
}
int main()
{
   int x = 5;
   show(x);
   printf("Outside: x = %d", x);
   return 0;
}
```

Output:

```
Inside : n = 105
Outside: x = 5
```

Step-by-Step Flow

- The call is reached; each argument expression is evaluated to a value.
- A new stack frame is created; each parameter is created and initialised with its value.
- The function body runs using only the copies.
- return sends the result back; the stack frame (and all copies) disappear.

- The caller's original variables are exactly as they were.

Advantages and Limitations

Advantages	Limitations
Original data cannot be corrupted (no side effects)	The function cannot modify the caller's variables
Arguments may be constants or expressions	Only one value can be returned through return
Easy to understand, trace and test	Copying large data (big structures) costs time and memory

Rule of thumb: if the function only needs to *look at* a value (square, interest, isPrime, grade, average), use pass by value. If it must *change* the caller's variable or return several results, pass the address (Chapter 22).

Getting a changed value back with pass by value: return it and assign it — x = doubleIt(x); This is the safe alternative when only one value must change.

C Passes Everything by Value — Even Pointers and Arrays

C has only pass by value. When we pass an address (a pointer), the **address** is what is copied; the function can then use that address to reach the caller's data. This is how "pass by reference" is simulated (Chapter 22). An array name passed to a function becomes a pointer to its first element, so the function can modify the original array elements.

20.2 Solved Examples and Debugging

Solved Examples — Easy

What is pass by value? What will the program print?

The value 5 of x is copied into n. Squaring n changes only the copy, so x is still 5 after the call.

```
#include <stdio.h>
void square(int n)
{
   n = n * n;
   printf("Square = %d\n", n);
}
int main()
{
   int x = 5;
   square(x);
   printf("Original value = %d", x);
   return 0;
}
```

Output:

```
Square = 25
Original value = 5
```

Write a program to find simple interest using pass by value. (p = 5000, r = 5, t = 2)

The three values p, r and t are copied into the parameters. The function only reads them and returns the result: SI = p × r × t / 100.

```
#include <stdio.h>
float interest(float p, float r, float t)
{
   return (p * r * t) / 100;
}
int main()
{
   float p = 5000, r = 5, t = 2;
   float result = interest(p, r, t);
   printf("Simple Interest = %.2f", result);
   return 0;
}
```

Output:

```
Simple Interest = 500.00
```

Solved Examples — Medium

Write a function to double a number using pass by value, and show that the original number is unchanged.

doubleValue() doubles its own copy and returns it. The caller receives the result through the return value; its own n is untouched.

```
#include <stdio.h>
int doubleValue(int n)
{
   n = n * 2;
   return n;
}
int main()
{
   int n = 5;
   int r = doubleValue(n);
   printf("n = %d, result = %d", n, r);
   return 0;
}
```

Output:

```
n = 5, result = 10
```

Predict the output. How many times is the parameter modified, and what finally reaches main()?

x starts as 4 (a copy of a). x++ makes it 5, x *= 2 makes it 10, and 10 is returned. a stays 4; b receives 10.

```
#include <stdio.h>
int f(int x)
{
   x++;
   x *= 2;
   return x;
}
int main()
{
   int a = 4;
   int b = f(a);
   printf("%d %d", a, b);
   return 0;
}
```

Output:

```
4 10
```

Solved Examples — HOTS

A programmer writes swap(int a, int b) and calls swap(x, y). Trace the program and explain why x and y are not exchanged.

a and b are copies. The three-line swap really does exchange a and b (see the “inside” line), but those are the copies. x and y in main() are different memory locations, so they never change. Swapping the caller’s variables needs their addresses — pass by reference (Chapter 22).

```
#include <stdio.h>
void swap(int a, int b)
{
   int t = a;
   a = b;
   b = t;
   printf("Inside:  a = %d, b = %d\n", a, b);
}
int main()
{
   int x = 10, y = 20;
   swap(x, y);
   printf("Outside: x = %d, y = %d", x, y);
   return 0;
}
```

Output:

```
Inside:  a = 20, b = 10
Outside: x = 10, y = 20
```

Design a helper function isPrime(int n) using pass by value. Use it to print all prime numbers from 1 to 20. Why is pass by value the right choice here?

isPrime only has to examine n, never modify it, so a copy is enough. It returns 1 for prime and 0 otherwise. Numbers below 2 are not prime, and a number n is checked for divisors from 2 up to n/2.

```
#include <stdio.h>
int isPrime(int n)
{
    int i;
    if (n < 2)
        return 0;
    for (i = 2; i <= n / 2; i++)
    {
        if (n % i == 0)
            return 0;
    }
    return 1;
}
int main()
{
    int n;
    for (n = 1; n <= 20; n++)
        if (isPrime(n))
            printf("%d ", n);
    return 0;
}
```

Output:

```
2 3 5 7 11 13 17 19
```

Compute the sum of the first n even numbers with a function that receives n by value, and show that n in main() is unchanged even though the function counts n down to 0.

The ith even number is 2 × i. sumEven() consumes its own copy of n (n--), adding 2 × n each time, and returns the total: for n = 5, 10 + 8 + 6 + 4 + 2 = 30. main()'s n is still 5.

```
#include <stdio.h>
int sumEven(int n)
{
   int sum = 0;
   while (n > 0)
   {
      sum = sum + 2 * n;
      n--;
   }
   return sum;
}
int main()
{
   int n = 5;
   int result = sumEven(n);
   printf("Sum = %d, n after = %d", result, n);
   return 0;
}
```

Output:

```
Sum = 30, n after = 5
```

Debugging Corner — Find the Bug (Solved)

DEBUG 1

The salary should rise by 10% to 33000.00, but the program still prints 30000.00. Why?

```
#include <stdio.h>
void raise(float salary)
{
   salary = salary * 1.1;
}
int main()
{
   float salary = 30000;
   raise(salary);
   printf("Salary = %.2f", salary);
   return 0;
}
```

Output of the buggy program: Salary = 30000.00

What is wrong?

- **Pass by value changes only the copy.** The raised salary is lost when raise() ends. Either return the new value and assign it, or pass the address (pass by reference).

Corrected code:

```
#include <stdio.h>
float raise(float salary)
{
   return salary * 1.1;
}
int main()
{
   float salary = 30000;
   salary = raise(salary);
   printf("Salary = %.2f", salary);
   return 0;
}
```

Output:

```
Salary = 33000.00
```

DEBUG 2

This cube program does not compile. Find the two problems in the function.

```
#include <stdio.h>
int cube(int);
int main()
{
   int n = 5;
   printf("%d", cube(n));
   return 0;
}
int cube(int)
{
   return x * x * x;
}
```

Output of the buggy program: Compilation error (the program does not build)

What is wrong?

- **The parameter has no name.** In a definition, the parameter must be named (int x) so the body can use it.
- **x is undeclared.** Because the parameter was never named, x does not exist inside the function.

Corrected code:

```
#include <stdio.h>
int cube(int);
int main()
{
   int n = 5;
   printf("%d", cube(n));
   return 0;
}
int cube(int x)
{
   return x * x * x;
}
```

Output:

```
125
```

Debug It Yourself — Practice (answers in Appendix B)

DEBUG IT 1

The number should become 10, but 5 is printed. Where did the doubled value go?

```
#include <stdio.h>
void doubleIt(int n)
{
   n = n * 2;
}
int main()
{
   int num = 5;
   doubleIt(num);
   printf("%d", num);
   return 0;
}
```

Hint: *Which variable did n = n * 2 change?*

DEBUG IT 2

The user types 25, yet the program prints 0. Why?

```
#include <stdio.h>
void readNumber(int n)
{
   scanf("%d", &n);
}
int main()
{
   int num = 0;
   readNumber(num);
   printf("%d", num);
   return 0;
}
```

Hint: *scanf stores the value into n. Is n the same variable as num?*

20.3 Important Questions

Exam Questions — Model Answers (2 / 5 / 10 marks)

Q1. Define pass by value. (2 marks)

In pass by value a copy of the actual argument's value is passed to the formal parameter; changes made to the parameter do not affect the original variable.

Q2. Explain pass by value with an example and a memory diagram. (5 marks)

- Define it (copy of value is passed).
- Show void show(int n) { n = n + 100; } called as show(x) with x = 5.
- Draw x at one address holding 5 and n at another address holding a copy; n becomes 105 while x stays 5.
- State that the copy is destroyed when the function ends.

Q3. Why does a swap function written with pass by value not swap the caller's variables? (5 marks)

The function swaps its own local copies a and b; the caller's x and y are at different addresses and are never touched. To swap the originals the addresses must be passed (pass by reference).

Q4. List the advantages and limitations of pass by value. (5 marks)

- Advantages: safe from side effects, simple, accepts constants and expressions.
- Limitations: cannot modify the caller's variables, only one value returned, large data costs copying time.

Q5. Write a program with a function that tries to swap two numbers using pass by value. Show the output and explain it. Then give the corrected version. (10 marks)

The first version prints swapped values inside the function and unchanged values in main. The corrected version passes addresses.

```
#include <stdio.h>
void swapValue(int a, int b)
{
   int t = a;  a = b;  b = t;
   printf("Inside swapValue : %d %d\n", a, b);
}
void swapRef(int *a, int *b)
{
   int t = *a;  *a = *b;  *b = t;
}
int main()
{
   int x = 10, y = 20;
   swapValue(x, y);
   printf("After swapValue  : %d %d\n", x, y);
   swapRef(&x, &y);
   printf("After swapRef    : %d %d", x, y);
   return 0;
}
```

Output:

```
Inside swapValue : 20 10
After swapValue  : 10 20
After swapRef    : 20 10
```

Interview and Placement Questions — Short Answers

Q6. Is C pass by value or pass by reference?

Always pass by value. Pass by reference is simulated by passing the address of a variable (a pointer), and the pointer itself is passed by value.

Q7. Why does swap(a, b) not work, and how do you fix it?

The function swaps copies. Pass &a and &b and swap *p and *q.

Q8. How can a pass-by-value function give a changed value back?

Return it and assign: x = f(x);

Q9. Why is passing a large structure by value inefficient?

The whole structure is copied on every call. Passing a pointer copies only the address.

Q10. If you pass an array to a function, is it passed by value?

The array name decays to a pointer to its first element, and that pointer is passed by value; so the function can modify the original elements.

20.4 Multiple Choice Questions

1. In pass by value, the function receives

- a) a copy of the argument's value
- b) the original variable itself
- c) the address of the argument
- d) the name of the argument

Answer: a) a copy of the argument's value (The original variable is not accessible through the parameter.)

2. void change(int x){ x = 20; } int a = 10; change(a); printf("%d", a); prints

- a) Garbage value
- b) 0
- c) 20

- d) 10

Answer: d) 10 (Only the copy x was changed.)

3. Changes made to a formal parameter in pass by value are

- a) permanent in the caller
- b) rejected by the compiler
- c) local to the called function
- d) copied back automatically

Answer: c) local to the called function (They vanish when the function returns.)

4. Which function is best written using pass by value?

- a) void reset(int *counter)
- b) int isPrime(int n)
- c) void readInto(int *n)
- d) void swap(int *a, int *b)

Answer: b) int isPrime(int n) (isPrime only reads n; it never modifies it.)

5. What is printed? int f(int x){ x++; x *= 2; return x; } int a = 4; printf("%d %d", a, f(a));

- a) 5 10
- b) 4 10
- c) 10 10
- d) 4 5

Answer: b) 4 10 (a is unchanged (4); f(4) returns (4+1)×2 = 10.)

20.5 Solved Programs

Program 1: The Original Is Not Changed

Problem: Show that changing a parameter does not change the caller's variable.

Approach: The function works on a copy, so main()'s variable keeps its value.

```
#include <stdio.h>
void doubleIt(int n) {
    n = n * 2;
    printf("Inside  : %d\n", n);
}
```

```
int main() {
    int a = 7;
    doubleIt(a);
    printf("Outside : %d\n", a);
    return 0;
}
```

Output:

```
Inside  : 14
Outside : 7
```

Program 2: Getting the Change Back with return

Problem: Use the returned value to update a variable.

Approach: The function returns the new value and the caller stores it.

```
#include <stdio.h>
int increment(int n) { return n + 1; }
int main() {
    int x = 10;
    int y = increment(x);
    printf("x = %d, y = %d\n", x, y);
    x = increment(x);
    printf("x = %d\n", x);
    return 0;
}
```

Output:

```
x = 10, y = 11
x = 11
```

20.6 Practice Problems — Try It Yourself

Attempt these without looking at the solved examples. Write the program, run it, and compare the output with your hand-trace.

1. Predict the output: void inc(int n){ n++; } int a = 7; inc(a); inc(a); printf("%d", a);
2. Write a program with a function area(int l, int b) that returns the area of a rectangle using pass by value.

3. Write isEven(int n) that returns 1 or 0, and use it to print all even numbers from 1 to 15.
4. Write a function that receives a number by value and returns the sum of its digits. Show that the original number is unchanged.
5. Predict the output: int add(int a, int b){ a = a + b; return a; } int x = 3, y = 4; printf("%d %d %d", add(x, y), x, y);
6. Draw a memory diagram showing how the value of x is copied into n for square(x). Give two reasons why pass by value is the safe default.

Chapter 21
Pointers in C

A **pointer** is a special variable that stores the **memory address** of another variable instead of a direct value. Every variable occupies a memory location with an address; a pointer lets the program reach that location.

Symbol	Name	Meaning	For int x = 10; int *p = &x;
&	Address-of	Gives the memory address of a variable	&x → address of x
*	Dereference	Accesses the value stored at an address	*p → 10
int *p	Pointer declaration	p can hold the address of an int	p → address of x
p++	Pointer arithmetic	Moves to the next element of the same type	step size = sizeof(int)

Common mistake: *dereferencing a pointer that has not been given a valid address (uninitialised or NULL). Rule: never dereference an invalid or NULL pointer — check if (p != NULL) first.*

21.1 Theory — Understand It First

The Big Idea

Analogy: A locker has a **number** and holds something **inside**. A pointer is a slip of paper with the locker number written on it, not the item itself. With the slip you can go to the locker, look at the item, or replace it.

Technical Concept: A **pointer** is a variable whose value is the **memory address** of another variable. The pointer's type tells the compiler what kind of data lives at that address (int *p points to an int).

Where It Is Used: Pass by reference, arrays and strings, dynamic memory (malloc), data structures such as linked lists and trees, and hardware access. scanf("%d", &x) works only because it receives the address of x.

> **Why We Use It:** To let a function change the caller's variables, to return several results, to access array elements efficiently, to avoid copying large data, and to build structures whose size is decided at run time.

Memory Model and the Two Operators

Memory is a long row of byte-sized cells, each with an address. A variable occupies one or more consecutive cells (an int usually 4 bytes).

```
int x = 10;   int *p = &x;

  variable   address   value
  x          1000      10       <- p points here
  p          2000      1000     <- p stores the ADDRESS of x

&x  = 1000  (address of x)      *p = 10  (value at address stored in p)
```

Operator	Name	Meaning	With int x = 10; int *p = &x;
&	Address-of	Address of a variable	&x gives the address of x
*	Dereference (indirection)	Value at an address	*p gives 10; *p = 20 changes x
int *p	Pointer declaration	p can hold the address of an int	—

*The * has two jobs. In a declaration (int *p) it says "p is a pointer". In an expression (*p) it means "go to the address and get the value". Also *&x is the same as x.*

Declaring and Initialising Pointers

Statement	Meaning
int *p;	p is a pointer to int (currently holds garbage — a wild pointer)
int *p = &x;	declare and initialise with the address of x
int *p = NULL;	p points to nothing (safe initial value)
float *f; char *c;	pointers to float and char — each can hold addresses of that type only

Statement	Meaning
int* a, b;	**Trap:** only a is a pointer; b is an ordinary int. Write int *a, *b; for two pointers.

```
#include <stdio.h>
int main()
{
   int x = 10, y = 25;
   int *p = &x;
   printf("%d\n", *p);   /* 10 */
   *p = 50;              /* changes x */
   p = &y;               /* p now points to y */
   printf("%d %d", x, *p);
   return 0;
}
```

Output:

```
10
50 25
```

Size of Pointers and Pointer Types

Every pointer stores an address, so all pointers on a machine have the **same size** (typically 8 bytes on a 64-bit system, 4 bytes on a 32-bit system) no matter what they point to. The pointed-to type matters for *dereferencing* and *arithmetic*, not for the pointer's own size.

```
#include <stdio.h>
int main()
{
   printf("%d\n", sizeof(int *) == sizeof(char *));
   printf("%d\n", sizeof(int *) == sizeof(double *));
   printf("%d", sizeof(int));
   return 0;
}
```

Output:

```
1
1
4
```

Pointer Arithmetic

Adding 1 to a pointer moves it forward by **one element of the pointed type**, not one byte: p + 1 adds sizeof(*p) bytes.

Operation	Allowed?	Meaning
p + n, p - n, p++, p--	Yes	Move forward / backward n elements
p2 - p1	Yes	Number of elements between two pointers into the same array
p1 < p2, p1 == p2	Yes	Compare addresses
p1 + p2, p * 2, p / 2	No	Meaningless; compile error

```
#include <stdio.h>
int main()
{
    int a[6] = {10, 20, 30, 40, 50, 60};
    int *p = a;          /* points to a[0] */
    int *q = &a[4];      /* points to a[4] */
    printf("%d\n", *(p + 3));   /* 40 */
    printf("%d\n", q - p);      /* 4 elements apart */
    p = p + 2;
    printf("%d %d", *p, *(p - 1));
    return 0;
}
```

Output:

```
40
4
30 20
```

Pointers and Arrays

- The name of an array is (in most expressions) the **address of its first element**: a is the same as &a[0].
- a[i] is the same as *(a + i) — the compiler converts one into the other.
- An array name is a **constant address**: a++ is illegal, but p++ is fine for a pointer p.
- sizeof(a) gives the size of the whole array; sizeof(p) gives only the size of a pointer.

Array	Pointer
Fixed block of memory with its own storage	Single variable holding an address
Name cannot be reassigned (a = b is an error)	Can be reassigned (p = q)
sizeof(a) = total bytes of all elements	sizeof(p) = size of an address
Size fixed at declaration	Can point to any block, including malloc'd memory

```
#include <stdio.h>
int main()
{
    int a[5] = {1, 2, 3, 4, 5};
    int *p = a;
    printf("%d %d\n", sizeof(a), sizeof(a) / sizeof(a[0]));
    printf("%d\n", sizeof(p) == sizeof(void *));
    printf("%d %d %d", a[2], *(a + 2), p[2]);
    return 0;
}
```

Output:

```
20 5
1
3 3 3
```

Special Pointers: NULL, void * and the Dangerous Ones

Kind	Description	Danger / use
NULL pointer	Points to nothing; value 0	Dereferencing it crashes. Test with if (p != NULL).
void pointer	void *p — can hold the address of any type	Cannot be dereferenced or used in arithmetic without a cast. Used by malloc.
Wild pointer	Declared but never initialised	Holds a random address; writing through it is undefined behaviour.
Dangling pointer	Points to memory that no longer exists (a local variable after its function returned, or freed memory)	Using it is undefined behaviour. Set to NULL after free().

Pointer to Pointer and const with Pointers

A pointer to a pointer stores the address of another pointer: int **q = &p; then *q is p and **q is x. It is needed when a function must change *where a pointer points*.

Declaration	Can change the value (*p)?	Can change where p points?
int *p	Yes	Yes
const int *p (pointer to constant)	No	Yes
int *const p (constant pointer)	Yes	No
const int *const p	No	No

*Trick to read it: read right to left. "int *const p" = p is a constant pointer to int.*

Precedence Traps: *p++, (*p)++, *++p

Expression	Meaning	Effect
*p++	*(p++) — use the value, then move p	Value read; p advances
(*p)++	Increment the value p points to	Value changes; p stays
*++p	*(++p) — move p first, then read	p advances; new value read
++*p	Increment the value, then use it	Value changes

```
#include <stdio.h>
int main()
{
   int a[3] = {10, 20, 30};
   int *p = a;
   printf("%d ", *p++);   /* prints 10, p -> a[1] */
   printf("%d ", *p);     /* 20 */
   printf("%d ", *++p);   /* p -> a[2], prints 30 */
   (*p)++;              /* a[2] becomes 31 */
   printf("%d", a[2]);
   return 0;
}
```

Output:

```
10 20 30 31
```

Preview: Dynamic Memory with Pointers

malloc() reserves memory at run time and returns its address as a void *. The memory must be released with free() when finished, otherwise it is a **memory leak**.

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
   int n = 4, i, sum = 0;
   int *p = (int *)malloc(n * sizeof(int));
   if (p == NULL)
      return 1;
   for (i = 0; i < n; i++)
   {
      p[i] = (i + 1) * 10;
      sum = sum + p[i];
   }
   printf("Sum = %d", sum);
   free(p);
   p = NULL;
   return 0;
}
```

Output:

```
Sum = 100
```

21.2 Solved Examples and Debugging

Solved Examples — Easy

What is a pointer? What do the operators & and * do? Show them with int x = 10; int *p = &x;

p holds the address of x. &x is that address; *p is the value stored at the address in p, which is 10. Because p contains &x, the comparison p == &x is true (1). (To print an address use %p with a cast to void *; the actual number differs on every run, so it is not shown here.)

```
#include <stdio.h>
int main()
{
   int x = 10;
   int *p = &x;
   printf("x  = %d\n", x);
   printf("*p = %d\n", *p);
   printf("p == &x ? %d", p == &x);
   return 0;
}
```

Output:

```
x  = 10
*p = 10
p == &x ? 1
```

Predict the output. How can a pointer change a variable?

p points to x, so *p and x are the same memory location. Assigning to *p changes x itself.

```
#include <stdio.h>
int main()
{
   int x = 10;
   int *p = &x;
   printf("Before: x = %d\n", x);
   *p = 25;
   printf("After:  x = %d", x);
   return 0;
}
```

Output:

```
Before: x = 10
After:  x = 25
```

Solved Examples — Medium

Predict the output. Explain how pointer arithmetic moves through an array.

p starts at a[0] (10). p++ moves to the next int, a[1] (20). p += 2 moves two elements ahead to a[3] (40). *(p − 1) is then a[2] (30). The step is always one element of the pointed type, not one byte.

```
#include <stdio.h>
int main()
{
   int a[5] = {10, 20, 30, 40, 50};
   int *p = a;
   printf("%d ", *p);
   p++;
   printf("%d ", *p);
   p += 2;
   printf("%d ", *p);
   printf("%d", *(p - 1));
   return 0;
}
```

Output:

```
10 20 40 30
```

Traverse an array with a pointer, printing every element and the sum. Which relationship is used?

a[i] is exactly the same as *(a + i). With p = a, *(p + i) reaches element i without changing p.

```
#include <stdio.h>
int main()
{
   int a[5] = {10, 20, 30, 40, 50};
   int *p = a;
   int i, sum = 0;
   for (i = 0; i < 5; i++)
   {
      printf("%d ", *(p + i));
      sum = sum + *(p + i);
   }
   printf("\nSum = %d", sum);
   return 0;
}
```

Output:

```
10 20 30 40 50
Sum = 150
```

Solved Examples — HOTS

Pointer to pointer: predict the output and show what **q, *q and q hold.

q stores the address of p, p stores the address of x. *q is the content of p (the address of x), and **q is the value in x. Therefore **q = 99 changes x, and x, *p and **q all print 99.

```
#include <stdio.h>
int main()
{
   int x = 10;
   int *p = &x;
   int **q = &p;
   **q = 99;
   printf("%d %d %d", x, *p, **q);
   return 0;
}
```

Output:

```
99 99 99
```

Predict the output and explain the difference between *p = *q; and p = q;

*p = *q copies the **value** that q points to into the place p points to (x becomes 10). p = q copies the **address**, so p now points to y; the next *p = 3 changes y, not x.

```
#include <stdio.h>
int main()
{
   int x = 5, y = 10;
   int *p = &x, *q = &y;
   *p = *q;
   p = q;
   *p = 3;
   printf("x = %d, y = %d", x, y);
   return 0;
}
```

Output:

```
x = 10, y = 3
```

Show that a[i], *(a + i) and i[a] mean the same thing.

The compiler turns a[i] into *(a + i). Addition is commutative, so *(i + a) is the same, and i[a] is legal C. Never write i[a] in real programs — but knowing it proves the pointer–array relationship.

```
#include <stdio.h>
int main()
{
   int a[5] = {10, 20, 30, 40, 50};
   printf("%d %d %d", a[2], *(a + 2), 2[a]);
   return 0;
}
```

Output:

```
30 30 30
```

Debugging Corner — Find the Bug (Solved)

DEBUG 1

This program crashes with a segmentation fault. Why?

```
#include <stdio.h>
int main()
{
   int *p = NULL;
   printf("%d", *p);
   return 0;
}
```

Output of the buggy program: Runtime crash (segmentation fault)

What is wrong?

- **NULL pointer dereference.** p holds no valid address, so *p tries to read memory that does not belong to the program. Always check before dereferencing, or make p point to a real variable first.

Corrected code:

```
#include <stdio.h>
int main()
{
   int *p = NULL;
   if (p != NULL)
      printf("%d", *p);
   else
      printf("p is NULL, not dereferenced");
   return 0;
}
```

Output:

```
p is NULL, not dereferenced
```

DEBUG 2

The programmer wants to increase x by 1 through the pointer, but x stays 5. Why?

```
#include <stdio.h>
int main()
{
   int x = 5;
   int *p = &x;
   *p++;
   printf("x = %d", x);
   return 0;
}
```

Output of the buggy program: x = 5

What is wrong?

- **Operator precedence.** ++ binds tighter than *, so *p++ means *(p++): the pointer p moves to the next address and the old value is discarded. x is never touched. Use (*p)++ to increment the value p points to.

Corrected code:

```
#include <stdio.h>
int main()
{
   int x = 5;
   int *p = &x;
   (*p)++;
   printf("x = %d", x);
   return 0;
}
```

Output:

```
x = 6
```

Debug It Yourself — Practice (answers in Appendix B)

DEBUG IT 1

The program is supposed to store 10 through the pointer. Why is it dangerous?

```
#include <stdio.h>
int main()
{
   int *p;
   *p = 10;
   printf("%d", *p);
   return 0;
}
```

Hint: *Where does p point before it is assigned an address?*

DEBUG IT 2

The compiler reports an incompatible integer-to-pointer conversion. Fix the declaration.

```
#include <stdio.h>
int main()
{
   int x = 10;
   int *p = x;
   printf("%d", *p);
   return 0;
}
```

Hint: *What kind of value must be stored in a pointer?*

21.3 Important Questions

Exam Questions — Model Answers (2 / 5 / 10 marks)

Q1. What is a pointer? Write the declaration of a pointer to float. (2 marks)

A pointer is a variable that stores the memory address of another variable. float *p;

Q2. What do the operators & and * do? (2 marks)

& gives the address of a variable; * (dereference) gives the value stored at an address.

Q3. Explain pointer arithmetic with an example. (5 marks)

- p + 1 moves by one element of the pointed type (sizeof(int) bytes for an int pointer).
- Allowed: p+n, p−n, p++, p−−, p2−p1, comparisons.
- Not allowed: adding two pointers, multiplying or dividing a pointer.
- Example: int a[]={10,20,30,40}; int *p=a; *(p+2) is 30; (a+3)−p is 3.

Q4. Differentiate between an array and a pointer. (5 marks)

- An array is a block of memory with a fixed size; a pointer is a variable holding an address.
- An array name is a constant and cannot be reassigned; a pointer can.
- sizeof(array) is the whole array; sizeof(pointer) is the address size.
- Both support the same subscript form: a[i] is *(a+i).

Q5. What are NULL, void, wild and dangling pointers? (5 marks)

- NULL: points to nothing.
- void *: generic pointer for any type; must be cast before dereferencing.
- Wild: uninitialised pointer with a random address.
- Dangling: points to memory that has ended (local variable out of scope, freed memory).

Q6. Write a program that uses a pointer to find the sum and the largest of the elements of the array {12, 45, 7, 89, 23}. (10 marks)

Set p = a and use *(p + i) to read each element. Keep the running sum and the maximum.

```
#include <stdio.h>
int main()
{
   int a[5] = {12, 45, 7, 89, 23};
   int *p = a;
   int i, sum = 0, max = *p;
   for (i = 0; i < 5; i++)
   {
      sum = sum + *(p + i);
      if (*(p + i) > max)
         max = *(p + i);
   }
   printf("Sum = %d\nLargest = %d", sum, max);
   return 0;
}
```

Output:

```
Sum = 176
Largest = 89
```

Interview and Placement Questions — Short Answers

Q7. What is the size of a pointer?

The size of an address on the machine: usually 8 bytes on 64-bit and 4 bytes on 32-bit systems, the same for all pointer types.

Q8. What is the difference between int *p, q; and int *p, *q;?

In the first only p is a pointer and q is an int; in the second both are pointers.

Q9. Explain *p++, (*p)++ and *++p.

*p++ reads the value then advances the pointer; (*p)++ increments the value; *++p advances the pointer then reads.

Q10. What is a dangling pointer and how do you avoid it?

A pointer to memory that no longer exists. Never return the address of a local variable; set pointers to NULL after free().

Q11. What is a void pointer? Can you dereference it?

A generic pointer that can hold any address. It cannot be dereferenced or used in arithmetic until cast to a concrete pointer type.

Q12. What is the difference between a NULL pointer and an uninitialised pointer?

NULL is a defined "points to nothing" value that can be tested; an uninitialised pointer holds a random address and cannot be tested safely.

Q13. Can you add two pointers? Subtract?

Adding two pointers is illegal. Subtracting two pointers into the same array gives the number of elements between them.

Q14. What is the difference between a[i] and i[a]?

None: both mean *(a + i). i[a] is legal but never used.

21.4 Multiple Choice Questions

1. A pointer is a variable that stores

- a) a float value only
- b) the size of a variable
- c) the memory address of another variable
- d) a copy of a function

Answer: c) the memory address of another variable (It points to another memory location.)

2. The & operator in C gives the

- a) value at an address
- b) logical AND of two pointers
- c) remainder after division
- d) address of a variable

Answer: d) address of a variable (&x means "address of x".)

3. With int x = 10; int *p = &x; the expression *p gives

- a) 10
- b) 0
- c) the address of x
- d) the address of p

Answer: a) 10 (* dereferences: it gives the value stored at the address.)

4. int x = 10; int *p = &x; *p = 50; printf("%d", x); prints

- a) 50
- b) 10
- c) Compilation error
- d) the address of x

Answer: a) 50 (p points to x, so *p and x are the same location.)

5. int a[] = {10, 20, 30}; int *p = a; printf("%d", *(p + 2)); prints

- a) 20
- b) 12
- c) 30
- d) 10

Answer: c) 30 (*(p + 2) is the third element, a[2].)

6. Which statement declares a pointer to an int?

- a) pointer int p;
- b) int &p;
- c) int p*;
- d) int *p;

Answer: d) int *p; (The * in the declaration makes p a pointer.)

7. int **q = &p; — here q stores

- a) the address of the integer x
- b) the address of the pointer p
- c) the value of p squared
- d) the value of x

Answer: b) the address of the pointer p (A pointer to a pointer holds the address of another pointer.)

8. Which check makes it safe to dereference a pointer p?

- a) if (p == 0 && *p)
- b) if (p++)
- c) if (&p)

- d) if (p != NULL)

Answer: d) if (p != NULL) (Never dereference a NULL or invalid pointer.)

9. int *p, q; Which statement is true?

- a) the declaration is invalid
- b) p is a pointer to int and q is an int
- c) both p and q are pointers
- d) q is a pointer and p is an int

Answer: b) p is a pointer to int and q is an int (The * binds to the name p only.)

10. int a[] = {10, 20, 30}; int *p = a; printf("%d", *++p); prints

- a) 10
- b) 11
- c) 30
- d) 20

Answer: d) 20 (++p moves to a[1], then it is dereferenced.)

11. Which pointer cannot be dereferenced directly?

- a) char *
- b) int *
- c) float *
- d) void *

Answer: d) void * (The compiler does not know the size of the pointed object.)

12. const int *p; Which operation is NOT allowed?

- a) *p = 5;
- b) p = &x;
- c) printf("%d", *p);
- d) p++;

Answer: a) *p = 5; (p points to constant data: the value cannot be changed through p.)

13. int a[5]; The expression a[2] is equivalent to

- a) *(a + 2)
- b) *a + 2

- c) a(2)
- d) &a + 2

Answer: a) *(a + 2) (The compiler converts subscripts to pointer arithmetic.)

14. For an int array, the pointers p = &a[1] and q = &a[4] give q − p equal to

- a) 12
- b) 1
- c) 4
- d) 3

Answer: d) 3 (Pointer subtraction counts elements, not bytes.)

21.5 Solved Programs

Program 1: Value, Address and Dereference

Problem: Read and modify a variable through a pointer.

Approach: *p reads or writes the value at the address stored in p.

```
#include <stdio.h>
int main() {
    int x = 25;
    int *p = &x;
    printf("x  = %d\n", x);
    printf("*p = %d\n", *p);
    *p = 40;
    printf("x after *p = 40 : %d\n", x);
    return 0;
}
```

Output:

```
x  = 25
*p = 25
x after *p = 40 : 40
```

Program 2: Pointer Arithmetic on an Array

Problem: Walk through an array using a pointer.

Approach: p + 1 moves to the next int, and the array name is the address of its first element.

```
#include <stdio.h>
int main() {
    int a[] = {10, 20, 30, 40};
    int *p = a;
    printf("%d %d %d\n", *p, *(p + 1), *(p + 3));
    p = p + 2;
    printf("p now points to %d\n", *p);
    return 0;
}
```

Output:

```
10 20 40
p now points to 30
```

21.6 Practice Problems — Try It Yourself

Attempt these without looking at the solved examples. Write the program, run it, and compare the output with your hand-trace.

1. Declare int x = 42 and a pointer p to it. Print x, *p and whether p == &x.
2. Predict: int x = 10; int *p = &x; *p = *p + 5; printf("%d", x);
3. Using only pointer notation, find and print the largest element of {12, 45, 7, 89, 23}.
4. Print the array {5, 10, 15, 20, 25} in reverse order using a pointer that starts at the last element and moves backwards.
5. Predict: int a[] = {5, 10, 15, 20}; int *p = a + 1; printf("%d %d", *p, *(p + 2));
6. Draw the memory diagram for int x = 10; int *p = &x; int **q = &p; and state the values of q, *q and **q.
7. List three situations that can lead to an invalid pointer dereference and how to avoid each.

Chapter 22
Pass by Reference (Using Pointers)

C has no separate pass-by-reference mechanism like C++. It is **implemented by passing the address of a variable** using pointers. The parameter is a pointer that receives the address; the function uses *parameter to reach — and permanently change — the caller's variable.

Rule of thumb: need only the value → pass by value. Need to modify the original (or send back several results) → pass the address.

Feature	Pass by value	Pass by reference (pointer)
Function parameter	int n	int *n
Function call	f(num)	f(&num)
Access the value	n	*n
What is received	Copy of the value	Address of the variable
Can modify the original?	No	Yes
Example	square(x)	swap(&x, &y)

Common mistake: *writing n = n + 5 instead of *n = *n + 5 inside the function. The first changes the local pointer; only the second changes the caller's variable.*

22.1 Theory — Understand It First

The Big Idea

Analogy: Instead of mailing a photocopy of your cupboard, you give your friend your **house address** and the key. Now they can go in and rearrange the cupboard itself. Passing an address lets the function reach — and change — the original variable.

Technical Concept: C implements **pass by reference** by passing the **address** of a variable. The formal parameter is a **pointer**; the actual argument is the **address-of expression** (&x). Inside the function, *parameter reads and writes the caller's variable.

Where It Is Used: scanf("%d", &x) — scanf modifies x through its address. Also swap, functions returning several results, functions that fill or modify arrays, and passing large data without copying it.

> **Why We Use It:** To change the caller's variables permanently, to return more than one result, and to avoid copying large amounts of data.

How It Works in Memory

```
main():  int x = 20;   change(&x);       change(int *n):  *n = 100;

  address 1000          address 2000
 +---------+          +-----------+
 | x = 20  | <---------- | n = 1000 |   n holds the ADDRESS of x
 +---------+   points   +-----------+
             to
 *n = 100  goes to address 1000  ->  x becomes 100
```

Step	Pass by value	Pass by reference (pointer)
Function header	void f(int n)	void f(int *n)
Function call	f(x);	f(&x);
Using the value inside	n	*n
What is copied	The value	The address
Can change caller's x?	No	Yes

- Remember the three-part recipe: **(1)** parameter is a pointer type, **(2)** the call passes &variable, **(3)** inside the function use *parameter.

Arrays Are Passed by Address

When an array is passed, the function receives the address of its first element, so it works on the **original** array. The declarations int a[] and int *a are identical for a parameter. Because only an address arrives, the function does not know the length — always pass the size as a second parameter.

```
#include <stdio.h>
void doubleAll(int a[], int n)
{
   int i;
   for (i = 0; i < n; i++)
      a[i] = a[i] * 2;
   printf("%d\n", sizeof(a) == sizeof(int *));   /* a is a pointer here */
}
int main()
{
   int v[4] = {1, 2, 3, 4};
   int i;
   doubleAll(v, 4);
   for (i = 0; i < 4; i++)
      printf("%d ", v[i]);
   return 0;
}
```

Output:

```
1
2 4 6 8
```

Inside the function sizeof(a) is the size of a pointer, not of the array. Never compute the array length with sizeof inside a function that received the array as a parameter.

Returning Several Results

Pass the inputs by value and one pointer for every result. The function writes its answers through the pointers and usually returns void.

```
#include <stdio.h>
void minMax(int a[], int n, int *min, int *max)
{
   int i;
   *min = *max = a[0];
   for (i = 1; i < n; i++)
   {
      if (a[i] < *min) *min = a[i];
      if (a[i] > *max) *max = a[i];
   }
}
int main()
{
   int v[5] = {34, 7, 89, 21, 56};
   int lo, hi;
   minMax(v, 5, &lo, &hi);
   printf("Min = %d, Max = %d", lo, hi);
   return 0;
}
```

Output:

```
Min = 7, Max = 89
```

Swapping — Three Ways and Their Traps

Method	Code (inside swap, a and b are int *)	Trap
Temporary variable	t = *a; *a = *b; *b = t;	None — safest
Arithmetic	*a = *a + *b; *b = *a - *b; *a = *a - *b;	Possible overflow; fails if a and b are the same variable
XOR	*a ^= *b; *b ^= *a; *a ^= *b;	If a and b point to the same variable the value becomes 0

```
#include <stdio.h>
void swapXor(int *a, int *b)
{
   *a = *a ^ *b;
   *b = *a ^ *b;
   *a = *a ^ *b;
}
int main()
{
   int x = 6, y = 9;
   swapXor(&x, &y);
   printf("%d %d\n", x, y);
   swapXor(&x, &x);      /* same variable twice! */
   printf("%d", x);
   return 0;
}
```

Output:

```
9 6
0
```

Pointer Parameters: Passing a Pointer by Value and Pointer to Pointer

A pointer parameter is itself a copy. Changing *where it points* inside the function does not change the caller's pointer. To change the caller's pointer you must pass its address — a pointer to pointer.

```
#include <stdio.h>
void setByValue(int *p, int *t) { p = t; }       /* changes the copy only */
void setByRef(int **pp, int *t) { *pp = t; }     /* changes the caller's pointer */
int main()
{
   int a = 1, b = 2;
   int *p = &a;
   setByValue(p, &b);
   printf("%d\n", *p);    /* still a: 1 */
   setByRef(&p, &b);
   printf("%d", *p);      /* now b: 2 */
   return 0;
}
```

Output:

```
1
2
```

Returning Pointers and Read-only Pointer Parameters

- A function may return a pointer, but **never the address of a local variable** — it is destroyed on return (dangling pointer).
- Safe to return: the address of a static or global variable, an element of an array supplied by the caller, or malloc'd memory.
- Use **const** for a pointer parameter the function must only read: void print(const int *a, int n). Any attempt to write through it becomes a compile error.

```
#include <stdio.h>
int *largest(int a[], int n)
{
    int i, *best = &a[0];
    for (i = 1; i < n; i++)
        if (a[i] > *best)
            best = &a[i];
    return best;                /* points into the caller's array */
}
int main()
{
    int v[5] = {12, 45, 7, 89, 23};
    int *p = largest(v, 5);
    *p = 0;                     /* changes the array element itself */
    printf("%d %d", v[3], *p);
    return 0;
}
```

Output:

```
0 0
```

22.2 Solved Examples and Debugging

Solved Examples — Easy

Write a function that changes the caller's variable to 100 using pass by reference. What is printed?

The call change(&x) sends the address of x. Inside, n holds that address, and *n = 100 writes 100 into x itself.

```
#include <stdio.h>
void change(int *n)
{
   *n = 100;
}
int main()
{
   int x = 20;
   printf("Before = %d\n", x);
   change(&x);
   printf("After = %d", x);
   return 0;
}
```

Output:

```
Before = 20
After = 100
```

Write a C program to increase a number by 5 using pass by reference.

The function receives &n and adds 5 to the value stored at that address.

```
#include <stdio.h>
void increase(int *n)
{
   *n = *n + 5;
}
int main()
{
   int n = 10;
   increase(&n);
   printf("Result = %d", n);
   return 0;
}
```

Output:

```
Result = 15
```

Solved Examples — Medium

Write a function to swap two integers using pass by reference.

swap() receives the addresses of x and y. The temp variable holds one value while the two are exchanged through the pointers, so the change is visible in main().

```
#include <stdio.h>
void swap(int *a, int *b)
{
   int temp;
   temp = *a;
   *a = *b;
   *b = temp;
}
int main()
{
   int x = 10, y = 20;
   printf("Before swap: x = %d, y = %d\n", x, y);
   swap(&x, &y);
   printf("After swap: x = %d, y = %d", x, y);
   return 0;
}
```

Output:

```
Before swap: x = 10, y = 20
After swap: x = 20, y = 10
```

A function can return only one value directly. Write calculate(a, b, &sum, &diff) that sends back both the sum and the difference.

The two result variables are passed by address; the function writes its answers into them. The return type is void because the results travel through the pointers.

```
#include <stdio.h>
void calculate(int a, int b, int *sum, int *diff)
{
   *sum = a + b;
   *diff = a - b;
}
int main()
{
   int x = 20, y = 8;
   int sum, diff;
   calculate(x, y, &sum, &diff);
   printf("Sum = %d\n", sum);
   printf("Difference = %d", diff);
   return 0;
}
```

Output:

```
Sum = 28
Difference = 12
```

Write findValues(a, b, &largest, &smallest) that returns both the largest and the smallest of two numbers.

a and b only need to be read, so they are passed by value. largest and smallest must be written, so they are passed by address.

```
#include <stdio.h>
void findValues(int a, int b, int *largest, int *smallest)
{
   if (a > b)
   {
      *largest = a;
      *smallest = b;
   }
   else
   {
      *largest = b;
      *smallest = a;
   }
}
int main()
{
   int a = 15, b = 25;
   int large, small;
   findValues(a, b, &large, &small);
   printf("Largest = %d\n", large);
   printf("Smallest = %d", small);
   return 0;
}
```

Output:

```
Largest = 25
Smallest = 15
```

Solved Examples — HOTS

Compare isPrime(int n) with isPrime(int *n). Which is the better design for prime checking, and why? Write the pointer version.

Both work, but the pointer version is unnecessary: prime checking only reads the number. The pointer version needs &num at the call and (*n) in every expression, and it would allow the function to modify the caller's variable by accident. Use pass by value here; keep pointers for functions that must change the caller's data.

```
#include <stdio.h>
int isPrime(int *n)
{
   int i;
   if (*n < 2)
      return 0;
   for (i = 2; i <= (*n) / 2; i++)
   {
      if ((*n) % i == 0)
         return 0;
   }
   return 1;
}
int main()
{
   int num = 17;
   if (isPrime(&num))
      printf("%d is a prime number", num);
   else
      printf("%d is not a prime number", num);
   return 0;
}
```

Output:

```
17 is a prime number
```

Write stats(a, b, &sum, &product, &average) that gives three results back to main() for a = 6 and b = 4.

Three results cannot be returned directly, so three addresses are passed. Note the mixed types: the sum and product are int, but the average must be float (10 / 2.0 = 5.0), so its pointer is float *.

```
#include <stdio.h>
void stats(int a, int b, int *sum, int *product, float *average)
{
   *sum = a + b;
   *product = a * b;
   *average = (a + b) / 2.0;
}
int main()
{
   int s, p;
   float avg;
   stats(6, 4, &s, &p, &avg);
   printf("Sum = %d, Product = %d, Average = %.2f", s, p, avg);
   return 0;
}
```

Output:

```
Sum = 10, Product = 24, Average = 5.00
```

Predict the output of calling swap twice, and of calling swap(&x, &x). What does the self-swap do?

The first swap exchanges 3 and 8; the second swaps them back. In swap(&x, &x) both pointers refer to the same variable, so temp = 3, then x = x (3), then x = temp (3) — x is unchanged.

```
#include <stdio.h>
void swap(int *a, int *b)
{
   int temp = *a;
   *a = *b;
   *b = temp;
}
int main()
{
   int x = 3, y = 8;
   swap(&x, &y);
   printf("after 1: %d %d\n", x, y);
   swap(&x, &y);
   printf("after 2: %d %d\n", x, y);
   swap(&x, &x);
   printf("self-swap: x = %d", x);
   return 0;
}
```

Output:

```
after 1: 8 3
after 2: 3 8
self-swap: x = 3
```

Write applyDiscount(&price, percent) that permanently reduces the price. Use it on ₹2000 with a 15% discount.

The price is modified, so its address is passed; the percentage is only read, so it is passed by value. Discount = 2000 × 15 / 100 = 300, new price = 1700.

```
#include <stdio.h>
void applyDiscount(float *price, float percent)
{
   *price = *price - (*price * percent / 100);
}
int main()
{
   float price = 2000;
   applyDiscount(&price, 15);
   printf("Price after discount = %.2f", price);
   return 0;
}
```

Output:

```
Price after discount = 1700.00
```

Debugging Corner — Find the Bug (Solved)

DEBUG 1

The swap function runs without error, yet x and y stay 10 and 20 after the call. Find the bug.

```
#include <stdio.h>
void swap(int *a, int *b)
{
   int *t = a;
   a = b;
   b = t;
}
int main()
{
   int x = 10, y = 20;
   swap(&x, &y);
   printf("x = %d, y = %d", x, y);
   return 0;
}
```

Output of the buggy program: x = 10, y = 20

What is wrong?

- **Only the local pointers were swapped.** a and b are local copies of the addresses; exchanging them changes where a and b point, not the values in x and y. Swap the values with *a and *b.

Corrected code:

```
#include <stdio.h>
void swap(int *a, int *b)
{
   int t = *a;
   *a = *b;
   *b = t;
}
int main()
{
   int x = 10, y = 20;
   swap(&x, &y);
   printf("x = %d, y = %d", x, y);
   return 0;
}
```

Output:

```
x = 20, y = 10
```

DEBUG 2

After the swap both variables hold 20. What went wrong?

```
#include <stdio.h>
void swap(int *a, int *b)
{
   *a = *b;
   *b = *a;
}
int main()
{
   int x = 10, y = 20;
   swap(&x, &y);
   printf("x = %d, y = %d", x, y);
   return 0;
}
```

Output of the buggy program: x = 20, y = 20

What is wrong?

- **No temporary variable.** *a = *b overwrites x with 20, destroying the original 10. The next line then copies that 20 into y. Save one value in temp before overwriting it.

Corrected code:

```
#include <stdio.h>
void swap(int *a, int *b)
{
   int temp = *a;
   *a = *b;
   *b = temp;
}
int main()
{
   int x = 10, y = 20;
   swap(&x, &y);
   printf("x = %d, y = %d", x, y);
   return 0;
}
```

Output:

```
x = 20, y = 10
```

Debug It Yourself — Practice (answers in Appendix B)

DEBUG IT 1

increase(&x) is called but x is still 10. Why?

```
#include <stdio.h>
void increase(int *n)
{
   n = n + 5;
}
int main()
{
   int x = 10;
   increase(&x);
   printf("%d", x);
   return 0;
}
```

Hint: *Is it the pointer or the pointed-to value that changes?*

DEBUG IT 2

The compiler complains about the argument of swap(x, y). Fix the call.

```
#include <stdio.h>
void swap(int *a, int *b)
{
   int t = *a;
   *a = *b;
   *b = t;
}
int main()
{
   int x = 1, y = 2;
   swap(x, y);
   printf("%d %d", x, y);
   return 0;
}
```

Hint: *What does the function expect — a value or an address?*

22.3 Important Questions

Exam Questions — Model Answers (2 / 5 / 10 marks)

Q1. Define pass by reference. How is it achieved in C? (2 marks)

In pass by reference the address of the actual argument is passed so the function can change the original variable. In C it is achieved by passing &variable to a pointer parameter and using *parameter inside the function.

Q2. Differentiate between pass by value and pass by reference. (5 marks)

(Draw the table.) Value: copy passed, parameter int n, call f(x), cannot change original. Reference: address passed, parameter int *n, call f(&x), use *n, can change original and return several results.

Q3. Write a function to swap two integers using pointers. (5 marks)

See program below.

```
#include <stdio.h>
void swap(int *a, int *b)
{
   int temp = *a;
   *a = *b;
   *b = temp;
}
int main()
{
   int x = 10, y = 20;
   swap(&x, &y);
   printf("x = %d, y = %d", x, y);
   return 0;
}
```

Output:

```
x = 20, y = 10
```

Q4. How are arrays passed to functions in C? Why must the size be passed separately? (5 marks)

- Only the address of the first element is passed (the array decays to a pointer).
- The function therefore works on the original array and its changes are permanent.
- The function cannot know the length (sizeof gives the pointer size), so the length is passed as another parameter.

Q5. Write a function that gives back the quotient and the remainder of a division together, and a program that uses it for 47 ÷ 5. (10 marks)

Two results are needed, so two pointer parameters are used while the dividend and divisor are passed by value.

```
#include <stdio.h>
void divide(int a, int b, int *quotient, int *remainder)
{
   *quotient = a / b;
   *remainder = a % b;
}
int main()
{
   int q, r;
   divide(47, 5, &q, &r);
   printf("Quotient = %d, Remainder = %d", q, r);
   return 0;
}
```

Output:

```
Quotient = 9, Remainder = 2
```

Interview and Placement Questions — Short Answers

Q6. Does C support pass by reference?

Not as a language feature. C simulates it by passing a pointer; the pointer is still passed by value.

Q7. Why does scanf("%d", &x) need &, and what happens with scanf("%d", x)?

scanf must write into x, so it needs x's address (pass by reference). Passing x passes its value as if it were an address and usually crashes.

Q8. How can you swap two numbers without a temporary variable? What is the catch?

Using +/− or XOR through pointers. The catch: if both pointers refer to the same variable the XOR version gives 0, and the arithmetic version can overflow.

Q9. What happens when you pass an array to a function and modify it?

The original array is changed because only its address is passed.

Q10. When do you need a pointer to a pointer as a parameter?

When the function must change the caller's pointer itself (for example to make it point somewhere else or to allocate memory for it).

Q11. What is wrong with returning the address of a local variable?

The local variable is destroyed when the function returns, leaving a dangling pointer.

Q12. What is the use of const in void print(const int *a, int n)?

It promises that the function will only read the array; writes through a become compile errors.

22.4 Multiple Choice Questions

1. In C, pass by reference is implemented by

- a) using the keyword ref
- b) passing the variable name as a string
- c) passing the address of the variable using pointers
- d) using the & symbol in the function header

Answer: c) passing the address of the variable using pointers (C has no separate reference parameters; addresses are passed by value.)

2. For void swap(int *a, int *b), the correct call is

- a) swap(x*, y*);
- b) swap(&x, &y);
- c) swap(x, y);
- d) swap(*x, *y);

Answer: b) swap(&x, &y); (The function expects addresses, so & is used at the call.)

3. What can a pass-by-reference (pointer) function do that a pass-by-value function cannot?

- a) Modify the caller's variable
- b) Use a loop
- c) Call another function
- d) Return a value

Answer: a) Modify the caller's variable (It can write through the address it received.)

4. void inc(int *n){ *n = *n + 1; } int a = 5; inc(&a); inc(&a); printf("%d", a); prints

- a) 7
- b) 5
- c) 8

- d) 6

Answer: a) 7 (Each call changes a itself: 5 → 6 → 7.)

5. To send two results back from one function in C, the usual technique is

- a) using the keyword double return
- b) returning an array name
- c) two return statements in a row
- d) pointer (reference) parameters

Answer: d) pointer (reference) parameters (Only one value can be returned directly.)

6. void g(int a[]){ a[0] = 99; } int v[3] = {1,2,3}; g(v); printf("%d", v[0]); prints

- a) 99
- b) Compilation error
- c) 0
- d) 1

Answer: a) 99 (The array decays to a pointer, so the original element is changed.)

7. Inside a function with the parameter int a[], sizeof(a) gives the

- a) size of one element
- b) size of the whole array
- c) number of elements
- d) size of a pointer

Answer: d) size of a pointer (a is really int *a; the length must be passed separately.)

8. A function that returns the address of one of its local (auto) variables creates a

- a) null pointer
- b) wild pointer
- c) dangling pointer
- d) void pointer

Answer: c) dangling pointer (The local variable is destroyed on return.)

9. In pass by reference using pointers, the actual argument in the call is written as

- a) variable[]
- b) &variable

- c) variable()
- d) *variable

Answer: b) &variable (The address-of operator supplies the address.)

10. void swap(int *a, int *b) with XOR swapping is called as swap(&x, &x); the value of x becomes

- a) 0
- b) garbage
- c) 1
- d) unchanged

Answer: a) 0 (x ^ x is 0, and the remaining steps keep it 0.)

11. In the call scanf("%d", &n), the argument &n is an example of

- a) a macro
- b) passing by reference (address)
- c) a recursive call
- d) passing by value only of n

Answer: b) passing by reference (address) (scanf must write into n, so it needs its address.)

22.5 Solved Programs

Program 1: Swap Using Pointers

Problem: Exchange two integers through a function.

Approach: The function receives addresses, so it changes the caller's variables.

```
#include <stdio.h>
void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}
int main() {
    int x = 5, y = 9;
    printf("Before: %d %d\n", x, y);
    swap(&x, &y);
    printf("After : %d %d\n", x, y);
```

```
    return 0;
}
```

Output:

```
Before: 5 9
After : 9 5
```

Program 2: Two Results from One Function

Problem: Return the smaller and larger of two numbers.

Approach: A function can return only one value directly, so two results go back through pointer parameters.

```
#include <stdio.h>
void minMax(int a, int b, int *min, int *max) {
    if (a < b) { *min = a; *max = b; }
    else       { *min = b; *max = a; }
}
int main() {
    int lo, hi;
    minMax(14, 9, &lo, &hi);
    printf("min = %d, max = %d\n", lo, hi);
    return 0;
}
```

Output:

```
min = 9, max = 14
```

22.6 Practice Problems — Try It Yourself

Attempt these without looking at the solved examples. Write the program, run it, and compare the output with your hand-trace.

1. Write a function that doubles a number permanently using a pointer parameter. Test with x = 12 (result 24).
2. Write a function that swaps two float variables using pointers.
3. Write minMax(int a[], int n, int *min, int *max) to return the smallest and largest of an array. (An array name already behaves like an address.)

4. Write splitTime(int totalMinutes, int *hours, int *mins). For 135 minutes it must give 2 hours and 15 minutes.
5. Write rectangle(int l, int b, int *area, int *peri). For l = 8 and b = 5 it must return area 40 and perimeter 26.
6. Predict: void f(int *a, int b){ *a = *a + b; b = b + 1; } int x = 5, y = 5; f(&x, y); printf("%d %d", x, y);
7. Decide whether each task needs pass by value or a pointer: (a) print the square, (b) swap two numbers, (c) read a number into the caller's variable, (d) compute interest, (e) reset a counter to 0.

Chapter 23
Helper Functions and Modular Programs

Modular programming means breaking a large problem into small, independent modules (functions). A **helper function** follows the **Single Responsibility Principle**: it does exactly one job, does it well, and returns a clear result. main() then reads like a short outline of the solution.

Design steps: (1) identify the task, (2) create a helper function with suitable parameters, (3) choose the return type (int, float, char or void) from the result needed, (4) call it from main() and use the result.

Advantage of modular programming	Why it helps
Improves readability	main() shows the plan; details live in named functions
Simplifies debugging	A wrong result can be traced to one small function
Promotes reuse	isPrime() written once can be used in many programs
Reduces complexity	Each function is small enough to understand at once
Makes testing easier	Each helper can be tested with its own sample values

Common mistake: *letting a helper do too much (calculate, print and read input in one function). A calculation helper should return its result and leave printing to a display function or to main().*

23.1 Theory — Understand It First

The Big Idea

Analogy: A relay team: each runner runs only their leg and hands the baton (the result) to the next. Nobody runs the whole race. A helper function is one runner — it takes what it needs, does its single job and hands the result on.

Technical Concept: A **helper function** is a small function that solves one sub-problem for a larger program. **Modular programming** builds a program from such independent modules, with main() acting as the coordinator. The guiding rule is the **Single Responsibility Principle**.

Where It Is Used: Marks and grade processing, billing, salary slips, prime / palindrome / Armstrong checks, unit conversions — every non-trivial program, and every placement coding problem that says “write a function to …”.

> **Why We Use It:** Readability, reuse, easy debugging, easy testing, team work, and simpler maintenance.

Anatomy of a Function

```
float  calculateAverage ( int total , int n )  {
^^^^^  ^^^^^^^^^^^^^^^^  ^^^^^^^^^^^^^^^^^     <- parameter list
return  function name                    (each with type + name)
type    (verb that says the job)

   return (float) total / n;     <- body ends with return
}
```

Part	Rule
Return type	int, float, char, double … or void if nothing is returned
Name	A verb or question: calculateTotal, displayBill, isPrime, findLargest
Parameter list	Types and names, separated by commas; (void) when there are none
Body	Local declarations and statements inside { }
return	Gives back one value of the return type and ends the function

Design Checklist for a Helper Function

- Does it do **exactly one job**? Can its purpose be said in one short sentence?
- Does the **name** describe the job (verb or is…)?
- Are the **parameters** exactly the data it needs — no more, no less?
- Does it **return** the result instead of printing it (unless it is a display function)?
- Does it avoid **global variables**?
- Does it handle **special cases** (zero, negative, empty, division by zero)?
- Has it been tested with **normal and boundary** values?

Choosing the Return Type

The function computes…	Return type	Example prototype
A count or whole-number total	int	int calculateTotal(int a[], int n)
A value with decimals (average, area, interest)	float / double	float calculateAverage(int total, int n)
A single letter (grade, option)	char	char findGrade(float average)

The function computes…	Return type	Example prototype
A yes/no answer (is prime? is leap year?)	int (1 or 0)	int isPrime(int n)
Nothing, only shows output	void	void displayScorecard(...)
Several results	void + pointer parameters	void minMax(int a[], int n, int *min, int *max)

A yes/no function returning 1 (true) or 0 (false) reads naturally in conditions: **if (isPrime(n)) ….** In C, any non-zero value counts as true.

Helpers Calling Helpers and Call Trees

A helper may call other helpers, which keeps every function small. The calling relationship forms a tree:

```
main
 |-- readMarks
 |-- calculateTotal
 |-- calculateAverage
 |-- calculateGrade
 |-- displayScorecard

main
 |-- countPrimes ------> isPrime   (a helper that uses a helper)
```

Testing Helper Functions

Test each helper alone before joining it to the program. Prepare a table of input, expected result and actual result, always including **boundary** values.

```
#include <stdio.h>
int isPrime(int n)
{
   int i;
   if (n < 2) return 0;
   for (i = 2; i * i <= n; i++)      /* up to the square root */
      if (n % i == 0) return 0;
   return 1;
}
int main()
{
   int test[7] = {0, 1, 2, 3, 4, 17, 25};
   int i;
   for (i = 0; i < 7; i++)
      printf("isPrime(%d) = %d\n", test[i], isPrime(test[i]));
   return 0;
}
```

Output:

```
isPrime(0) = 0
isPrime(1) = 0
isPrime(2) = 1
isPrime(3) = 1
isPrime(4) = 0
isPrime(17) = 1
isPrime(25) = 0
```

*Efficiency (placement favourite): checking divisors only up to √n (i * i <= n) instead of n/2 or n makes isPrime run in O(√n) time instead of O(n).*

Function vs Macro (Common Interview Trap)

	Function	Macro (#define)
Handled by	Compiler at run time (a real call)	Preprocessor, as text substitution before compiling
Type checking	Yes	No
Speed	Small call overhead	No call overhead, but code is repeated

	Function	Macro (#define)
Safety	Arguments evaluated once	Arguments may be evaluated several times; precedence bugs

```
#include <stdio.h>
#define SQUARE(x) x * x
#define SAFE_SQUARE(x) ((x) * (x))
int square(int x) { return x * x; }
int main()
{
    printf("%d\n", SQUARE(2 + 3));        /* 2 + 3 * 2 + 3 = 11 */
    printf("%d\n", SAFE_SQUARE(2 + 3));   /* 25 */
    printf("%d", square(2 + 3));          /* 25 */
    return 0;
}
```

Output:

```
11
25
25
```

23.2 Solved Examples and Debugging

Solved Examples — Easy

Design a helper function findLargest() that returns the largest of three numbers, and use it from main().

Task: find the largest. Helper: int findLargest(int a, int b, int c). Result: returned to main().

```
#include <stdio.h>
int findLargest(int a, int b, int c)
{
   if (a >= b && a >= c)
      return a;
   else if (b >= c)
      return b;
   else
      return c;
}
int main()
{
   int a = 12, b = 45, c = 30, largest;
   largest = findLargest(a, b, c);
   printf("Largest = %d", largest);
   return 0;
}
```

Output:

```
Largest = 45
```

Write a function isEven() that returns 1 if a number is even and 0 otherwise, and use it in an if statement.

Returning 1 or 0 lets the call be used directly as a condition: if (isEven(12)) … because non-zero means true.

```
#include <stdio.h>
int isEven(int n)
{
   if (n % 2 == 0)
      return 1;
   else
      return 0;
}
int main()
{
   if (isEven(12)) printf("12 is Even\n");
   else            printf("12 is Odd\n");
   if (isEven(7))  printf("7 is Even");
   else            printf("7 is Odd");
   return 0;
}
```

Output:

```
12 is Even
7 is Odd
```

Solved Examples — Medium

Design helper functions to calculate the total, average and grade of a student from three marks (85, 78, 92).

Three small helpers, each with arguments and a return value, chained in main(): total → average → grade. The grade function returns a char.

```
#include <stdio.h>
int calculateTotal(int m1, int m2, int m3)
{
   return m1 + m2 + m3;
}
float calculateAverage(int total)
{
   return total / 3.0;
}
char findGrade(float average)
{
   if (average >= 90)      return 'A';
   else if (average >= 75) return 'B';
   else if (average >= 60) return 'C';
   else if (average >= 50) return 'D';
   else                    return 'F';
}
int main()
{
   int total;
   float average;
   char grade;
   total = calculateTotal(85, 78, 92);
   average = calculateAverage(total);
   grade = findGrade(average);
   printf("Total = %d\n", total);
   printf("Average = %.2f\n", average);
   printf("Grade = %c", grade);
   return 0;
}
```

Output:

```
Total = 255
Average = 85.00
Grade = B
```

Electricity bill: first 100 units ₹2/unit, next 100 units ₹3/unit, above 200 units ₹5/unit. Write calculateBill(units) and find the bill for 250 units.

Slab billing is cumulative: 100 × 2 = 200, then 100 × 3 = 300, then the remaining 50 units × 5 = 250. Total = 200 + 300 + 250 = **₹750**.

```
#include <stdio.h>
float calculateBill(int units)
{
    float bill;
    if (units <= 100)
        bill = units * 2;
    else if (units <= 200)
        bill = 100 * 2 + (units - 100) * 3;
    else
        bill = 100 * 2 + 100 * 3 + (units - 200) * 5;
    return bill;
}
int main()
{
    int units = 250;
    printf("Units = %d\n", units);
    printf("Bill = Rs. %.2f", calculateBill(units));
    return 0;
}
```

Output:

```
Units = 250
Bill = Rs. 750.00
```

The sample answer in the reference PPT shows Rs. 850.00 for 250 units; the correct value by the stated slabs is Rs. 750.00 (verified by running the program).

Write a helper isLeapYear(year) that returns 1 or 0 and test it for 2024, 1900 and 2000.

A year is a leap year if it is divisible by 400, or divisible by 4 but not by 100. So 2000 is leap, 1900 is not (divisible by 100 but not 400), 2024 is leap.

```
#include <stdio.h>
int isLeapYear(int year)
{
   if ((year % 400 == 0) || (year % 4 == 0 && year % 100 != 0))
      return 1;
   else
      return 0;
}
int main()
{
   int y[3] = {2024, 1900, 2000};
   int i;
   for (i = 0; i < 3; i++)
   {
      if (isLeapYear(y[i])) printf("%d Leap Year\n", y[i]);
      else                  printf("%d Not a Leap Year\n", y[i]);
   }
   return 0;
}
```

Output:

```
2024 Leap Year
1900 Not a Leap Year
2000 Leap Year
```

Solved Examples — HOTS

Design a modular calculator with add, subtract, multiply and divide functions. The divide function must protect against division by zero.

Four helpers with arguments and a return value. divide() checks the divisor first; if it is 0 it reports an error and returns 0 instead of dividing.

```
#include <stdio.h>
float add(float a, float b)      { return a + b; }
float subtract(float a, float b) { return a - b; }
float multiply(float a, float b) { return a * b; }
float divide(float a, float b)
{
   if (b == 0)
   {
      printf("Error: division by zero\n");
      return 0;
   }
   return a / b;
}
int main()
{
   float a = 20, b = 5;
   printf("Addition = %.2f\n", add(a, b));
   printf("Subtraction = %.2f\n", subtract(a, b));
   printf("Multiplication = %.2f\n", multiply(a, b));
   printf("Division = %.2f\n", divide(a, b));
   printf("Division = %.2f", divide(a, 0));
   return 0;
}
```

Output:

```
Addition = 25.00
Subtraction = 15.00
Multiplication = 100.00
Division = 4.00
Error: division by zero
Division = 0.00
```

Write separate helper functions to convert Celsius → Fahrenheit, Fahrenheit → Celsius and Celsius → Kelvin. Test with 25 °C and 98.6 °F.

F = C × 9/5 + 32, C = (F − 32) × 5/9, K = C + 273.15. Writing 9 / 5 with a float C keeps the division floating-point because C × 9 is a float before the division.

```
#include <stdio.h>
float celsiusToFahrenheit(float c) { return (c * 9 / 5) + 32; }
float fahrenheitToCelsius(float f) { return (f - 32) * 5 / 9; }
float celsiusToKelvin(float c)     { return c + 273.15; }
int main()
{
   printf("25 C  = %.2f F\n", celsiusToFahrenheit(25));
   printf("25 C  = %.2f K\n", celsiusToKelvin(25));
   printf("98.6 F = %.2f C", fahrenheitToCelsius(98.6));
   return 0;
}
```

Output:

```
25 C  = 77.00 F
25 C  = 298.15 K
98.6 F = 37.00 C
```

Build two helpers: isPrime(n) and countPrimes(limit) that uses isPrime(). How many primes are there up to 30?

A helper can call another helper. countPrimes() loops from 2 to limit and counts every n for which isPrime(n) returns 1. The primes up to 30 are 2, 3, 5, 7, 11, 13, 17, 19, 23, 29 — ten of them.

```
#include <stdio.h>
int isPrime(int n)
{
    int i;
    if (n < 2) return 0;
    for (i = 2; i <= n / 2; i++)
        if (n % i == 0) return 0;
    return 1;
}
int countPrimes(int limit)
{
    int n, count = 0;
    for (n = 2; n <= limit; n++)
        if (isPrime(n))
            count++;
    return count;
}
int main()
{
    printf("Primes up to 30: %d", countPrimes(30));
    return 0;
}
```

Output:

```
Primes up to 30: 10
```

Employee salary: HRA = 20% and DA = 10% of the basic salary. Decompose the problem into helper functions and compute the gross salary for a basic salary of ₹30,000.

Sub-tasks: calculate HRA, calculate DA, calculate gross, display. HRA = 6000, DA = 3000, Gross = 30000 + 6000 + 3000 = 39000. The display function receives four values and returns nothing.

```
#include <stdio.h>
float calculateHRA(float basic)   { return basic * 0.20; }
float calculateDA(float basic)    { return basic * 0.10; }
float calculateGross(float basic, float hra, float da)
{
   return basic + hra + da;
}
void displaySalary(float basic, float hra, float da, float gross)
{
   printf("Basic Salary = %.2f\n", basic);
   printf("HRA = %.2f\n", hra);
   printf("DA = %.2f\n", da);
   printf("Gross Salary = %.2f", gross);
}
int main()
{
   float basic = 30000, hra, da, gross;
   hra = calculateHRA(basic);
   da = calculateDA(basic);
   gross = calculateGross(basic, hra, da);
   displaySalary(basic, hra, da, gross);
   return 0;
}
```

Output:

```
Basic Salary = 30000.00
HRA = 6000.00
DA = 3000.00
Gross Salary = 39000.00
```

Debugging Corner — Find the Bug (Solved)

DEBUG 1

The average of 245 over 3 subjects is printed as 81.00 instead of 81.67. The helper looks correct — what is wrong?

```
#include <stdio.h>
int calculateAverage(int total)
{
   return total / 3;
}
int main()
{
   float average = calculateAverage(245);
   printf("Average = %.2f", average);
   return 0;
}
```

Output of the buggy program: Average = 81.00

What is wrong?

- **Wrong return type and integer division.** total / 3 is an integer division (81), and an int function cannot carry the decimals anyway. Make the helper float and divide by 3.0.

Corrected code:

```
#include <stdio.h>
float calculateAverage(int total)
{
   return total / 3.0;
}
int main()
{
   float average = calculateAverage(245);
   printf("Average = %.2f", average);
   return 0;
}
```

Output:

```
Average = 81.67
```

DEBUG 2

The program declares 0 and 1 to be prime. Why?

```
#include <stdio.h>
int isPrime(int n)
{
   int i;
   for (i = 2; i <= n / 2; i++)
      if (n % i == 0)
         return 0;
   return 1;
}
int main()
{
   printf("%d %d %d", isPrime(0), isPrime(1), isPrime(2));
   return 0;
}
```

Output of the buggy program: 1 1 1

What is wrong?

- **Missing check for n < 2.** For 0 and 1 the loop body never runs, so the function falls through to return 1. Numbers below 2 are not prime and must return 0 first.

Corrected code:

```
#include <stdio.h>
int isPrime(int n)
{
   int i;
   if (n < 2)
      return 0;
   for (i = 2; i <= n / 2; i++)
      if (n % i == 0)
         return 0;
   return 1;
}
int main()
{
   printf("%d %d %d", isPrime(0), isPrime(1), isPrime(2));
   return 0;
}
```

Output:

```
0 0 1
```

DEBUG 3

The grade should be the letter B but the program prints a number. Why?

```
#include <stdio.h>
char findGrade(float average)
{
   if (average >= 90) return 'A';
   else if (average >= 75) return 'B';
   else return 'F';
}
int main()
{
   char grade = findGrade(85);
   printf("Grade = %d", grade);
   return 0;
}
```

Output of the buggy program: Grade = 66

What is wrong?

- **Wrong format specifier.** A char is stored as its ASCII code (B = 66). %d prints the number; %c prints the character.

Corrected code:

```
#include <stdio.h>
char findGrade(float average)
{
   if (average >= 90) return 'A';
   else if (average >= 75) return 'B';
   else return 'F';
}
int main()
{
   char grade = findGrade(85);
   printf("Grade = %c", grade);
   return 0;
}
```

Output:

```
Grade = B
```

Debug It Yourself — Practice (answers in Appendix B)

DEBUG IT 1

The program says 4 is prime. Find the exact bug.

```
#include <stdio.h>
int isPrime(int n)
{
    int i;
    if (n < 2)
        return 0;
    for (i = 2; i < n / 2; i++)
        if (n % i == 0)
            return 0;
    return 1;
}
int main()
{
    printf("%d", isPrime(4));
    return 0;
}
```

Hint: *For n = 4, how many times does the loop run when the condition is i < 2?*

DEBUG IT 2

For 250 units the bill prints Rs. 250.00 instead of Rs. 750.00. Why?

```
#include <stdio.h>
float calculateBill(int units)
{
   float bill;
   if (units <= 100)
      bill = units * 2;
   else if (units <= 200)
      bill = (units - 100) * 3;
   else
      bill = (units - 200) * 5;
   return bill;
}
int main()
{
   printf("Rs. %.2f", calculateBill(250));
   return 0;
}
```

Hint: *Slab billing is cumulative: the earlier slabs are still charged.*

23.3 Important Questions

Exam Questions — Model Answers (2 / 5 / 10 marks)

Q1. What is a helper function? (2 marks)

A small function that performs one specific sub-task for a larger program, such as isPrime() or calculateAverage().

Q2. State the Single Responsibility Principle. (2 marks)

Every function should do exactly one job and do it well.

Q3. Write the steps to design a helper function. Illustrate with a function that finds the largest of three numbers. (5 marks)

- Identify the task: find the largest.
- Choose parameters: three ints.
- Choose return type: int.
- Write the body with comparisons and return.
- Call it from main() and test with normal and boundary values.

```
int findLargest(int a, int b, int c)
{
   if (a >= b && a >= c) return a;
   else if (b >= c)      return b;
   else                  return c;
}
```

Q4. Explain the advantages of modular programming. (5 marks)

- Readability: main() reads like an outline.
- Debugging: an error is traced to one function.
- Reuse: write once, call many times.
- Complexity: small pieces are easy to understand.
- Testing and team work: functions can be tested and written independently.

Q5. Write a modular program to compute the net salary: HRA 20%, DA 10% and PF 12% of the basic salary ₹30,000. (10 marks)

Five helpers: HRA, DA, PF, net salary and display. Net = basic + HRA + DA − PF = 30000 + 6000 + 3000 − 3600 = 35400.

```
#include <stdio.h>
float calcHRA(float basic) { return basic * 0.20; }
float calcDA(float basic)  { return basic * 0.10; }
float calcPF(float basic)  { return basic * 0.12; }
float netSalary(float basic, float hra, float da, float pf)
{
   return basic + hra + da - pf;
}
void display(float net)
{
   printf("Net Salary = Rs. %.2f", net);
}
int main()
{
   float basic = 30000;
   float hra = calcHRA(basic), da = calcDA(basic), pf = calcPF(basic);
   display(netSalary(basic, hra, da, pf));
   return 0;
}
```

Output:

```
Net Salary = Rs. 35400.00
```

Interview and Placement Questions — Short Answers

Q6. Why should a helper return a value instead of printing it?

A returned value can be reused, stored, compared and tested; printed output cannot.

Q7. How do you make an isPrime function efficient?

Check divisors only up to the square root of n (i * i <= n). Handle n < 2 first and skip even divisors for further speed. Complexity O(√n).

Q8. What is the difference between a function and a macro?

A function is a real call checked by the compiler; a macro is textual substitution by the preprocessor with no type checking, and SQUARE(2+3) without brackets gives 11.

Q9. Where should prototypes be placed in a large program?

At the top of the file or, better, in a header (.h) file that is #included wherever the functions are used.

Q10. What is the difference between a helper that returns 1/0 and one that prints "Yes/No"?

The 1/0 helper can be used in conditions and reused; the printing one cannot.

23.4 Multiple Choice Questions

1. Which is an advantage of modular programming?

- a) No need to test the program
- b) No need for any parameters
- c) Improved readability and easier debugging
- d) Programs automatically run faster

Answer: c) Improved readability and easier debugging (Small functions are easier to read, test and reuse.)

2. The most suitable return type for a helper that calculates the average of marks is

- a) int
- b) float
- c) char
- d) void

Answer: b) float (An average can have a fractional part.)

3. A correct isPrime(n) must return 0 for

- a) n = 7
- b) n = 3
- c) n = 1
- d) n = 2

Answer: c) n = 1 (Numbers below 2 are not prime.)

4. Electricity slabs: first 100 units at ₹2, next 100 at ₹3, above 200 at ₹5. The bill for 250 units is

- a) ₹850
- b) ₹750
- c) ₹1250
- d) ₹500

Answer: b) ₹750 (100×2 + 100×3 + 50×5 = 200 + 300 + 250.)

5. float f(int t){ return t / 3; } printf("%.2f", f(245)); prints

- a) 81.00
- b) 82.00
- c) 0.00
- d) 81.67

Answer: a) 81.00 (Integer division gives 81 before conversion to float.)

6. #define SQ(x) x*x printf("%d", SQ(3+1)); prints

- a) 7
- b) 16
- c) 12
- d) 8

Answer: a) 7 (The macro expands to 3+1*3+1 = 7. Without brackets, precedence bites.)

23.5 Solved Programs

Program 1: isPrime Helper

Problem: Write a helper that returns 1 for a prime and 0 otherwise, and use it twice.

Approach: Only divisors up to the square root of n need to be tested.

```
#include <stdio.h>
int isPrime(int n) {
    if (n < 2) return 0;
    for (int i = 2; i * i <= n; i++)
        if (n % i == 0) return 0;
    return 1;
}
int main() {
    printf("17 is %s\n", isPrime(17) ? "prime" : "not prime");
    printf("18 is %s\n", isPrime(18) ? "prime" : "not prime");
    return 0;
}
```

Output:

```
17 is prime
18 is not prime
```

Program 2: A Helper Calling a Helper

Problem: Compute a squared sum using one helper inside another.

Approach: sumOfSquares() reuses square() instead of repeating the multiplication.

```
#include <stdio.h>
int square(int x) { return x * x; }
int sumOfSquares(int a, int b) { return square(a) + square(b); }
int main() {
    printf("3^2 + 4^2 = %d\n", sumOfSquares(3, 4));
    return 0;
}
```

Output:

```
3^2 + 4^2 = 25
```

23.6 Practice Problems — Try It Yourself

Attempt these without looking at the solved examples. Write the program, run it, and compare the output with your hand-trace.

1. Modular student result: write helpers for total of four subjects, percentage, and pass/fail (pass if percentage ≥ 40). Display all three.
2. Employee net salary: gross = basic + 20% HRA + 10% DA; deduct 12% of basic as PF. Write one helper for each step.
3. Write helpers simpleInterest(p, r, t) and compoundInterest(p, r, t) and compare them for 10000, 5%, 2 years.
4. Write a helper that returns 1 if a number is a palindrome (121, 1331) and 0 otherwise, and use it to test 121 and 123.
5. Extend the electricity bill: if the bill amount exceeds ₹500, add a 10% surcharge. Use calculateBill(), calculateCharges() and displayBill().
6. Write isPrime() and nextPrime(n) that returns the smallest prime greater than n. Test with n = 14.
7. Design (as a table of function name, arguments, return type) the helpers for an online-shopping bill: item total, discount, tax, final amount, display.

Chapter 24
Recursive Thinking and Recursion

Recursive thinking means solving a problem by solving a smaller version of the same problem. In C it is implemented by a function that **calls itself**. Every recursive solution has two parts: a **base case** (the simplest problem, solved directly — it stops the recursion) and a **recursive case** (reduce the problem and call the function again).

Recipe: **Identify → Reduce → Repeat → Stop at the base case.** Each call is kept on the **call stack**; when the base case is reached the calls return in reverse order. Without a base case, the calls never stop and the stack overflows.

Problem	Base case	Recursive case
Factorial n!	0! = 1	n × (n − 1)!
Sum of 1..n	sum(0) = 0	n + sum(n − 1)
Fibonacci F(n)	F(0) = 0, F(1) = 1	F(n − 1) + F(n − 2)
GCD(a, b)	gcd(a, 0) = a	gcd(b, a % b)
Power x^n	$x^0 = 1$	$x \times x^{n-1}$

Common mistake: *forgetting the base case, or writing a recursive call that does not make the problem smaller (for example factorial(n) instead of factorial(n − 1)). Both give endless recursion and a stack overflow crash.*

24.1 Theory — Understand It First

The Big Idea

Analogy: Two mirrors facing each other, or a set of **Russian dolls**: to open the big doll you open a smaller one of the same kind, and again, until you reach the tiniest doll that does not open (the base case). Then you close them back one by one.

Technical Concept: **Recursion** is a technique in which a function **calls itself** to solve a smaller instance of the same problem. It needs a **base case** (solved directly, stops the recursion) and a **recursive case** (reduces the problem and calls itself). Each step must move closer to the base case.

Where It Is Used: Factorial, Fibonacci, GCD, power, digit problems, searching (binary search), sorting (merge sort, quick sort), tree and graph traversal, Tower of Hanoi, backtracking (maze, N-Queens), file-system traversal.

> **Why We Use It:** Some problems are naturally defined in terms of smaller copies of themselves; recursion gives short, clear solutions for them.

Anatomy of a Recursive Function

Part	Role	In factorial
Base case	Simplest case, solved directly; stops the calls	if (n == 0) return 1;
Recursive case	Reduces the problem and calls the function again	return n * factorial(n - 1);
Progress	Every call must get closer to the base case	n decreases by 1 each call

Recipe: Identify → Reduce → Repeat → Stop. Ask: (1) How can I express the answer using the answer to a smaller problem? (2) What is the smallest problem whose answer I know? (3) Does each call shrink the problem?

How Recursion Works: The Call Stack

Each call gets its own **stack frame** holding its parameters, local variables and the return address. Frames are pushed on the way down and popped on the way back, in reverse order (last in, first out).

```
factorial(3)
  = 3 * factorial(2)            Calls go DOWN (frames pushed)
      = 2 * factorial(1)
          = 1 * factorial(0)
              = 1           <- base case reached
          = 1 * 1 = 1           Results come back UP (frames popped)
      = 2 * 1 = 2
  = 3 * 2 = 6

Stack at deepest point:  | fact(0) |
                         | fact(1) |
                         | fact(2) |
                         | fact(3) |
                         |  main   |
```

If there is no base case, or the call does not shrink the problem, frames keep being pushed until the stack is full: ***stack overflow*** *(the program crashes).*

Types of Recursion

Type	Meaning	Example
Direct	Function calls itself	factorial(n) calls factorial(n−1)
Indirect (mutual)	f calls g and g calls f	isEven ↔ isOdd
Tail	The recursive call is the **last** action; nothing remains to do after it returns	factorial with an accumulator
Non-tail (head)	Work remains after the call returns	n * factorial(n−1)
Linear	One recursive call per invocation	factorial, sum of digits
Tree	Two or more recursive calls per invocation	fib(n−1) + fib(n−2), Hanoi

```
#include <stdio.h>
int isOdd(int n);
int isEven(int n) { if (n == 0) return 1; return isOdd(n - 1); }
int isOdd(int n)  { if (n == 0) return 0; return isEven(n - 1); }

int factTail(int n, int acc)          /* tail recursive */
{
   if (n == 0) return acc;
   return factTail(n - 1, acc * n);
}
int main()
{
   printf("%d %d\n", isEven(10), isOdd(7));
   printf("%d", factTail(5, 1));
   return 0;
}
```

Output:

```
1 1
120
```

Recursion vs Iteration

Aspect	Recursion	Iteration (loop)
Mechanism	Function calls itself	for / while / do-while
Termination	Base case	Loop condition becomes false
Memory	A stack frame per call (more)	Fixed (less)

Aspect	Recursion	Iteration (loop)
Speed	Slower (call overhead)	Faster
Code	Short, close to the mathematical definition	Longer for some problems
Risk	Stack overflow	Infinite loop
Best for	Trees, divide-and-conquer, backtracking	Simple repetition, counting

Cost of Recursion: Fibonacci Calls and Memoization

The plain Fibonacci function is a **tree recursion**: it recomputes the same values again and again, so the number of calls grows exponentially (about 2^n). Storing results already computed (**memoization**) removes the repetition.

```
#include <stdio.h>
int calls = 0;
int fib(int n)
{
   calls++;
   if (n <= 1) return n;
   return fib(n - 1) + fib(n - 2);
}
long long memo[50];
long long fibMemo(int n)
{
   if (n <= 1) return n;
   if (memo[n] != 0) return memo[n];
   memo[n] = fibMemo(n - 1) + fibMemo(n - 2);
   return memo[n];
}
int main()
{
   int r = fib(5);
   printf("fib(5) = %d using %d calls\n", r, calls);
   calls = 0;
   fib(10);
   printf("fib(10) needs %d calls\n", calls);
   printf("fibMemo(40) = %lld", fibMemo(40));
   return 0;
}
```

Output:

```
fib(5) = 5 using 15 calls
fib(10) needs 177 calls
fibMemo(40) = 102334155
```

Recurrence Relations for the Standard Problems

Problem	Recurrence	Base case	Calls
Factorial	$n! = n \times (n-1)!$	$0! = 1$	$n + 1$
Sum 1..n	$S(n) = n + S(n-1)$	$S(0) = 0$	$n + 1$
Fibonacci	$F(n) = F(n-1) + F(n-2)$	$F(0)=0, F(1)=1$	$\approx 2^n$
GCD	gcd(a, b) = gcd(b, a mod b)	gcd(a, 0) = a	$\approx \log b$
Power x^n	$x^n = x \times x^{n-1}$	$x^0 = 1$	$n + 1$
Fast power	$x^n = (x^{n/2})^2$ (× x if n odd)	$x^0 = 1$	$\approx \log n$
Sum of digits	sd(n) = n mod 10 + sd(n / 10)	n = 0 → 0	digits + 1
Binary search	search the half that can contain the key	range empty / found	$\approx \log n$
Tower of Hanoi	$H(n) = 2 \cdot H(n-1) + 1$ moves	n = 0 → 0 moves	$2^n - 1$

24.2 Solved Examples and Debugging

Solved Examples — Easy

Define the base case and the recursive case. Write a recursive factorial and trace factorial(5).

Base case: 0! = 1. Recursive case: n! = n × (n − 1)!. The calls go down 5 → 4 → 3 → 2 → 1 → 0, then the results come back up: 1, 1, 2, 6, 24, 120.

```
#include <stdio.h>
int factorial(int n)
{
   if (n == 0)
      return 1;
   return n * factorial(n - 1);
}
int main()
{
   printf("5! = %d", factorial(5));
   return 0;
}
```

Output:

```
5! = 120
```

Write a recursive function for the sum of the first n natural numbers.

sum(5) = 5 + sum(4) = 5 + 4 + sum(3) … down to sum(0) = 0. The result is 5 + 4 + 3 + 2 + 1 = 15.

```
#include <stdio.h>
int sum(int n)
{
   if (n == 0)
      return 0;
   return n + sum(n - 1);
}
int main()
{
   printf("Sum = %d", sum(5));
   return 0;
}
```

Output:

```
Sum = 15
```

Solved Examples — Medium

Print the first 8 terms of the Fibonacci series using recursion. Identify the two base cases.

F(n) = F(n − 1) + F(n − 2) with base cases F(0) = 0 and F(1) = 1. Two base cases are needed because each term depends on the two before it.

```
#include <stdio.h>
int fib(int n)
{
   if (n == 0) return 0;
   if (n == 1) return 1;
   return fib(n - 1) + fib(n - 2);
}
int main()
{
   int i;
   for (i = 0; i < 8; i++)
      printf("%d ", fib(i));
   return 0;
}
```

Output:

```
0 1 1 2 3 5 8 13
```

Count the digits of a number recursively.

Base case: a number below 10 has one digit. Recursive case: removing the last digit (n / 10) leaves a number with one digit fewer, so count(n) = 1 + count(n / 10).

```
#include <stdio.h>
int countDigits(int n)
{
   if (n < 10)
      return 1;
   return 1 + countDigits(n / 10);
}
int main()
{
   printf("Digits = %d", countDigits(12345));
   return 0;
}
```

Output:

```
Digits = 5
```

Write a recursive function power(x, n) for x^n ($n \geq 0$) and compute 2^5.

$x^0 = 1$ is the base case; $x^n = x \times x^{n-1}$ is the recursive case. $2^5 = 2 \times 2^4 = \ldots = 32$.

```
#include <stdio.h>
int power(int x, int n)
{
   if (n == 0)
      return 1;
   return x * power(x, n - 1);
}
int main()
{
   printf("2^5 = %d", power(2, 5));
   return 0;
}
```

Output:

```
2^5 = 32
```

Solved Examples — HOTS

Find the GCD of 48 and 18 using Euclid's method recursively. Trace every call.

gcd(a, b) = gcd(b, a % b), and gcd(a, 0) = a. Trace: gcd(48, 18) → gcd(18, 12) → gcd(12, 6) → gcd(6, 0) = 6.

```
#include <stdio.h>
int gcd(int a, int b)
{
   if (b == 0)
      return a;
   return gcd(b, a % b);
}
int main()
{
   printf("GCD = %d", gcd(48, 18));
   return 0;
}
```

Output:

```
GCD = 6
```

Reverse a number recursively (1234 → 4321) using a second parameter that builds the answer.

Each call moves the last digit of n onto the end of rev: rev = rev × 10 + n % 10, and n shrinks by n / 10. When n reaches 0 (base case), rev holds the reversed number. Trace: (1234, 0) → (123, 4) → (12, 43) → (1, 432) → (0, 4321).

```
#include <stdio.h>
int reverse(int n, int rev)
{
   if (n == 0)
      return rev;
   return reverse(n / 10, rev * 10 + n % 10);
}
int main()
{
   printf("%d", reverse(1234, 0));
   return 0;
}
```

Output:

```
4321
```

Check whether a number is a palindrome by reversing it recursively and comparing with the original.

A number is a palindrome if it equals its reverse. The reverse helper is reused; the checking helper only compares.

```
#include <stdio.h>
int reverse(int n, int rev)
{
   if (n == 0)
      return rev;
   return reverse(n / 10, rev * 10 + n % 10);
}
int isPalindrome(int n)
{
   return n == reverse(n, 0);
}
int main()
{
   if (isPalindrome(121)) printf("121 is a Palindrome\n");
   else                   printf("121 is not a Palindrome\n");
   if (isPalindrome(123)) printf("123 is a Palindrome");
   else                   printf("123 is not a Palindrome");
   return 0;
}
```

Output:

```
121 is a Palindrome
123 is not a Palindrome
```

Predict the output. Why does the function print the numbers twice, once on the way down and once on the way back up?

The first printf runs before the recursive call (going down: 3, 2, 1). When show(0) hits the base case it returns, and each earlier call resumes at the statement after its recursive call, so the second printf runs on the way back up (1, 2, 3). This is the call stack unwinding.

```
#include <stdio.h>
void show(int n)
{
   if (n == 0)
      return;
   printf("%d ", n);
   show(n - 1);
   printf("%d ", n);
}
int main()
{
   show(3);
   return 0;
}
```

Output:

```
3 2 1 1 2 3
```

Debugging Corner — Find the Bug (Solved)

DEBUG 1

factorial(5) crashes the program. What is missing?

```
#include <stdio.h>
int factorial(int n)
{
   return n * factorial(n - 1);
}
int main()
{
   printf("Factorial = %d", factorial(5));
   return 0;
}
```

Output of the buggy program: Runtime crash (segmentation fault)

What is wrong?

- **No base case.** factorial keeps calling itself with 4, 3, 2, 1, 0, −1, −2 … and never stops, until the call stack overflows (segmentation fault). Add a base case: if (n == 0) return 1;

Corrected code:

```
#include <stdio.h>
int factorial(int n)
{
   if (n == 0)
      return 1;
   return n * factorial(n - 1);
}
int main()
{
   printf("Factorial = %d", factorial(5));
   return 0;
}
```

Output:

```
Factorial = 120
```

DEBUG 2

The sum of 1 + 2 + 3 should be 6, but the program prints 7. Why?

```
#include <stdio.h>
int sum(int n)
{
   if (n == 0)
      return 1;
   return n + sum(n - 1);
}
int main()
{
   printf("Sum = %d", sum(3));
   return 0;
}
```

Output of the buggy program: Sum = 7

What is wrong?

- **Wrong base-case value.** The base case returns 1, so an extra 1 is added at the bottom: 3 + 2 + 1 + 1 = 7. The sum of no numbers is 0, so the base case must return 0.

Corrected code:

```
#include <stdio.h>
int sum(int n)
{
   if (n == 0)
      return 0;
   return n + sum(n - 1);
}
int main()
{
   printf("Sum = %d", sum(3));
   return 0;
}
```

Output:

```
Sum = 6
```

Debug It Yourself — Practice (answers in Appendix B)

DEBUG IT 1

power(2, 3) crashes. What is wrong with the recursive call?

```
#include <stdio.h>
int power(int x, int n)
{
   if (n == 0)
      return 1;
   return x * power(x, n);
}
int main()
{
   printf("%d", power(2, 3));
   return 0;
}
```

Hint: *Is the problem getting smaller in every call?*

DEBUG IT 2

fib(5) should be 5 but 8 is printed. Which base case is wrong?

```
#include <stdio.h>
int fib(int n)
{
   if (n <= 1)
      return 1;
   return fib(n - 1) + fib(n - 2);
}
int main()
{
   printf("%d", fib(5));
   return 0;
}
```

Hint: *What should F(0) be in the Fibonacci series 0, 1, 1, 2, 3, 5?*

24.3 Important Questions

Exam Questions — Model Answers (2 / 5 / 10 marks)

Q1. Define recursion. What is a base case? (2 marks)

Recursion is a technique in which a function calls itself to solve smaller instances of the same problem. The base case is the simplest case, solved directly, that stops the recursion.

Q2. Differentiate between recursion and iteration. (5 marks)

(Write the table.) Recursion uses function calls and a base case, needs a stack frame per call and is shorter but slower; iteration uses loops, constant memory and is faster; recursion can overflow the stack, iteration can loop infinitely.

Q3. Trace the execution of factorial(4). (5 marks)

- Calls: fact(4) → fact(3) → fact(2) → fact(1) → fact(0).
- fact(0) returns 1 (base case).
- fact(1) = 1×1 = 1; fact(2) = 2×1 = 2; fact(3) = 3×2 = 6; fact(4) = 4×6 = 24.

Q4. Explain the types of recursion with examples. (5 marks)

- Direct: factorial.
- Indirect: isEven and isOdd calling each other.
- Tail: recursive call is the last statement (factorial with accumulator).
- Linear vs tree: one call per invocation (factorial) versus two (Fibonacci).

Q5. Write a recursive program to find the GCD of two numbers and trace gcd(48, 18). (10 marks)

Euclid's recurrence gcd(a, b) = gcd(b, a % b) with gcd(a, 0) = a. Trace: gcd(48,18) → gcd(18,12) → gcd(12,6) → gcd(6,0) = 6.

```
#include <stdio.h>
int gcd(int a, int b)
{
   if (b == 0)
      return a;
   return gcd(b, a % b);
}
int main()
{
   printf("GCD(48, 18) = %d", gcd(48, 18));
   return 0;
}
```

Output:

```
GCD(48, 18) = 6
```

Interview and Placement Questions — Short Answers

Q6. What is a stack overflow in recursion and when does it occur?

The call stack runs out of memory because calls keep being added. It occurs when there is no base case or the call never reaches it.

Q7. What is tail recursion and why is it useful?

The recursive call is the last operation. A compiler can reuse the same frame, making it as efficient as a loop (tail-call optimisation).

Q8. What is the time complexity of the plain recursive Fibonacci? How do you improve it?

About $O(2^n)$. Use memoization or an iterative loop to bring it to O(n).

Q9. Convert a recursive function to an iterative one — when is it hard?

Linear recursion converts easily to a loop. Tree recursion (Hanoi, tree traversal) needs an explicit stack.

Q10. Can main() be called recursively?

It is legal in C, but not recommended; it must have a base case like any other recursive function.

Q11. What does a recursive function that prints before and after the recursive call output?

Statements before the call run on the way down (in call order); statements after it run on the way back up (in reverse order).

Q12. What is the number of moves in Tower of Hanoi with n disks?

$2^n - 1$.

24.4 Multiple Choice Questions

1. Every recursive function must have

- a) a global variable
- b) at least two parameters
- c) a loop inside it
- d) a base case

Answer: d) a base case (The base case stops the recursion.)

2. A recursive function without a base case usually ends in

- a) compile-time success and correct output
- b) a faster program
- c) stack overflow
- d) garbage collection

Answer: c) stack overflow (Calls pile up on the stack until memory runs out.)

3. If factorial(0) = 1 and factorial(n) = n × factorial(n − 1), then factorial(4) is

- a) 16
- b) 4
- c) 10
- d) 24

Answer: d) 24 (4 × 3 × 2 × 1 × 1 = 24.)

4. Using Euclid's method, gcd(48, 18) is

- a) 6
- b) 18
- c) 3

- d) 12

Answer: a) 6 (gcd(48,18) → gcd(18,12) → gcd(12,6) → gcd(6,0) = 6.)

5. In the Fibonacci series with F(0) = 0 and F(1) = 1, the value of F(6) is

- a) 13
- b) 8
- c) 5
- d) 6

Answer: b) 8 (0, 1, 1, 2, 3, 5, 8 — the 7th term (index 6).)

6. Recursive function calls are kept in the

- a) output buffer
- b) call stack
- c) header file
- d) hard disk

Answer: b) call stack (Each active call occupies a stack frame.)

7. void fun(int n){ if (n > 0){ printf("%d", n); fun(n - 1); printf("%d", n); } } fun(2) prints

- a) 1221
- b) 2211
- c) 2112
- d) 2121

Answer: c) 2112 (Down: 2, 1; then back up: 1, 2.)

8. int h(int n){ if (n <= 0) return 0; return n + h(n - 2); } h(7) returns

- a) 7
- b) 16
- c) 10
- d) 28

Answer: b) 16 (7 + 5 + 3 + 1 + 0 = 16.)

9. Calls made by plain recursive fib(5) (counting every call, with fib(0) = 0, fib(1) = 1)

- a) 25
- b) 8

- c) 15
- d) 5

Answer: c) 15 (The call tree has 15 nodes; Fibonacci recursion is exponential.)

10. Minimum number of moves in Tower of Hanoi with 4 disks

- a) 7
- b) 8
- c) 16
- d) 15

Answer: d) 15 ($2^n - 1 = 15$.)

11. What is wrong with: int f(int n) { if (n == 0) return 1; return n * f(n); }

- a) The recursive call does not reduce n, so there is infinite recursion
- b) It has no return type
- c) It needs a loop
- d) Nothing is wrong

Answer: a) The recursive call does not reduce n, so there is infinite recursion (The problem never gets closer to the base case.)

12. The time complexity of binary search (recursive) on n sorted elements is

- a) O(1)
- b) O(log n)
- c) $O(n^2)$
- d) O(n)

Answer: b) O(log n) (Each call halves the range.)

13. Which of these is a tail-recursive formulation of factorial?

- a) return fact(n + 1)
- b) return n × fact(n − 1)
- c) fact(n, acc) returns fact(n − 1, acc × n)
- d) a loop with while

Answer: c) fact(n, acc) returns fact(n − 1, acc × n) (The recursive call is the last operation; nothing is left to do after it.)

24.5 Solved Programs

Program 1: Factorial

Problem: Find 5! recursively.

Approach: Base case: factorial(0) = 1. Recursive case: n * factorial(n - 1).

```
#include <stdio.h>
int factorial(int n) {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}
int main() {
    printf("5! = %d\n", factorial(5));
    return 0;
}
```

Output:

```
5! = 120
```

Program 2: Sum of Digits

Problem: Add the digits of 2749 recursively.

Approach: The last digit is n % 10; the remaining digits are n / 10; the base case is n == 0.

```
#include <stdio.h>
int sumDigits(int n) {
    if (n == 0) return 0;
    return n % 10 + sumDigits(n / 10);
}
int main() {
    printf("Sum of digits of 2749 = %d\n", sumDigits(2749));
    return 0;
}
```

Output:

```
Sum of digits of 2749 = 22
```

24.6 Practice Problems — Try It Yourself

Attempt these without looking at the solved examples. Write the program, run it, and compare the output with your hand-trace.

1. Write a recursive function that prints the numbers from n down to 1. Then change it so it prints 1 up to n.
2. Write a recursive function that returns the sum of the digits of a number (for example 1234 → 10).
3. Write both a recursive and an iterative factorial. Trace factorial(4) by hand on the call stack.
4. Draw the call tree of fib(5). How many calls are made altogether?
5. Write a recursive power(x, n) and use it to compute 3^4 (81). What is the base case?
6. Find the GCD of 54 and 24 recursively. List every call with its arguments.
7. Write a recursive function that prints the binary form of a decimal number (13 → 1101).

Chapter 25
Scope, Lifetime and Storage Classes

Two questions decide how a variable behaves: **where can it be seen?** (scope) and **how long does it live?** (lifetime). These are asked in almost every exam and interview on functions, because parameters, local variables, global variables and static variables all behave differently.

Term	Meaning
Scope	The region of the program where a variable name can be used
Lifetime	The period during which the variable exists in memory
Local (auto) variable	Declared inside a function or block; scope = that block; lifetime = while the block runs; initial value = garbage
Global variable	Declared outside every function; scope = rest of the file; lifetime = whole program; initial value = 0
Static local variable	Declared with static inside a function; scope = that function; lifetime = whole program; keeps its value between calls; initial value = 0

Common mistake: *assuming a local variable keeps its value between calls. A normal local is recreated (with garbage) on every call; only a static local remembers.*

25.1 Theory — Understand It First

The Big Idea

Analogy: A whiteboard in one classroom (local) is visible only to that class and is wiped when the class ends. A notice board in the corridor (global) is visible to everyone all day. A locker with your name (static) belongs to your room but keeps your things even when you leave.

Technical Concept: Scope is the region of the program in which a name is visible; lifetime is the period during which the variable exists in memory. The storage classes auto, register, static and extern control both, together with the default initial value.

Where It Is Used: Counters that must survive between calls (static), configuration shared by many functions (global/extern), and loop variables that must vanish after the loop (block scope).

> **Why We Use It:** It explains why a function cannot see main()'s variables, why a copy changes while the original does not, why a counter resets on every call, and why global variables make programs hard to debug.

The Four Storage Classes

Storage class	Declared	Stored in	Initial value	Scope	Lifetime
auto	Inside a block (default)	Stack	Garbage	Block	Until block ends
register	Inside a block	CPU register (request)	Garbage	Block	Until block ends
static	Inside a function / at file level	Data segment	0	Block / file	Whole program
extern	Declaration of a variable defined elsewhere	Data segment	0	Global	Whole program

- **register** is only a request to the compiler for speed; the address of a register variable cannot be taken (&r is an error).
- **extern** says "this variable is defined elsewhere (another file or later in this file)"; it does not create a new variable.
- A global variable can be modified by any function — convenient but risky (high coupling). Prefer passing parameters.
- When a local variable has the same name as a global one, the local **shadows** the global inside its block.

Static Local Variable — Remembering Between Calls

```
#include <stdio.h>
void counterAuto(void)
{
   int c = 0;          /* recreated at every call */
   c++;
   printf("auto: %d  ", c);
}
void counterStatic(void)
{
   static int c = 0;    /* initialised once, then remembered */
   c++;
   printf("static: %d\n", c);
}
int main()
{
   counterAuto();   counterStatic();
   counterAuto();   counterStatic();
   counterAuto();   counterStatic();
   return 0;
}
```

Output:

```
auto: 1  static: 1
auto: 1  static: 2
auto: 1  static: 3
```

Global Variables and Shadowing

```
#include <stdio.h>
int x = 100;                /* global */
void show(void)
{
   printf("show: x = %d\n", x);      /* uses the global */
}
int main()
{
   int x = 5;               /* local x shadows the global */
   printf("main: x = %d\n", x);
   show();
   {
      int x = 50;           /* inner block shadows again */
      printf("inner: x = %d\n", x);
   }
   printf("main again: x = %d", x);
   return 0;
}
```

Output:

```
main: x = 5
show: x = 100
inner: x = 50
main again: x = 5
```

extern — Using a Variable Defined Elsewhere

```
#include <stdio.h>
extern int total;         /* declaration only: no new variable */
void addToTotal(int n)
{
   total = total + n;
}
int total = 10;           /* the single definition */
int main()
{
   addToTotal(5);
   addToTotal(7);
   printf("total = %d", total);
   return 0;
}
```

Output:

```
total = 22
```

25.2 Solved Examples and Debugging

Solved Examples — Easy

Define the scope of a variable. What is the difference between a local and a global variable?

Scope is the region of the program in which a variable can be used. A local variable is declared inside a function and can be used only there; a global variable is declared outside all functions and can be used by every function that follows its declaration. The function below cannot see main()'s local variable, but both functions use the global g.

```
#include <stdio.h>
int g = 7;                  /* global */
void show(void)
{
   int local = 3;           /* local to show */
   printf("g = %d, local = %d\n", g, local);
}
int main()
{
   show();
   g = g + 1;
   printf("g = %d", g);
   return 0;
}
```

Output:

```
g = 7, local = 3
g = 8
```

Write a function that counts how many times it has been called, using a static variable.

A static local is created once and keeps its value between calls, so it can count the calls.

```
#include <stdio.h>
int callCount(void)
{
   static int count = 0;
   count++;
   return count;
}
int main()
{
   callCount();
   callCount();
   printf("Called %d times", callCount());
   return 0;
}
```

Output:

```
Called 3 times
```

Solved Examples — Medium

Predict the output and explain which variable each printf uses.

The parameter x of f() is a separate local variable that shadows the global x. f(10) makes it 10, then 11, and prints 11. The global x (5) is never touched, so main() prints 5.

```
#include <stdio.h>
int x = 5;
void f(int x)
{
   x = x + 1;
   printf("f: x = %d\n", x);
}
int main()
{
   f(10);
   printf("main: x = %d", x);
   return 0;
}
```

Output:

```
f: x = 11
main: x = 5
```

A function must keep a running total of all the amounts passed to it across calls. Write it with a static variable and with a global variable, and compare the two.

Both give the same answer, but the static variable is hidden inside the function so no other function can change it by accident, while the global is open to the whole file.

```
#include <stdio.h>
int gTotal = 0;
void addGlobal(int n) { gTotal = gTotal + n; }
int addStatic(int n)
{
    static int total = 0;
    total = total + n;
    return total;
}
int main()
{
    addGlobal(10);  addGlobal(20);
    addStatic(10);
    printf("global = %d, static = %d", gTotal, addStatic(20));
    return 0;
}
```

Output:

```
global = 30, static = 30
```

Solved Examples — HOTS

Use a static variable inside a recursive function to count the number of recursive calls made while computing factorial(5).

A static variable is shared by all the recursive calls (there is only one copy), unlike an ordinary local, which is a new variable in every stack frame. It is incremented once per call: calls for n = 5, 4, 3, 2, 1, 0 = 6 calls.

```
#include <stdio.h>
int factorial(int n)
{
   static int calls = 0;
   calls++;
   if (n == 0)
   {
      printf("calls = %d\n", calls);
      return 1;
   }
   return n * factorial(n - 1);
}
int main()
{
   printf("5! = %d", factorial(5));
   return 0;
}
```

Output:

```
calls = 6
5! = 120
```

Predict the output. Explain why the two counters behave differently.

Each call to withLocal() creates a new c initialised to 10, so it always prints 11. withStatic() initialises its static c to 10 only once; the later calls continue from the previous value: 11, 12.

```
#include <stdio.h>
void withLocal(void)  { int c = 10;        c++; printf("%d ", c); }
void withStatic(void) { static int c = 10; c++; printf("%d ", c); }
int main()
{
   withLocal();  withLocal();
   withStatic(); withStatic();
   return 0;
}
```

Output:

```
11 11 11 12
```

Debugging Corner — Find the Bug (Solved)

DEBUG 1

The function is meant to number the tickets 1, 2, 3 but always prints 1. Why?

```
#include <stdio.h>
int nextTicket(void)
{
   int ticket = 0;
   ticket++;
   return ticket;
}
int main()
{
   printf("%d %d %d", nextTicket(), nextTicket(), nextTicket());
   return 0;
}
```

Output of the buggy program: 1 1 1

What is wrong?

- **Ordinary local variable.** ticket is created and set to 0 at every call, so it can never remember the previous value. Declare it static: static int ticket = 0;

Corrected code:

```
#include <stdio.h>
int nextTicket(void)
{
   static int ticket = 0;
   ticket++;
   return ticket;
}
int main()
{
   int a = nextTicket();
   int b = nextTicket();
   int c = nextTicket();
   printf("%d %d %d", a, b, c);
   return 0;
}
```

Output:

```
1 2 3
```

DEBUG 2

addToTotal() updates the global total, but main still prints 0. Why?

```
#include <stdio.h>
int total = 0;
void addToTotal(int n)
{
   total = total + n;
}
int main()
{
   int total = 0;
   addToTotal(25);
   printf("Total = %d", total);
   return 0;
}
```

Output of the buggy program: Total = 0

What is wrong?

- **Shadowing.** The local int total in main() hides the global total, so main prints its own local variable, which was never changed. Remove the local declaration so the global is used.

Corrected code:

```
#include <stdio.h>
int total = 0;
void addToTotal(int n)
{
   total = total + n;
}
int main()
{
   addToTotal(25);
   printf("Total = %d", total);
   return 0;
}
```

Output:

```
Total = 25
```

Debug It Yourself — Practice (answers in Appendix B)

DEBUG IT 1

The program prints a garbage value instead of 1 on the first call. What is missing?

```
#include <stdio.h>
int main()
{
   int count;
   count++;
   printf("%d", count);
   return 0;
}
```

Hint: *What value does an uninitialised local variable hold?*

DEBUG IT 2

The compiler says "'result' undeclared" in main(). Why?

```
#include <stdio.h>
void compute(void)
{
   int result = 42;
}
int main()
{
   compute();
   printf("%d", result);
   return 0;
}
```

Hint: *Where is result declared, and where is it used?*

25.3 Important Questions

Exam Questions — Model Answers (2 / 5 / 10 marks)

Q1. Differentiate between scope and lifetime of a variable. (2 marks)

Scope is the region where the name can be used; lifetime is the period for which the variable exists in memory.

Q2. Explain the four storage classes in C. (5 marks)

(Write the table of auto, register, static and extern with default value, scope and lifetime.)

Q3. Differentiate between local and global variables. (5 marks)

- Local: declared inside a function, visible only there, created on each call, garbage initial value, on the stack.
- Global: declared outside all functions, visible to every function after its declaration, lives for the whole program, initial value 0, in the data segment.

Q4. What is the use of a static variable inside a function? Show with a program. (5 marks)

It retains its value between calls and is initialised only once — useful for counters. See the counterStatic() program above (output 1, 2, 3).

Interview and Placement Questions — Short Answers

Q5. What is the default initial value of a local, a global and a static variable?

Local: garbage. Global and static: 0.

Q6. Where are local, global and static variables stored?

Local: stack. Global and static: the data segment (initialised or uninitialised/BSS).

Q7. What does static mean on a global variable or a function?

It gives internal linkage: the name is visible only inside that source file.

Q8. What is variable shadowing?

A variable in an inner scope with the same name hides the outer one inside that scope.

Q9. Why are global variables discouraged?

Any function can change them, so the data flow is hidden, coupling increases, and bugs are hard to trace and test.

25.4 Multiple Choice Questions

1. int f(int n){ static int c = 0; c += n; return c; } f(1); f(2); printf("%d", f(3)); prints

- a) 0
- b) 3
- c) 9
- d) 6

Answer: d) 6 (The static c keeps its value: 1, 3, 6.)

2. The default initial value of an uninitialised static or global int is

- a) 1
- b) −1
- c) 0
- d) garbage

Answer: c) 0 (Static storage is zero-initialised.)

3. A static local variable is initialised

- a) only when the program ends
- b) only once, and keeps its value between calls
- c) at every call
- d) never

Answer: b) only once, and keeps its value between calls (Its lifetime is the whole program.)

4. Which statement is true about variable scope?

- a) A local variable is visible in all functions

- b) A global variable can never be shadowed
- c) Parameters are global
- d) A local variable shadows a global variable of the same name inside its block

Answer: d) A local variable shadows a global variable of the same name inside its block (The inner declaration hides the outer one.)

5. Which storage class gives a variable that is visible only inside its function yet keeps its value between calls?

- a) register
- b) static
- c) auto
- d) extern

Answer: b) static (static local.)

25.5 Solved Programs

Program 1: static Remembers, auto Forgets

Problem: Compare a static local variable with an ordinary local variable.

Approach: A static local is created once and keeps its value between calls; an auto local is recreated each time.

```
#include <stdio.h>
void visit(void) {
    static int count = 0;
    count++;
    printf("Visit %d\n", count);
}
void normal(void) {
    int c = 0;
    c++;
    printf("Normal %d\n", c);
}
int main() {
    visit(); visit(); visit();
    normal(); normal();
    return 0;
}
```

Output:

```
Visit 1
Visit 2
Visit 3
Normal 1
Normal 1
```

Program 2: Local Variable Hides the Global

Problem: Show shadowing of a global variable by a local one.

Approach: Inside show(), the name total refers to the local variable.

```
#include <stdio.h>
int total = 100;
void show(void) {
    int total = 5;
    printf("Local total  = %d\n", total);
}
int main() {
    show();
    printf("Global total = %d\n", total);
    return 0;
}
```

Output:

```
Local total  = 5
Global total = 100
```

25.6 Practice Problems — Try It Yourself

Attempt these without looking at the solved examples. Write the program, run it, and compare the output with your hand-trace.

1. Write a function that returns the next even number each time it is called (2, 4, 6, …) using a static variable.
2. Predict the output: int x = 10; void f(){ x = x * 2; } main: f(); f(); printf("%d", x);
3. Write a program with a global variable and a local variable of the same name; show which one each printf uses.
4. List three differences between auto and static local variables.
5. Write a function that prints how many times it has been called (use static). Call it five times.

6. Why can main() not use a variable declared inside another function? Show the compiler error.

Chapter 26
Placement Logic Bank — Standard Programs Using Functions

These are the classic logic programs that appear again and again in university exams, campus placement tests and interviews. Every program is written the modular way — one helper function per job — and every one comes with its **logic** (the idea in plain steps), the **code**, and the **verified output**. Learn the logic first, then rewrite the code yourself without looking.

Almost every digit problem uses the same two-step trick: **n % 10** gives the last digit and **n / 10** removes it. Learn this pair well.

#	Program	#	Program
1	Sum of digits	12	Count how many times a digit occurs
2	Reverse a number and check for palindrome	13	Digital root
3	Armstrong number	14	Fast exponentiation x^n in O(log n)
4	Perfect number	15	Swap two numbers without a third variable
5	Strong number	16	Largest and second largest in an array
6	Prime numbers in a range	17	Reverse an array in place using two pointers
7	Prime factorisation	18	Linear search and recursive binary search
8	GCD and LCM	19	Tower of Hanoi
9	Factorial and nCr	20	Pattern printing with a function
10	Fibonacci series	21	Simple series
11	Decimal to binary	22	Leap-year check and number of days in a month

26.1 Solved Programs — Standard Programs Using Functions

Program 1 — Sum of digits

Logic

- Take the last digit with n % 10 and add it to sum.
- Remove the last digit with n / 10.
- Repeat until n becomes 0.

```
#include <stdio.h>
int sumOfDigits(int n)
{
    int sum = 0;
    while (n > 0)
    {
        sum = sum + n % 10;
        n = n / 10;
    }
    return sum;
}
int main()
{
    printf("Sum of digits of 4587 = %d", sumOfDigits(4587));
    return 0;
}
```

Output:

```
Sum of digits of 4587 = 24
```

Program 2 — Reverse a number and check for palindrome

Logic

- rev = 0. Repeatedly: rev = rev * 10 + n % 10; n = n / 10.
- A number is a palindrome if the reverse equals the original.

```
#include <stdio.h>
int reverse(int n)
{
    int rev = 0;
    while (n > 0)
    {
        rev = rev * 10 + n % 10;
        n = n / 10;
    }
    return rev;
}
int isPalindrome(int n)
{
    return n == reverse(n);
}
int main()
{
    printf("reverse(1234) = %d\n", reverse(1234));
    printf("12321 palindrome? %d\n", isPalindrome(12321));
    printf("1234 palindrome? %d", isPalindrome(1234));
    return 0;
}
```

Output:

```
reverse(1234) = 4321
12321 palindrome? 1
1234 palindrome? 0
```

Program 3 — Armstrong number (e.g. 153 = $1^3 + 5^3 + 3^3$)

Logic

- Count the digits d of n.
- Add (each digit)^d for every digit.
- If the sum equals n, it is an Armstrong number.

```
#include <stdio.h>
int countDigits(int n)
{
    int c = 0;
    while (n > 0) { c++; n = n / 10; }
    return c;
}
int power(int b, int e)
{
    int r = 1;
    while (e-- > 0) r = r * b;
    return r;
}
int isArmstrong(int n)
{
    int d = countDigits(n), sum = 0, t = n;
    while (t > 0)
    {
        sum = sum + power(t % 10, d);
        t = t / 10;
    }
    return sum == n;
}
int main()
{
    int i;
    printf("153 -> %d, 154 -> %d\n", isArmstrong(153), isArmstrong(154));
    printf("Armstrong numbers up to 500: ");
    for (i = 1; i <= 500; i++)
        if (isArmstrong(i))
            printf("%d ", i);
    return 0;
}
```

Output:

```
153 -> 1, 154 -> 0
Armstrong numbers up to 500: 1 2 3 4 5 6 7 8 9 153 370 371 407
```

Program 4 — Perfect number (6 = 1 + 2 + 3)

Logic

- Add all proper divisors of n (divisors from 1 to n/2).
- If the sum equals n, n is perfect.

```
#include <stdio.h>
int isPerfect(int n)
{
    int i, sum = 0;
    for (i = 1; i <= n / 2; i++)
        if (n % i == 0)
            sum = sum + i;
    return n > 1 && sum == n;
}
int main()
{
    int i;
    printf("Perfect numbers up to 1000: ");
    for (i = 1; i <= 1000; i++)
        if (isPerfect(i))
            printf("%d ", i);
    return 0;
}
```

Output:

```
Perfect numbers up to 1000: 6 28 496
```

Program 5 — Strong number (145 = 1! + 4! + 5!)

Logic

- Write a factorial helper.
- Add the factorial of every digit.
- If the sum equals n it is a strong number.

```
#include <stdio.h>
int factorial(int n)
{
   int f = 1;
   while (n > 1) { f = f * n; n--; }
   return f;
}
int isStrong(int n)
{
   int sum = 0, t = n;
   while (t > 0)
   {
      sum = sum + factorial(t % 10);
      t = t / 10;
   }
   return sum == n;
}
int main()
{
   printf("145 -> %d, 146 -> %d", isStrong(145), isStrong(146));
   return 0;
}
```

Output:

```
145 -> 1, 146 -> 0
```

Program 6 — Prime numbers in a range (efficient test)

Logic

- n < 2 is not prime.
- Try divisors i = 2, 3, … while i * i <= n.
- If any divides n, it is not prime.

```
#include <stdio.h>
int isPrime(int n)
{
    int i;
    if (n < 2) return 0;
    for (i = 2; i * i <= n; i++)
        if (n % i == 0) return 0;
    return 1;
}
void printPrimes(int lo, int hi)
{
    int n;
    for (n = lo; n <= hi; n++)
        if (isPrime(n))
            printf("%d ", n);
}
int main()
{
    printPrimes(10, 50);
    return 0;
}
```

Output:

```
11 13 17 19 23 29 31 37 41 43 47
```

Program 7 — Prime factorisation (60 = 2 × 2 × 3 × 5)

Logic

- Start with divisor d = 2.
- While d divides n, print d and divide n by d.
- Otherwise increase d. Stop when n becomes 1.

```
#include <stdio.h>
void primeFactors(int n)
{
    int d = 2;
    while (n > 1)
    {
        if (n % d == 0)
        {
            printf("%d ", d);
            n = n / d;
        }
        else
            d++;
    }
}
int main()
{
    printf("Prime factors of 60: ");
    primeFactors(60);
    printf("\nPrime factors of 97: ");
    primeFactors(97);
    return 0;
}
```

Output:

```
Prime factors of 60: 2 2 3 5
Prime factors of 97: 97
```

Program 8 — GCD and LCM

Logic

- GCD by Euclid: while b != 0 { r = a % b; a = b; b = r; } the answer is a.
- LCM = (a × b) / GCD.

```
#include <stdio.h>
int gcd(int a, int b)
{
   while (b != 0)
   {
      int r = a % b;
      a = b;
      b = r;
   }
   return a;
}
int lcm(int a, int b)
{
   return a / gcd(a, b) * b;
}
int main()
{
   printf("GCD(36, 60) = %d\n", gcd(36, 60));
   printf("LCM(36, 60) = %d", lcm(36, 60));
   return 0;
}
```

Output:

```
GCD(36, 60) = 12
LCM(36, 60) = 180
```

*Writing a / gcd(a, b) * b instead of a * b / gcd avoids overflow for large numbers.*

Program 9 — Factorial and nCr

Logic

- n! = 1 × 2 × … × n (0! = 1).
- nCr = n! / (r! × (n − r)!).

```
#include <stdio.h>
long long factorial(int n)
{
    long long f = 1;
    int i;
    for (i = 2; i <= n; i++) f = f * i;
    return f;
}
long long nCr(int n, int r)
{
    return factorial(n) / (factorial(r) * factorial(n - r));
}
int main()
{
    printf("5! = %lld\n", factorial(5));
    printf("10! = %lld\n", factorial(10));
    printf("5C2 = %lld", nCr(5, 2));
    return 0;
}
```

Output:

```
5! = 120
10! = 3628800
5C2 = 10
```

int overflows beyond 12!; long long is safe up to 20!.

Program 10 — Fibonacci series (iterative)

Logic

- Keep the last two terms a and b.
- next = a + b; then shift: a = b, b = next.

```
#include <stdio.h>
void printFibonacci(int n)
{
    int a = 0, b = 1, i, next;
    for (i = 0; i < n; i++)
    {
        printf("%d ", a);
        next = a + b;
        a = b;
        b = next;
    }
}
int main()
{
    printFibonacci(10);
    return 0;
}
```

Output:

```
0 1 1 2 3 5 8 13 21 34
```

Program 11 — Decimal to binary (recursive) and to any base

Logic

- Divide by the base and remember the remainder.
- The answer is the remainders read **in reverse**; recursion gives the reverse for free: recurse first, print after.

```
#include <stdio.h>
void toBase(int n, int base)
{
    if (n == 0)
        return;
    toBase(n / base, base);
    printf("%c", "0123456789ABCDEF"[n % base]);
}
int main()
{
    printf("13  in binary = ");  toBase(13, 2);
    printf("\n255 in octal  = "); toBase(255, 8);
    printf("\n255 in hex    = "); toBase(255, 16);
    return 0;
}
```

Output:

```
13  in binary = 1101
255 in octal  = 377
255 in hex    = FF
```

Program 12 — Count how many times a digit occurs

Logic

- Look at the last digit; if it equals d add 1.
- Remove the last digit and repeat.

```
#include <stdio.h>
int countDigit(int n, int d)
{
    int count = 0;
    while (n > 0)
    {
        if (n % 10 == d)
            count++;
        n = n / 10;
    }
    return count;
}
int main()
{
    printf("Digit 7 occurs %d times in 7177457", countDigit(7177457, 7));
    return 0;
}
```

Output:

```
Digit 7 occurs 4 times in 7177457
```

Program 13 — Digital root (recursive): repeat sum of digits until one digit remains

Logic

- If n < 10 it is already a single digit.
- Otherwise find the digit sum and apply the same function to it.

```
#include <stdio.h>
int digitSum(int n)
{
    return n == 0 ? 0 : n % 10 + digitSum(n / 10);
}
int digitalRoot(int n)
{
    if (n < 10)
        return n;
    return digitalRoot(digitSum(n));
}
int main()
{
    printf("digitalRoot(9875) = %d", digitalRoot(9875));
    return 0;
}
```

Output:

```
digitalRoot(9875) = 2
```

Program 14 — Fast exponentiation x^n in O(log n)

Logic

- $x^0 = 1$.
- Compute half = x^(n/2) once.
- If n is even the answer is half × half; if odd, half × half × x.

```
#include <stdio.h>
long long power(long long x, int n)
{
    long long half;
    if (n == 0)
        return 1;
    half = power(x, n / 2);
    if (n % 2 == 0)
        return half * half;
    return half * half * x;
}
int main()
{
    printf("2^10 = %lld\n", power(2, 10));
    printf("3^13 = %lld", power(3, 13));
    return 0;
}
```

Output:

```
2^10 = 1024
3^13 = 1594323
```

Program 15 — Swap two numbers without a third variable

Logic

- Arithmetic: a = a + b; b = a − b; a = a − b.
- Always guard the case where both pointers refer to the same variable.

```
#include <stdio.h>
void swap(int *a, int *b)
{
   if (a == b)
      return;              /* guard: same variable */
   *a = *a + *b;
   *b = *a - *b;
   *a = *a - *b;
}
int main()
{
   int x = 15, y = 40;
   swap(&x, &y);
   printf("x = %d, y = %d\n", x, y);
   swap(&x, &x);
   printf("x = %d", x);
   return 0;
}
```

Output:

```
x = 40, y = 15
x = 40
```

Program 16 — Largest and second largest in an array

Logic

- Keep two variables: first and second, both very small.
- For every element: if it is bigger than first → second = first, first = element; else if it is bigger than second (and not equal to first) → second = element.

```
#include <stdio.h>
void twoLargest(int a[], int n, int *first, int *second)
{
   int i;
   *first = *second = -2147483647;
   for (i = 0; i < n; i++)
   {
      if (a[i] > *first)
      {
         *second = *first;
         *first = a[i];
      }
      else if (a[i] > *second && a[i] != *first)
         *second = a[i];
   }
}
int main()
{
   int v[6] = {34, 89, 21, 89, 56, 7};
   int f, s;
   twoLargest(v, 6, &f, &s);
   printf("Largest = %d, Second largest = %d", f, s);
   return 0;
}
```

Output:

```
Largest = 89, Second largest = 56
```

Program 17 — Reverse an array in place using two pointers

Logic

- One pointer at the start, one at the end.
- Swap the two elements, move the pointers towards each other, stop when they meet.

```
#include <stdio.h>
void reverseArray(int *a, int n)
{
   int *lo = a, *hi = a + n - 1, t;
   while (lo < hi)
   {
      t = *lo;
      *lo = *hi;
      *hi = t;
      lo++;
      hi--;
   }
}
int main()
{
   int v[5] = {1, 2, 3, 4, 5}, i;
   reverseArray(v, 5);
   for (i = 0; i < 5; i++)
      printf("%d ", v[i]);
   return 0;
}
```

Output:

```
5 4 3 2 1
```

Program 18 — Linear search and recursive binary search

Logic

- Linear: check every element; return the index or −1.
- Binary (sorted array): compare the key with the middle; recurse into the left or right half; stop when the range is empty.

```
#include <stdio.h>
int linearSearch(int a[], int n, int key)
{
   int i;
   for (i = 0; i < n; i++)
      if (a[i] == key)
         return i;
   return -1;
}
int binarySearch(int a[], int lo, int hi, int key)
{
   int mid;
   if (lo > hi)
      return -1;
   mid = (lo + hi) / 2;
   if (a[mid] == key) return mid;
   if (key < a[mid])  return binarySearch(a, lo, mid - 1, key);
   return binarySearch(a, mid + 1, hi, key);
}
int main()
{
   int v[10] = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91};
   printf("linear(16) = %d\n", linearSearch(v, 10, 16));
   printf("binary(23) = %d\n", binarySearch(v, 0, 9, 23));
   printf("binary(40) = %d", binarySearch(v, 0, 9, 40));
   return 0;
}
```

Output:

```
linear(16) = 4
binary(23) = 5
binary(40) = -1
```

Program 19 — Tower of Hanoi (n = 3 needs $2^3 - 1 = 7$ moves)

Logic

- To move n disks from A to C using B: (1) move n−1 disks A → B, (2) move disk n A → C, (3) move n−1 disks B → C.
- Base case: n = 0, nothing to move.

```
#include <stdio.h>
void hanoi(int n, char from, char to, char via)
{
   if (n == 0)
      return;
   hanoi(n - 1, from, via, to);
   printf("Move disk %d from %c to %c\n", n, from, to);
   hanoi(n - 1, via, to, from);
}
int main()
{
   hanoi(3, 'A', 'C', 'B');
   return 0;
}
```

Output:

```
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

Program 20 — Pattern printing with a function (right triangle and pyramid)

Logic

- Outer loop = rows. Inner loops print spaces then stars.
- Row i of a pyramid of n rows: n − i spaces and 2i − 1 stars.

```
#include <stdio.h>
void triangle(int n)
{
   int i, j;
   for (i = 1; i <= n; i++)
   {
      for (j = 1; j <= i; j++)
         printf("*");
      printf("\n");
   }
}
void pyramid(int n)
{
   int i, j;
   for (i = 1; i <= n; i++)
   {
      for (j = 1; j <= n - i; j++) printf("");
      for (j = 1; j <= 2 * i - 1; j++) printf("*");
      printf("\n");
   }
}
int main()
{
   triangle(3);
   pyramid(3);
   return 0;
}
```

Output:

```
*
**
***
  *
 ***
*****
```

Program 21 — Simple series: sum of 1 + 1/2 + 1/3 + … + 1/n (harmonic)

Logic

- Loop i from 1 to n adding 1.0 / i.
- Use 1.0 (not 1) so the division is a floating-point division.

```
#include <stdio.h>
double harmonic(int n)
{
    double sum = 0;
    int i;
    for (i = 1; i <= n; i++)
        sum = sum + 1.0 / i;
    return sum;
}
int main()
{
    printf("H(4) = %.4f", harmonic(4));
    return 0;
}
```

Output:

```
H(4) = 2.0833
```

Program 22 — Leap-year check and number of days in a month

Logic

- Leap year: divisible by 400, or divisible by 4 but not by 100.
- February has 29 days in a leap year, 28 otherwise; April, June, September, November have 30; the rest 31.

```
#include <stdio.h>
int isLeap(int y)
{
   return (y % 400 == 0) || (y % 4 == 0 && y % 100 != 0);
}
int daysInMonth(int m, int y)
{
   if (m == 2)
      return isLeap(y) ? 29 : 28;
   if (m == 4 || m == 6 || m == 9 || m == 11)
      return 30;
   return 31;
}
int main()
{
   printf("Feb 2024: %d days\n", daysInMonth(2, 2024));
   printf("Feb 1900: %d days\n", daysInMonth(2, 1900));
   printf("Apr 2025: %d days", daysInMonth(4, 2025));
   return 0;
}
```

Output:

```
Feb 2024: 29 days
Feb 1900: 28 days
Apr 2025: 30 days
```

26.2 Important Questions

Q1. Why does an efficient prime test check divisors only up to the square root of n?

If n = a x b with a <= b, then a <= sqrt(n). So every composite number has a divisor not larger than its square root, and testing beyond it is wasted work. This cuts the steps from n to sqrt(n).

Q2. State Euclid's algorithm for GCD and find gcd(48, 18).

gcd(a, b) = gcd(b, a % b) until b becomes 0. For 48 and 18: (48,18) -> (18,12) -> (12,6) -> (6,0), so the GCD is 6. The LCM is a x b / gcd = 144.

Q3. How do you reverse a number and test for a palindrome?

Repeat: take digit = n % 10, set rev = rev * 10 + digit, then n = n / 10 until n is 0. The number is a palindrome if rev equals the original number.

Q4. What is an Armstrong number? Check 153.

A number equal to the sum of its digits each raised to the power of the number of digits. 1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153, so 153 is an Armstrong number.

26.3 Multiple Choice Questions

1. Which is the best reason to make isPrime(n) check divisors only up to √n?

- a) Because primes are square numbers
- b) Because n / 2 would give wrong answers
- c) Because the loop is shorter to type
- d) A larger factor always pairs with a smaller one that is found earlier

Answer: d) A larger factor always pairs with a smaller one that is found earlier (If n has a factor greater than √n it must also have one smaller than √n.)

2. The LCM of 12 and 18 is

- a) 6
- b) 36
- c) 72
- d) 216

Answer: b) 36 (gcd = 6, so lcm = 12 x 18 / 6 = 36.)

3. Which of these is a perfect number?

- a) 12
- b) 18
- c) 28
- d) 30

Answer: c) 28 (28 = 1 + 2 + 4 + 7 + 14.)

4. Which of these is an Armstrong number?

- a) 152
- b) 153
- c) 154
- d) 155

Answer: b) 153 (1 + 125 + 27 = 153.)

26.4 Practice Problems — Try It Yourself

1. **Write a function that counts the digits of a number.**
2. **Write a program that checks whether 145 is a strong number (sum of digit factorials equals the number).**
3. **Write a recursive function for the sum of the first n natural numbers.**
4. **Print the first ten Fibonacci numbers using a function.**
5. **Write a function that returns the largest of three numbers, and another that prints the multiplication table of n.**

Chapter 27
Unit 2 Revision — Practice MCQs and Quick Revision Sheet

27.1 Practice MCQs — Try It Yourself

Attempt without help. The answer key is in Appendix C.

1. float getPrice(void) is which type of function?

a) Arguments, no return value
b) No arguments, with a return value
c) No arguments, no return value
d) Arguments, with a return value

2. int add(int a, int b) { return a + b; } What happens for the call add(2)?

a) Prints garbage
b) Returns 0
c) Compilation error — too few arguments
d) Returns 2

3. void g(int n){ n = n * 2; } int x = 3; g(x); printf("%d", x); prints

a) 6
b) 0
c) Compilation error
d) 3

4. void f(int *p){ *p += 2; } int x = 1; f(&x); f(&x); f(&x); printf("%d", x); prints

a) 7
b) 1
c) 5
d) 3

5. Which statement is true about C?

a) All arguments are passed by reference
b) Arrays cannot be passed to functions
c) Pass by reference is simulated by passing addresses (pointers)
d) It has a built-in & parameter for references

6. int a[] = {2, 4, 6, 8}; int *p = a + 1; printf("%d", *p + *(p + 1)); prints

a) 12
b) 8
c) 6
d) 10

7. int f(int n){ if (n == 0) return 0; return n % 10 + f(n / 10); } f(123) returns

a) 321
b) 6
c) 3
d) 123

8. int add(int a, int b){ return a + b; } printf("%d", add(add(1, 2), add(3, 4))); prints

a) 10
b) 4
c) 7
d) 3

9. A void function is called as display(5); Which is true?

a) It can only be called from main()
b) It cannot have parameters
c) It must return an int
d) Its result cannot be assigned to a variable

10. Which declaration is NOT valid for a pointer to an integer variable x?

a) int *p; p = &x;
b) int *p = NULL;
c) int *p = x;
d) int *p = &x;

11. int f(int x){ static int s = 1; s = s * x; return s; } f(2); f(3); printf("%d", f(4)); prints

a) 24
b) 4
c) 9
d) 8

12. int a[] = {5, 10, 15, 20}; int *p = a; p += 2; printf("%d %d", *p, *(p - 1)); prints

a) 10 5
b) 15 10
c) 15 20
d) 10 15

13. A function that must change the caller's pointer itself (for example make it point elsewhere) needs a parameter of type

a) int
b) float
c) pointer to pointer
d) void

14. void print(int n){ if (n == 0) return; print(n - 1); printf("%d ", n); } print(4) prints

a) 4
b) 0 1 2 3
c) 4 3 2 1
d) 1 2 3 4

15. #define MAX(a,b) a>b?a:b Which is the safest macro design?

a) #define MAX a b
b) #define MAX(a,b) (((a)>(b))?(a):(b))
c) #define MAX(a,b) a>b?a:b
d) #define MAX(a,b) a,b

16. Which variable has the longest lifetime?

a) A static local variable
b) A register variable
c) A function parameter
d) An auto local variable

17. int f(int n){ return n <= 1 ? 1 : n * f(n - 1); } f(5) returns

a) 120
b) 15
c) 24
d) 5

18. Which call correctly passes the array arr of 10 ints to void show(int *a, int n)?

a) show(arr[], 10);
b) show(arr, 10);
c) show(*arr, 10);
d) show(&arr, 10);

27.2 Quick Revision Sheet — Rules, Formulas and Traps

One-page memory aids for the night before the exam or the interview.

Function types, parameters and passing

Topic	Remember
4 function types	void f(void) · void f(int) · int f(int) · int f(void)
Declaration / definition / call	type f(types); · type f(params){body} · f(args);
Parameter vs argument	Parameter = in the definition; argument = in the call; matched by position
Pass by value	Copy of value · int n · f(x) · original unchanged
Pass by reference	Address · int *n · f(&x) · use *n · original changed
Arrays as parameters	Passed as address; changes are permanent; pass the length separately
Return	One value only; several → pointer parameters; never return the address of a local

Pointer cheat sheet

Expression	Meaning
&x *p *&x	address of x · value at p · same as x
p + 1	next element (adds sizeof(type) bytes)
q − p	number of elements between two pointers (same array)
a[i] *(a+i) i[a]	all identical
*p++ (*p)++ *++p	read then move · increment value · move then read
int *p, q;	only p is a pointer
const int *p / int *const p	cannot change the value / cannot change the pointer
NULL · void * · wild · dangling	nothing · generic · uninitialised · points to dead memory

Recursion and logic recipes

Task	Recipe
Last digit / remove last digit	n % 10 and n / 10
Reverse a number	rev = rev * 10 + n % 10
Number of digits	while (n) { c++; n /= 10; }
GCD / LCM	gcd(a,b) = gcd(b, a % b) · lcm = a / gcd * b
Prime test	n ≥ 2 and no divisor with i * i <= n
Binary conversion	divide by 2; remainders in reverse (recurse first, print after)

Task	Recipe
Hanoi moves / Fibonacci calls	$2^n - 1$ moves · about 2^n calls (plain recursion)
Recursion needs	base case + a call that shrinks the problem

Fifteen common traps

1. Expecting a returned value to be used without storing or printing it.
2. Integer division: 245 / 3 is 81; use (float) or 3.0.
3. Returning a float from an int function truncates it.
4. Arguments in the wrong order — they are matched by position.
5. void function result assigned to a variable (compile error).
6. Prototype and definition with different types (conflicting types).
7. Code written after return is never executed.
8. Swap with pass by value changes only copies.
9. n = n + 5 instead of *n = *n + 5 with a pointer parameter.
10. Dereferencing NULL or an uninitialised pointer.
11. *p++ moves the pointer, not the value.
12. sizeof(array parameter) is the pointer size, not the array size.
13. Missing or wrong base case; recursive call that does not shrink the problem.
14. Local variables forget their value between calls; use static.
15. Macro without brackets: SQUARE(2+3) gives 11, not 25.

Appendix A — Debug It Yourself: Answer Key

Try every "Debug It Yourself" program on your own first — compile it, read the message, predict the output. Then check your reasoning here. Each entry names the bug, then shows only the corrected line(s) and the output after the fix.

Section 14.3

Debug It 1 — The compiler reports an error on the line after the mistake. Where is the real bug?

Answer: A semicolon is missing after int age = 20. Every declaration ends with ;. GCC reports the error at the next token, so the real mistake is on the previous line.

```
int age = 20;
```

Output after the fix: 20 5.8

Debug It 2 — The variable is declared, yet the compiler says it is undeclared.

Answer: C is case-sensitive: Total and total are two different names. Use the same spelling in the declaration and in printf().

```
printf("Total = %d", Total);
```

Output after the fix: Total = 100

Section 14.5

Debug It 1 — The average of 70, 75, 80 and 90 should be 78.75. Why is 78.00 printed?

Answer: The sum and 4 are both int, so the division is an integer division (315 / 4 = 78) and only then converted to float. Divide by 4.0 or cast the sum to float.

```
float avg = (m1 + m2 + m3 + m4) / 4.0;
```

Output after the fix: Average = 78.75

Debug It 2 — 50000 times 50000 should be 2500000000. The output is wrong. Why?

Answer: The true product 2,500,000,000 is larger than the biggest int (2,147,483,647), so the int overflows and a wrong (often negative) value is printed. Use long long and %lld, and convert BEFORE multiplying.

```
long long product = (long long)a * b;
printf("Product = %lld", product);
```

Output after the fix: Product = 2500000000

Section 14.8

Debug It 1 — The compiler rejects this printf(). What is missing?

Answer: A comma is missing between the format string and the value: printf("Sum = %d\n", a + b);

```
printf("Sum = %d\n", a + b);
```

Output after the fix: Sum = 10

Debug It 2 — The user types 20 and Enter, but the program never waits for the grade. Why?

Answer: After the number, the Enter key (a newline character) stays in the buffer and %c happily reads it as the grade. Put a space before %c — scanf(" %c", &grade); — so leading whitespace is skipped.

```
scanf(" %c", &grade);
```

Output after the fix: Age: Grade: 20 B

Section 15.2

Debug It 1 — Find the invalid operator.

Answer: C has no === operator. Equality is ==.

```
if (a == b)
```

Output after the fix: Equal

Debug It 2 — Where do the brackets of an if belong?

Answer: The if needs a single pair of parentheses around the entire condition: if ((marks >= 40) && (attendance >= 75)).

```
if ((marks >= 40) && (attendance >= 75))
```

Output after the fix: Eligible

Section 15.3

Debug It 1 — 45 marks out of 60 should give 75 percent. Why is 0 printed?

Answer: 45 / 60 is integer division and gives 0, so 0 * 100 is 0. Multiply first, or make one operand real: obtained * 100.0 / total.

```
float percent = obtained * 100.0 / total;
```

Output after the fix: Percentage = 75.00

Debug It 2 — This program crashes when the count is 0. Guard it.

Answer: Integer division by zero is undefined and normally crashes the program (floating point). Check the divisor before dividing.

```
if (count != 0)
   printf("Average = %d", total / count);
else
   printf("Cannot divide by zero");
```

Output after the fix: Cannot divide by zero

Section 15.4

Debug It 1 — Mathematically 0.1 + 0.2 equals 0.3, yet the program prints "Not equal".

Answer: 0.1 and 0.2 cannot be stored exactly, so the sum differs from 0.3 by a tiny amount. Compare real numbers with a tolerance: fabs(x - 0.3) < 0.000001.

```
#include <math.h>
   if (fabs(x - 0.3) < 0.000001)
```

Output after the fix: Equal

Debug It 2 — The programmer wants "a is not equal to b". What does =! do?

Answer: a =! b means a = (!b): b is 3 (true) so !b is 0, which is assigned to a and the condition is false. The correct not-equal operator is !=.

```
if (a != b)
```

Output after the fix: Same

Section 15.5

Debug It 1 — This will not compile. Fix the range check.

Answer: The right side of && is missing its left operand. Write marks > 90 && marks < 100.

```
if (marks > 90 && marks < 100)
```

Output after the fix: Excellent

Debug It 2 — x is 3, so "x is not 5" should print. It does not.

Answer: !x == 5 is (!x) == 5 = 0 == 5 = false. Write x != 5 or !(x == 5).

```
if (x != 5)
```

Output after the fix: x is not 5

Section 15.6

Debug It 1 — Why does this not compile?

Answer: The left side of = must be a variable (a place that can store a value), not an expression. Write c = a + b.

```
c = a + b;
```

Output after the fix: 7

Debug It 2 — The chained declaration fails. Why?

Answer: b is used before it exists. Declare both: int a, b; a = b = 10;

```
int a, b;
a = b = 10;
```

Output after the fix: 10 10

Section 15.7

Debug It 1 — The next token should be 101 but 100 is printed.

Answer: Post-increment gives the OLD value (100) and changes the variable afterwards. Use ++token, or token + 1.

```
printf("Next token: %d", ++token);
```

Output after the fix: Next token: 101

Debug It 2 — The programmer expects 10 or 11. Why is the result unreliable?

Answer: n is modified twice in one statement (by ++ and by =), which is undefined behaviour. Separate the steps: n = n * 2; n++; (or whatever you actually mean).

```
n = n * 2;
n++;
```

Output after the fix: 11

Section 15.8

Debug It 1 — Bit 3 of 12 (binary 1100) IS set, but the program says it is clear.

Answer: == binds tighter than &, so the test becomes n & ((1<<3) == 1) = n & 0. Even with brackets, a set bit 3 gives 8, not 1. Use (n & (1 << 3)) != 0.

```
if ((n & (1 << 3)) != 0)
```

Output after the fix: Bit 3 is set

Debug It 2 — The result should be 21 (5 shifted left by 2, plus 1).

Answer: + has HIGHER precedence than <<, so this is 5 << 3 = 40. Bracket the shift: (5 << 2) + 1.

```
int y = (5 << 2) + 1;
```

Output after the fix: 21

Section 15.9

Debug It 1 — The maximum of 20, 5 and 30 should be 30. Why is 20 printed?

Answer: It parses as a > b ? a : (b > c ? b : c). When a > b, a is chosen without ever being compared with c. Compare the winner with c: (a > b) ? ((a > c) ? a : c) : ((b > c) ? b : c).

```
int max = (a > b) ? ((a > c) ? a : c) : ((b > c) ? b : c);
```

Output after the fix: Max = 30

Debug It 2 — The text should be printed, but a number appears.

Answer: "big" and "small" are strings, so the format specifier must be %s.

```
printf("%s", x > 5 ? "big" : "small");
```

Output after the fix: big

Section 15.10

Debug It 1 — The value of a should become 6, but it stays 5.

Answer: *p++ increments the POINTER (moves it to the next int), not the value. To increase the value use (*p)++.

```
(*p)++;
```

Output after the fix: 6

Debug It 2 — An array of 3 elements should report a count of 3, but 12 is printed.

Answer: sizeof(arr) is the total size in bytes (3 x 4 = 12). Divide by the size of one element: sizeof(arr) / sizeof(arr[0]).

```
int n = sizeof(arr) / sizeof(arr[0]);
```

Output after the fix: Elements = 3

Section 15.11

Debug It 1 — The area of a ring (outer radius R, inner radius r) is wrong.

Answer: 3.14 multiplies only R*R. The difference of the squares must be bracketed: 3.14 * (R * R - r * r).

```
float area = 3.14 * (R * R - r * r);
```

Output after the fix: Area = 50.24

Debug It 2 — Half of n should be 4.0 for n = 8, but 0 is printed.

Answer: 1 / 2 is integer division = 0, and it happens first. Write n / 2 or 0.5 * n.

```
float half = n / 2;
```

Output after the fix: 4.0

Section 16.3

Debug It 1 — The alert appears even though the temperature is safe.

Answer: A semicolon after the condition makes the if do nothing; the printf then always runs. Remove the semicolon.

```
if (temp > 40)
```

Debug It 2 — The compiler complains about the if line.

Answer: The condition must be inside parentheses: if (x > 5).

```
if (x > 5)
```

Output after the fix: big

Section 16.4

Debug It 1 — An 18-year-old is refused although 18 is the voting age.

Answer: Off-by-one at the boundary: "18 or above" needs >= 18, not > 18.

```
if (age >= 18)
```

Output after the fix: Eligible to vote

Debug It 2 — The compiler reports "else without a previous if".

Answer: Two statements need braces { }. Without them the if controls only the first printf and the else has lost its if.

```
if (x > 0) {
} else
```

Output after the fix: Positive number

Section 16.5

Debug It 1 — The user exists but has the wrong password. The message is misleading.

Answer: The else pairs with if (passOk), so "Unknown user!" appears for a wrong password. Use braces and give each failure its own message.

```
if (userOk) {
   else
      printf("Wrong password!");
} else
```

Output after the fix: Wrong password!

Debug It 2 — The origin (0, 0) is reported as "On the X-axis".

Answer: The most specific case (the origin) must be tested first. Here y == 0 already catches it, and the last branch can never be reached.

```
if (x == 0 && y == 0)
   printf("Origin");
else if (y == 0)
```

Output after the fix: Origin

Section 16.6

Debug It 1 — This will not compile. What is wrong after else?

Answer: A plain else has no condition. Write else printf("Fail"); or else if (marks < 40).

```
Else
```

Output after the fix: Fail

Debug It 2 — a = 30, b = 30, c = 10: the maximum is 30, but 10 is printed.

Answer: With a tie, neither a > b nor b > a is true, so control falls into the last else. Use >= in the comparisons.

```
if (a >= b && a >= c)
else if (b >= a && b >= c)
```

Output after the fix: Max = 30

Section 16.7

Debug It 1 — The compiler reports a duplicate case value.

Answer: Each case constant must be unique. The second case 2 should probably be case 3.

```
    case 1: printf("One");   break;
    case 2: printf("Two");   break;
    case 3: printf("Three"); break;
```

Output after the fix: Two

Debug It 2 — Why is a variable not accepted as a case label?

Answer: Case labels must be constant expressions. Use the literal 5 (or a #define / enum constant) instead of the variable limit.

```
#define LIMIT 5
  int n = 5;
    case LIMIT: printf("At limit"); break;
```

Output after the fix: At limit

Section 16.8

Debug It 1 — A student with 95 marks gets no grade.

Answer: switch matches exact values only, so only exactly 90 gives an A. Use marks / 10 (with case 10 and case 9) or an if-else-if ladder.

```
if (marks >= 90)      printf("Grade A");
else if (marks >= 80) printf("Grade B");
else                  printf("No grade");
```

Output after the fix: Grade A

Debug It 2 — Several values should share one message, but the compiler rejects this.

Answer: C has no comma syntax in case. Stack the labels: case 1: case 2: printf("Low"); break;

```
case 1:
case 2: printf("Low");  break;
default: printf("Other");
```

Output after the fix: Low

Appendix B — Debug It Yourself: Answer Key

Try every "Debug It Yourself" program on your own first — compile it, read the message, predict the output. Then check your reasoning here. Each entry names the bug, then shows only the corrected lines and the output after the fix.

Chapter 17 — Decomposition

Debug It 1 — This program does not even compile. What does the compiler complain about?

Output of the buggy program: Compilation error (the program does not build)

Answer: A void function returns nothing, so its result cannot be assigned to total (error: void value not ignored as it ought to be). Either make the function int and return a + b + c, or do not assign its result.

```
int calculateTotal(int a, int b, int c)
{
   return a + b + c;
}
```

Output after the fix: Total = 245

Debug It 2 — For an average of 85 the program prints Grade = C. Why?

Output of the buggy program: Grade = C

Answer: The conditions are in the wrong order: 85 ≥ 60 is true, so C is returned immediately and the other branches are never examined. Test the highest threshold first (≥ 90, then ≥ 75, then ≥ 60).

```
   if (average >= 90) return 'A';
   else if (average >= 75) return 'B';
   else if (average >= 60) return 'C';
   else return 'F';
```

Output after the fix: Grade = B

Chapter 18 — Functions in C

Debug It 1 — The compiler stops with "void value not ignored as it ought to be". Which line is wrong and why?

Output of the buggy program: Compilation error (the program does not build)

Answer: display() returns nothing, so there is no value to store in r. Call it as a statement: display();

```
   display();
```

Output after the fix: Hello

Debug It 2 — The call add(5) is rejected by the compiler. What is wrong?

Output of the buggy program: Compilation error (the program does not build)

Answer: add() is defined with two parameters but called with one argument ("too few arguments to function 'add'"). The number and types of arguments must match the parameters: add(5, 3).

```
add(5, 3)
```

Output after the fix: 8

Chapter 19 — Parameters vs Arguments and Return Values

Debug It 1 — The area of a circle of radius 5 should be 78.50, but the program prints 78. Why?

Output of the buggy program: Area = 78

Answer: The function computes 78.5 but its return type is int, so the fractional part is truncated. Declare the function as float and print with %.2f.

```
float calculateArea(float radius)
printf("Area = %.2f", calculateArea(5));
```

Output after the fix: Area = 78.50

Debug It 2 — The compiler complains about "too many arguments to function 'add'". Fix it.

Output of the buggy program: Compilation error (the program does not build)

Answer: add() has two parameters but is called with three arguments. Either call add(2, 3), or redefine add() with three parameters (int a, int b, int c) and return a + b + c.

```
add(2, 3)
```

Output after the fix: 5

Chapter 20 — Pass by Value

Debug It 1 — The number should become 10, but 5 is printed. Where did the doubled value go?

Output of the buggy program: 5

Answer: It changed the local copy n, which vanishes when the function returns. Return the value (int doubleIt(int n) { return n * 2; } and num = doubleIt(num);) or pass &num.

```
int doubleIt(int n)
{
   return n * 2;
}
   num = doubleIt(num);
```

Output after the fix: 10

Debug It 2 — The user types 25, yet the program prints 0. Why?

Output of the buggy program: 0

Answer: scanf writes into the parameter n, which is a copy. num keeps 0. To read into the caller's variable, pass its address: void readNumber(int *n) { scanf("%d", n); } called as readNumber(&num);

```
void readNumber(int *n)
{
   scanf("%d", n);
}
readNumber(&num);
```

Output after the fix: 25

Chapter 21 — Pointers in C

Debug It 1 — The program is supposed to store 10 through the pointer. Why is it dangerous?

Output of the buggy program: Undefined behaviour — usually a crash (segmentation fault)

Answer: p is uninitialised, so it holds a random address. Writing through it is undefined behaviour (usually a crash). Make it point to a real variable first: int x; int *p = &x; *p = 10;

```
   int x;
   int *p = &x;
   *p = 10;
```

Output after the fix: 10

Debug It 2 — The compiler reports an incompatible integer-to-pointer conversion. Fix the declaration.

Output of the buggy program: Compilation error (the program does not build)

Answer: A pointer must be initialised with an address, not a value. x is an int, so int *p = x is wrong; write int *p = &x;. (Newer GCC versions treat this as an error; older ones only warn and the program then crashes.)

```
int *p = &x;
```

Output after the fix: 10

Chapter 22 — Pass by Reference (Using Pointers)

Debug It 1 — increase(&x) is called but x is still 10. Why?

Output of the buggy program: 10

Answer: n = n + 5 moves the local pointer 5 ints away (pointer arithmetic) and never touches x. Use *n = *n + 5 to change the value at the address.

```
   *n = *n + 5;
```

Output after the fix: 15

Debug It 2 — The compiler complains about the argument of swap(x, y). Fix the call.

Output of the buggy program: Compilation error (the program does not build)

Answer: The parameters are int *, but swap(x, y) passes int values (error: incompatible integer to pointer conversion). Pass addresses: swap(&x, &y);

```
swap(&x, &y);
```

Output after the fix: 2 1

Chapter 23 — Helper Functions and Modular Programs

Debug It 1 — The program says 4 is prime. Find the exact bug.

Output of the buggy program: 1

Answer: The loop condition i < n / 2 should be i <= n / 2. For n = 4 the loop starts at i = 2 and 2 < 2 is false, so the divisor 2 is never tried and 4 is reported as prime.

```
i <= n / 2
```

Output after the fix: 0

Debug It 2 — For 250 units the bill prints Rs. 250.00 instead of Rs. 750.00. Why?

Output of the buggy program: Rs. 250.00

Answer: Only the last slab is charged. Each higher slab must add the cost of all full lower slabs: 100 × 2 + (units − 100) × 3 for the middle slab, and 100 × 2 + 100 × 3 + (units − 200) × 5 for the top slab.

```
bill = 100 * 2 + (units - 100) * 3;
bill = 100 * 2 + 100 * 3 + (units - 200) * 5;
```

Output after the fix: Rs. 750.00

Chapter 24 — Recursive Thinking and Recursion

Debug It 1 — power(2, 3) crashes. What is wrong with the recursive call?

Output of the buggy program: Runtime crash (segmentation fault)

Answer: The recursive call passes n unchanged, so n never reaches 0 and the base case is never met (stack overflow). Reduce the problem: power(x, n − 1).

```
power(x, n - 1);
```

Output after the fix: 8

Debug It 2 — fib(5) should be 5 but 8 is printed. Which base case is wrong?

Output of the buggy program: 8

Answer: The base case returns 1 for both n = 0 and n = 1, but F(0) = 0. Returning n for n ≤ 1 gives F(0) = 0 and F(1) = 1, which makes fib(5) = 5.

```
    return n;
```

Output after the fix: 5

Chapter 25 — Scope, Lifetime and Storage Classes

Debug It 1 — The program prints a garbage value instead of 1 on the first call. What is missing?

Output of the buggy program: Undefined behaviour — usually a crash (segmentation fault)

Answer: A local (auto) variable is not initialised; it holds garbage, so count++ gives a garbage-plus-one result (undefined behaviour). Initialise it: int count = 0;

```
int count = 0;
```

Output after the fix: 1

Debug It 2 — The compiler says "'result' undeclared" in main(). Why?

Output of the buggy program: Compilation error (the program does not build)

Answer: result is local to compute() and does not exist in main(). Return it from the function (int compute(void) { return 42; }) or make it global.

```
int compute(void)
{
   int result = 42;
   return result;
}
   int result = compute();
   printf("%d", result);
```

Output after the fix: 42

Appendix C — Practice MCQs: Answer Key

Check these only after attempting all the practice MCQs of Section 2.12.

Q	Answer
1	b) No arguments, with a return value
2	c) Compilation error — too few arguments
3	d) 3
4	a) 7
5	c) Pass by reference is simulated by passing addresses (pointers)
6	d) 10
7	b) 6
8	a) 10
9	d) Its result cannot be assigned to a variable
10	c) int *p = x;
11	a) 24
12	b) 15 10
13	c) pointer to pointer
14	d) 1 2 3 4
15	b) #define MAX(a,b) (((a)>(b))?(a):(b))
16	a) A static local variable
17	a) 120
18	b) show(arr, 10);